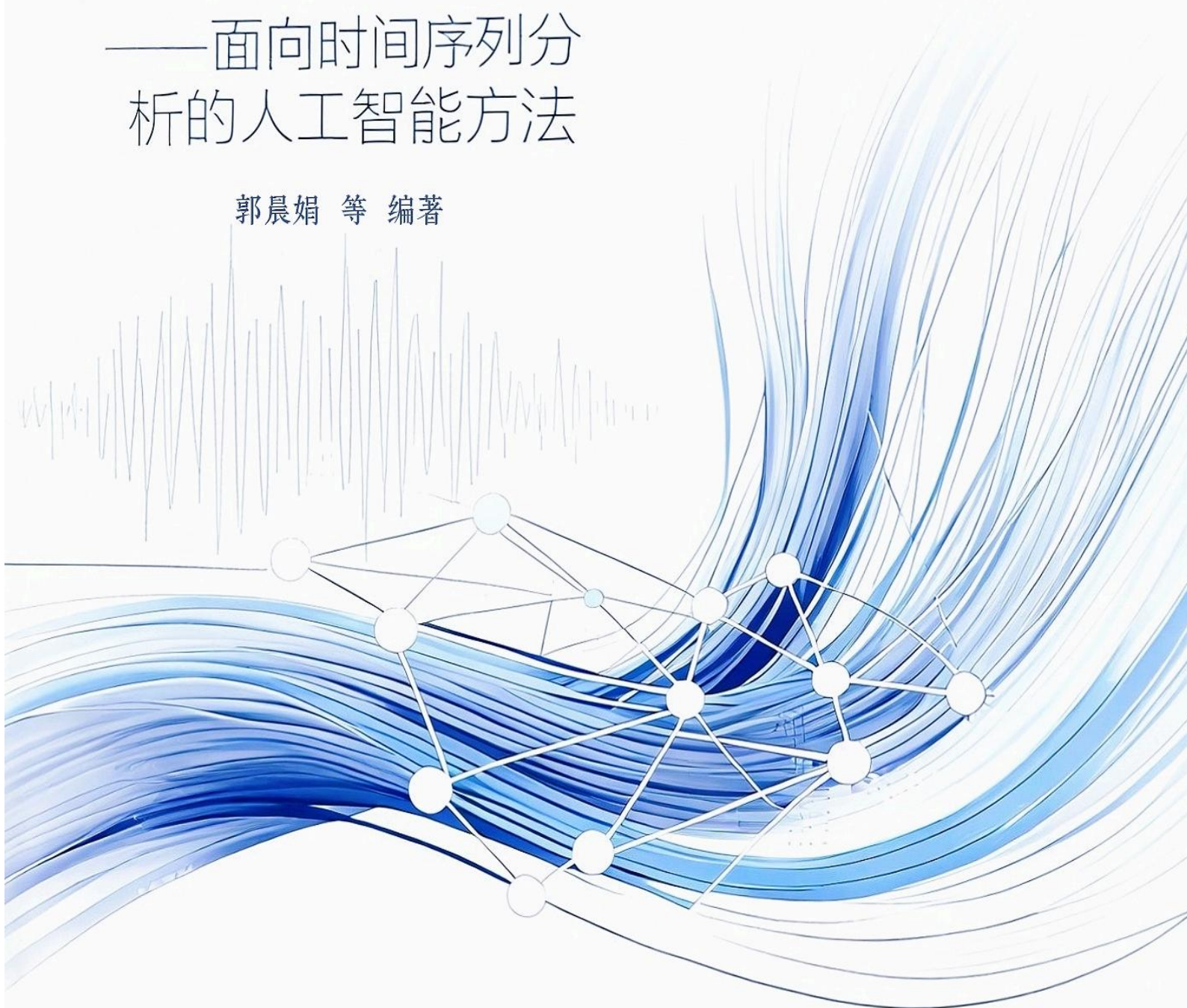


AI for Time Series

时序智能

——面向时间序列分析的人工智能方法

郭晨娟 等 编著



编著团队介绍

主编

郭晨娟

华东师范大学 数据科学与工程学院

教授 博士生导师 国家级青年人才 上海市领军人才

副主编

杨 彬

华东师范大学 数据科学与工程学院

二级教授 博士生导师 国家级领军人才

胡吉林

华东师范大学 数据科学与工程学院

教授 博士生导师 国家级青年人才

树 扬

华东师范大学 数据科学与工程学院

助理教授 硕士生导师 晨晖学者

编著组

该书编著工作由华东师范大学决策智能实验室 (Decision Intelligence Lab) 成员共同参与完成，负责专著的章节编写、案例设计、实验验证与配套资源开发。主编和副主编对各章节拟定提纲，组织研究章节难点与问题，并对全章统稿、修改并最终定稿。

其他参与编著成员如下（按照姓名音序排列）：

陈 鹏	陈宇轩	成涵吟	丁逸飞
高洪帆	胡诗彦	黄姗姗	金建新
雷 智	李北步	李 哲	李正宇
陆骏凯	卢雨凝	邱翔飞	邱钰莹
申屠琦超	田锦东	王益杭	汪琳丰
汪思远	吴行健	严思雨	徐榕荟
张睿桐	郑兴泽		

目 录

1 介绍	2
1.1 读者对象	3
1.2 本书结构	3
1.3 初识时间序列	3
1.4 时间序列的定义和分类	5
1.5 时间序列算法的发展历程	6
参考文献	9
2 基础概念	16
2.1 时间序列数据概述	17
2.1.1 时间序列基本特性	17
2.1.2 数据来源	21
2.2 时间序列数据治理	22
2.2.1 时间序列缺失值处理	22
2.2.2 时间序列平稳化处理	25
2.2.3 时间序列平滑处理	29
2.2.4 时间序列分解处理	31
2.3 时间序列数据建模角度	35
2.3.1 从时域角度建模	35
2.3.2 从频域角度建模	37
2.4 基础深度学习模型	39
2.4.1 多层感知机	39
2.4.2 卷积神经网络	40
2.4.3 循环神经网络	41
2.4.4 时间卷积网络	42
2.4.5 图卷积网络	43
2.4.6 Transformer	44
参考文献	46
3 时间序列预测	54
3.1 时间序列预测定义及流程	55
3.2 时间序列预测模型训练与评估	57
3.3 时间序列预测模型	58

3.3.1 统计学习方法	58
3.3.2 机器学习方法	59
3.3.3 多层感知机	59
3.3.4 循环神经网络	62
3.3.5 卷积神经网络	64
3.3.6 Transformer	69
3.4 时间序列通道关系	76
3.4.1 通道关系的不同策略	77
3.4.2 基于 Transformer 的通道依赖建模	78
3.4.3 基于图神经网络的通道依赖建模	80
3.4.4 基于多层感知机和卷积神经网络的通道依赖建模	81
3.4.5 相关性的不同特征	82
3.4.6 总结	83
3.5 时间序列概率预测	84
参考文献	89
4 时间序列异常检测	99
4.1 时间序列异常检测定义及流程	100
4.2 时间序列异常类型	101
4.3 时间序列异常检测模型训练与评估	102
4.3.1 基于有监督分类的异常检测方法	103
4.3.2 基于无监督重构的异常检测方法	105
4.3.3 基于无监督预测的异常检测方法	107
4.3.4 其他异常检测方法	108
4.4 时间序列异常检测模型	108
4.4.1 统计方法	108
4.4.2 经典机器学习方法	109
4.4.3 卷积神经网络	109
4.4.4 Transformer	113
4.4.5 自编码器	118
4.4.6 变分自编码器	124
4.4.7 循环神经网络	133
4.4.8 图神经网络	137
4.4.9 对比学习	140
参考文献	145

5 时间序列分类	154
5.1 时间序列分类定义及流程	155
5.2 时间序列分类模型训练与评估	156
5.3 时间序列分类模型	158
5.3.1 经典机器学习方法	158
5.3.2 多层感知机	165
5.3.3 卷积神经网络	166
5.3.4 循环神经网络	171
5.3.5 图神经网络	173
5.3.6 注意力机制	176
5.4 时间序列分类自监督方法	182
5.4.1 基于序列的对比学习	182
5.4.2 基于实例的对比学习	185
5.4.3 掩码-重构学习	185
参考文献	186
6 自动化时间序列分析	193
6.1 时间序列分析自动化简介	194
6.2 时间序列分析模型自动选择	195
6.2.1 模型自动选择的定义及流程	195
6.2.2 模型自动选择策略	196
6.2.3 基于搜索的模型自动选择	197
6.2.4 基于元学习的模型自动选择	199
6.3 时间序列分析模型自动集成	202
6.3.1 模型自动集成的定义及流程	202
6.3.2 模型自动集成策略	203
6.3.3 数据预处理方法	205
6.3.4 输出集成方法	206
6.4 时间序列分析模型自动设计	207
6.4.1 模型自动设计的定义及流程	207
6.4.2 模型自动设计策略	208
6.4.3 任务特定的模型自动设计	209
6.4.4 可泛化的模型自动设计	214
参考文献	218
7 时间序列基础模型	221

7.1 基于预训练的时间序列基础模型	222
7.1.1 模型定义及流程	222
7.1.2 模型训练及评估	223
7.1.3 模型关键技术	224
7.2 基于大语言模型的时间序列基础模型	233
7.2.1 模型定义及流程	233
7.2.2 模型训练及评估	234
7.2.3 模型关键技术	235
参考文献	239
8 时间序列评测基准	243
8.1 评测基准的背景与意义	244
8.2 时间序列评测基准的发展脉络	245
8.3 时间序列评测基准框架	248
8.4 数据层	250
8.4.1 时序数据集收集	250
8.4.2 时序数据集筛选	251
8.4.3 时序数据集划分	252
8.5 方法层	254
8.5.1 设计原则	254
8.5.2 方法类别	254
8.6 评估层	256
8.6.1 评估设置	256
8.6.2 评估指标的选择	259
8.7 报告层	262
参考文献	266
9 神经微分方程时间序列分析	269
9.1 微分方程与时间序列	270
9.1.1 微分方程简介	270
9.1.2 微分方程求解方法	272
9.1.3 时间依赖型微分方程与时间序列	273
9.2 神经微分方程	275
9.2.1 神经微分方程简介	275
9.2.2 反向传播算法	276
9.3 相关应用研究	277

参考文献	281
------------	-----

Preprint
Decision
Intelligence
Lab

1 介绍

本章作者为：郑兴泽、郭晨娟、杨彬。

本章主编为郭晨娟。

本章首先介绍本书的面向对象和结构，随后引入时间序列和主要分析任务，接着介绍时间序列的定义、分类及其算法的发展历程。

1.1 读者对象

作为深度学习背景下，时间序列分析的入门书籍，本书在撰写过程中注重可读性，期望入门读者完全可以较轻松地掌握相关概念。同时，对于有经验的读者，也可以从中发现许多实用新颖的知识。如果读者在阅读本书前，已经了解了高等数学、概率论与数理统计、线性代数、算法分析与设计等知识，那么理解过程会更加容易。

1.2 本书结构

本书的目的是让读者理解时间序列分析的基本概念和基本原理，重点关注应用深度学习解决实际问题的方法。全书共有 9 章。

第 1 章首先介绍了时间序列的基础知识，涵盖了时间序列的概念、定义和分类，以及时间序列分析的概念和任务。另外，还介绍了时间序列算法从统计方法到深度学习的发展历程。

第 2 章介绍了时间序列的基础概念。涵盖了时间序列数据的特性、可视化、来源、处理方法和特征工程。然后，介绍了应用于时间序列分析的基础深度学习模型，以及两种时间序列数据建模角度。

第 3 章介绍了时间序列预测任务。包括时间序列预测的定义、流程和关键模型，以及不同通道处理策略的建模方法。

第 4 章介绍了时间序列异常检测任务。包括时间序列异常检测的定义、流程、异常类型、检测方法和相关模型。

第 5 章介绍了时间序列分类任务。包括时间序列分类的定义和流程，以及基于机器学习、深度学习和自监督的分类方法。

第 6 章介绍了自动化时间序列分析。包括模型的自动选择、自动集成和自动设计。

第 7 章介绍了时间序列大模型。包括时间序列基础模型的定义、流程和架构，以及相关模型的介绍。

第 8 章介绍了时间序列分析的基准评测。

第 9 章介绍了神经微分方程。包括微分方程的定义和求解方法，以及神经微分方程在时间序列中的应用。

1.3 初识时间序列

时间序列是通过对某一特定实体（Entity）的指标在时间维度上的采样，记录其随时间变化的动态过程。具体地，时间序列的数据来源于对某个实体（如物理系统、

生物体等)在不同时间点上进行测量或记录,形成一系列有序的数据点序列。这些数据点代表了实体在各个采样时刻的状态或行为,可以是数值型(如温度值)或类别型(如晴天、雨天、多云等表示的天气状况)。时间序列记录了这些数据点随时间推移的变化趋势,因此每个数据点都关联到一个特定的时间戳。

时间序列分析是通过对历史时间序列的研究,理解数据的内在规律,以支持未来的下游任务和决策。随着数字化进程的不断推进,时间序列分析广泛应用于金融、交通、医疗、气象、能源和物联网等多个领域,主要任务包括:时间序列预测、时间序列异常检测、时间序列分类、时间序列插值、时间序列聚类,各个任务在不同领域有不同应用。图 1.1 为各时间序列分析任务的示例。

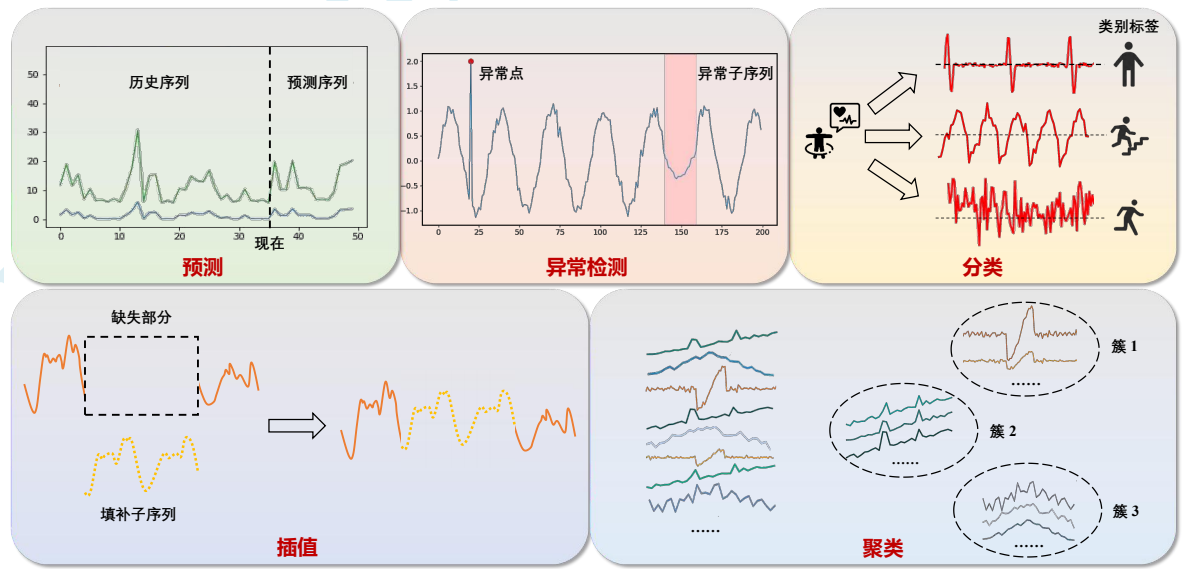


图 1.1: 时间序列分析任务示例。

时间序列预测 (Time Series Forecasting) 是一项基于历史数据推断未来数据的关键任务,通过分析过去的序列来预测未来趋势,广泛应用于股票市场分析、交通流量预测和能源需求预测等领域。在工业领域,石油产量预测是上游油气供应链中的核心环节,通过油藏性质等相关因素预测未来产量,从而判断产量的可持续开采时间 [46]。此外,随着云计算需求的快速增长,合理分配云资源以满足用户需求成为确保系统稳定性和成本控制的关键,这需要通过预测来“提前掌握用户需求”,从而优化扩缩容决策 [42]。

时间序列异常检测 (Time Series Anomaly Detection) 从时间序列数据中识别与正常模式显著不同的点或子序列,在设备故障检测、医疗诊断等领域具有重要意义。工业设备的运行数据反映了其工作状态,通过实时监控设备运行情况,自动识别超出正常范围的异常波动,提醒运维人员设备可能出现故障,进而采取预防性维修或更换措施,避免设备的严重故障和生产中断 [30]。心电图 (ECG) 信号常用于评估心脏健康

状况，正常情况下，ECG 波形具有规律性，但当心脏发生异常，如心律失常时，ECG 波形会出现不规则变化，通过实时监测患者的心脏状况，检测是否存在偏离正常模式的情况，可以帮助医生做出快速诊断并进行干预 [59]。

时间序列分类 (Time Series Classification) 将时间序列数据分配到预定义的类别中，在医疗诊断、行为识别等领域有广泛应用。如前文所述，心电图 (ECG) 信号通过异常检测可以初步识别异常情况，进一步地，ECG 通过分类可以识别具体的心脏疾病类型 [59, 13]。类似地，癫痫患者的脑电图 (EEG) 信号通过分类可以识别具体的癫痫类型 [59]。此外，对由佩戴在人体左脚踝、右脚踝、腰部和胸部的四个传感器不恒定测量得到的 3D 位置数据进行分类，可以识别相应的人体行为，包括行走、坐着、躺着等 [50, 58]。

时间序列插值 (Time Series Imputation) 在时间序列中填补缺失的数据点，广泛应用于环境监测、气象、影音数据恢复等领域。气象站在不同地点收集温度、降水量等气象数据，由于设备故障、网络传输中断或其他原因，某些时间点的数据可能缺失，通过填补缺失值，可以生成完整的气象记录，从而用于气候研究和预测 [55]。在音频信号中，可能会出现由于噪声或干扰导致的瞬时失真，通过对音频信号中丢失的部分进行插值，以提升音质 [47]。

时间序列聚类 (Time Series Clustering) 将相似的时间序列聚合为同一组，广泛应用于生物学、行为分析等领域。通过对基因表达数据进行聚类，可以表明基因群对特定细胞过程的共同贡献 [18]。此外，时间序列聚类还可以作为预处理的一步，因为在某些情况下，需要处理的时间序列非常多，分别对其进行分析的工作量过于庞大，此时可以先将这些时间序列聚类，然后再对每一组中的所有序列建模进行下游任务 [1]。

时间序列分析在各个领域中发挥着重要作用。未来，随着机器学习和深度学习技术的不断进步，以及大数据和云计算的进一步发展，时间序列分析的应用将更加广泛和深入。

1.4 时间序列的定义和分类

时间序列是一串按采样时间戳排序的一组向量，可以用公式1表示为：

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times N} \quad (1)$$

其中， $\mathbf{x}_t \in \mathbb{R}^N$ 是在时间戳 t 上的观测向量，包含 N 个变量。

均匀采样时间序列与非均匀采样时间序列：每两个观测向量之间的采样时间间隔可以是恒定的 (Regular)，也可以是非恒定的 (Irregular)。恒定时间间隔指时间序列中的每两个相邻数据点之间的时间间隔是固定的，保持一致，即采样时间是规则的，这种时间序列是均匀采样时间序列；而非恒定时间间隔指时间序列中的每两个相邻数

据点之间的时间间隔不固定，可能变化，即采样时间是不规则的，可能由某些事件触发，或者取决于数据采集的条件，这种时间序列是非均匀采样时间序列。

单变量时间序列与多变量时间序列：时间序列根据每个数据点中的变量数量，可以分为单变量（Univariate）时间序列（ $N=1$ ）和多变量（Multivariate）时间序列（ $N>1$ ）。单变量时间序列指仅包含一个变量在不同时间点上的观测值的序列，是一串有序的数据点。多变量时间序列指同时包含多个相关变量在不同时间点上的观测值的序列，是多条单变量时间序列。

单变量时间序列和多变量时间序列广泛存在于现实生活中。某地的气温形成的序列是单变量时间序列，而多地的交通流量形成的序列是多变量时间序列。当时间序列包含多个变量时，它们之间的相互关系值得考虑，因为它们之间不一定相互独立。图 1.2 展示了四个地点一周的交通流量时间序列 [11]。地点 1 和地点 2 位于同一条街道，与位于另一条街道的地点 3 和地点 4 相比，表现出不同的模式。而地点 1 和地点 2 由于位于同一条街道，因此表现出相似的模式。同理，地点 3 和地点 4 也是如此。

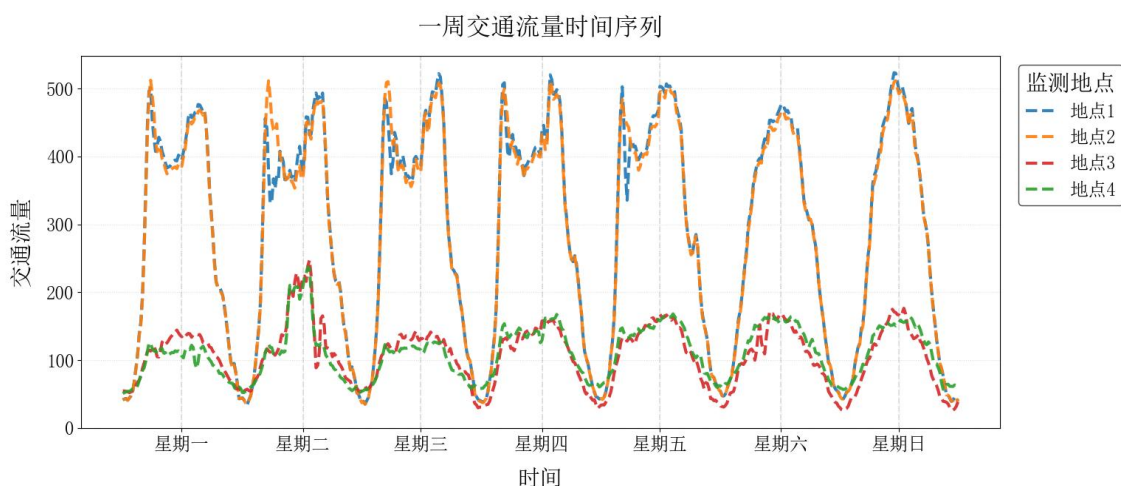


图 1.2: 一周交通流量时间序列。

1.5 时间序列算法的发展历程

时间序列算法的发展经历了从传统统计方法到现代深度学习的演变，可以分为四个主要阶段：统计学习、机器学习、深度学习和预训练大模型，每个阶段都有其优势与不足，持续推动着时间序列分析技术的进步。表 1.1 总结了这四个阶段的时间序列算法。

统计学习（Statistical Learning）方法 [2, 6, 25, 22, 23]，如自回归移动平均和季节性分解，建立在稳健的统计理论之上，其模型的参数具有明确的统计意义，容易理

表 1.1: 时间序列算法总结。

方法	模型	优势	劣势	代表作
统计学习	统计学习模型	理论基础稳健，参数可解释性强	难以处理非线性关系和多变量	ARIMA [6]、ETS [25]、Theta [22]、VAR [23]
机器学习	机器学习模型	模型多样，可处理非线性关系	依赖特征工程，易过拟合	XGBoost [9, 57]、GBRT [17]、随机森林 [7, 39]、LightGBM [31]
深度学习	多层感知器 (MLP)	结构简单，支持并行计算	长期依赖捕捉弱，时间顺序处理差	NLinear [56]、DLinear [56]
	卷积神经网络 (CNN)	局部特征提取强，短期依赖捕捉强	全局依赖捕捉弱，时间顺序敏感性低	TimesNet [53]、TIMEMIXER++ [51]
	循环神经网络 (RNN)	时序依赖捕捉强	梯度问题严重，训练慢	DeepAR [45]、NSIBF [15]
	时序卷积网络 (TCN)	解决 RNN 长期依赖问题，并行高效	需深层网络，计算开销大	TCN [4]、ModernTCN [37]
	图卷积网络 (GCN)	时空依赖捕捉强	计算复杂，依赖图结构	DGCNN [48]、GraphSleepNet [26]
	Transformer	全局信息捕捉强，并行处理	计算资源消耗大	Informer [60]、Autoformer [54]、Triformer [10]、PatchTST [40]
大模型	预训练大模型	泛化能力强，多任务适用	数据和计算资源需求大，可解释性差	TimesFM [12]、Moirai [52]、Chronos [3]、TimeGPT-1 [21]、MOMENT [24]、UniTS [20]、Time-LLM [27]、Lag-llama [43]、Timer [36]

解和解释，适用于简单的平稳时间序列。但是统计模型基于线性假设，难以处理非线性关系和复杂的数据模式，需要根据数据特性手动选择合适的模型，并且统计模型通常对异常值较敏感，这可能影响预测准确性。此外，统计学习方法在处理多变量时序时效果不佳，通常只能处理单一变量。

机器学习 (Machine Learning) 方法 [38, 5, 16, 9, 57, 17, 7, 39, 31] 提供了多样化的模型选择，引入了非线性模型，如支持向量机和决策树，适用于多变量时间序列，能够处理复杂的非线性关系，还可以通过特征提取和选择，提升模型性能，适用于更多实际应用场景。但是许多机器学习模型的性能往往依赖于人工设计的特征且容易过拟合，特别是在数据稀疏或样本不足的情况下。

深度学习 (Deep Learning) 方法 [35, 19, 8, 41, 56, 45, 60, 61, 54, 40, 10] 可以根据其骨干架构分为：基于多层感知器的、基于卷积神经网络的、基于循环神经网络的、基于时序卷积网络的、基于图卷积网络的和基于 Transformer 的。

- 多层感知器 (MLP) [44] 设计简单，支持并行计算，适合短期依赖和简单任务。但它对长时间序列数据的处理能力有限，难以有效捕捉长期依赖关系，缺乏对时间顺序的处理能力。
- 卷积神经网络 (CNN) [33] 具有优秀的特征提取能力，擅长提取局部特征，捕捉短期依赖。但它在捕捉全局依赖方面存在局限性，对序列的时间顺序敏感性较低。
- 循环神经网络 (RNN) [14] 特别适合处理序列数据，能够捕捉时间上的依赖关系。但它训练时面临梯度消失和爆炸问题，训练时间长，长序列处理效果不佳。
- 时序卷积网络 (TCN) [4] 克服了 RNN 的长期依赖问题，并行计算高效。但为了有效捕捉长期依赖关系，它通常需要更深的网络结构，这会导致计算开销的增加。
- 图卷积网络 (GCN) [32] 能够捕捉空间依赖性和时间依赖性，适合处理多变量的复杂任务。但它的计算复杂度较高，依赖图的拓扑结构。
- Transformer [49] 利用强大的自注意力机制，能够并行处理序列，具备全局信息捕捉能力，能够同时处理局部和全局依赖。但它的计算资源消耗大，对数据量需求较高。

预训练大模型 (Pretrained Large Model) [34, 28, 29, 12, 52, 3, 21, 24, 20, 27, 43, 36] 能够通过海量数据预训练提升模型性能，减少训练时间，利用在大规模数据上学习到丰富的特征，提高模型的泛化能力，可用于多种下游任务。但它需要大量数据和计算资源进行预训练，在特定任务上微调时可能会面临过拟合问题，并且其可解释性仍然是一个挑战。

时间序列算法的发展历程从统计学习的可解释性和理论基础逐步演进到深度学习的强大建模能力，虽然深度学习和预训练大模型在捕捉复杂模式方面表现优秀，但同时也面临可解释性、计算成本等问题。因此，在实际应用中，选择合适的时间序列分析方法取决于具体任务的需求、数据特性和可用资源。

参考文献

- [1] Saeed Aghabozorgi, Ali Seyed Shirkhorshidi, and Teh Ying Wah. Time-series clustering—a decade review. *Information Systems*, 53:16–38, 2015.
- [2] Theodore W Anderson. *The statistical analysis of time series*. John Wiley & Sons, 2011.
- [3] Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Türkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda-Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [4] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [5] Gianluca Bontempi, Souhaib Ben Taieb, and Yann-Aël Le Borgne. Machine learning strategies for time series forecasting. In *Proceedings of the Business Intelligence*, pages 62–77, 2012.
- [6] George EP Box and David A Pierce. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *Journal of the American Statistical Association (JASA)*, 65(332):1509–1526, 1970.
- [7] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [8] Cristian Challu, Kin G Olivares, Boris N Oreshkin, Federico Garza Ramirez, Max Mergenthaler Canseco, and Artur Dubrawski. Nhits: Neural hierarchical interpolation for time series forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 6989–6997, 2023.
- [9] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 785–794, 2016.
- [10] Razvan-Gabriel Cirstea, Chenjuan Guo, Bin Yang, Tung Kieu, Xuanyi Dong, and Shirui Pan. Triformer: Triangular, variable-specific attentions for long se-

- quence multivariate time series forecasting. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1994–2001, 2022.
- [11] Razvan-Gabriel Cirstea, Bin Yang, Chenjuan Guo, Tung Kieu, and Shirui Pan. Towards spatio-temporal aware traffic time series forecasting. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 2900–2913, 2022.
- [12] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [13] Hoang Anh Dau, Eamonn Keogh, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, Yanping, Bing Hu, Nurjahan Begum, Anthony Bagnall, Abdullah Mueen, Gustavo Batista, and Hexagon-ML. The ucr time series classification archive, 2018.
- [14] Jeffrey L Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
- [15] Cheng Feng and Pengwei Tian. Time series anomaly detection for cyber-physical systems via neural system identification and bayesian filtering. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 2858–2867, 2021.
- [16] Jan Alexander Fischer, Philipp Pohl, and Dietmar Ratz. A machine learning approach to univariate time series forecasting of quarterly earnings. *Review of Quantitative Finance and Accounting*, 55:1163–1179, 2020.
- [17] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [18] André Fujita, Patricia Severino, Kaname Kojima, João Ricardo Sato, Alexandre Galvão Patriota, and Satoru Miyano. Functional clustering of time series gene expression data by granger causality. *BMC Systems Biology*, 6(1):137, 2012.
- [19] John Cristian Borges Gamboa. Deep learning for time-series analysis. *arXiv preprint arXiv:1701.01887*, 2017.
- [20] Shanghua Gao, Teddy Koker, Owen Queen, Thomas Hartvigsen, Theodoros Tsiligkaridis, and Marinka Zitnik. Units: A unified multi-task time series model. *arXiv preprint arXiv:2403.00131*, 2024.

- [21] Azul Garza and Max Mergenthaler-Canseco. Timegpt-1. *arXiv preprint arXiv:2310.03589*, 2023.
- [22] Federico Garza, Max Mergenthaler Canseco, Cristian Challú, and Kin G Olivares. Statsforecast: Lightning fast forecasting with statistical and econometric models. In *Proceedings of the PyCon*, page 66, 2022.
- [23] Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. In *Proceedings of the Conference on Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Track on Datasets and Benchmarks)*, 2021.
- [24] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. MOMENT: A family of open time-series foundation models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [25] Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with exponential smoothing: the state space approach*. Springer Science & Business Media, 2008.
- [26] Ziyu Jia, Youfang Lin, Jing Wang, Ronghao Zhou, Xiaojun Ning, Yuanlai He, and Yaoshuai Zhao. Graphsleepnet: Adaptive spatial-temporal graph convolutional networks for sleep stage classification. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1324–1330, 2020.
- [27] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-llm: Time series forecasting by reprogramming large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [28] Ming Jin, Qingsong Wen, Yuxuan Liang, Chaoli Zhang, Siqiao Xue, Xue Wang, James Zhang, Yi Wang, Haifeng Chen, Xiaoli Li, Shirui Pan, Vincent S. Tseng, Yu Zheng, Lei Chen, and Hui Xiong. Large models for time series and spatio-temporal data: A survey and outlook. *arXiv preprint arXiv:2310.10196*, 2023.
- [29] Ming Jin, Yifan Zhang, Wei Chen, Kexin Zhang, Yuxuan Liang, Bin Yang, Jindong Wang, Shirui Pan, and Qingsong Wen. Position: What can large language models tell us about time series analysis. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

- [30] Pooja Kamat and Rekha Sugandhi. Anomaly detection for predictive maintenance in industry 4.0-a survey. *E3S Web of Conferences*, 170:02007, 2020.
- [31] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 3146–3154, 2017.
- [32] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [33] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE*, pages 2278–2324, 1998.
- [34] Yuxuan Liang, Haomin Wen, Yuqi Nie, Yushan Jiang, Ming Jin, Dongjin Song, Shirui Pan, and Qingsong Wen. Foundation models for time series analysis: A tutorial and survey. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 6555–6565, 2024.
- [35] Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: a survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.
- [36] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [37] Donghao Luo and Xue Wang. ModernTCN: A modern pure convolution structure for general time series analysis. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2024.
- [38] Ricardo P Masini, Marcelo C Medeiros, and Eduardo F Mendes. Machine learning advances for time series forecasting. *Journal of Economic Surveys*, 37(1):76–111, 2023.
- [39] Jie Mei, Dawei He, Ronald Harley, Thomas Habetler, and Guannan Qu. A random forest method for real-time price forecasting in new york electricity mar-

- ket. In *Proceedings of the IEEE Power & Energy Society General Meeting | Conference & Exposition (PES)*, pages 1–5, 2014.
- [40] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [41] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [42] Zhicheng Pan, Yihang Wang, Yingying Zhang, Sean Bin Yang, Yunyao Cheng, Peng Chen, Chenjuan Guo, Qingsong Wen, Xiduo Tian, Yunliang Dou, et al. Magicscaler: Uncertainty-aware, predictive autoscaling. *Proceedings of the Very Large Data Bases Endowment*, 16(12):3808–3821, 2023.
- [43] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, Anderson Schneider, Sahil Garg, Alexandre Drouin, Nicolas Chapados, Yuriy Nevmyvaka1, and Irina Rish. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.
- [44] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [45] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. Deepar: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.
- [46] Leonid B Sheremetov, Arturo González-Sánchez, Itzamá López-Yáñez, and Andrew V Ponomarev. Time series forecasting: applications to the upstream oil and gas supply chain. *International Federation of Automatic Control*, 46(9):957–962, 2013.
- [47] Paris Smaragdis, Bhiksha Raj, and Madhusudana Shashanka. Missing data imputation for time-frequency representations of audio signals. *Journal of Signal Processing Systems*, 65(3):361–370, 2011.

- [48] Tengfei Song, Wenming Zheng, Peng Song, and Zhen Cui. Eeg emotion recognition using dynamical graph convolutional neural networks. *IEEE Transactions on Affective Computing*, 11(3):532–541, 2018.
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [50] Vedrana Vidulin, Mitja Lustrek, Bostjan Kaluza, Rok Piltaver, and Jana Krivec. Localization data for person activity. University of California, Irvine Machine Learning Repository (UCI ML Repository), 2010.
- [51] Shiyu Wang, Jiawei Li, Xiaoming Shi, Zhou Ye, Baichuan Mo, Wenze Lin, Sheng-tong Ju, Zhixuan Chu, and Ming Jin. TimeMixer++: A general time series pattern machine for universal predictive analysis. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2025.
- [52] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 53140–53164, 2024.
- [53] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. TimesNet: Temporal 2D-variation modeling for general time series analysis. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2023.
- [54] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 22419–22430, 2021.
- [55] Ceylan Yozgatligil, Sipan Aslan, Cem Iyigun, and Inci Batmaz. Comparison of missing value imputation methods in time series: the case of turkish meteorological data. *Theoretical and Applied Climatology*, 112(1):143–167, 2013.
- [56] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 11121–11128, 2023.

- [57] Lingyu Zhang, Wenjie Bian, Wenyi Qu, Liheng Tuo, and Yunhai Wang. Time series forecast of sales volume based on xgboost. *Journal of Physics: Conference Series (JPCS)*, 1873(1):012067, 2021.
- [58] Weijia Zhang, Chenlong Yin, Hao Liu, Xiaofang Zhou, and Hui Xiong. Irregular multivariate time series forecasting: A transformable patching graph neural networks approach. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [59] Weiqi Zhang, Jiexia Ye, Ziyue Li, Jia Li, and Fugee Tsung. Dualtime: A dual-adapter multimodal language model for time series representation. *arXiv preprint arXiv:2406.06620*, 2024.
- [60] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 11106–11115, 2021.
- [61] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 27268–27286, 2022.

2 基础概念

2.1 节作者为：郑兴泽、郭晨娟、树扬、杨彬；

2.2 节作者为：汪思远、郑兴泽、郭晨娟、树扬、杨彬；

2.3 节作者为：汪思远、郭晨娟、树扬、杨彬；

2.4 节作者为：郑兴泽、郭晨娟、树扬、杨彬。

本章主编为郭晨娟、树扬。

本章介绍时间序列的基础概念。2.1 章节介绍了时间序列数据的基本特性和主要来源。2.2 章节介绍了四种时间序列数据治理方法：缺失值处理、平稳化处理、平滑处理和分解处理。2.3 章节分别从时域和频域的角度介绍了时间序列数据的建模方式。2.4 章节简要介绍了常见时序分析深度学习模型的原理，这包括多层感知机、卷积神经网络、循环神经网络、时间卷积网络、图卷积网络和 Transformer 的原理。

2.1 时间序列数据概述

本章节解析时间序列数据的基本特性，并结合金融、医疗、物联网等领域的实际场景，明确数据的产生逻辑，帮助读者建立对时间序列数据的系统性基础认知。

2.1.1 时间序列基本特性

来自不同领域的时间序列具有不同的特性，包括趋势性、季节性、平稳性、漂移性、转移和相关性，本小节将通过定义和可视化对其进行描述。理解这些特性不仅有助于理解数据的内在规律，也决定了选择何种方法来进行数据处理、模型构建和分析，从而影响结果的准确性和可靠性。

对于时间序列 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times N}$ ，使用 Loess 分解法 [9] 可以将其分解为趋势项 (**T**)、季节项 (**S**) 和其他部分 (**R**): $\mathbf{X} = \mathbf{T} + \mathbf{S} + \mathbf{R}$ 。Loess 分解方法 [9] 是一种基于局部加权多项式回归的非参数时间序列分解技术，其核心原理是通过在滑动窗口内对数据点进行动态加权拟合实现序列成分分离。该方法针对时间序列中每个数据点，采用三次方权重函数对邻近数据赋予随距离递减的权重，构建局部多项式模型以估计趋势成分；对于季节成分，则通过周期内局部拟合捕捉周期性波动模式。最终将原始序列稳定分离为趋势项（反映长期变化）、季节项（体现周期波动）和其他部分（包含随机噪声）。

趋势性 (Trend) 是指时间序列长期变化或模式的总体方向，如图 2.1 所示。参考解释方差 [48]——一种评估特定因素对数据波动解释能力的统计指标，其核心是通过量化“特定因素（如本处的趋势项）所能解释的数据变异占总变异的比例”来衡量该因素对数据波动的贡献程度，本质上表现为“解释方差与总方差（解释方差 + 未解释方差）的比值”，趋势性定义如下：

$$Trend = \max\left(0, 1 - \frac{var(\mathbf{R})}{var(\mathbf{X} - \mathbf{S})}\right) \quad (2)$$

该指标本质是解释方差在时间序列分解中的具体应用：分母 $var(\mathbf{X} - \mathbf{S})$ 代表去除季节项后序列的总变异，分子 $var(\mathbf{R})$ 代表未被趋势项和季节项解释的变异， $1 - \frac{var(\mathbf{R})}{var(\mathbf{X} - \mathbf{S})}$ 正是基于解释方差的核心逻辑，计算趋势项对去除季节项后序列变异的解释比例，比值越小，趋势项解释力越强。

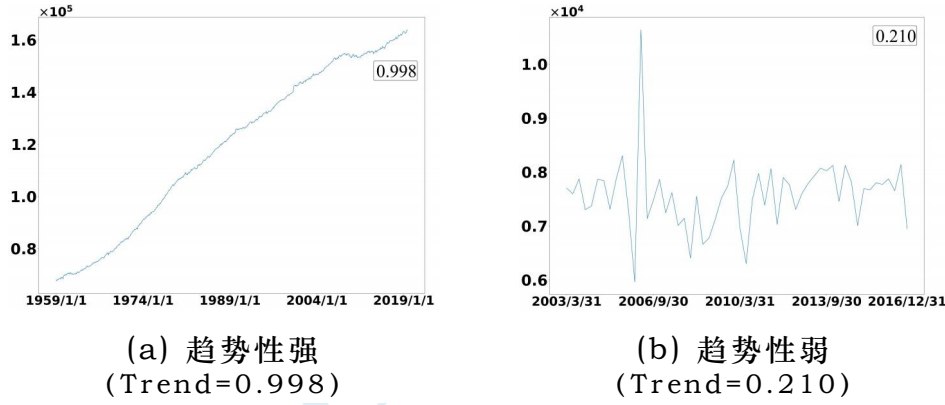


图 2.1: 时间序列的趋势性示例。

季节性 (Seasonality) 是指时间序列在特定时间间隔内重复变化的现象，如图 2.2 所示。与趋势性的度量相似，季节性定义如下：

$$Seasonality = \max\left(0, 1 - \frac{var(\mathbf{R})}{var(\mathbf{X} - \mathbf{T})}\right) \quad (3)$$

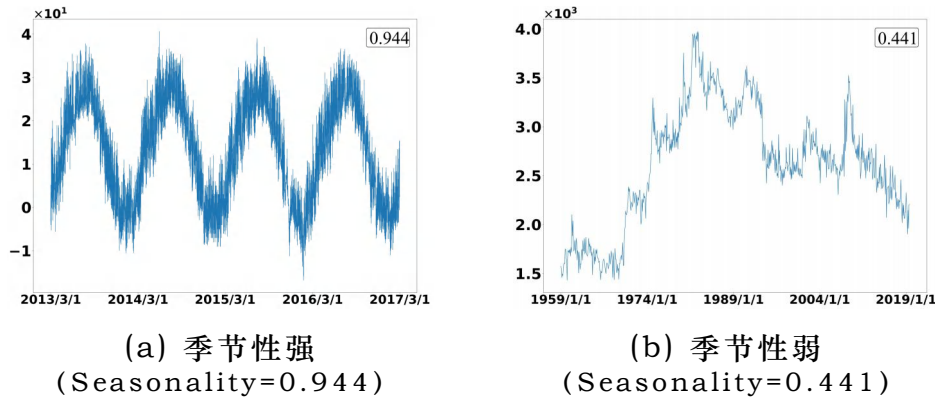


图 2.2: 时间序列的季节性示例。

平稳性 (Stationarity) 是指时间序列的各阶统计特征不随时间的变化而变化，如图 2.3 所示。时间序列的均值和方差对于所有观测值都是恒定的，协方差仅依赖于观测值之间的距离。严平稳时间序列在实际中很少见，通常采用宽平稳性条件 [37][47]。

非平稳性 (Non-Stationarity) 是指时间序列的统计特性不恒定，随着时间的推移而变化。这种情况通常使得时间序列更为复杂，需要通过特定的方法进行处理。可以采用 Augmented Dick-Fuller(ADF) 检验统计量 [16] 来定量平稳性，计算公式如下：

$$Stationarity = \begin{cases} True, & ADF(\mathbf{X}) < 0.05 \\ False, & ADF(\mathbf{X}) > 0.05 \end{cases} \quad (4)$$

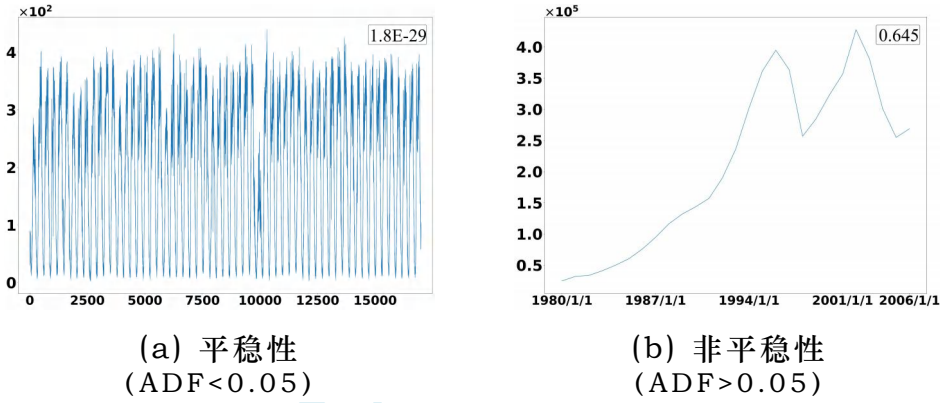


图 2.3: 时间序列的平稳性示例。

漂移性 (Shifting) 是指时间序列数据分布随时间发生系统性偏移的现象，表现为序列的整体水平（如中位数）在不同时间段发生显著变化，如图 2.4所示。这种行为可能源于系统内部的结构变化、外部影响或随机事件的发生。漂移性的计算过程如下 [52]：

1. 序列归一化：对时间序列 $\mathbf{X} \in \mathbb{R}^{T \times 1}$ 进行 Z-score 归一化，得到 $\mathbf{Z} \in \mathbb{R}^{T \times 1}$ ，消除量纲差异；
2. 生成阈值集合：计算 $\mathbf{Z}$ 的最小值 $Z_{\min}$ 与最大值 $Z_{\max}$ ，生成由 m 个等间隔阈值构成的集合 $S = \{s_i \mid s_i = Z_{\min} + (i-1) \cdot \frac{Z_{\max} - Z_{\min}}{m}, 1 \leq i \leq m\}$ ，覆盖序列全分布区间；
3. 分布重心定位与归一化：对每个阈值 $s_i \in S$ ，筛选出 $\mathbf{Z}$ 中大于 s_i 的所有时间点索引集合 K ，计算 K 的中位数 M_i （反映该阈值下的分布重心），并对所有 M_i 进行 Min-Max 归一化，得到归一化后的中位数集合 M' ；
4. 确定漂移值：取 M' 的中位数作为最终漂移值 *Shifting*。

转移 (Transition) 是指时间序列中模式变化的规律性程度，通过符号化序列的转移矩阵协方差迹（trace of covariance）量化，如图 2.5所示。该特性捕捉序列中存在的规律性和可识别的固定模式特征，例如趋势性和季节性的明确表现，或者趋势性和季节性同时存在。转移的计算过程如下 [52]：

1. 计算降采样步长：计算时间序列 $\mathbf{X} \in \mathbb{R}^{T \times 1}$ 自相关函数的第一个过零点 τ ，这反映了序列最显著的周期性间隔，用于降低数据冗余；
2. 序列降采样：以 τ 为步长对 $\mathbf{X}$ 进行降采样，得到简化序列 $\mathbf{Y} \in \mathbb{R}^{T' \times 1}$ （ T' 为降采样后的时间步长），聚焦核心模式特征；

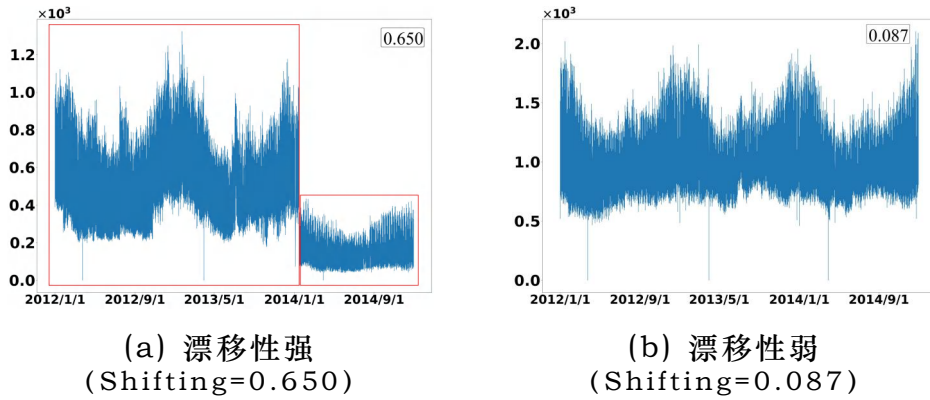


图 2.4: 时间序列的漂移性示例。

3. 序列符号化：对 $\mathbf{Y}$ 按数值大小排序，得到索引序列 $\mathbf{I} = \text{argmax}(\mathbf{Y})$ ，再将 $\mathbf{I}$ 划分为 3 类状态，由此生成符号化序列 $\mathbf{Z} \in \{0, 1, 2\}^{T' \times 1}$ ：对每个时间步 j ， $Z[j] = \text{floor}\left(\frac{I[j]}{\frac{1}{3}T'}\right)$ ，即根据数值大小通过等间隔阈值将序列映射为 3 个状态符号；
4. 构建转移矩阵：统计符号化序列 $\mathbf{Z}$ 中相邻符号的转换次数，生成 3×3 转移矩阵 $\mathbf{M}$ ($M[a][b]$ 表示从符号 a 转换到符号 b 的次数)，再归一化得到 $\mathbf{M}' = \frac{1}{T'}\mathbf{M}$ (转移概率矩阵)；
5. 计算转移值：计算 $\mathbf{M}'$ 列间的协方差矩阵 $\mathbf{C}$ ，取其迹作为最终转移值 $\text{Transition} = \text{tr}(\mathbf{C})$ 。

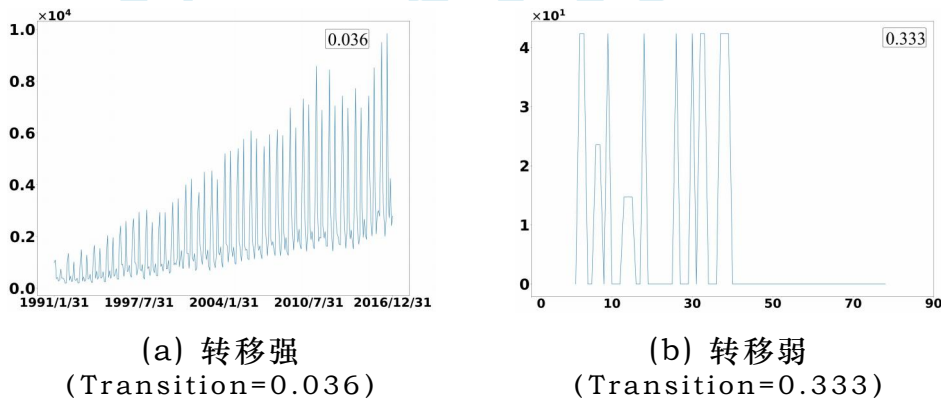


图 2.5: 时间序列的转移示例。

相关性 (Correlation) 是指多变量时间序列中不同变量具有共同趋势或模式的可能性，表明它们受到相似因素的影响或存在某种潜在关系，如图 2.6 所示。相关性可以通过计算所有变量对之间的 Pearson 相关系数 (PCCs) [10] 的均值和方差得到，

计算公式如下：

$$Correlation = mean(\mathbf{p}) + \frac{1}{1 + var(\mathbf{p})} \quad (5)$$

其中， $\mathbf{p}$ 是所有变量对之间的 Pearson 相关系数。

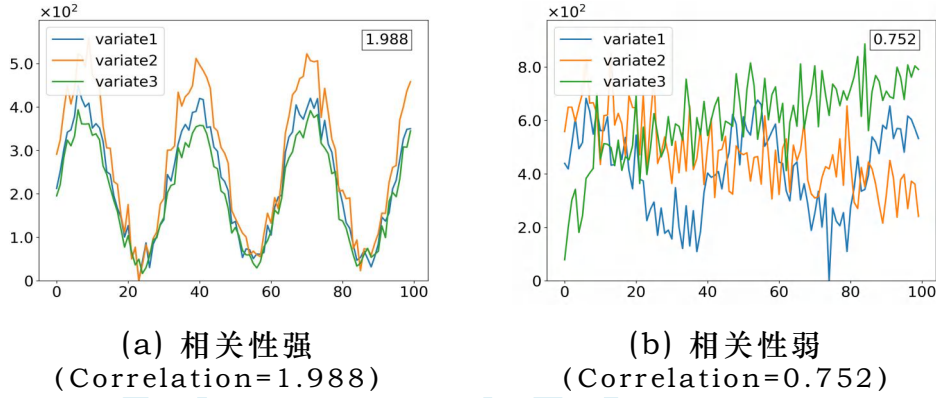


图 2.6: 时间序列的相关性示例。

2.1.2 数据来源

时间序列数据来源广泛，包括金融、医疗健康、交通、能源和自然环境等领域，通用的数据集档案有 Monash 时间序列预测档案 [20]、UCR 时间序列分类档案 [13]。具体地，有以下主要来源：

- 金融：股票市场数据，比如股票价格、成交量、开盘价等，通常来源于证券交易所；宏观经济数据，比如国内生产总值、消费者价格指数、通货膨胀率等，由政府机构发布。常见的数据集包括 Exchange [33]、Bitcoin [20]、FRED-MD [46]。
- 医疗健康：患者的生理数据，比如心电图、血压、血糖水平等，由医疗设备监测和记录；可穿戴设备作为健康管理的工具，可以收集心率、步数、睡眠数据等。常见的数据集包括 Hospital [26]、ILI [65]、Covid-19 [50]。
- 交通：车辆流量数据，比如车辆流量、速度、事故信息等，由交通摄像头和传感器记录；公交车、地铁等公共交通工具的实时位置、运行时间、车厢拥挤程度等，由交通部门提供。常见的数据集包括 Traffic [65]、PEMS [56]、PEMS-BAY [39]。
- 能源：电力消耗数据由电力公司监控，包括每日、每小时的电力负荷；可再生能源数据，比如太阳能、风能发电量随时间的变化数据，用于能源预测和调度。常见的数据集包括 ETT [69]、Electricity [59]、Solar [33]。
- 自然环境：气象站数据，比如气温、湿度、降雨量、风速、气压、空气质量等，通常由国家气象局或气象研究机构提供；遥感数据，比如云层覆盖、温度分布、气

象灾害（如飓风、雷暴等）等动态变化数据，由卫星或无人机等设备收集。常见的数据集包括 Weather [65]、KDD Cup 2018 [20]、Sunspot [20]。

- 商务：销售数据，比如每个商品的销售量、销售额、毛利率、价格波动、用户评价和购买频率等，来自购物平台的交易记录。常见的数据集包括 M5 [1]、Hierarchical Sales [45]、Dominick [20]。
- 网络：用户行为数据，比如用户在网页上的点击、搜索、浏览历史和收藏内容的添加、删除等行为数据，通过客户端或服务端日志捕获。常见的数据集包括 Wiki2000 [19]、Web Traffic [44]。
- 物联网：智能农业设备，通过土壤湿度、温度、气象数据等进行农业监控和优化管理；智能家居设备，比如智能空调、智能照明系统、智能恒温装置等，也能产生时间序列数据，由相关传感器收集。常见的数据集包括 Sensordata [42]。

2.2 时间序列数据治理

2.2.1 时间序列缺失值处理

在进行时间序列分析之前，选择适当的历史数据集是关键，但大多数数据集往往存在缺失值问题。缺失值是指在数据集中某些时间点或变量缺失观测值，可能表现为空白、NaN 等。缺失值的产生有多种原因，具体包括：(1) 传感器故障：在某些时段或某些设备上，传感器出现故障，导致数据丢失或无法读取；(2) 网络问题：数据在传输过程中可能由于网络不稳定或中断，导致部分数据丢失；(3) 多变量时序中的不同采样频率：当多种变量采用不同的采样频率时，在时间戳对齐过程中可能会出现低频采样数据缺失，尤其在高频数据与低频数据进行对比时尤为明显。例如在分析电力负荷与气温关系时，电力负荷数据每分钟采集一次（高频），而气温数据每小时采集一次（低频）。当需要分析气温变化对电力负荷瞬时波动的影响时，必须将两者对齐到分钟级别，这导致气温数据在 59/60 的时间点上显示为缺失值，严重影响了相关性计算和因果分析的准确性。这些因素共同导致了数据集的不完整性。

从统计属性角度，缺失值可分为三类。第一类是**完全随机缺失 (Missing Completely At Random, MCAR)**：缺失值的产生与观测值或未观测值完全无关。也就是说，数据的缺失是完全随机的，不受任何因素的影响。例如，传感器因随机的网络丢包导致数据传输失败，这种缺失与传感器测量值本身或时间点无关。第二类是**随机缺失 (Missing At Random, MAR)**：缺失值的产生与观测值相关，但与未观测值无关。比如，温度传感器在低温环境下（如冬季夜间）更容易出现电池电量不足而停止工作，此时缺失与已观测的季节、时间等变量相关，但与具体的未观测温度值无关。换句话说，缺失数据可以通过已有的观测数据进行推测。第三类是**非随机缺失 (Missing Not**

At Random, MNAR)：缺失值的产生与未观测的变量相关，缺失通常是非随机的。比如，股价监控系统在市场极端波动时可能因为过载而停止记录，或者环境监测设备在污染严重时因传感器损坏而无法工作，此时缺失值的产生与未观测的极端数值本身密切相关。通过这些例子，我们可以更好地理解不同类型的缺失值，以及如何根据实际情况进行相应的处理。

缺失值的存在可能会给时间序列分析带来一系列问题，例如：(1) 影响模型的训练效果：许多机器学习和统计模型对缺失值非常敏感，直接带入缺失数据会导致模型表现不佳，甚至无法训练。(2) 偏差产生：如果缺失值未能合理填充或处理，可能会引入数据偏差，导致分析结果不具代表性。(3) 降低预测准确性：缺失值会影响模型的预测能力，尤其是在时间序列数据中，若数据缺失处理不当，会使得预测的结果不稳定且不准确。因此，合理处理缺失值是时序分析前必不可少的步骤。

缺失值处理方法主要分为两类：删除和填充。第一大类方法是删除法：删除法是最直接的方法，即直接丢弃缺失值对应的时间点，保留其余完整的时间序列数据点。然而，这种方法可能导致数据点之间的采样间隔不一致，影响时间序列的连贯性。为减小删除法带来的负面影响，可以使用时间长短记忆网络（Time-aware Long Short Term Memory, T-LSTM）[4]。T-LSTM 会将数据点之间的时间间隔和数据点本身作为输入传递给模型，通过将时间间隔作为额外特征，帮助模型在处理缺失值后仍能保持时间序列的连贯性。第二大类是填充法：填充法较删除法更精确，可结合缺失值前后的数据对缺失值进行估计。填充法可进一步分为基于传统统计和基于机器学习两类。下面我们对填充法进行详细阐述。

填充法中的第一类是基于传统统计的填充方法，包括：

- **就近填充法 (Nearest Neighbor Imputation)**：代表方法有前推法和后推法，分别使用缺失值前面和后面的观测值填补缺失值。这两种方法简单快速，适用于缺失值较少且数据变化平缓的情况。
- **统计量填充法 (Statistical Imputation)**：诸如使用均值、中值和常用值等数据的统计特征对缺失值进行填充。这些方法基于数据的统计特性，适用于数据分布较为集中且缺失机制为随机缺失的情况。
- **线性插值法 (Linear Interpolation)**：包括一元线性回归、多元线性回归和岭回归等方法。这些方法通过建立变量内或变量间的线性关系，拟合回归模型并对缺失值进行预测。

这三种方法的效果差异如图 2.7 所示。

- **基于统计模型的方法**：这一类方法主要基于现有的统计概率模型，比如期望最大化 (Expectation Maximization, EM) [15]、链式方程多重插补 (Multivariate Imputation by Chained Equations, MICE) [2]、概率主成分分析 (Probabilistic

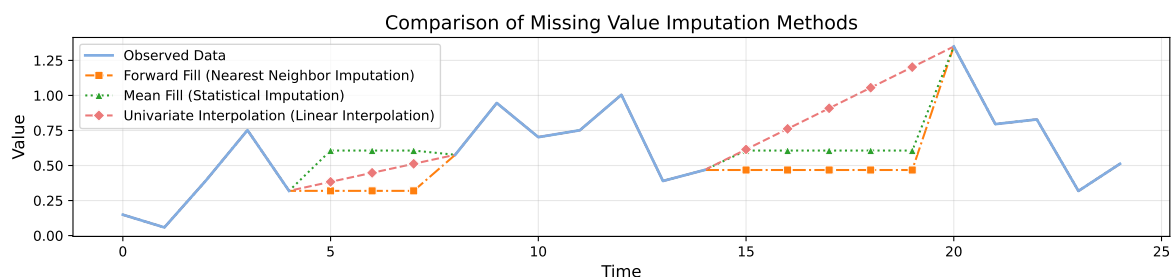


图 2.7: 就近填充、统计量填充和线性插值法填补缺失值。

Principal Component Analysis, PPCA)、卡尔曼滤波器 (Kalman filter)、差分整合移动平均自回归模型 (Autoregressive Integrated Moving Average, ARIMA) [5] 等, 对时间序列中有效的数据进行分析, 估计出概率模型的缺失参数, 然后结合模型和分析结果估计出缺失值。其中, 卡尔曼滤波器和差分整合移动平均自回归模型 (ARIMA) 等还结合时间序列的季节性引申出了季节性卡尔曼滤波器 (Seasonal Kalman filter) 和季节性差分整合移动平均自回归模型 (Seasonal Autoregressive Integrated Moving Average, SARIMA) 方法, 可以更好地分析时间序列的趋势性和季节性成分, 更好地对缺失值进行估计。

- 基于专业知识: 结合时间序列来源领域的专业领域知识对时间序列缺失值进行估计也是一种常见方法。例如, 在电力负荷数据中, 可以根据工作日与周末的用电模式差异、季节性用电特征以及特殊节假日的影响, 结合历史同期数据和天气因素来推测并填补缺失值。这种方法具备更高的可解释性, 但过于依赖领域知识并需要花费大量人力物力。
- 针对多变量时序的缺失值填补方法: 利用多个相关时间序列之间的相互关系, 通过矩阵分解技术, 如奇异值分解 (Singular Value Decomposition, SVD) 或非负矩阵分解 (Non-negative Matrix Factorization, NMF) [36], 来提取全局特征, 构建低秩近似矩阵, 从而填补缺失值。这种方法能够利用数据的多变量关联性, 提高填补的准确性和一致性。区别于前面提到既适用于单变量时序也适用于多变量时序的方法, 这类方法仅适用于多变量时序, 对单变量时序不适用。

第二类是基于机器学习和深度学习的填充方法:

- 基于相似性匹配: 这类方法的核心思想是利用时间序列之间的相似性模式, 通过寻找与目标序列最相似的参考序列来推断缺失值。代表性方法包括 k 近邻 (k -Nearest Neighbors, KNN) [12], 它通过计算序列间的距离度量找到 k 个最相似的序列; 以及动态时间规整算法 (Dynamic Time Warping, DTW) [28], 它能够处理不同长度和速度的序列对齐问题。这些方法无需复杂的模型训练, 直接利用

数据间的相似性进行填充。

- 基于预测的判别式方法：这类方法通过学习历史观测值与未来值之间的映射关系，直接预测缺失位置的值。典型代表包括循环神经网络（Recurrent Neural Network, RNN）[17]，它通过循环连接捕捉时序依赖；以及 Encoder-Decoder 架构，它先将输入序列编码为固定维度的表示，再解码生成目标值。这类判别式模型专注于学习确定性的预测函数，特别适合处理具有明确时序规律的数据。
- 基于生成的概率建模方法：这类方法不直接预测缺失值，而是学习整个数据的潜在概率分布，从而能够生成多种可能的填充值。主要方法包括生成对抗网络（Generative Adversarial Networks, GAN）[21]，它通过生成器和判别器的对抗训练学习数据分布；以及变分自编码器（Variational Autoencoder, VAE）[30]，它通过变分推断学习数据的潜在表示。这类生成式模型的优势在于能够建模数据的不确定性，更适合处理具有复杂随机性的时间序列。

时间序列分析中的缺失值处理至关重要，合理的缺失值处理方法可显著提高模型的稳定性和预测能力。通过结合传统统计方法与机器学习技术，依据数据特性和缺失机制选择适合的处理策略，不仅能减少缺失值带来的偏差，还能提升分析的可靠性和结果的精准性。

2.2.2 时间序列平稳化处理

在前面，我们已经定义了时间序列的平稳性。在时间序列分析中，我们通常关注序列的平稳性，即序列的统计特征在时间上是否保持稳定。**严平稳 (Strict Stationarity)** 和 **宽平稳 (Weak Stationarity)** 是时间序列中两种重要的平稳性质。严平稳序列要求序列的联合概率分布不随时间平移而改变。具体而言，对于任意时间点的集合，其联合分布与这些时间点整体平移后的联合分布完全相同。这是一个非常强的条件，意味着序列的所有统计特性（包括所有阶矩）都不随时间变化。宽平稳序列的要求则相对宽松，主要关注序列的平均水平和波动幅度保持稳定，不会随时间推移而上升或下降，并且不同时间点之间的相关性只取决于它们相隔多久，而不取决于具体是哪个时间段。简单来说，宽平稳序列虽然会有起伏变化，但其基本的统计行为模式是稳定的。但是在现实场景中，大多数时间序列既非严平稳，也非宽平稳，其分布通常随时间变化。

为了能更好地理解时间序列的内在模式，平稳化处理成为必要步骤。常见的平稳化方法分为传统的基于统计的平稳化方法以及深度学习平稳化方法。

基于统计的平稳化方法包括：

- 差分法：差分是通过计算相邻观测值之间的差值来消除时间序列中的趋势或周期性成分。一阶差分是用当前值减去前一个值，可以消除线性趋势；对于更复杂的

趋势，可能需要进行二阶或更高阶差分。季节性差分则是用当前值减去一个周期前的值，用于消除季节性模式。例如，股票价格通常呈现上升或下降趋势，通过差分转换为价格变化量（收益率），往往能得到更平稳的序列。差分法的优点是简单有效，特别适合处理具有确定性趋势的时间序列。

- 对数转换法：对数变换通过取对数来压缩数据的动态范围，特别适用于处理数值变化幅度随水平增加而增大的时间序列。例如，经济数据常常表现出这种特性——当 GDP 较小时波动幅度小，当 GDP 增长后波动幅度也随之增大。对数变换可以将这种“异方差性”（方差随时间变化）转化为更稳定的形式。此外，对数变换还能将指数增长转化为线性增长，使得数据更容易分析和建模。需要注意的是，对数变换只能应用于正值数据。
- 标准归一化（Z-score 标准化）：提供了一种简单而有效的平稳化方式，通过标准化观测值的分布，减轻训练集和测试集之间的分布偏差，同时保持数据的内在变化。

$$\mu = \frac{1}{T} \sum_{t=1}^T x_t, \quad \sigma = \sqrt{\frac{1}{T} \sum_{t=1}^T (x_t - \mu)^2}, \quad x'_t = \frac{x_t - \mu}{\sigma} \quad (6)$$

其中： $\mu, \sigma \in \mathbb{R}^N$ 是时间序列 X 的均值向量和标准差向量， X' 是时间序列 X 归一化后的观测向量，具有零均值和单位方差。

这种归一化方法使得处理后的数据在每个维度上均值为 0、标准差为 1，有助于提高模型的训练稳定性和泛化能力。差分法、对数转换法和 Z-score 方法的效果差异如图 2.8 所示。

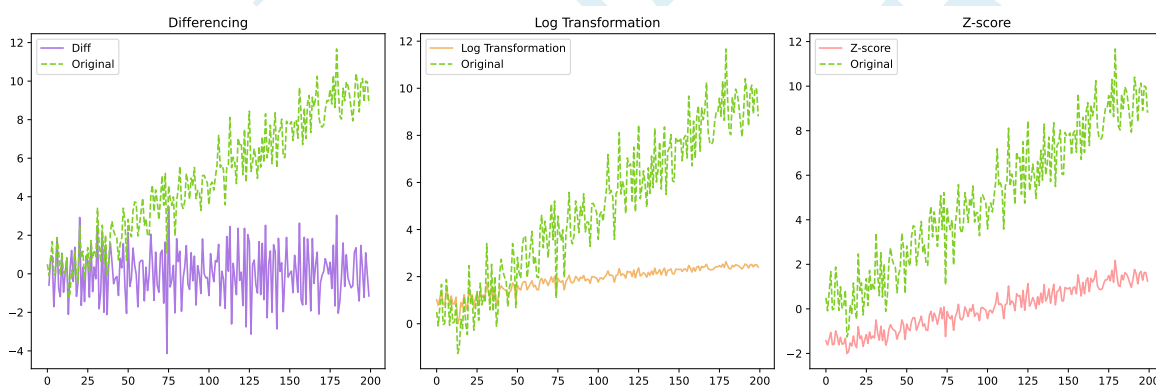


图 2.8: 同一条非平稳时间序列分别经过差分法、对数转换法和 Z-score 方法进行平稳化处理。

随着时间序列数据复杂度的增加和模型对数据分布稳定性要求的提高，传统平稳化方法逐渐被更具自适应性和灵活性的深度学习平稳化技术所取代。这些方法能够动

态调整归一化参数，捕捉非平稳时间序列中的分布变化，有效提高模型在实际任务中的适应性和泛化能力。以下是几种**深度学习平稳化技术**中具有代表性的方法：

- 深度自适应输入标准化 (Deep Adaptive Input Normalization, DAIN) [51]：通过神经网络（如 LSTM、Transformer 等）的隐藏状态动态生成归一化参数。这些参数依赖于输入数据的上下文信息，能够根据时间序列的当前状态和历史模式进行调整。

$$\begin{aligned} (X^i)' &= (X^i - \alpha^i) \oslash \beta^i, \quad (X^i)'' = (X^i)' \odot \gamma^i, \\ \alpha^i &= \frac{1}{T} \sum_{t=1}^T x_t^i, \quad \beta^i = \sqrt{\frac{1}{T} \sum_{t=1}^T (x_t^i - \alpha^i)^2}, \quad \alpha^i = W_a \alpha^i, \quad \beta^i = W_b \beta^i, \\ c^i &= \frac{1}{T} \sum_{t=1}^T (x_t^i)', \quad \gamma^i = \text{sigm}(W_c c^i + d) \end{aligned} \quad (7)$$

时间序列数据 $X^i = (x_1^i, x_2^i, \dots, x_T^i)$ 中 i 表示第 i 个时间序列。首先，计算时间序列 i 的平均特征向量 α^i ，然后通过权重矩阵 W_a 学习得到自适应平移参数 α^i 。这一步将数据平移到适当的特征空间区域。接着，计算时间序列 i 的标准差向量 β^i ，然后通过权重矩阵 W_b 学习得到自适应缩放参数 β^i 。这一步根据数据的方差对数据进行线性缩放。将平移和缩放后的数据进行归一化，得到 $(X^i)'$ 。计算归一化后数据的平均特征向量 c^i ，然后通过权重矩阵 W_c 和偏置 d 学习得到门控权重向量 γ^i 。最后，将门控权重应用于归一化后的数据，得到最终的归一化输出 $(X^i)''$ 。 $\oslash$ 表示逐元素除法 (Hadamard 除法)， $\odot$ 表示逐元素乘法 (Hadamard 乘法)， $\text{sigm}(x) = \frac{1}{1+e^{-x}}$ 是 *Sigmoid* 函数，用于将门控权重限制在 (0,1) 范围内。

- 可逆实例规范化 (Reversible Instance Normalization, RevIN) [29]：通过计算输入序列的均值向量 μ 和标准差向量 σ 并引入自适应参数向量 γ 和 β ，实现了更灵活的归一化。旨在解决时间序列预测中的数据分布变化问题。其主要特点是归一化过程是可逆的，即在预测完成后，可以将归一化后的数据恢复到原始分布。

$$\begin{aligned} \mu &= \frac{1}{T} \sum_{t=1}^T x_t, \quad \sigma = \sqrt{\frac{1}{T} \sum_{t=1}^T (x_t - \mu)^2}, \quad X' = \frac{X - \mu}{\sigma} r, \\ r &= X - X', \quad X'' = X' + r \end{aligned} \quad (8)$$

首先，计算每个时间序列的均值向量 μ ，作为该通道的中心化参考。接着，计算每个通道的标准差 σ ，用于衡量数据的波动程度。然后，对每个通道的数据进行归一化处理，通过减去均值并除以标准差，得到归一化后的数据 X' 。为了保留原始数据的特性，计算残差项 r ，即原始数据与归一化数据的差值。最后，将归一化后的数据与残差项相加，得到最终的归一化输出 X'' 。这种方法既实现了数据的归一化，又保留了原始数据的重要信息。

- 非平稳变换器 (Non-Stationary Transformer) [41] 针对过度平稳化可能削弱模型对关键时间依赖关系捕捉能力的问题, 提出了一个包含两个互补部分的通用框架: 序列平稳化 (Series Stationarization) 和去平稳化注意力 (De-stationary Attention)。

序列平稳化通过归一化模块处理输入时间序列 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times N}$, 使其遵循稳定分布以提高可预测性。归一化过程为:

$$\mu = \frac{1}{T} \sum_{t=1}^T \mathbf{x}_t, \quad \sigma^2 = \frac{1}{T} \sum_{t=1}^T (\mathbf{x}_t - \mu)^2, \quad \mathbf{x}'_t = \frac{1}{\sigma} \odot (\mathbf{x}_t - \mu) \quad (9)$$

其中 $\mu, \sigma \in \mathbb{R}^{N \times 1}$ 分别为序列的均值和标准差, 得到平稳化后的序列 $\mathbf{X}' = (\mathbf{x}'_1, \mathbf{x}'_2, \dots, \mathbf{x}'_T)$ 。

为避免过度平稳化导致的信息损失, 去平稳化注意力机制通过学习去平稳化因子来恢复原始序列的非平稳特性:

$$\begin{aligned} \log \tau &= \text{MLP}(\sigma, \mathbf{X}), \quad \Delta = \text{MLP}(\mu, \mathbf{X}) \\ \text{Attn}(\mathbf{Q}', \mathbf{K}', \mathbf{V}', \tau, \Delta) &= \text{Softmax} \left(\frac{\tau \mathbf{Q}' \mathbf{K}'^\top + \mathbf{1} \Delta^\top}{\sqrt{d_k}} \right) \mathbf{V}' \end{aligned} \quad (10)$$

其中, $\tau \in \mathbb{R}^+$ 为正缩放标量, $\Delta \in \mathbb{R}^T$ 为偏移向量, 它们通过多层感知机从原始序列 $\mathbf{X}$ 的统计信息 μ 和 σ 中学习得到。这种设计使模型既能受益于平稳化序列的可预测性, 又能保持原始序列的内在时间依赖关系。

- 分片自适应归一化 (Slicing Adaptive Normalization, SAN) [43] 提出了一种基于分片的归一化框架, 通过对时间序列进行局部处理来更好地保留每个时间段的内在模式。与全局归一化不同, SAN 将输入序列划分为多个非重叠的分片, 每个分片根据其局部统计量独立归一化。

对于输入时间序列 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times N}$, SAN 首先将其划分为 M 个长度为 τ 的分片 $\{\mathbf{X}_j\}_{j=1}^M$, 其中 $M = T/\tau$ 。对每个分片计算局部统计量并进行归一化:

$$\mu_j = \frac{1}{\tau} \sum_{t=1}^{\tau} \mathbf{x}_{j,t}, \quad \sigma_j^2 = \frac{1}{\tau} \sum_{t=1}^{\tau} (\mathbf{x}_{j,t} - \mu_j)^2, \quad \bar{\mathbf{X}}_j = \frac{1}{\sigma_j + \epsilon} \odot (\mathbf{X}_j - \mu_j) \quad (11)$$

其中 $\mu_j, \sigma_j \in \mathbb{R}^N$ 是第 j 个分片的均值和标准差, ϵ 是一个小常数用于数值稳定。

为了恢复输出序列的非平稳性, SAN 引入了统计预测模块来估计未来分片的分布。该模块采用残差学习策略, 基于输入序列的全局均值 $\rho = \frac{1}{T} \sum_{t=1}^T \mathbf{x}_t$ 和局部统计量预测未来分布:

$$\begin{aligned} \hat{\mu} &= \mathbf{W}_1 \odot \text{MLP}(\mu - \rho, \bar{\mathbf{X}} - \rho) + \mathbf{W}_2 \odot \rho \\ \hat{\sigma} &= \text{MLP}(\sigma, \bar{\mathbf{X}}) \end{aligned} \quad (12)$$

其中 $\boldsymbol{\mu} = [\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_M] \in \mathbb{R}^{N \times M}$ 包含所有输入分片的均值, $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^N$ 是可学习的权重向量。

最后, 利用预测的统计量对模型输出进行反归一化。对于输出序列的每个分片 $\bar{\mathbf{Y}}_j$:

$$\mathbf{Y}'_j = \bar{\mathbf{Y}}_j \odot (\hat{\boldsymbol{\sigma}}_j + \boldsymbol{\epsilon}) + \hat{\boldsymbol{\mu}}_j \quad (13)$$

SAN 采用两阶段训练策略: 首先训练统计预测模块至收敛, 然后冻结该模块并训练预测模型。这种设计将非平稳预测任务解耦为统计预测和平稳预测两个子任务, 使框架具有良好的灵活性和通用性。

这些平稳化方法通过考虑时间序列的平稳性和非平稳性特征, 显著提升了模型在非平稳时间序列预测中的泛化能力和准确性, 使时间序列模型具备更强的处理现实任务的能力。

2.2.3 时间序列平滑处理

时间序列平滑处理通常是对多个相邻时序点的数据进行操作, 每个时刻平滑后的值由该时刻和相邻时序点平滑前的值计算得到, 以减少数据中的噪声, 从而更好地捕捉其趋势性和周期性成分。下面介绍几种主要的平滑处理方法。

移动平均 (Moving Average): 通过计算特定时间窗口内数据的平均值来平滑数据。移动平均方法可分为简单移动平均和加权移动平均。

- 简单移动平均 (Simple Moving Average): 通过在大小为 N 的特定窗口内取数据的平均值来平滑数据, 公式如下:

$$\mathbf{s}_t = \frac{1}{N} \sum_{i=t-N+1}^t \mathbf{x}_i \quad (14)$$

其中, $\mathbf{s}_t$ 是第 t 个时间点的平滑值, $\mathbf{x}_i$ 是第 i 个时间点的观测值, N 是窗口长度, 如窗口大小为 7, 则 $N=7$ 。

- 加权移动平均 (Weigthed Moving Average): 每个数据点被赋予不同的权重, 通常靠近当前时刻的数据点权重更大, 公式如下:

$$\mathbf{s}_t = \frac{\sum_{i=t-N+1}^t \mathbf{w}_i \cdot \mathbf{x}_i}{\sum_{i=t-N+1}^t \mathbf{w}_i} \quad (15)$$

其中, $\mathbf{w}_i$ 是第 i 个时间点的权重。

指数平滑 (Exponential Smoothing): 指数平滑由移动平均发展而来, 可以看作是一种特殊的加权移动平均, 其基本思想是将权重按照指数级进行衰减。按平滑的次数, 常见的指数平滑可分为一次指数平滑、二次指数平滑、三次指数平滑。

- 一次指数平滑 (Simple Exponential Smoothing)：适用于无趋势性和季节性特征的序列。通过指数加权对数据进行平滑，历史数据的权重随着时间推移按指数递减，因此较新的数据点的影响更大，公式如下：

$$\mathbf{s}_t = \alpha \mathbf{x}_t + (1 - \alpha) \mathbf{s}_{t-1} \quad (16)$$

$$\mathbf{s}_0 = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad (17)$$

其中， $\mathbf{s}_0$ 是平滑初始值， n 是数据点的个数，通常 $n=3$ 。 $\alpha \in (0, 1)$ 是水平平滑系数，决定当前值和历史值的权重分配，值越大越关注近期数据。

- 二次指数平滑 (Holt Exponential Smoothing)：适用于有趋势性特征但无季节性特征的序列。在一次指数平滑的基础上加入了趋势性成分，公式如下：

$$\mathbf{s}_t = \alpha \mathbf{x}_t + (1 - \alpha)(\mathbf{s}_{t-1} + \mathbf{b}_{t-1}) \quad (18)$$

$$\mathbf{b}_t = \beta(\mathbf{s}_t - \mathbf{s}_{t-1}) + (1 - \beta)\mathbf{b}_{t-1} \quad (19)$$

$$\mathbf{s}_0 = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad (20)$$

$$\mathbf{b}_0 = \frac{1}{n-1} \sum_{i=2}^n (\mathbf{x}_i - \mathbf{x}_{i-1}) \quad (21)$$

其中， $\mathbf{b}_t$ 是第 t 个时间点的趋势成分， $\mathbf{b}_0$ 是趋势初始值，通常 $n=3$ 。 $\beta \in (0, 1)$ 是趋势平滑系数，值越大越关注近期趋势变化。

- 三次指数平滑 (Holt-Winters Exponential Smoothing)：也称为 Holt-Winters 方法 [24, 64]，适用于有趋势性特征且有季节性特征的序列。在二次指数平滑的基础上进一步加入了季节性成分，分为加法模型 (Additive Model) 和乘法模型 (Multiplicative Model)，加法模型的季节性波动幅度固定不变，乘法模型的季节性波动幅度随着数据水平的变化而变化，公式如下：

加法模型：

$$\mathbf{s}_t = \alpha(\mathbf{x}_t - \mathbf{p}_{t-L}) + (1 - \alpha)(\mathbf{s}_{t-1} + \mathbf{b}_{t-1}) \quad (22)$$

$$\mathbf{b}_t = \beta(\mathbf{s}_t - \mathbf{s}_{t-1}) + (1 - \beta)\mathbf{b}_{t-1} \quad (23)$$

$$\mathbf{p}_t = \gamma(\mathbf{x}_t - \mathbf{s}_{t-1} - \mathbf{b}_{t-1}) + (1 - \gamma)\mathbf{p}_{t-L} \quad (24)$$

$$\mathbf{s}_0 = \frac{1}{L} \sum_{i=1}^L \mathbf{x}_i \quad (25)$$

$$\mathbf{b}_0 = \frac{1}{L} \left(\frac{1}{L} \sum_{i=L+1}^{2L} \mathbf{x}_i - \frac{1}{L} \sum_{i=1}^L \mathbf{x}_i \right) \quad (26)$$

$$\mathbf{p}_0[j] = \mathbf{x}_j - \mathbf{s}_0 \quad (j = 1, 2, \dots, L) \quad (27)$$

乘法模型：

$$s_t = \alpha \frac{x_t}{p_{t-L}} + (1 - \alpha)(s_{t-1} + b_{t-1}) \quad (28)$$

$$b_t = \beta(s_t - s_{t-1}) + (1 - \beta)b_{t-1} \quad (29)$$

$$p_t = \gamma \frac{x_t}{s_{t-1} + b_{t-1}} + (1 - \gamma)p_{t-L} \quad (30)$$

$$s_0 = \frac{1}{L} \sum_{i=1}^L x_i \quad (31)$$

$$b_0 = \frac{1}{L} \left(\frac{1}{L} \sum_{i=L+1}^{2L} x_i - \frac{1}{L} \sum_{i=1}^L x_i \right) \quad (32)$$

$$p_0[j] = \frac{x_j}{s_0} \quad (j = 1, 2, \dots, L) \quad (33)$$

$$\bar{p}_0 = \frac{1}{L} \sum_{i=1}^L p_0[i], \quad p_0[j] = \frac{p_0[j]}{\bar{p}_0} \quad (34)$$

其中， p_t 是第 t 个时间点的季节成分，季节初始值 $p_0 = [p_0(1), p_0(2), \dots, p_0(L)]$ ， L 是季节周期长度（如一年有 12 个月）。 $\gamma \in (0, 1)$ 是季节平滑系数，值越大越关注近期季节模式。

图 2.9 是时序平滑处理的示例，分别使用移动平均方法和三种指数平滑方法平滑原始时间序列。

2.2.4 时间序列分解处理

时间序列分解处理是将序列分解为不同成分，以更好地理解数据的复杂变化。下面介绍几种主要的分解处理方法。

第一类方法是季节性-趋势分解法 (Seasonal-Trend Decomposition)。对于时间序列 X ，使用季节性-趋势分解函数 f 将其分解如下：

$$f(X) = T, S, C, I \quad (35)$$

其中，**T** 是长期趋势项，表示序列明显的长期趋势。**S** 是季节项，表示序列表现出与季节相关的稳定的周期变化，即每年相同季节都会有相同幅度和方向的变化，总是具有固定的已知频率。**C** 是循环项，表示序列呈由高到低，再由低到高的反复循环的波动，循环的幅度不规则，频率不固定。**I** 是不规则项，表示不可预期的偶然因素对时间序列的影响。通常，认为时间序列包含三个部分：趋势项、季节项和其他部分（除趋势项和季节项以外的其他内容）。

一般地，季节性-趋势分解法有两种分解模型：加法模型和乘法模型 [27]。

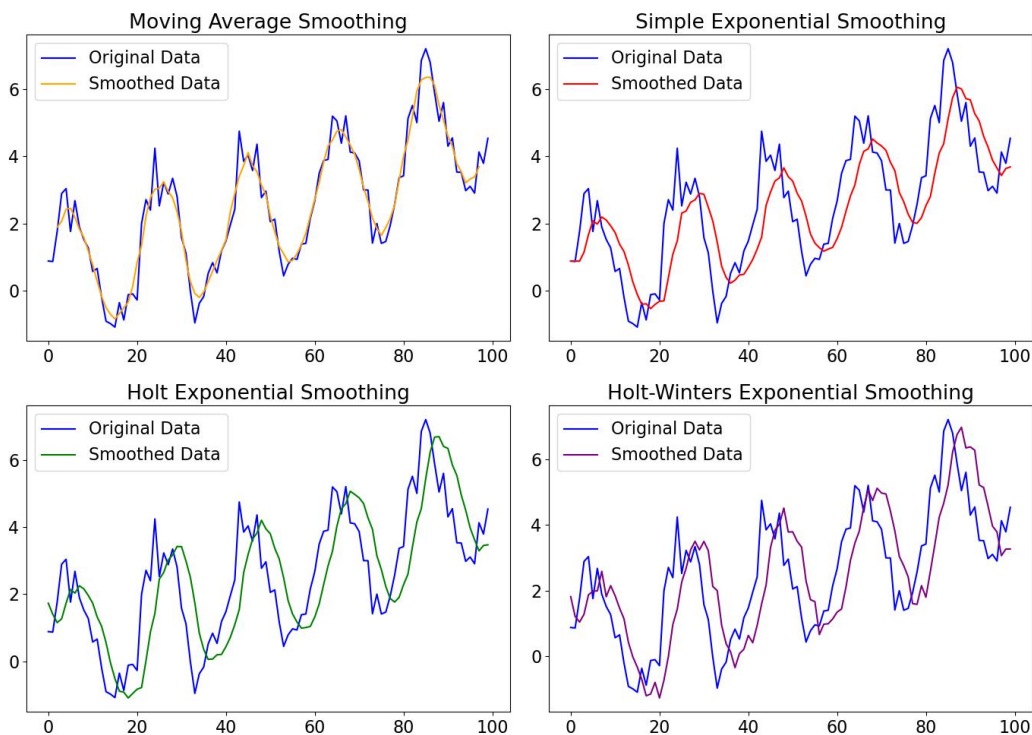


图 2.9: 时间序列平滑处理的示例。

- **加法模型 (Additive Model)**: 假设三种变动因素是相互独立的, 则时间序列各期发展水平是各个影响因素相加之和, 可以表示如下:

$$\mathbf{X} = \mathbf{T} + \mathbf{S} + \mathbf{R} \quad (36)$$

其中, $\mathbf{T}$ 是趋势项, $\mathbf{S}$ 是季节项, $\mathbf{R}$ 是其他部分。

- **乘法模型 (Multiplicative Model)**: 假设三种变动因素存在着某种关系, 互不独立, 相互影响, 时间序列各期发展水平是各个影响因素相乘之积, 可以表示如下:

$$\mathbf{X} = \mathbf{T} \times \mathbf{S} \times \mathbf{R} \quad (37)$$

对于乘法模型, 可以取对数将其转化为加法模型, 即:

$$\log(\mathbf{X}) = \log(\mathbf{T}) + \log(\mathbf{S}) + \log(\mathbf{R}) \quad (38)$$

季节性-趋势分解法的实现方法有多种, 常用的方法包括经典分解法、RobustSTL 分解法、Loess 分解法等 [63, 9, 14, 9], 它们都基于加法模型或者乘法模型进行分解。下面对经典分解法和 RobustSTL 分解法的计算过程进行简要介绍。

经典分解法起源于 20 世纪 20 年代, 通过移动平均将时间序列分解为趋势项、季节项和其他成分。它通常假设季节性成分在每个周期内都是相同的, 步骤相对简单,

同时也是很多其他分解算法的基础。其中需要使用 m 阶移动平均，即 m -MA，公式如下：

$$\hat{\mathbf{x}}_t = \frac{1}{m} \sum_{j=-k}^k \mathbf{x}_{t+j} \quad (39)$$

其中 $m = 2k + 1$ ，即时刻 t 的移动平均值为向前 k 个值和向后 k 个值的均值。当阶数为偶数时，移动平均线是非对称的。为了达到对称性的要求，可以使用中心化移动平均。通常， $2 \times m$ -MA 阶移动平均等价于一个第一项和最后一项权重取 $\frac{1}{2m}$ ，其他所有项权重都取 $\frac{1}{m}$ 的 $m+1$ 阶移动平均。

经典分解法的计算过程可以总结如下。

- 求时间序列的趋势项 $\mathbf{T}$ ：如果 m 为偶数，连续使用两次 m -MA 阶移动平均计算；如果 m 为奇数，则使用一次 m -MA 阶移动平均计算。
- 得到去掉趋势项的时间序列 $\mathbf{D}$ ： $\mathbf{D} = \mathbf{X} - \mathbf{T}$ 。
- 求时间序列的季节项 $\mathbf{S}$ ：对同一周期的数据求均值即可。比如对于月度数据，要计算三月的季节项，对 $\mathbf{D}$ 中所有三月的数据求均值即可。由此循环计算到 $\mathbf{D}$ 的长度，就得到了 $\mathbf{D}$ 的所有季节项。
- 求时间序列的其他部分 $\mathbf{R}$ ： $\mathbf{R} = \mathbf{X} - \mathbf{T} - \mathbf{S}$ 。

同理，对于乘法模型，将上述加法模型的第二点改成 $\mathbf{D} = \mathbf{X} \times \mathbf{T}$ ，第四点改成 $\mathbf{R} = \mathbf{X} \div (\mathbf{T} \times \mathbf{S})$ 即可。

RobustSTL 分解法 [63] 相较于经典分解法更为稳健，可以有效处理随时间变化的季节性成分。计算过程可以总结如下。

- 去噪：通过双边滤波去除噪声，保留重要特征。
- 求时间序列的趋势项 $\mathbf{T}$ ：通过求解具有稀疏正则化的最小绝对偏差 (Least Absolute Deviations, LAD) 回归得到趋势项。
- 求时间序列的季节项 $\mathbf{S}$ ：通过非局部季节性滤波得到季节项，克服季节性波动和偏移。
- 调整成分：调整得到的趋势项和季节项，得到更新的成分，然后求时间序列的其他部分 $\mathbf{R}$ 。

通过不断重复以上步骤就可以得到最终的分解结果。乘性模型的分解也可以类似地得到。

图 2.10 是一个季节性-趋势分解法的应用示例，将时间序列分解为趋势 (Trend)、季节性 (Seasonal)、残差 (Residual) 三部分。

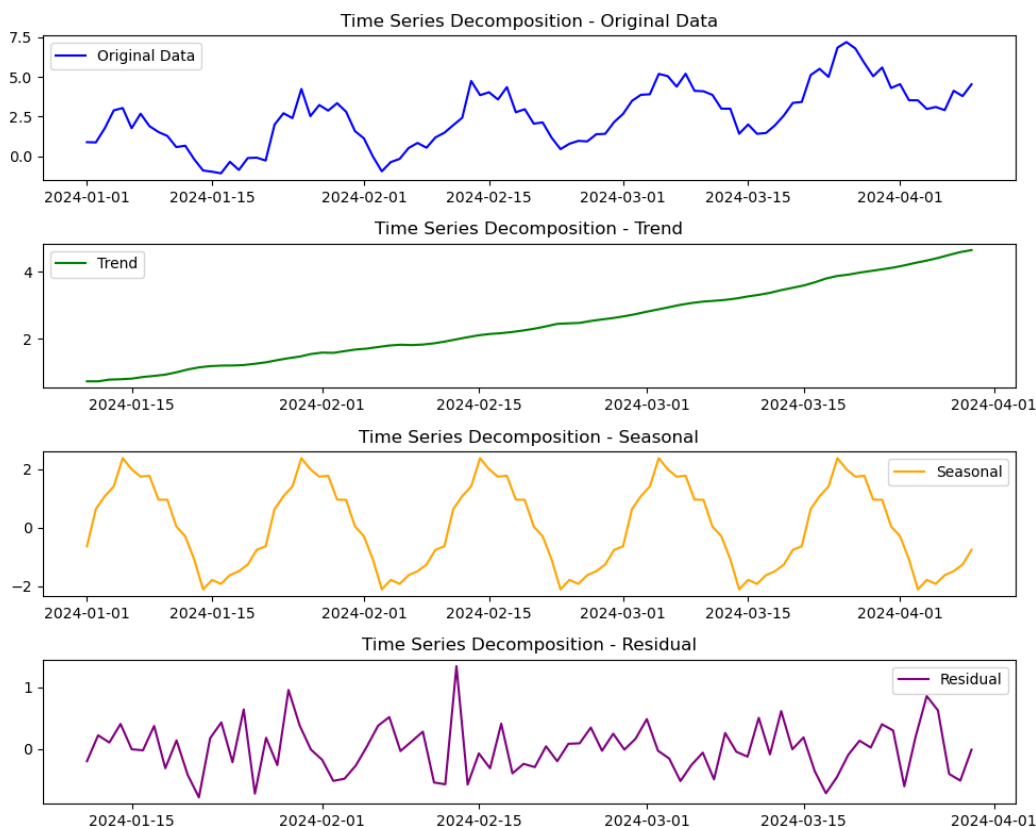


图 2.10: 季节性-趋势分解法的应用示例。

第二类方法是基函数扩展 (Basis Expansion)。基函数扩展的主要思想是将时间序列的非线性变化表示为一组预定义或数据驱动基函数的线性组合：

$$\mathbf{X} = \sum_{i=1}^k \theta_i \cdot \mathbf{v}_i \quad (40)$$

其中， $\mathbf{v}_i$ 是基函数向量， θ_i 是扩展系数（基函数权重）， k 是基函数个数。通过基函数的加权组合揭示复杂时序的潜在模式。

基函数的选取直接决定了模型对时序特征的捕获能力，根据是否依赖领域先验，可分为预定义基函数和数据驱动基函数两类 [49]。

- **预定义基函数**：通过融入时间序列特性（如趋势、季节性），直接约束模型输出符合现实规律，典型代表为多项式基和傅里叶基，适用于可解释性需求较高的场景。
 - 多项式基：针对时间序列中的趋势特征（如商品销量的长期增长），采用低阶多项式（通常为 1-3 次）作为基函数，即原始时序的趋势成分可表示为该基函数的线性组合。

- 傅里叶基：针对时间序列中的季节性特征（如日用电量的 24 小时周期），采用正弦或余弦函数（傅里叶级数）作为基函数，即原始时序的季节性成分可表示为该基函数的线性组合，不同频率的正余弦函数可匹配不同周期的季节性。

- **数据驱动基函数**：当时间序列模式复杂（如无固定周期的不规则波动）时，模型通过神经网络自主学习基函数，无需人工定义，灵活性较强。

第三类方法是矩阵分解（Matrix Factorization）。矩阵分解针对多变量时间序列，将高维时间序列矩阵分解为两个低秩矩阵——变量特征矩阵与时间依赖矩阵的乘积 [66, 61]。

多变量时间序列可以表示为矩阵（每一行代表不同时刻的观测值，每一列代表不同的变量） $\mathbf{X} \in \mathbb{R}^{T \times N}$ ，其中 T 表示时间步数， N 表示变量数量。矩阵分解的目标是将原始高维矩阵 $\mathbf{X}$ 近似表示为两个低秩基矩阵的乘积：

$$\mathbf{X} \approx \mathbf{F} \cdot \hat{\mathbf{X}} \quad (41)$$

其中， $\mathbf{F} \in \mathbb{R}^{k \times N}$ 是**变量特征矩阵**，每一列 $\mathbf{F}_{:,n} \in \mathbb{R}^k$ 对应第 n 个变量的低维特征向量，刻画该变量与其他变量的关联模式，例如，交通数据中相邻路段的列向量相似，反映其空间相关性。 $\hat{\mathbf{X}} \in \mathbb{R}^{T \times k}$ 是**时间依赖矩阵**，每一行 $\hat{\mathbf{X}}_{t,:} \in \mathbb{R}^k$ 对应第 t 时刻的时间特征向量，刻画该时刻下 k 个变量特征的动态变化，例如，早高峰与晚高峰时刻的行向量相似，反映时序周期性。 $k \ll \min(T, N)$ ，是控制降维维度的超参数。

矩阵分解通过“低秩基矩阵组合”处理多变量时序，具有两大核心优势：

- **降维**：将原始 $T \times N$ 矩阵转化为 $T \times k$ 与 $k \times N$ 矩阵，参数数量从 $O(T \cdot N)$ 降至 $O(T \cdot k + k \cdot N)$ （ $k \ll N$ 时降幅显著），有利于提高计算效率。
- **解耦变量与时间**：变量特征矩阵捕获变量关联，时间依赖矩阵捕获时序动态，在降维的同时保留关键信息。

2.3 时间序列数据建模角度

在时间序列分析中主要可以分为两种建模角度，一种是最常见的**从时域角度建模**，另一种是**从频域角度建模**。时域和频域是时间序列分析中两种不同的表示方式。时域分析让我们直接观察时间序列的原始信息，而频域分析则能帮助识别时间序列中的周期性和频率特性。

2.3.1 从时域角度建模

在时域中，数据点按照时间顺序排列，能够让人直观地观察到序列值随时间的变化。这种表示方式包含了关于时间序列的初始信息，如趋势、周期性和噪声等特

征。在时域中可以直接使用各种传统统计分析方法来理解时间序列的特性，例如计算均值、方差、自相关函数等，以分析序列的整体水平、波动性以及其内部的相关性。以上介绍的全都是时序在时域上的相关特性，常见的时域模型包括自回归（Auto-regressive, AR）模型、移动平均（Moving Average Model, MA）模型和自回归移动平均（Auto-regressive Moving Average, ARMA）模型。

而在时间序列分析中，时域上的建模角度又可以细分为**点粒度级（point-wise）**、**片段粒度级（patch-wise）**和**序列粒度级（series-wise）**。这种细分方式是基于原始数据输入模型时进行的切分处理，对应图中的三种切分方式。将模型用 f 抽象表示， θ 是模型的可学习参数， $\hat{Y}$ 表示模型的输出。

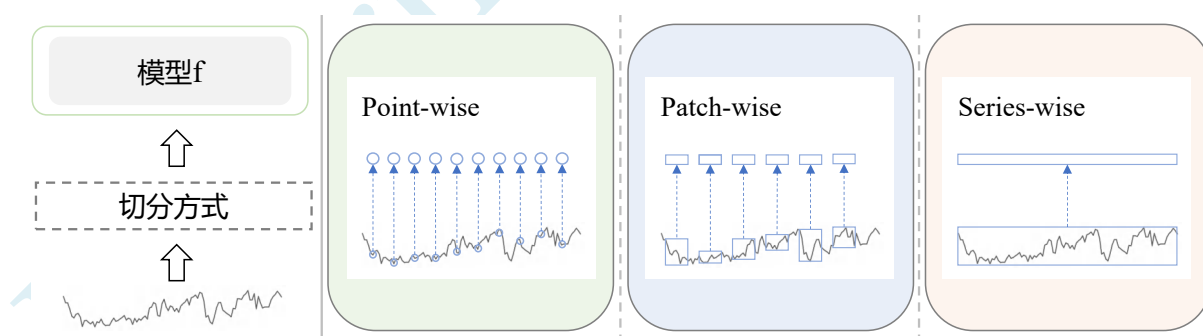


图 2.11: 不同粒度建模。

点级（Point-wise）建模是一种专注于单个时间点数据的细粒度时间序列分析方法，它将每个时间点视为独立的输入或特征进行处理，通常用于单点预测或分类任务，而不考虑数据之间的关联。

$$\hat{Y} = f_{\theta}(x_1, x_2, \dots, x_T) \quad (42)$$

点级建模适合短期预测或不依赖于长时间关系的任务，如异常检测、数据清洗等。其计算相对简单，适合高频数据或需要实时响应的任务。例如，LogSparse [38] 采样方法是一种针对高频时间序列数据的稀疏采样技术，它通过选择性地保留具有代表性的数据点，大幅减少了数据量，从而提高了计算效率和模型的响应速度。这种特性使得 LogSparse 在处理需要实时响应的高频数据时具有显著优势。

然而，点级建模的缺点在于点之间相互独立，没有上下文信息，忽略了时间序列中潜在的长时依赖关系，可能无法捕捉到复杂的趋势和周期性。而 Informer [69] 模型虽然本质上也是一种点级建模，但它通过引入 ProbSparse 自注意力机制和生成式解码器，有效地捕捉了时间序列中的长时依赖关系。

块级（Patch-wise）建模将时间序列分为多个较短的连续片段或“补丁”（patch），每个片段包含若干个连续的时间点。此方法在局部时间窗口内提取特征，因此可以捕捉到短期趋势和局部模式，是比点级建模更粗粒度的建模方法。Triformer [8] 是经典

的块级建模工作之一，它通过滑动窗口将时间序列切分为长度为 P 的 M 个片段，其中第 i 个片段表示为 $\mathbf{p}_i \in \mathbb{R}^P$ ，长度为 T 的时间序列切分后， $N = \lfloor \frac{T-P}{S} \rfloor + 2$ 。这篇文章中，在切片之前，会在原始序列的末尾填充 S 个重复的数值，这些数值为原始序列的最后一个值。类似的，卷积神经网络中的感受野也是一种类似的块级粒度时序建模概念。

$$\hat{\mathbf{Y}} = f_{\theta}(\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_M) \quad (43)$$

块级建模适用于中短期预测任务或包含局部趋势和周期的时间序列分析，如信号处理、传感器数据分析等。块级建模通过局部时间窗口捕捉短期依赖关系，能够获得较好的信息利用率，适合用于卷积神经网络 (Convolutional Neural Network, CNN) [34] 等模型。缺点是片段的长度选择较为关键。过短会丢失依赖信息，过长则会增加计算量，并可能引入噪声。此外，块级建模还被应用于一些先进的深度学习模型中，比如 Autoformer [65]，它采用了块级建模的方法，并将传统时序分解和时序自相关思想融入到 Transformer 中，采用 FFT 加快自相关的计算效率，适用于长期时间序列预测任务。又比如 Pathformer [6]，考虑了块 (Patch) 大小对预测性能的影响，通过自适应地选择合适的块大小进行建模以达到更好的预测效果。

序列级 (Series-wise) 建模将整个时间序列作为一个整体进行处理和建模，关注序列中所有点的长期依赖关系。它通常用于长序列分析或全局趋势预测。

$$\hat{\mathbf{Y}} = f_{\theta}(\mathbf{x}) \quad (44)$$

序列级建模适用于长时依赖、全局趋势分析的任务，如金融时间序列、气候变化趋势预测等。优点是能够捕捉到长期的时间依赖性，适合处理具有复杂趋势和周期的时间序列。以 TimeXer[62] 模型为例，它除了使用块级建模，还使用了序列级建模，这使得在处理复杂的时间序列预测任务时，能够同时考虑除目标变量的其他变量，捕捉到长期的时间依赖性，从而更准确地预测未来的时序波动。序列级建模往往结合 RNN、LSTM 或 Transformer 等架构。比如 iTransformer[40] 使用了像 Transformer 这样的自注意力机制来进行序列级建模，以捕获长程依赖关系。但是同时由于需要处理全局信息，因此计算成本较高，对模型的计算能力要求较大。

这三种方法中点级建模最细，块级次之，序列级最粗。然后在提取时间依赖的时候，点级几乎无依赖，块级捕捉短期依赖，序列级可捕捉长期依赖。在实际应用中，模型选择应基于任务需求和数据特性，点级用于实时和局部特征检测，块级适合局部模式识别，序列级则关注全局特征和趋势。

2.3.2 从频域角度建模

除了从时域角度建模，另一个角度是从频域建模。频域分析是一种通过将时域数据转换为频率表示来理解时间序列的技术。频域分析关注的是信号的频率成分，通过

将时域信号应用傅里叶变换或其他频域方法，时间序列被分解为不同频率的振荡成分。

频域表示有助于识别时间序列中的周期性和频率特征。通过频域分析，可以得到时间序列中不同频率成分的振幅和相位信息，从而进一步理解时间序列的周期性行为、频率特性和噪声组成。常见的频域模型包括傅里叶级数和傅里叶变换模型。以快速傅里叶变换为例，从频域角度建模的过程如下：首先，对时间序列进行快速傅里叶变换（Fast Fourier Transform, FFT） [11]，将其从时域映射到频域。在频域中，可以更清晰地识别和分析时间序列的周期性成分和频率特征。基于这些信息，构建模型并进行预测，得到的预测值仍为频域表示。预测完成后，再将预测结果通过逆快速傅里叶变换（Inverse Fast Fourier Transform, IFFT）映射回时域，从而得到最终的预测值。

$$\hat{\mathbf{Y}} = IFFT(f(FFT(\mathbf{X}))) \quad (45)$$

通过傅里叶变换，我们可以将一个时间序列分解为一系列正弦和余弦波的叠加，从而识别出序列中的周期性和频率成分。这在分析复杂的周期性行为和频率特性时非常有用。例如下图 2.12，我们有一个心跳监测器记录的 10 秒数据，原始信号包含心跳信号（约 1.17 Hz，即 70 次/分钟）、呼吸造成的低频波动（0.3 Hz）以及仪器的高频噪声（10 Hz）。在时域中，这三种信号混合在一起形成了复杂的波形，肉眼很难从这个混乱的时域信号中分辨出心跳频率。但通过傅里叶变换后，在频域图中会清晰地显示出三个峰值：0.3 Hz（呼吸）、1.17 Hz（心跳主峰）、10 Hz（噪声），立即就能识别出心跳的准确频率。对一个连续的时序，傅里叶变换计算如下：

$$FT(\mathbf{X}) = \int_{-\infty}^{\infty} \mathbf{x}_t e^{-j2\pi f t} dt \quad (46)$$

其中： $FT(\mathbf{X})$ 表示信号在频率 f 上的频域表示。 $\mathbf{x}_t$ 是时间域中的信号。 $e^{-j2\pi f t}$ 是复指数项，包含频率信息。 j 是虚数单位（即 $j = \sqrt{-1}$ ）。

需要注意的是，时域和频域是互相关联的概念，而非截然分割的。我们可以先将时域数据转换为频域数据进行分析，然后将频域结果转换回时域，以获得更全面的理解和预测能力。

总而言之，时域和频域是时间序列分析中两种不同但相关的表示方式。时域分析可以让我们直观地观察到时间序列的原始信息，而频域分析则可以帮助我们识别时间序列中的周期性和频率特性。

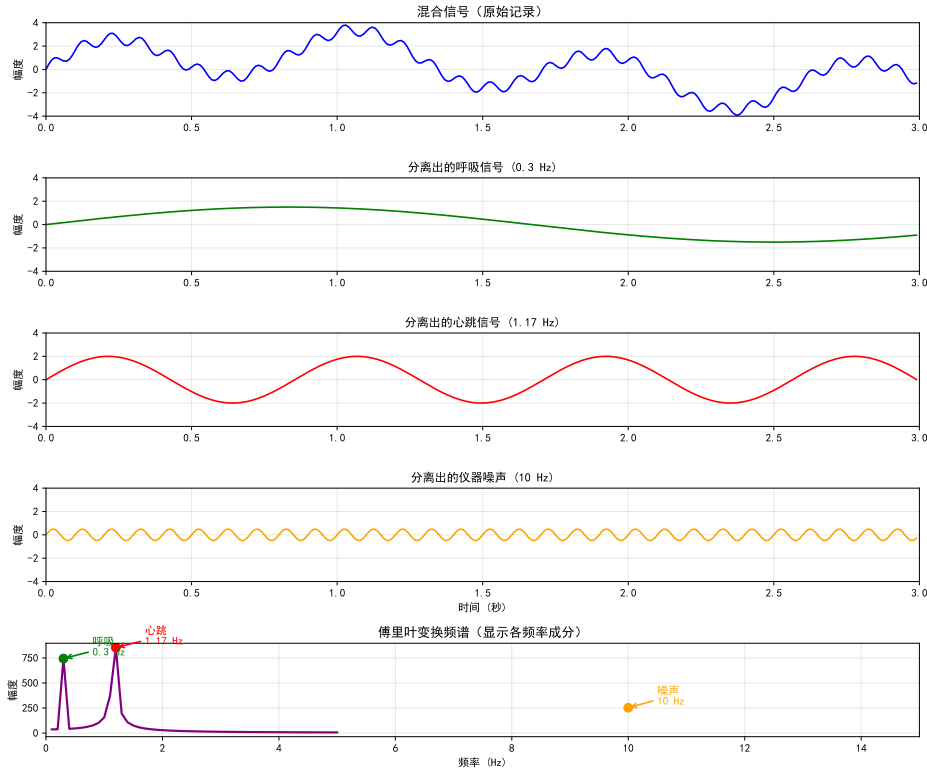


图 2.12: 通过傅里叶变换将心跳监测器数据分解为三个独立的频率成分。

2.4 基础深度学习模型

2.4.1 多层感知机

多层感知机 (Multi-Layer Perceptron, MLP) 是一种前馈人工神经网络, 由 David Rumelhart 等人 [54] 在 1986 年提出, 它的前身是单层感知机, 一种受生物神经元启发设计的二元线性分类器, 由 Frank Rosenblatt [53] 在 1957 年提出。MLP 由输入层、隐藏层和输出层组成, 输入层接收数据, 隐藏层处理数据, 输出层输出结果。输入层和输出层之间可以包含一至多个隐藏层, 其中的神经元称为隐藏单元, 输出称为隐藏变量, 其结构如图 2.13 所示。

在 MLP 中, 隐藏层可以表示为:

$$\mathbf{z}_j^{(l)} = \sum_i \mathbf{w}_{ji}^{(l)} \mathbf{h}_i^{(l-1)} + \mathbf{b}_j^{(l)} \quad (47)$$

其中, $\mathbf{z}_j^{(l)}$ 是第 l 层第 j 个节点的输出, $\mathbf{h}_i^{(l-1)}$ 是前一层第 i 个节点的输出, $\mathbf{w}_{ji}^{(l)}$ 是连接上一层第 i 个神经元到第 l 层第 j 个神经元的权重, $\mathbf{b}_j^{(l)}$ 是第 l 层第 j 个节点的偏置。

隐藏单元的输出通过激活函数 ϕ 进一步计算, 得到最终输出:

$$\mathbf{h}_j^{(l)} = \phi(\mathbf{z}_j^{(l)}) \quad (48)$$

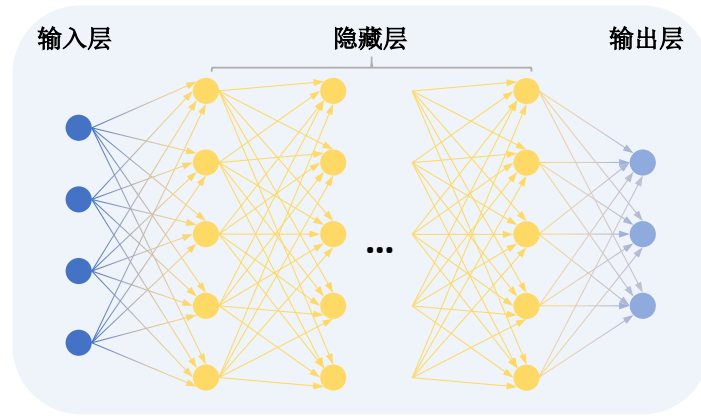


图 2.13: 多层感知机结构图。

2.4.2 卷积神经网络

卷积神经网络（Convolutional Neural Network, CNN）可以看作是 MLP 的变形，将 MLP 的全连接用局部连接替代。其提出可以追溯到二十世纪 80~90 年代，1979 年 Kunihiko Fukushima [18] 提出了视觉识别模式的多层架构 Neocognitron，即 CNN 的鼻祖，1998 年 Yann LeCun 等人 [35] 在此基础上进行了改进，提出了著名的 LeNet 神经网络。此后，更加强化的 CNN 网络被相继提出 [32, 68, 55, 57, 23, 25, 58]。CNN 的基本结构包括输入层、卷积层、池化层、全连接层、输出层等，输入层接收数据，卷积层提取数据局部特征，池化层对卷积层输出的特征图进行下采样，全连接层将局部特征组合成全局特征，输出层输出结果，其结构如图 2.14 所示。

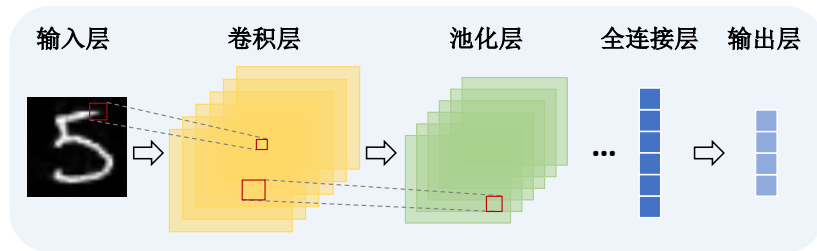


图 2.14: 卷积神经网络结构图。

卷积的操作类似于图像处理中的空间滤波，卷积核在图像内移动，在每一个位置卷积核权重和对应位置的图像像素相乘并求和，即卷积核在图像的每个局部区域逐次进行逐元素乘积和求和运算，以得到卷积后的图像的每个像素值。给定二维图像作为输入，对于 (i, j) 处的像素点，二维卷积运算可以表示为：

$$\mathbf{Y}_{ij} = \sum_{a=0}^{n-1} \sum_{b=0}^{m-1} \mathbf{w}_{ab} \mathbf{X}_{(i+a)(j+b)} \quad (49)$$

其中，卷积核的大小为 $n \times m$ ， a 和 b 是卷积核的索引。 w_{ab} 是权重矩阵 w 在 (a, b) 处的权重值。 $X_{(i+a)(j+b)}$ 和 Y_{ij} 分别是卷积前 $(i+a, j+b)$ 处和卷积后 (a, b) 处的像素值。

2.4.3 循环神经网络

循环神经网络是一种专门用于处理序列数据的神经网络模型。与传统的前馈神经网络不同，这种网络具有循环连接，可以记住输入序列中的上下文信息。循环神经网络的核心思想在于，判定过去的隐变量是否会对当前的样本产生影响，这样的考虑赋予了其处理依赖性较高的数据的能力，因此在时间序列分析、自然语言处理和语音识别等领域表现出色。接下来介绍常见的三类循环神经网络模型的结构和异同。

原始循环神经网络 Vanilla RNN[67] 是最早提出的循环神经网络，用于捕获长依赖关系和上下文信息，是序列数据任务的关键组件。RNN 的结构如图 2.15 所示，其状态更新可以形式化为以下形式：

$$h_t = \sigma(w_{xh}x_t + w_{hh}h_{t-1} + b_h) \quad (50)$$

其中， h_t 表示时间步 t 的隐藏状态向量， $\sigma(\cdot)$ 为激活函数，通常为 $\tanh$ 或者 ReLU 。而输出层可以形式化为：

$$y_t = \phi(w_{hy}h_t + b_y) \quad (51)$$

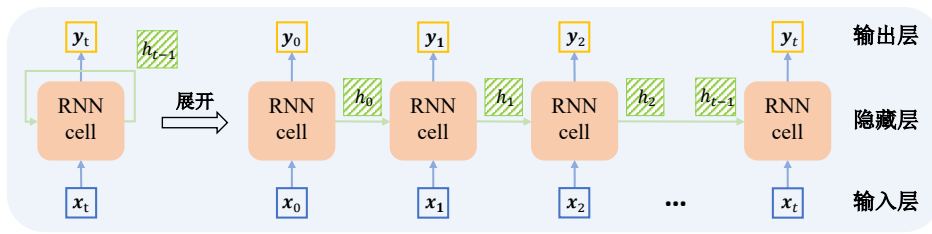


图 2.15: 循环神经网络结构图。

RNN 通常通过一种称为时间反向传播 (Backpropagation Through Time, BPTT) 的迭代训练过程进行优化。当在时间上展开后，RNN 可被看作一个非常深的神经网络，且不同时间步的参数共享。在每个时间步，梯度通过链式法则不断进行矩阵乘法，这可能导致梯度在传播过程中呈指数级衰减（即梯度消失）或爆炸（即梯度爆炸）。为了解决这些问题，人们提出了二阶结构的循环模型，包括长短期记忆网络 (LSTM) 和门控循环单元 (GRU)，用以提升深层网络在长序列上的学习能力。

长短期记忆网络 LSTM (Long Short-Term Memory) [22] 是一种改进的循环神经网络，专门设计用来解决标准 RNN 在处理长序列时易发生的梯度消失和梯度爆炸问题。通过引入记忆单元和三个门结构（遗忘门、输入门和输出门），LSTM 能够动态地选择

性记忆或丢弃信息，从而有效捕获长时依赖关系。三个门控机构可以形式化表示为：

$$\mathbf{f}_t = \sigma(\mathbf{w}_f \mathbf{x}_t + \mathbf{u}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (52)$$

$$\mathbf{i}_t = \sigma(\mathbf{w}_i \mathbf{x}_t + \mathbf{u}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (53)$$

$$\mathbf{o}_t = \sigma(\mathbf{w}_o \mathbf{x}_t + \mathbf{u}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (54)$$

$$(55)$$

状态更新公式为：

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{w}_c \mathbf{x}_t + \mathbf{u}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (56)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (57)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (58)$$

与朴素的 RNN 一样，最后输出的 $\mathbf{h}_t$ 为当前隐藏状态。通过这一系列方法，LSTM 实现了长短期记忆的融合。

门控循环单元 GRU (Gated Recurrent Unit) [7] 是 RNN 的另一种广泛使用的变体，它与 LSTM 类似，能够控制信息的流动，并在多个时间步之间记忆上下文。其形式化表示为：

$$\mathbf{z}_t = \sigma(\mathbf{w}_z \mathbf{x}_t + \mathbf{u}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \quad (59)$$

$$\mathbf{r}_t = \sigma(\mathbf{w}_r \mathbf{x}_t + \mathbf{u}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \quad (60)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{w}_h \mathbf{x}_t + \mathbf{u}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad (61)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (62)$$

GRU 简化了 LSTM 的三门控制策略，选择使用更新门 $\mathbf{z}_t$ 和重置门 $\mathbf{r}_t$ 进行控制，最后从多个候选隐藏状态 $\tilde{\mathbf{h}}_t$ 中选择当前隐藏状态 $\mathbf{h}_t$ 。

2.4.4 时间卷积网络

时间卷积网络 (Temporal Convolutional Network, TCN) 是一种旨在处理时间序列数据的 CNN，在 2018 年由 Shaojie Bai 等人 [3] 提出。TCN 以 CNN 为基础，使用一维全卷积网络架构，并引入了空洞因果卷积 (Dilated Causal Convolution) 和残差连接，其结构如图 2.16 所示。

给定滤波器 (Filter) $\mathbf{f} = (f_1, f_2, \dots, f_K)$ ，即在卷积操作中用于提取局部特征的权重矩阵。对于序列 $\mathbf{x} = (x_1, x_2, \dots, x_T)$ ，因果卷积和空洞卷积分别定义如下：

$$(\mathbf{f} * \mathbf{x})_{(x_t)} = \sum_{k=1}^K f_k x_{t-K+k} \quad (63)$$

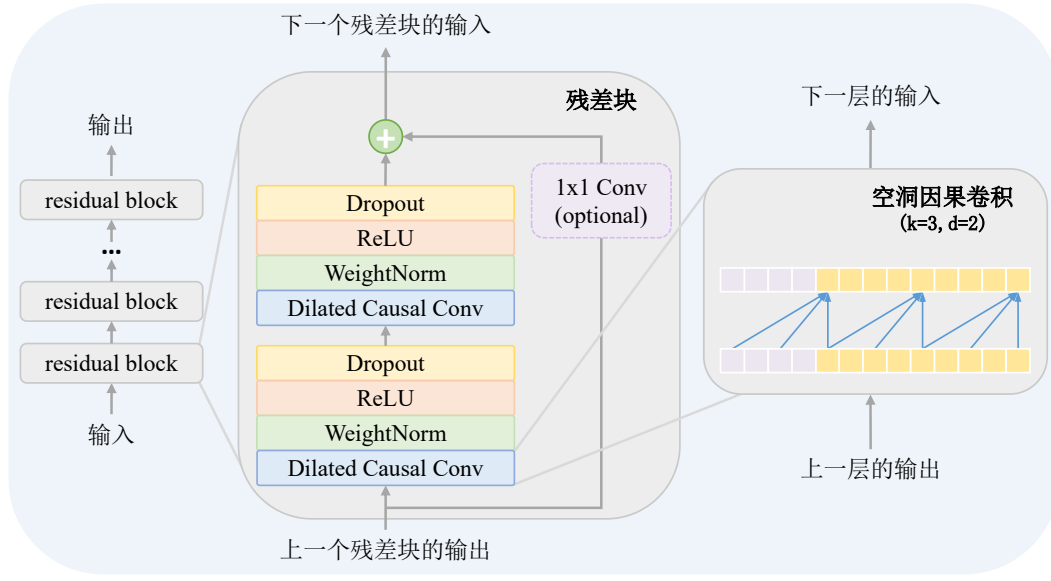


图 2.16: 时间卷积网络结构图。

$$(\mathbf{f} * d \mathbf{x})_{(x_t)} = \sum_{k=1}^K f_k x_{t-(K-k)d} \quad (64)$$

其中, x_t 是输入序列中时刻 t 的值, f_k 是滤波器中的第 k 个元素, d 是膨胀因子 (dilation factor), 表示滤波器中每两个相邻元素之间的间隔。

2.4.5 图卷积网络

图卷积网络 (Graph Convolutional Network, GCN) 是一种基于图结构的非欧氏数据的 CNN, 在 2017 年由 Thomas Kipf 等人 [31] 提出。GCN 是谱图卷积的一阶局部近似, 仅依赖距离中心节点 k 步距离之内的节点, 每个卷积层处理一阶邻域信息, 通过堆叠多个卷积层处理多阶邻域信息, 其结构如图 2.17 所示。

每个节点下一层的信息由该点前一层的信息和相邻节点的信息加权加和并经过非线性变换得到。假设数据有 N 个节点, 每个节点有 D 个特征, 这些节点的特征组成一个 $N \times D$ 的特征矩阵 $\mathbf{X}$, 各节点之间的关系组成一个 $N \times N$ 的邻接矩阵 $\mathbf{A}$, $\mathbf{X}$ 和 $\mathbf{A}$ 作为模型的输入, 经过模型处理后得到 $N \times F$ 的特征矩阵 $\mathbf{Z}$ 。网络的层间传播规则表示为:

$$\mathbf{H}^{(l)} = f(\mathbf{H}^{(l-1)}, \mathbf{A}) = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l-1)} \mathbf{w}^{(l)} \right) \quad (65)$$

其中, $\mathbf{H}^{(l-1)}$ 、 $\mathbf{H}^{(l)}$ 分别代表上一层卷积层输出的节点表示和当前层卷积层输出的节点表示, 则 $\mathbf{H}^{(0)} = \mathbf{X}$, $\mathbf{H}^{(L)} = \mathbf{Z}$, L 是层数。 $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$, $\mathbf{I}_N$ 是 N 维单位矩阵。 $\tilde{\mathbf{D}}$ 是 $\tilde{\mathbf{A}}$ 的度矩阵。 $\mathbf{w}^{(l)}$ 是当前层中的权重矩阵。 σ 是非线性激活函数。

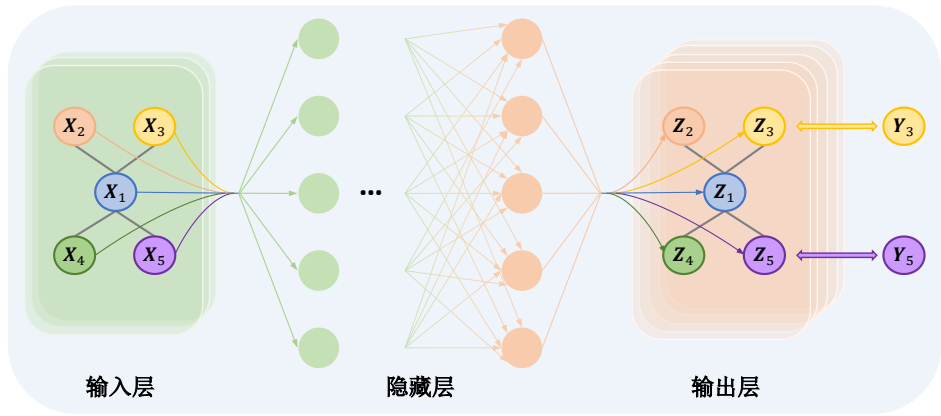


图 2.17: 图卷积网络结构图。

2.4.6 Transformer

Transformer 是一种完全基于自注意力机制的神经网络，用于处理序列数据，在 2017 年由谷歌公司团队 [60] 提出。相比于 RNN，Transformer 具有更好的并行性能和更短的训练时间，在自然语言处理领域中得到了广泛应用。Transformer 主要由输入部分（嵌入层与位置编码层）、多层编码器、多层解码器和输出部分（线性层与 Softmax 层）四大部分组成，其结构如图 2.18 所示。

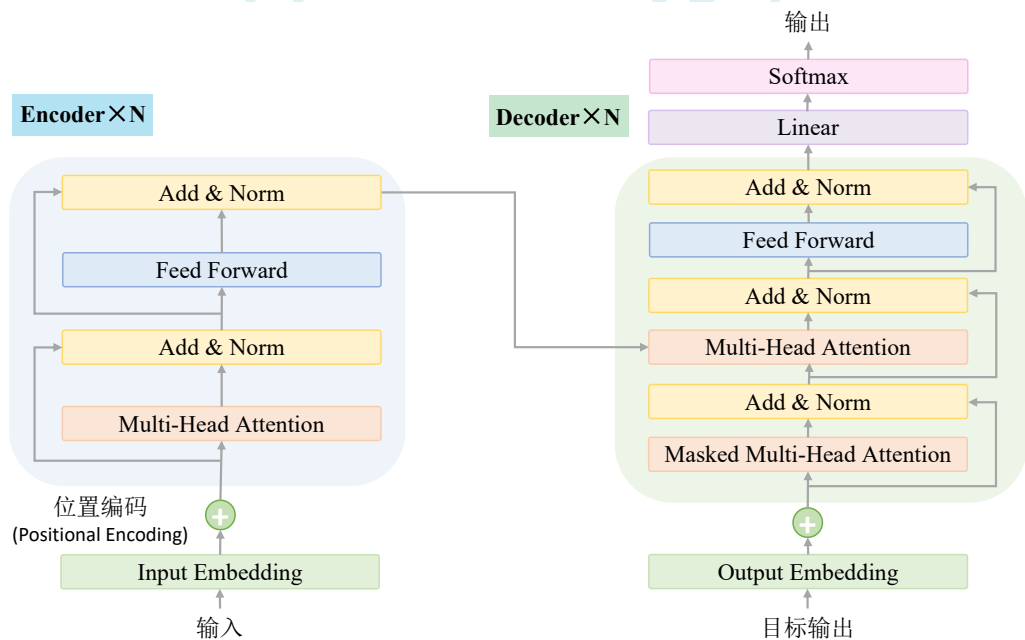


图 2.18: Transformer 结构图。

Transformer 的核心在于自注意力机制（Self-Attention Mechanism），首先计算每个位置与其他位置之间的注意力权重，即每个位置对其他位置的重要性。然后将每

个位置向量与注意力权重相乘，再将它们相加，得到加权和向量。最后加权和向量经过线性变换，得到最终的输出向量，计算公式如下：

$$Attention(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = Softmax(\frac{\mathbf{QK}^T}{\sqrt{d_k}})\mathbf{V} \quad (66)$$

其中， $\mathbf{Q}$ 是 Query 矩阵， $\mathbf{K}$ 是 Key 矩阵， $\mathbf{V}$ 是 Value 矩阵，它们都是对输入 $\mathbf{X}$ 进行线性变换。Query 矩阵代表信息需求，用于与 Key 矩阵进行匹配。Key 矩阵代表各个位置的标识信息，用于被 Query 矩阵查询匹配。Value 矩阵代表与 Key 矩阵相对应的实际值，当 Query 与某个 Key 匹配时，相应的 Value 将被用来计算输出。 d 是缩放因子。

Transformer 在自然语言处理领域的突破性进展源于多头注意力机制（Multi-Head Attention Mechanism）。每个编码器和解码器模块包含多个并行的注意力头，每个注意力头独立学习不同的注意力模式，使得模型可以同时捕捉词汇的语义多样性和句法结构。此外，Transformer 引入了位置编码（Position Encoding），通过正弦和余弦函数的组合编码位置信息，使模型能够感知序列顺序。

参考文献

- [1] Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. Gluonts: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- [2] Melissa J Azur, Elizabeth A Stuart, Constantine Frangakis, and Philip J Leaf. Multiple imputation by chained equations: what is it and how does it work? *International journal of methods in psychiatric research (IJMPR)*, 20(1):40–49, 2011.
- [3] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [4] Inci M. Baytas, Cao Xiao, Xi Zhang, Fei Wang, Anil K. Jain, and Jiayu Zhou. Patient subtyping via time-aware LSTM networks. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 65–74, 2017.
- [5] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.
- [6] Peng Chen, Yingying Zhang, Yunyao Cheng, Yang Shu, Yihang Wang, Qingsong Wen, Bin Yang, and Chenjuan Guo. Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting. In *Proceedings of the Conference on Learning Representations (ICLR)*, 2024.
- [7] Junyoung Chung, Caglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.
- [8] Razvan-Gabriel Cirstea, Chenjuan Guo, Bin Yang, Tung Kieu, Xuanyi Dong, and Shirui Pan. Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting. In *Proceedings of the Conference on Artificial Intelligence (IJCAI)*, pages 1994–2001, 2022.

- [9] Robert B Cleveland, William S Cleveland, Jean E McRae, and Irma Terpenning. Stl: A seasonal-trend decomposition. *Journal of Official Statistics*, 6(1):3–73, 1990.
- [10] Israel Cohen, Yiteng Huang, Jingdong Chen, Jacob Benesty, Jacob Benesty, Jingdong Chen, Yiteng Huang, and Israel Cohen. Pearson correlation coefficient. *Noise Reduction in Speech Processing*, pages 1–4, 2009.
- [11] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation (Math. Comp.)*, 19(90):297–301, 1965.
- [12] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory (TIT)*, 13(1):21–27, 1967.
- [13] Hoang Anh Dau, Eamonn Keogh, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, Yanping, Bing Hu, Nurjahan Begum, Anthony Bagnall, Abdullah Mueen, Gustavo Batista, and Hexagon-ML. The ucr time series classification archive, 2018. https://www.cs.ucr.edu/~eamonn/time_series_data_2018/.
- [14] Alysha M De Livera, Rob J Hyndman, and Ralph D Snyder. Forecasting time series with complex seasonal patterns using exponential smoothing. *Journal of the American Statistical Association (JASA)*, 106(496):1513–1527, 2011.
- [15] Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the royal statistical society: series B (methodological)(JRSS-B)*, 39(1):1–22, 1977.
- [16] Graham Elliott, Thomas J Rothenberg, and James H Stock. Efficient tests for an autoregressive unit root. *Econometrica*, 64(4):813–836, 1996.
- [17] Jeffrey L Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
- [18] Kunihiro Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.
- [19] Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic

- p forecasting with spline quantile function rnns. In
- Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*
- , pages 1901–1910, 2019.
- [20] Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. In *Proceedings of the Conference on Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Track on Datasets and Benchmarks)*, 2021.
- [21] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. Generative adversarial nets. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 2672–2680, 2014.
- [22] Alex Graves and Alex Graves. Long short-term memory. *Supervised sequence labelling with recurrent neural networks*, pages 37–45, 2012.
- [23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [24] Charles C Holt. Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting*, 20(1):5–10, 2004.
- [25] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708, 2017.
- [26] Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with exponential smoothing: the state space approach*. Springer Science & Business Media, 2008.
- [27] Robin John Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2018.
- [28] Fumitada Itakura. Minimum prediction residual principle applied to speech recognition. *IEEE Transactions on Signal Processing (TSP)*, 23(1):67–72, 2003.
- [29] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series fore-

- p>casting against distribution shift. In
- Proceedings of the International Conference on Learning Representations (ICLR)*
- , 2022.
- [30] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.
- [31] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [32] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 1106–1114, 2012.
- [33] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *Proceedings of the International ACM SIGIR Conference on Research & Development in Information Retrieval (SIGIR)*, pages 95–104, 2018.
- [34] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proc. IEEE*, 86(11):2278–2324, 1998.
- [35] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE*, pages 2278–2324, 1998.
- [36] Daniel D Lee and H Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *nature*, 401(6755):788–791, 1999.
- [37] Doyup Lee. Anomaly detection in multivariate non-stationary time series for automatic DBMS diagnosis. In *Proceedings of the IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 412–419, 2017.
- [38] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyu Zhou, Wenhui Chen, Yu-Xiang Wang, and Xifeng Yan. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 5244–5254, 2019.
- [39] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

- [40] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *Proceedings of The Conference on Learning Representations (ICLR)*, 2024.
- [41] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 9881–9893, 2022.
- [42] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [43] Zhiding Liu, Mingyue Cheng, Zhi Li, Zhenya Huang, Qi Liu, Yanhu Xie, and Enhong Chen. Adaptive normalization for non-stationary time series forecasting: A temporal slice perspective. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 14273–14292, 2023.
- [44] Maggie, Oren Anava, Vitaly Kuznetsov, and Will Cukierski. Web traffic time series forecasting. Kaggle, 2017. <https://kaggle.com/competitions/web-traffic-time-series-forecasting>.
- [45] Paolo Mancuso, Veronica Piccialli, and Antonio M Sudoso. A machine learning approach for forecasting hierarchical time series. *Expert Systems with Applications*, 182:115102, 2021.
- [46] Michael W McCracken and Serena Ng. Fred-md: A monthly database for macroeconomic research. *Journal of Business & Economic Statistics*, 34(4):574–589, 2016.
- [47] GP Nason. Stationary and non-stationary time series. In *Statistics in Volcanology (SIV)*, pages 129–142. 2006.
- [48] Kevin E O’Grady. Measures of explained variance: Cautions and limitations. *Psychological Bulletin*, 92(3):766, 1982.
- [49] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecast-

- ing. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [50] George Panagopoulos, Giannis Nikolentzos, and Michalis Vazirgiannis. Transfer graph neural networks for pandemic forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 4838–4845, 2021.
- [51] Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, 31(9):3760–3765, 2019.
- [52] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. TFB: towards comprehensive and fair benchmarking of time series forecasting methods. *Proceedings of the Very Large Data Bases Endowment*, 17(9):2363–2377, 2024.
- [53] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386, 1958.
- [54] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [55] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- [56] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 914–921, 2020.
- [57] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015.

- [58] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019.
- [59] Artur Trindade. ElectricityLoadDiagrams20112014. UCI Machine Learning Repository, 2015. DOI: <https://doi.org/10.24432/C58C86>.
- [60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [61] Yuxuan Wang, Haixu Wu, Jiayang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. Deep time series models: A comprehensive survey and benchmark. *arXiv preprint arXiv:2407.13278*, 2024.
- [62] Yuxuan Wang, Haixu Wu, Jiayang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. Timexer: Empowering transformers for time series forecasting with exogenous variables. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 469–498, 2024.
- [63] Qingsong Wen, Jingkun Gao, Xiaomin Song, Liang Sun, Huan Xu, and Shenghuo Zhu. Robuststl: A robust seasonal-trend decomposition algorithm for long time series. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 5409–5416, 2019.
- [64] Peter R Winters. Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3):324–342, 1960.
- [65] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 22419–22430, 2021.
- [66] Liang Xiong, Xi Chen, Tzu-Kuo Huang, Jeff G. Schneider, and Jaime G. Carbonell. Temporal collaborative filtering with bayesian probabilistic tensor factorization. In *Proceedings of the SIAM International Conference on Data Mining (SDM)*, pages 211–222, 2010.

- [67] Wojciech Zaremba. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.
- [68] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 818–833, 2014.
- [69] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 11106–11115, 2021.

Preprint
Decision
Intelligence
Lab

3 时间序列预测

3.1-3.2节作者为：李哲、郭晨娟、杨彬；

3.3节作者为：李哲、郑兴泽、雷智、王益杭、陈鹏、汪思远、郭晨娟、杨彬；

3.4节作者为：成涵吟、邱翔飞、吴行健、郭晨娟、杨彬；

3.5节作者为：高洪帆、郭晨娟、杨彬。

本章主编为郭晨娟。

本章主要介绍时间序列预测任务的相关知识。对于时间序列预测的具体应用场景而言,预测需求是多样的,时间序列预测主要被分为两类:**确定性预测 (Deterministic Forecasting)** 和**概率预测 (Probabilistic Forecasting)**。确定性预测以输出单一确定性值为目标,适用于需要明确行动方案的场景(如自动驾驶车辆需要预测前方车辆下一时刻的确切位置,以便控制系统能据此计算出精确的油门、刹车或转向指令);概率预测则提供未来值的概率分布或置信区间,适用于需要量化不确定性的场景(如医疗诊断中,预测一位患者未来 24 小时发生内出血的概率,以便医生制定诊疗方案)。

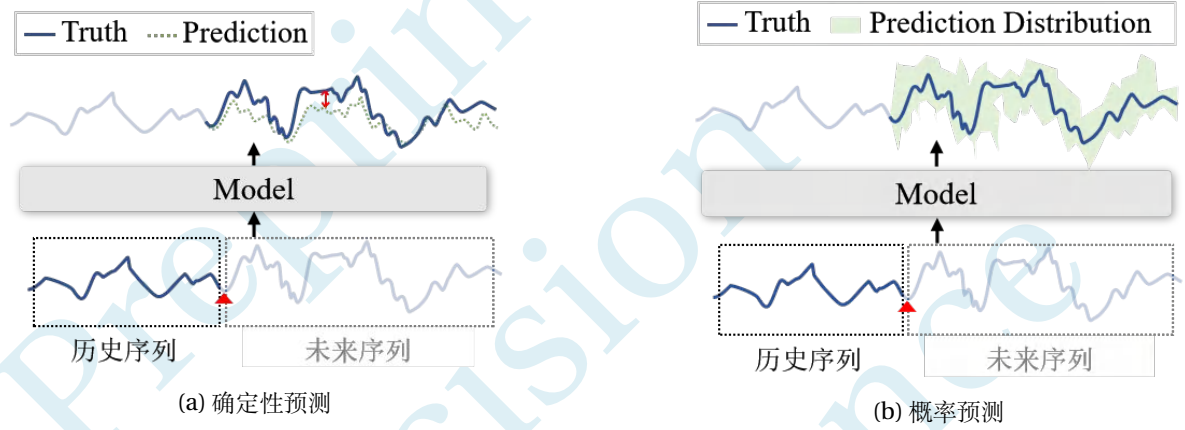


图 3.1: 时间序列预测的两种类型。

后续章节将重点关注端到端时间序列预测模型,具体内容安排如下: 3.1 章节给出了时间序列预测任务的背景、分类与工作流程。3.2 章节介绍时间序列预测模型的训练流程和评估策略。3.3 章节介绍了基于 MLP、基于 RNN、基于 CNN 以及基于 Transformer 的时间序列预测任务模型的通用性和特殊性。3.4 章节则是从通道关系的角度分别介绍了多变量时间序列数据的处理策略。针对于概率预测的特殊性,我们在 3.5 章节着重介绍了时间序列概率预测模型构建过程中的特殊性。

3.1 时间序列预测定义及流程

时间序列预测 (Time Series Forecasting) 是通过已有历史数据预测未来数值的任务,广泛应用于金融市场、天气预报、库存管理及城市交通等领域。时间序列预测任务是指根据历史 H 步序列 $\mathbf{X} = (\mathbf{x}_{t-H+1}, \mathbf{x}_{t-H+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{H \times N}$ 来预测未来 F 序列 $\hat{\mathbf{X}} = (\hat{\mathbf{x}}_{t+1}, \hat{\mathbf{x}}_{t+2}, \dots, \hat{\mathbf{x}}_{t+F}) \in \mathbb{R}^{F \times N}$, 可以用公式表示为:

$$\hat{\mathbf{X}} = f_{\theta}(\mathbf{X}) \quad (67)$$

其中, N 是变量数量, $f(\cdot)$ 是某种映射函数, θ 是可学习参数。

时间序列预测任务被划分为不同的预测类型:

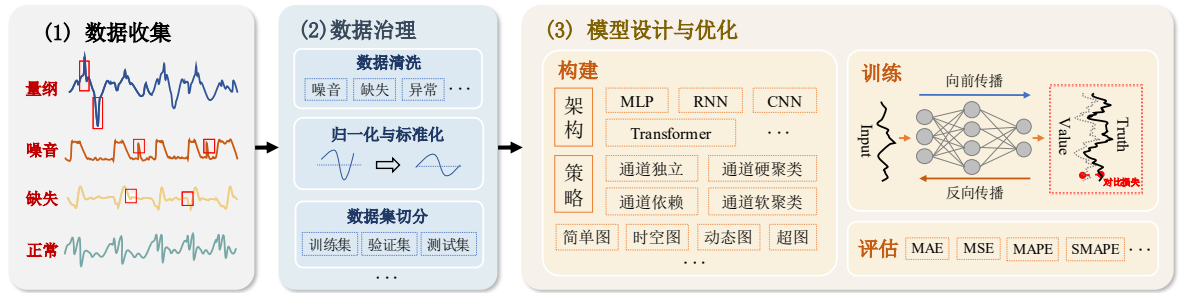


图 3.2: 时间序列预测任务流程。

- 根据输入变量个数分为**单变量时间序列预测** ($N = 1$) 和**多变量时间序列预测** ($N > 1$)。
 - 单变量时间序列预测 (Univariate Time Series Forecasting)**: 仅利用单一特征的历史数据进行未来预测, 专注于捕捉其自身的时间依赖关系 (如趋势、季节性)。例如, 仅用股票自身的历史价格预测其未来价格。
 - 多变量时间序列预测 (Multivariate Time Series Forecasting)**: 利用多个相关特征的历史数据进行未来预测, 不仅需要捕捉每个序列自身的模式, 还需要建模多个序列间的相互影响和依赖关系。例如, 同时利用温度、湿度、风速的历史数据来预测未来的降水量。
- 根据所需预测的未来时间序列长度分为**单步时间序列预测** ($F = 1$) 和**多步时间序列预测** ($F > 1$)。
 - 单步时间序列预测 (Single-step Time Series Forecasting)**: 模型仅预测未来相邻单一时间点。例如, 用过去 10 天的数据预测第 11 天的销售额。
 - 多步时间序列预测 (Multi-step Time Series Forecasting)**: 模型需要一次性预测未来多个时间点。例如, 用过去 10 天的数据预测接下来第 11 天到第 20 天的销售额。其核心挑战在于如何缓解误差累积问题。
- 根据根据预测范围分为**短期时间序列预测**和**长期时间序列预测**。
 - 短期时间序列预测 (Short-term Time Series Forecasting)**: 预测范围小于或相近于数据的一个主要季节性周期 (如一天、一周)。其核心目标是高精度地捕捉系统在近期未来的详细状态和波动。
 - 长期时间序列预测 (Long-term Time Series Forecasting)**: 预测范围显著超过数据的一个主要季节性周期。其核心目标是把握系统在遥远未来的总体趋势和宏观轮廓。核心目标在于解决误差累积、分布漂移和建模长期依赖关系。

图 3.2 展示了时间序列预测任务训练过程，通常包括以下几个步骤：1) 数据收集，从传感器或其他数据源获取的时间序列数据；2) 数据治理，旨在提高原始数据质量，包括数据清洗、缺失值处理、归一化和标准化等操作；3) 模型设计与优化，选择合适的模型架构（如 MLP、RNN、CNN、Transformer 等），通过前向传播和反向传播优化模型参数，学习训练数据集的变化规律，并在测试集上评估模型性能。

3.2 时间序列预测模型训练与评估

时间序列预测模型的目标是从给定的历史时间序列中学习潜在规律，并通过该规律对未来数据进行预测。在此过程中，模型通过学习输入序列的各种特征（如时间信息、序列位置信息、上下文信息等）来捕捉时间序列的动态变化，实现更精准的预测。

具体来说，根据一定的比例进行数据集划分，得到训练集 $\mathbf{D}_{train} = \{\mathbf{x}_i^{train}\}_{i=1}^{T_{train}}$ 、验证集 $\mathbf{D}_{val} = \{\mathbf{x}_i^{val}\}_{i=1}^{T_{val}}$ 和测试集 $\mathbf{D}_{test} = \{\mathbf{x}_i^{test}\}_{i=1}^{T_{test}}$ 。以训练集为例， $\mathbf{x}_i^{train} = (\mathbf{x}_{t-H+1}^{train}, \mathbf{x}_{t-H+2}^{train}, \dots, \mathbf{x}_t^{train}, \mathbf{x}_{t+1}^{train}, \dots, \mathbf{x}_{t+F}^{train}) \in \mathbb{R}^{(H+F) \times N}$ 为训练集中第 i 个样本在 t 时刻的输入输出序列， H 、 F 分别是历史序列长度和未来序列长度， T_{train} 是整个训练集的样本数量。训练和评估过程如下：

1. **训练阶段**：训练阶段的目标是最小化模型 f_θ 在训练集 $\mathbf{D}_{train}$ 上的预测误差。具体来说，首先是**特征提取**，模型通过对历史序列 $\{\mathbf{x}_i^{train}\}_{i=t-H+1}^t \in \mathbb{R}^{H \times N}$ 进行分析，从中提取关键特征信息，如序列的趋势性与季节性变化规律、多变量序列间的相互依赖关系、多尺度的复杂时间模式等。然后，模型基于捕捉到的数据特征进行**序列预测**，输出预测值 $\{\hat{\mathbf{x}}_i^{train}\}_{i=t+1}^{t+F} \in \mathbb{R}^{F \times N}$ 。为了提高模型预测效果，通常选择以模型预测值与真实值之间的误差来指导**模型优化**方向，误差越小，表明模型学习到的序列信息越多。常见的**损失函数**有：

- (a) **均方误差 (Mean Square Error, MSE)**：用于衡量模型整体预测的准确性，以最小化预测值与真实值之间的平方差为优化目标：

$$\mathcal{L}_{train} = \frac{1}{F} \sum_{i=t+1}^{t+F} (\mathbf{x}_i^{train} - \hat{\mathbf{x}}_i^{train})^2 \quad (68)$$

- (b) **平均绝对误差 (Mean Absolute Error, MAE)**：通过计算每个预测值和真实值之间的绝对差异，能够量化预测误差的大小，对异常值的敏感度较低，以最小化预测值与真实值之间的绝对差为优化目标：

$$\mathcal{L}_{train} = \frac{1}{F} \sum_{i=t+1}^{t+F} |\mathbf{x}_i^{train} - \hat{\mathbf{x}}_i^{train}| \quad (69)$$

需要注意的是，在模型训练过程中，出于避免模型 f_θ 在 $\mathbf{D}_{train}$ 过拟合或者筛选

最优超参数等目的，通常使用验证集 $\mathbf{D}_{val}$ 判断模型 f_{θ} 性能，获得**最优模型** f_{θ^*} ：

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{val}(\theta) \quad (70)$$

2. **评估阶段**：在模型评估阶段，使用测试集 $\mathbf{D}_{test}$ 检测模型性能。即使用训练阶段得到的最优模型 f_{θ^*} 建模历史序列 $\{\mathbf{x}_i^{test}\}_{i=t-H+1}^t \in \mathbb{R}^{H \times N}$ ，并输出预测值 $\{\hat{\mathbf{x}}_i^{test}\}_{i=t+1}^{t+F} \in \mathbb{R}^{F \times N}$ 。通过计算预测值与真实值之间的误差来量化模型的预测性能。**评估指标**需要结合具体任务需求选择，如异常值的敏感度等，以下是常见评估：

- (a) **平均绝对误差 (MAE)** 反映绝对误差的平均值，对异常值鲁棒：

$$\text{MAE} = \frac{1}{F} \sum_{i=t+1}^{t+F} |\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test}| \quad (71)$$

- (b) **均方误差 (MSE)** 衡量预测值与真实值的平方误差均值，对大误差敏感：

$$\text{MSE} = \frac{1}{F} \sum_{i=t+1}^{t+F} (\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test})^2 \quad (72)$$

- (c) **平均绝对百分比误差 (Mean Absolute Percentage Error, MAPE)** 以百分比形式表示误差，可以直观表示模型性能，但真实值为零时失效：

$$\text{MAPE} = \frac{1}{F} \sum_{i=t+1}^{t+F} \left| \frac{\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test}}{\mathbf{x}_i^{test}} \right| \times 100\% \quad (73)$$

- (d) **均方根误差 (Root Mean Square error, RMSE)** 为 MSE 的平方根：

$$\text{RMSE} = \sqrt{\frac{1}{F} \sum_{i=t+1}^{t+F} (\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test})^2} \quad (74)$$

3.3 时间序列预测模型

对于端到端时间序列预测模型的设计和训练，通过深入分析时间序列数据的特点，不断优化模型参数以最大化模型对某数据集的预测性能。这种模型的优势在于它能充分挖掘某一数据集的潜在规律，结合领域知识更好地理解 and 预测特定领域的变化趋势。接下来，本章节先简单介绍时间序列预测的统计学习和机器学习方法，再分别介绍基于 MLP、RNN、CNN 以及 Transformer 的时间序列预测任务的基本预测流程和模型关键技术。

3.3.1 统计学习方法

统计学习方法基于时间序列的历史观测值构建数学模型，通过对序列自身的统计特性（如自相关性、趋势性和季节性）进行建模来预测未来值。这类方法的核心思想

是假设时间序列的未来值可以由其过去观测值的线性组合或统计关系推导得出。经典的统计学习方法包括 ARIMA [3]、ETS [22]、Theta [18]、VAR [19] 等。这些方法在时间序列预测的早期研究中占据主导地位，具有理论基础坚实、模型可解释性强、参数意义明确等优势。然而，统计学习方法通常假设时间序列满足平稳性、线性关系等条件，在处理具有复杂非线性模式、多变量相互作用或突变趋势的时间序列时表现受限。

3.3.2 机器学习方法

随着机器学习技术的快速发展，基于机器学习的时间序列预测方法应运而生。这类方法不依赖于严格的数据分布假设，而是从数据中自动学习复杂的非线性模式和特征关系。具有代表性的机器学习方法包括 XGBoost [9, 78]、梯度提升回归树 (GBRT) [16]、随机森林 [4, 41] 以及 LightGBM [25] 等。这些方法能够捕捉时间序列中难以用简单统计模型描述的时序依赖模式和动态特征，在处理不同类型和长度的时间序列方面表现出较强的灵活性，通常能够提供比传统统计方法更高的预测精度。然而，机器学习方法的性能在很大程度上依赖于特征工程的质量，且需要谨慎调整超参数以避免过拟合问题。

3.3.3 多层感知机

多层感知机 (Multi-Layer Perceptron, MLP) [49] 具有强大的特征学习能力，通过多层非线性变换，将输入空间映射到输出空间，从而学习数据的复杂特征表示。因其结构简单且易于实现，在时间序列预测中得到广泛应用，具体结构如图 3.3 所示。尽管 MLP 本身没有捕捉序列信息的能力，但在加入特定数据处理技术后，它在时间序列预测任务上展现出了较好的性能。

在时间序列预测中，MLP 的优势可以总结如下——计算效率高：全连接结构相较于其他复杂模型更为简单，规避了复杂模型为捕获时序依赖引入的动态计算开销（比如 RNN 的循环单元、Transformer 的自注意力机制），适用于需要实时预测或高频数据更新的场景；易于解释：输入和输出之间的映射关系比较透明，有助于分析输入变量对预测结果的贡献和变量之间的关系；良好的扩展性：易于扩展和优化，比如通过嵌入其他特征提取模块来改善性能，适应不同类型的数据。

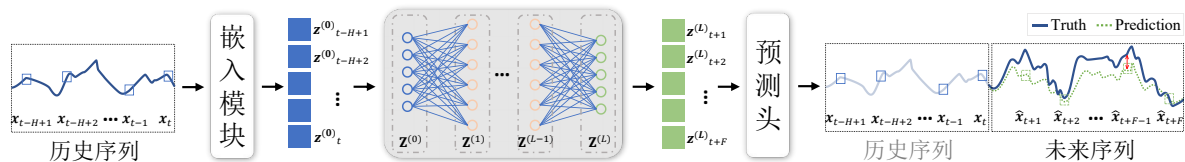


图 3.3: 基于多层感知机的时间序列模型架构。

基本预测流程 骨干网络为 MLP 的时间序列预测模型将时间序列的点粒度级作为输入。首先将时间序列每个时间点数据的值嵌入到高维表示 $\mathbf{Z}^{(0)} = (\mathbf{z}_1^{(0)}, \mathbf{z}_2^{(0)}, \dots, \mathbf{z}_H^{(0)}) \in \mathbb{R}^{H \times D}$ 。之后，嵌入表示 $\mathbf{Z}^{(0)}$ 输入 MLP 的第一层，每个神经元连接到嵌入向量中的特征点。那么，对于第 l 层，将第 $l-1$ 层的输出作为该层输入：

$$\mathbf{Z}^{(l)} = g^{(l)}(\mathbf{w}^{(l)}\mathbf{Z}^{(l-1)} + \mathbf{b}^{(l)}), l = 1, \dots, L \quad (75)$$

其中， $\mathbf{Z}^{(l-1)} \in \mathbb{R}^{H \times D}$ 是第 $l-1$ 层时间序列的特征表示， D 是其维度， $\mathbf{w}^{(l)} \in \mathbb{R}^{F \times H}$ 是权重矩阵， $\mathbf{b}^{(l)} \in \mathbb{R}^{F \times 1}$ 是偏置项， $g^{(l)}(\cdot)$ 是非线性激活函数。在经过 L 个隐藏层后得到 $\mathbf{Z}^{(L)} = (\mathbf{z}_1^{(L)}, \mathbf{z}_2^{(L)}, \dots, \mathbf{z}_F^{(L)}) \in \mathbb{R}^{F \times D}$ ，最终通过预测头输出时间序列的预测值：

$$\hat{\mathbf{X}} = \text{PredictionHead}(\mathbf{Z}^{(L)}) \quad (76)$$

基于 MLP 的时间序列预测模型关键技术 近年来，产生了许多基于 MLP 的时间序列模型。作为主干网络，MLP 一般不修改网络架构，只考虑层数的设计。在时间序列预测任务中，其关键技术如表 3.1 所示，这些技术在时间序列预测中发挥了重要作用。

表 3.1: 基于 MLP 的时间序列预测模型关键技术总结。

模型	年份	权重共享	通道处理策略	归一化	序列分解	特征提取设计	参数量
TSMixer [15]	2023	是	通道依赖	可逆实例归一化	无	时间和变量维度 信息交错混合	498.7K
RLinear [31]	2023	是	通道独立	可逆实例归一化	无	无	70.5K
NLinear [77]	2023	是	通道独立	NowNorm [57]	无	无	69.8K
DLinear [77]	2023	是	通道独立	无	趋势项 + 季节项	无	139.7K
TimeMixer [61]	2024	是	通道独立	可逆实例归一化	趋势项 + 季节项	多尺度融合	221.7K
FITS [71]	2024	是	通道独立	实例归一化	无	频域线性变换	4.5K~10K
SparseTSF [33]	2024	是	通道独立	MeanNorm	无	跨周期稀疏预测	0.15K

- 1. 归一化 (Normalisation)** 的目的是将不同量纲的数据转换到相同的尺度上，即相同的数值范围。FITS [71] 采用了实例归一化，序列减去其均值并除以其标准差。而大多数基于 MLP 的时间序列预测模型采用了可逆实例归一化，在实例归一化的基础上，引入具有参数的仿射变换。此外，NLinear [77] 为了解决数据中的分布漂移，先用序列的最后一个值减去输入，经过一个线性层后再加回减去的部分，这在 [57] 中被称为 NowNorm。SparseTSF [33] 在序列输入模型之前减去序列本身的均值，输出模型之后再加回减去的部分，我们将这一过程称为 MeanNorm。
- 2. 通道处理策略 (Channel Processing Strategy)** 应用于多变量时间序列，包括通道独立和通道依赖。通道独立策略将多变量时间序列简单地当作多个单变量时间序列，而不显示地考虑其之间的相关性，应用统一的预测模型进行处理，而通道依赖策略通过引入专门的模块来捕获通道间的相关性。大多数基于 MLP 的时间

序列预测模型都采用了通道独立策略，且 RLinear [31] 证明了通道独立策略有利于提升模型预测性能，而 TSMixer [15] 采用了通道依赖策略，弥补了通道独立策略未对多变量之间的依赖关系提取的不足。

3. **权重共享 (Weight Sharing)** 是在多变量之间使用相同的权重参数。可以用公式表示为：

$$\hat{\mathbf{X}}_i = \mathbf{w}\mathbf{X}_i + \mathbf{b} \quad (77)$$

其中， $\mathbf{X}_i$ 是第 i 个变量的历史时间序列， $\hat{\mathbf{X}}_i$ 是第 i 个变量的未来时间序列。 $\mathbf{w}$ 和 $\mathbf{b}$ 与 i 无关，在变量之间共享。

4. **序列分解 (Series Decomposition)** 将一个复杂的序列拆解为多个成分，详细介绍在章节 2.2.4。DLinear [77] 将原始序列分解为趋势项和季节项，分别作为单层线性层的输入。TimeMixer [61] 通过不同频率的下采样得到不同尺度的序列，并将其分解为趋势项和季节项。
5. **特征提取设计 (Feature Extraction Design)** 包括多种从原始数据中提取特征性信息的方法。**频域线性变换**是将输入时间序列通过傅里叶变换得到频率特征（详细介绍可参考章节 2.3.2），然后通过 MLP 线性变换得到新的频率特征 [71]。**跨周期稀疏预测**是在时间轴上进行跨周期的稀疏滑动预测，根据周期将原始序列下采样为等长历史子序列，分别对其进行预测 [33]。**多尺度融合**是一种创新的信息交互方式，对原始序列进行不同频率的下采样，得到不同尺度的序列，使用两种相反的信息流动方式对其进行处理 [61]。**时间和变量维度信息交错混合**是在时间和通道维度上使用 MLP，提取时间步之间的依赖关系和通道之间的依赖关系 [15]。
6. **参数量 (Number of Parameters)** 是模型中可训练参数的总数，用来衡量模型的大小。MLP 通常具有较小的参数量，这是其优势所在。基于公平实验设置 [71]，DLinear [77] 的参数量为 139.7K。FITS [71] 的参数量是 DLinear 的 3.22%~7.16%，通过选择适当的截止频率对原始序列进行截断，将包含噪声和冗余信息的高频分量过滤，压缩了模型大小，同时保留了主要信息。SparseTSF [33] 的参数量不到 1K，应用一个单层线性模型进行稀疏预测，有效减少了模型参数。与之相比，基于 Transformer 的模型的参数量可达数百万，例如 Pyraformer [36] 的参数量为 241.4M，Informer [81]、Autoformer [68] 和 FEDformer [82] 的参数量在 14.4M 至 20.7M 之间。

Toner 等人 [57] 对线性时间序列预测模型进行了分析。在时间序列预测任务中，线性模型达到了最先进的性能水平，但线性模型的变体之间的实际性能差异并不明显。作者分析了主流的线性模型所表达的函数集，证明了其本质是相同的，功能基本

上是等价的，与标准的无/弱约束线性回归无法区分，因此都可以重新解释为在适当扩展的特征集上进行的无约束线性回归。

3.3.4 循环神经网络

循环神经网络 (Recurrent Neural Network, RNN) 具有天然的序列建模能力，通过其循环连接结构对时间步进行迭代处理，能够显式地捕捉时间动态与上下文依赖关系。因其能够处理变长序列并建模时序动态，在时间序列预测中曾被广泛使用，基本结构如图 3.4 所示。尽管传统 RNN 存在梯度消失或爆炸等问题，但通过改进结构（如 LSTM、GRU）和训练策略，它在时间序列预测任务中仍表现出良好的性能。

在时间序列预测中，RNN 的优势可总结如下——时序建模能力强：循环连接结构使其能够记忆历史信息，实现对序列中长期和短期依赖关系的显式建模，适用于具有明显时间动态的数据；输入灵活性高：可处理不同长度的时间序列输入，无需固定时间窗口，对非均匀采样或变长序列具有良好的适应性；特征与状态融合自然：允许将外部特征与隐藏状态动态结合，在每个时间步融入上下文信息，增强模型对复杂时序模式的表达能力。

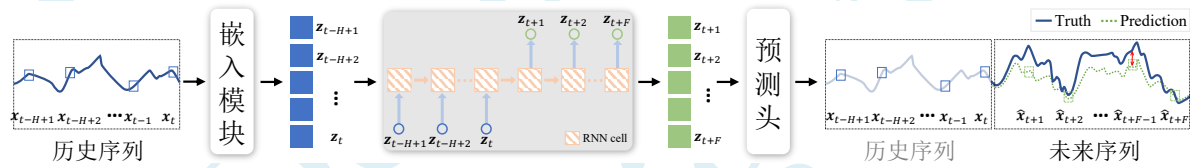


图 3.4: 基于循环神经网络的时间序列模型架构。

基本预测流程 骨干网络为 RNN 的时间序列预测模型以自回归 (Autoregressive) 的方式进行预测。首先，将时间序列每个时间点数据的值嵌入到高维表示 $\mathbf{Z}^{(0)} = (z_1^{(0)}, z_2^{(0)}, \dots, z_H^{(0)}) \in \mathbb{R}^{H \times D}$ 。之后，嵌入表示 $\mathbf{Z}^{(0)}$ 输入 RNN 层，按时间步依次处理并传递隐藏状态。那么，对于第 l 层，将第 $l-1$ 层的输出作为该层输入：

$$\mathbf{h}_t^{(l)} = g^{(l)} \left(\mathbf{w}^{(l)} \mathbf{z}_t^{(l-1)} + \mathbf{u}^{(l)} \mathbf{h}_{t-1}^{(l)} + \mathbf{b}^{(l)} \right), \quad t = 1, \dots, H \quad (78)$$

其中， $\mathbf{z}^{(l-1)} \in \mathbb{R}^D$ 是第 $l-1$ 层在 t 时刻的特征表示， $\mathbf{h}_{t-1}^{(l-1)} \in \mathbb{R}^D$ 为上一时间步的隐藏状态， $\mathbf{w}^{(l)}$ 和 $\mathbf{u}^{(l)}$ 分别为输入和隐藏状态的权重矩阵， $\mathbf{b}^{(l)} \in \mathbb{R}^{F \times 1}$ 是偏置项， $g^{(l)}(\cdot)$ 是非线性激活函数。经多轮迭代和多个隐藏层处理后，获取最终隐藏状态表示 $\mathbf{h}_{\text{H}}^{(l-1)} \in \mathbb{R}^D$ 。在预测阶段，该状态被传递至解码器。解码器以自回归方式工作，将上一步的预测输出作为下一步的输入，递归地生成多步预测结果：

$$\hat{x}_{H+f} = \text{PredictionHead}(\mathbf{h}_{H+f}^{(L)}), \quad f = 1, 2, \dots, F \quad (79)$$

其中， $\mathbf{h}_{H+f}^{(L)}$ 由 RNN 根据上一步的隐藏状态和上一步的预测值（或对应的嵌入）计算得到。

基于 RNN 的时间序列预测模型关键技术 近年来，产生了许多基于 RNN 的时间序列模型。在时间序列预测任务中，其关键技术如表 3.2 所示，这些技术在时间序列预测中发挥了重要作用。

表 3.2: 基于 RNN 的时间序列预测模型总结。

模型	年份	建模特性	嵌入方式	生成方式	支持概率预测
DeepAR [50]	2017	-	点嵌入	迭代生成	是
DA-RNN [44]	2017	-	点嵌入	迭代生成	否
LSTNet [27]	2018	长短期建模	点嵌入	迭代生成	是
C2FAR [2]	2023	多尺度建模	点嵌入	迭代生成	是
SegRNN [32]	2023	-	块嵌入	并行生成	否
PIAD-SRNN [42]	2025	周期趋势建模	点嵌入	直接生成	否
P-sLSTM [26]	2025	-	块嵌入	直接生成	否

1. **特征提取 (Feature Extraction)** 的目的是从时序数据中提取能够用来表达其语义的高维特征来增强时序预测。本节将介绍循环神经网络建模的时序特征提取设计，主要围绕建模时序特性以及嵌入方式两个方面，对代表性模型的技术特点进行阐述。

(a) **建模特性**是针对时间序列中蕴含的多种时间依赖关系与动态模式进行显式或隐式的表征建模过程。LSTNet [27] 使用卷积神经网络 (CNN) 和循环神经网络 (RNN) 来提取变量之间的短期局部依赖模式，并发现时间序列趋势的长期模式。C2FAR [2] 通过分层的方式逐步细化预测，首先在较粗的时间尺度上捕捉全局趋势，然后逐步过渡到更细的时间尺度以捕捉局部细节。PIAD-SRNN [42] 采用周期趋势分解来分别建模时序的周期和趋势特征。

(b) **嵌入方式**直接决定了模型所能捕获的时序粒度与动态特征。针对循环神经网络 (RNN) 类模型，常见的两种输入表示方式为点嵌入 (Point Embedding) 和块嵌入 (Patch Embedding)，二者在信息表达和建模能力上存在显著差异。点嵌入以逐点输入的形式将每个时间步的观测值依次送入 RNN，从而细粒度地捕捉序列的连续动态变化。然而，这种方式在建模长期依赖时容易出现信息衰减，对高频噪声敏感，且难以抽象局部模式，适应于短期的时序建模。代表模型有 DeepAR [50] LSTNet [27]，C2FAR [2] 等。块嵌入将相邻的若干时间点划分为一个时序片段，通过线性投影映射得到局部特征向量，再输入 RNN 进行高层时间建模。这种方式在输入阶段完成了局部上下文的聚

合，能够显著缓解长序列依赖问题，增强对局部趋势和周期模式的建模能力，同时提升鲁棒性，代表模型有 SegRNN [32], P-sLSTM [26]。

2. **预测设计 (Forecasting Design)** 的目的是通过不同的预测策略以满足实际应用中的多样化需求。本节将介绍基于循环神经网络建模的预测设计工作，主要围绕生成方式和概率预测能力两个方面，对代表性模型的技术特点进行阐述。

(a) **生成方式**决定了模型如何输出预测序列，直接影响预测的准确性、速度和自回归误差累积效应。迭代生成是早期 RNN 模型最经典的范式。模型逐步预测下一个时间步，并将该预测值（或真实值）作为输入反馈给模型，以进行后续预测。这种方式容易导致误差在多步预测中累积。代表模型包括 DeepAR [50]、DA-RNN [44]、LSTNet [27] 和 C2FAR [2]。并行生成旨在打破迭代生成的序列依赖性，一次性生成整个预测序列，极大提升了推理速度，尤其适合高吞吐量场景。SegRNN [32] 核心技术是通过分段循环，将长序列分割成较短的片段，并在这些片段上并行地进行 RNN 计算，最后通过重叠片段的方式进行融合，用“空间换时间”的思路，在保持 RNN 建模能力的同时实现了并行化。直接生成通过单次前向计算直接输出整个预测序列，结构最为简洁。P-sLSTM [26] 采用了块嵌入 (Patch Embedding) 的方式，将时间序列分段成块，作为模型输入。其核心是使用线性门控机制简化 LSTM 的结构，在减少计算量的同时，实现了高效且高性能的直接预测。

(b) **概率预测**能够提供预测值的不确定性估计，对风险敏感的应用（如金融、能源）尤为重要。支持概率预测的模型通过输出概率分布的参数或分位数来量化不确定性。如 DeepAR [50], LSTNet [27] 及 C2FAR [2]，它们通常假设数据服从一个参数分布（如高斯分布），模型直接输出该分布的参数（如均值和方差），从而可以从分布中采样得到预测区间。不支持概率预测的模型通常输出确定性的点预测值，适用于对确定性要求较高的场景或作为基准模型。DA-RNN [44], SegRNN [32], PIAD-SRNN [42] 及 P-sLSTM [26] 这些模型的设计初衷是提升点预测的准确性或效率。例如，DA-RNN 通过注意力机制提升精度，SegRNN 和 P-sLSTM 专注于预测速度的突破。

3.3.5 卷积神经网络

卷积神经网络 (Convolution Neural Network, CNN) [18] 具有优秀的特征提取能力，卷积的并行过程提供了更优的效率与性能平衡。Bai 等人首次将 CNN 框架迁移到时间序列任务上，称之为时间卷积网络 (Temporal Convolution Networks, TCN) [1]。根据卷积核的维度可以将后续研究分为基于一维卷积的时间序列预测模型和基于二维卷积的时间序列模型。

图 3.5展示了基于 CNN 时间序列预测模型的通用架构。简单来说，对于给定的一段历史序列 $\mathbf{X} = (\mathbf{x}_{t-H+1}, \mathbf{x}_{t-H+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{H \times N}$ （图中以 $N = 1$ 的单变量时间序列为例进行展示），首先通过嵌入模块将其映射到隐藏空间，具体包括对序列进行“切分”（Patching）、对时间标签进行编码等操作等。随后，将嵌入模块输出的向量表示传递到卷积层，卷积层通过具体的卷积操作捕捉对预测有帮助的关键特征。最后，这些关键特征被传递至预测头，作为模型的输出层，预测头将通过回归等方式生成最终的预测结果 $\hat{\mathbf{X}} = (\hat{\mathbf{x}}_{t+1}, \hat{\mathbf{x}}_{t+2}, \dots, \hat{\mathbf{x}}_{t+F}) \in \mathbb{R}^{F \times N}$ 。接下来，我们将详细介绍卷积过程。

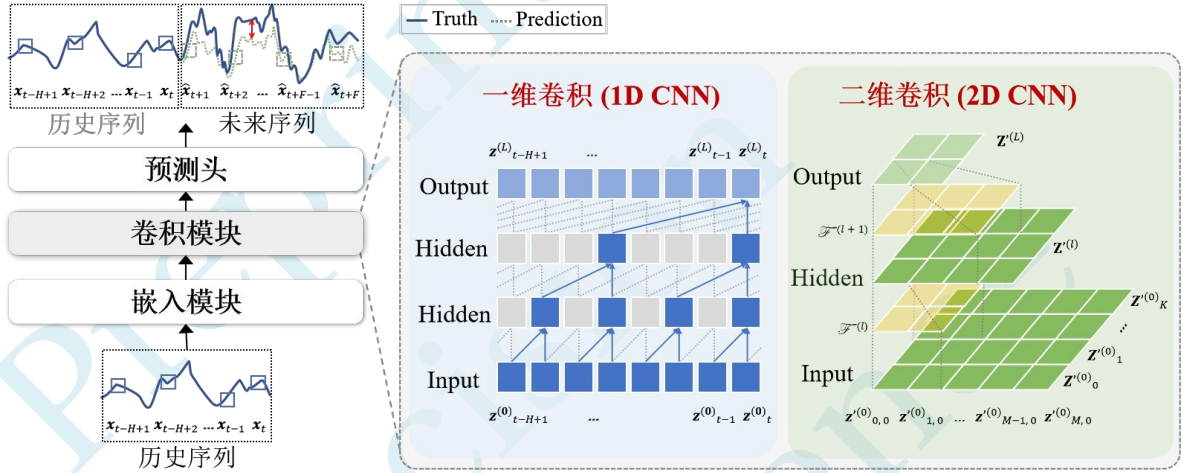


图 3.5: 基于 CNN 的时间序列模型通用架构。

基于一维卷积时间序列预测模型 对于时间序列而言，序列本身包含着时间维度的变化信息。当卷积核与时间序列进行滑动窗口计算时可以逐步捕捉序列的局部特征，如趋势和周期性等。为了捕捉时间序列的全局特征，可以通过堆积不同大小卷积核的卷积层，增大感受野，实现对时间序列不同尺度信息进行建模。

2018 年，Shaojie Bai 等人提出了 **空洞因果卷积**（Dilated Causal Convolution）来改进卷积神经网络在时间序列建模中的表现 [1]。其中，“因果性”保证了在时刻 t 的输出只依赖于过去及当前输入，而不会使用未来信息；“空洞性”则通过在卷积核元素之间引入一个膨胀因子（dilation factor），控制采样的间隔。在这种机制下，卷积能够在不增加卷积核大小和参数量的情况下快速扩大感受野（receptive field）。通过在多层网络中逐层增大膨胀因子（如 1, 2, 4, 8, ...），模型的感受野可以呈指数级增长，从而在较少层数下捕获长程的时间依赖关系。

对于输入的历史序列 $\mathbf{X}$ 通过嵌入模块将其映射到高维表示 $\mathbf{Z}^{(0)} = (\mathbf{z}_1^{(0)}, \mathbf{z}_2^{(0)}, \dots, \mathbf{z}_H^{(0)}) \in \mathbb{R}^{H \times N \times D}$ ，作为卷积模块的第一层输入。那么，给定第 l 层的卷积核 $\mathbf{F}^{(l)} = (\mathbf{f}_1^{(l)}, \mathbf{f}_2^{(l)}, \dots, \mathbf{f}_K^{(l)}) \in \mathbb{R}^{K \times D}$ 和第 $l-1$ 层的输出 $\mathcal{Z}^{(l-1)} = (\mathbf{z}_1^{(l-1)}, \mathbf{z}_2^{(l-1)}, \dots, \mathbf{z}_H^{(l-1)})$ ，卷积过程可以用公式

表示：

$$\mathbf{Z}^{(l)} = (\mathcal{F}^{(l)} * \mathbf{Z}^{(l-1)}) = \sum_{k=1}^K \mathbf{f}_k^{(l)} \mathbf{z}_{t-(K-k)d,i}^{(l-1)} \quad (80)$$

其中， K 是卷积核大小， $i = \{1, 2, \dots, N\}$ 表示序列的第 i 个通道， D 是隐藏层维度， d 是膨胀因子。

值得注意的，一维卷积除了可以沿时间维度进行特征提取，同时还可以沿通道维度提取跨模态信息。给定第 l 层的卷积核 $\mathcal{F}'^{(l)} = (\mathbf{f}_1'^{(l)}, \mathbf{f}_2'^{(l)}, \dots, \mathbf{f}_M'^{(l)}) \in \mathbb{R}^{M \times D}$ ，其卷积核大小为 M ，卷积过程可以定义为：

$$\mathbf{Z}^{(l)} = (\mathcal{F}'^{(l)} * \mathbf{Z}^{(l-1)}) = \sum_{m=1}^M \mathbf{f}_m'^{(l)} \mathbf{z}_{t,i-(M-m)d}^{(l-1)} \quad (81)$$

基于二维卷积时间序列预测模型 时间序列往往同时包含多种频率和周期成分，这些成分之间可能相互重叠并产生复杂的交互作用。为了有效建模这些特性，Haixu Wu 等人于 2023 年提出了 TimesNet [67]。该方法利用快速傅里叶变换（Fast Fourier Transformation, FFT）将一维时间序列分解为二维张量表示，从而能够在二维域中捕捉不同频率成分的局部和全局模式。

具体来讲，输入的历史序列的嵌入 $\mathbf{Z} \in \mathbb{R}^{H \times D}$ （简单起见以 $N = 1$ 的单变量时间序列为例描述二维卷积过程）通过 FFT 被分解为：

$$\mathbf{A}, \{f_1, f_2, \dots, f_k\}, \{p_1, p_2, \dots, p_k\} = \text{FFT}(\mathbf{Z}) \quad (82)$$

其中， k 为预定义的超参数表示有效频率数量， f_i 和 p_i 分布表示被筛选出来的频率和对应周期， $i = 1, 2, \dots, k$ 。随后， $\mathbf{Z}$ 将被重构为二维表征 $\mathbf{Z}'^{(0)} = (\mathbf{z}_1'^{(0)}, \mathbf{z}_2'^{(0)}, \dots, \mathbf{z}_k'^{(0)})$ ，其中 $\mathbf{z}_i'^{(0)} \in \mathbb{R}^{f_i \times p_i \times D}$ 。那么，给定第 l 层的第 i 个二维卷积核 $\mathcal{F}_i'^{(l)} \in \mathbb{R}^{K \times M \times D}$ 和第 $l-1$ 层输出的第 i 个重构的二维表征 $\mathbf{z}_i'^{(l-1)}$ ，卷积过程如下：

$$\mathbf{z}_i'^{(l)} = (\mathcal{F}_i'^{(l)} * \mathbf{z}_i'^{(l-1)}) = \sum_{p=1}^K \sum_{q=1}^M \mathcal{F}_{i,(p,q)}'^{(l)} \mathbf{z}_{i,(p,q)}'^{(l-1)} \quad (83)$$

其中 K, M 表示卷积核大小。在将 $\mathbf{z}_i'^{(l)}$ 重新恢复到一维 $\mathbf{z}_i^{(l)} \in \mathbb{R}^{T \times D}$ 后，依据 $\mathbf{A}^l$ 聚合所有表征的信息：

$$\mathbf{Z}^{(l)} = \sum_{i=1}^K \mathbf{A}_i \mathbf{z}_i^{(l)} \quad (84)$$

除了基于 FFT 分解得到的时频矩阵外，二维卷积的思想还可以自然地扩展到多变量时间序列的建模中。例如，对于 $\mathbf{Z}^{(0)} \in \mathbb{R}^{H \times N \times D}$ ，可以直接将时间维度 H 与通道维度 N 视作二维平面，从而同时捕获跨时间和跨变量的依赖关系。这为时间序列预测提供了更为灵活的建模方式。

表 3.3: 基于 CNN 的时间序列预测模型总结。

类别	模型	年份	数据域	局部特征	全局特征	特征融合
一维 卷积	DeepGLO [51]	2019	时域	空洞卷积	卷积层堆叠 + 基序列组合	-
	SCINet [34]	2022	时域	下采样 + 因果卷积	卷积层堆叠	拼接
	MICN [59]	2023	时域	下采样 + 因果卷积	大尺度卷积核	拼接
	ModernTCN [40]	2024	时域	因果卷积	大尺度卷积核	拼接
	LightCTS [28]	2023	时空域	空洞卷积	卷积层堆叠 + 最后时刻压缩	通道洗牌
二维 卷积	TimesNet [67]	2023	频域	二维卷积	拼接	加权聚合
	TIMEMIXER++ [60]	2024	频域	下采样 + 因果卷积	二维卷积 + 注意力机制	残差卷积 + 加权聚合

基于 CNN 的时间序列预测模型关键技术 近年来，涌现了许多基于 CNN 的时间序列预测模型，其关键技术如表 3.3 所示：

1. **数据域 (Data Domain)** 主要涉及数据的范围或类型，常用于描述数据的来源、内容、数据值的限制等。时间序列的表示方式主要有时域和频域两类，本书 2.3 章给出了这两种数据表示的详细解释。在**时域**中，数据点按照时间顺序排列，直接对时域数据进行分析可以直接观察时间序列的原始信息。沿时间维度对时域序列进行滑动窗口卷积计算是基于 CNN 的时间序列预测模型处理时域数据的较为常见的方式 [51, 34, 59, 40]。而在**频域**中，数据被表示为不同频率成分的组合，对频域数据的分析有助于识别时间序列中的周期性和频率特征。可以通过傅里叶变换 (Fast Fourier Transform, FFT) 等方法将时间序列分解为不同频率序列，再利用二维卷积提取这些频率序列中的特征 [67, 60]。

需要注意的是，在实际应用场景中，采集时间序列的传感器可能分布于不同的空间位置，而这些传感器的空间特征能够为时间序列的变化提供额外的空间维度信息，我们称这些数据来源于**时空域**。像 LightCTS [28] 等研究充分考虑时间序列的时空特征，同时对时间序列的时间特征和空间特征进行建模。相比于传统的将时间建模模块和空间建模模块交叉堆叠的方式，LightCTS 提出了一种新的设计思路：首先利用空洞卷积对序列的时间特征进行建模，然后通过注意力机制捕捉序列的空间特征。考虑到序列的时间维度通常远大于空间维度，这种设计有效避免了交叉堆叠网络结构中需要在空间建模模块中保留高维特征的问题，从而显著降低了模型的计算复杂度，并优化了在边缘设备上的部署效率。

2. **特征提取 (Feature Extraction)** 是从原始数据中提取有用信息的过程，旨在将高维、复杂的数据转化为更加简洁且富有表现力的形式，以便为后续的分析、建模或决策提供支持，包括**局部特征提取**和**全局特征提取**两种。

(a) **局部特征 (Local Features)** 是时间序列中短期或小范围内的模式，这些模式通常具有较高的时间依赖性。通过滑动窗口的方式，因果卷积和空洞卷

积可以很轻松的建模时间序列局部特征。在卷积基础上进行**下采样**操作，可以在保留重要的特征信息的同时减少数据的维度和大小，降低计算复杂度。MICN [59] 通过常规的“池化”操作实现下采样过程。而 SCINet [34] 认为根据时间索引将序列下采样为奇索引时间序列和偶索引时间序列，并不影响时间序列中的重要模式或趋势，通过迭代执行“下采样-卷积-子序列交互”过程，获得了不同尺度序列特征，扩充了数据的多样性。TIMEMIXER++ [60] 则效仿计算机视觉任务中的 Inception [56] 使用具有不同大小的卷积核对二维时间序列进行卷积操作。

- (b) **全局特征 (Global Features)** 描述时间序列的整体趋势、周期性和长期规律。基于 CNN 的时间序列预测模型建模序列全局特征的常规方式是**卷积层堆叠**。在低层卷积层中，网络关注短时间窗口内的局部变化；在高层卷积层中，网络将局部特征结合起来，形成更长时间跨度的模式 [34, 40]。卷积层的堆叠以一种简单而有效的方式捕捉序列的全局特征。然而，如果仅将最高层次的特征传递到后续网络，可能会丢失序列中的局部细节信息；而若将所有卷积层的输出都传播下去（如 TimesNet [67] 采用拼接操作聚合不同层次输出），则会导致计算复杂度的显著增加。

针对这一问题，基于一个固有的事实——离当前时刻较远的时间特征对当前时刻的预测贡献较小，可以被忽略——产生了**最后时刻压缩法** [28]。具体来讲，为了有效提取与当前时刻相关的全局特征，通过加和堆叠的卷积层中最靠近当前时刻的特征，并将这些特征作为全局特征用于后续模型组件，在有效减少计算复杂度的同时保持对重要时间特征的关注。

协变量通常指与主时间序列数据相关的额外变量或特征，这些特征可以帮助模型更好地进行预测，因此许多研究尝试在进行多层卷积操作之前提取序列的协变量作为额外全局特征。一种代表性的技术是**基序列组合**，使用矩阵分解技术从时间序列中提取具有显著时间特征模式的基序列，并通过基序列的线性组合来表征原始序列，将其作为全局协变量增强模型的预测能力，如 DeepGLO [51]。

由于不断堆叠卷积层会造成模型的复杂度的增加，一些研究认为**大尺度卷积核**是扩大感受野、捕捉全局特征的更有效方法 [40]。其中，MICN [59] 引入等距卷积的概念，通过在长度为 H 序列前拼接 $H-1$ 个占位符 0，使用同序列等长的卷积核执行卷积操作，捕获了序列中的全局特征。

随着深度学习技术的发展，许多研究采用了更为复杂的网络架构来增强全局特征建模能力，例如在卷积层之间插入**注意力机制**。TIMEMIXER++ [60] 利用双轴注意力机制提取时间序列中的长期特征，尤其是季节性变化和趋势特征。具体而言，模型首先通过二维卷积操作捕捉序列中的局部特征，再对序

列的不同维度（即行和列）进行注意力加权处理。其中，行轴注意力关注频率维度的变化，识别长期趋势，捕捉序列的趋势性特征；列轴注意力则侧重于关注不同时间段的变化，捕捉时间序列中的季节特性。

3. **特征融合 (Feature Fusion)** 是将不同来源、不同类型或不同视角的特征进行合并，以提高模型的性能或增强数据的表达能力。

拼接 (Concatenation) 是最简单的特征融合方式，即将多个特征向量或特征矩阵沿某个维度进行连接，形成一个更大的特征向量或矩阵 [40, 34, 59]。拼接操作本身不会显式地考虑特征之间的交互作用，会忽略不同特征之间的复杂关系。

通道洗牌 (Channel Shuffle) 作为一种在卷积神经网络 (CNN) 中常用的特征融合技术，可以优化特征通道之间的交互方式。该技术的核心思想是通过打乱、重排通道的顺序，增强通道之间的交流，从而提高模型的学习能力和表现。基于此，LightCTS [28] 采用特征分组洗牌策略，既减少了卷积过程中序列与卷积核全连接的数量，降低了计算复杂度，又在卷积操作后对输出特征通道进行洗牌，使得每一组中的特征在下次卷积时发生变化，实现了不同通道之间的特征融合，进一步提升了模型的表达能力。

加权聚合 (Weight Aggregation) 是处理频域数据更为常见的一种操作。以 FFT 为例，时域序列在被映射到频域后，得到了序列各个频率成分的振幅和相位信息。其中，显著频率在频谱上通常具有更高的振幅，反应了不同频率序列对时间序列变化的相对重要性，因此许多模型选择将振幅视为加权聚合中的权重因子 [60, 67]。更复杂的是，TIMEMIXER++ 为了更精准融合序列不同尺度的季节特性和趋势特征，采用了一种**残差卷积**的操作。具体来讲，对于多尺度的季节性时间序列，较长的模式可以被解释为较短模式的组合（例如，一年中的降水模式是由每月的变化组成的），因此使用自下而上的残差卷积可以融合从细粒度到粗粒度的季节模式。而与季节模式不同，对于多尺度的趋势时间序列，更粗的尺度更能够突出整体的趋势，因此采用自上而下的混合策略建模全局趋势。最后将这两个模块的输出拼接，执行 FFT 的加权聚合操作。

3.3.6 Transformer

早期基于深度学习的时序模型（如 RNN）依赖顺序传递信息，逐步更新隐藏状态。然而，由于梯度消失、信息瓶颈和无法并行处理等限制，这些模型在捕捉长时间跨度的依赖关系时表现不足。Transformer[58] 是一种以自注意力机制为核心的深度学习模型，最初应用于自然语言处理任务中。基于 Transformer 的时间序列预测模型依赖其核心的自注意力机制，拥有强大的全局建模能力，能够在输入序列的所有时间

步之间建立直接联系，灵活获取各个时间点的信息，从而更好地建模时序数据中的复杂模式和依赖性，在时序预测领域展现出广泛的应用前景与显著优势。

图 3.6展示了基于 Transformer 的时间序列预测模型的通用架构。对于给定的历史序列 $\mathbf{X} = (\mathbf{x}_{t-H+1}, \mathbf{x}_{t-H+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{H \times N}$ （图中以 $N=1$ 的单变量时间序列为例进行展示），首先运用包含线性变换、卷积等操作的嵌入模块，将输入的序列从原始维度变换到指定的隐藏空间维度，生成初始表示 $\mathbf{Z}_0 \in \mathbb{R}^{H \times D}$ ，其中 H 表示历史时间步数， D 表示嵌入维度。随后， $\mathbf{Z}_0$ 作为输入传递到主干网络（Transformer Backbone）中，通过自注意力机制捕获时间序列中的全局依赖关系和复杂模式，并生成表征（Representation）。最终由预测头（Prediction Head）通过线性变换将其映射到目标维度，生成预测结果 $\hat{\mathbf{X}} = (\hat{\mathbf{x}}_{t+1}, \hat{\mathbf{x}}_{t+2}, \dots, \hat{\mathbf{x}}_{t+F}) \in \mathbb{R}^{F \times N}$ 。模型通过优化预测结果与真实目标序列之间的误差，不断提高对时间序列复杂模式的拟合能力。

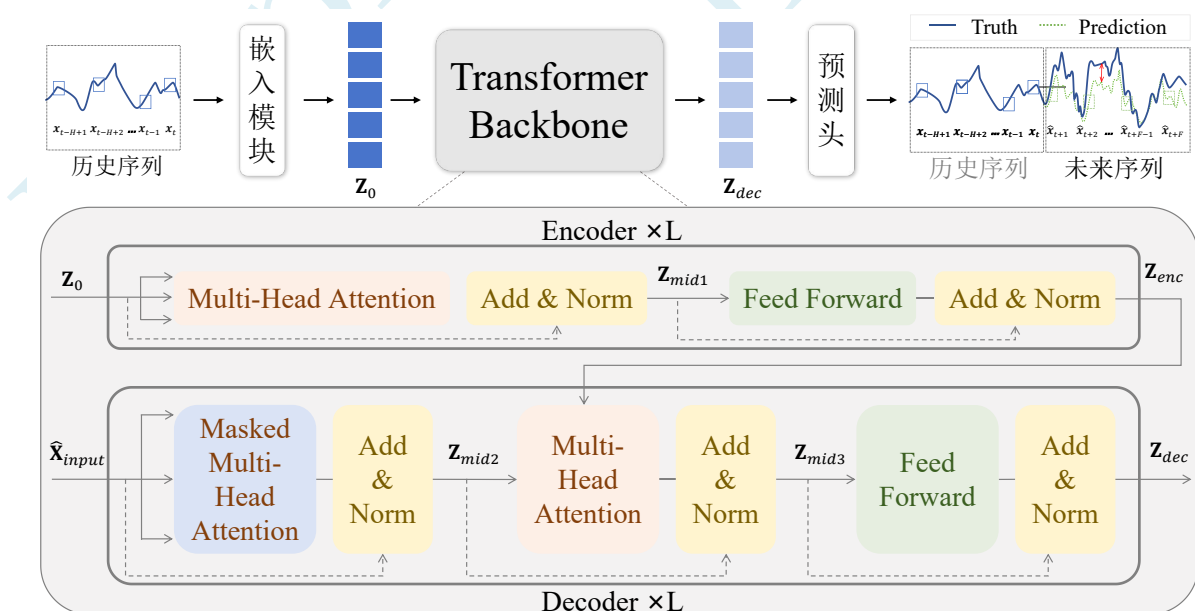


图 3.6: Transformer 结构。

接下来，我们将详细介绍 Transformer Backbone 模块。Transformer Backbone 通过堆叠多个相同结构的模块实现时序数据的特征提取与预测，主要由**编码器 (Encoder)**和**解码器 (Decoder)**组成。注意，并不是所有的基于 Transformer 的模型都包含编码器和解码器，此处我们以最通用的编码器-解码器架构为例进行说明。编码器主要用于捕获输入序列的全局时间步间依赖关系，而解码器则会结合编码器输出与目标序列的信息，进而生成未来时间步的预测结果。

编码器 在编码器中，输入序列的嵌入表示 $\mathbf{Z}_0 \in \mathbb{R}^{H \times D}$ 首先通过多头自注意力（Multi-Head Attention），捕获输入序列中各时间步之间的全局依赖关系，每个注意力头的

计算过程如下：

$$\mathbf{Q}_e = \mathbf{Z}_0 \mathbf{W}_Q, \mathbf{K}_e = \mathbf{Z}_0 \mathbf{W}_K, \mathbf{V}_e = \mathbf{Z}_0 \mathbf{W}_V \quad (85)$$

$$\text{Attention}(\mathbf{Q}_e, \mathbf{K}_e, \mathbf{V}_e) = \text{softmax}\left(\frac{\mathbf{Q}_e \mathbf{K}_e^\top}{\sqrt{d_k}}\right) \mathbf{V}_e \quad (86)$$

其中, $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{D \times d_k}$ 是模型的可学习参数, $\mathbf{Q}_e, \mathbf{K}_e, \mathbf{V}_e$ 分别表示查询、键和值, d_k 是每个注意力头的维度, softmax 是一种归一化函数, 用于将注意力分数转化为概率分布, 从而赋予不同时间步以不同的权重, 便于模型专注于相关信息。多个注意力头的输出拼接后通过线性变换生成最终的多头注意力输出：

$$\text{MultiHead}_{enc}(\mathbf{Q}_e, \mathbf{K}_e, \mathbf{V}_e) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \mathbf{W}_O \quad (87)$$

其中, $\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$ 表示第 i 个注意力头的输出, h 为注意力头数, $\text{Concat}(\cdot)$ 表示将各注意力头的输出进行拼接, $\mathbf{W}_O \in \mathbb{R}^{hd_k \times D}$ 是模型的可学习参数。随后, 通过残差连接 (Residual Connection) 将多头注意力的输入与输出相加, 缓解深层网络中的梯度消失问题, 并使用层归一化 (LayerNorm) 稳定网络训练, 获得中间表示 $\mathbf{Z}_{mid1}$ ：

$$\mathbf{Z}_{mid1} = \text{LayerNorm}(\mathbf{Z}_0 + \text{MultiHead}_{enc}(\mathbf{Q}_e, \mathbf{K}_e, \mathbf{V}_e)) \quad (88)$$

图 3.6 中的 Add & Norm 表示残差连接与层归一化, 经过残差连接和层归一化得到中间表示 $\mathbf{Z}_{mid1}$ 输入到前馈神经网络 (Feed Forward Network, FFN), 主要用于挖掘非线性特征并提升特征表达能力。通过残差连接和层归一化处理后, 生成编码器的最终输出：

$$\mathbf{Z}_{enc} = \text{LayerNorm}(\mathbf{Z}_{mid1} + \text{FFN}(\mathbf{Z}_{mid1})) \quad (89)$$

解码器 解码器的结构与编码器类似, 但自注意力机制有所不同, 解码器中使用的是掩码自注意力机制和编码器-解码器交叉注意力机制。解码器的初始输入为 $\hat{\mathbf{X}}_{input} \in \mathbb{R}^{F \times D}$, 其中 F 表示未来时间步长度, 该嵌入可以基于固定初始化标记、位置编码、部分历史序列或其他先验信息生成。这些初始输入随后会与编码器的输出结合, 通过掩码自注意力机制和编码器-解码器交叉注意力机制进行处理, 逐步生成完整的目标序列。具体细节如下：

首先, 解码器的输入序列的嵌入表示 $\hat{\mathbf{X}}_{input}$ 通过掩码自注意力机制进行处理。掩码自注意力机制确保解码器在生成目标序列的每个时间步时, 仅利用已生成的部分, 而屏蔽未来时间步的信息。参考前面的公式 85, 对输入 $\hat{\mathbf{X}}_{input}$ 进行变换, 可得 $\mathbf{Q}_d, \mathbf{K}_d, \mathbf{V}_d$, 然后再进行掩码注意力的计算：

$$\text{MaskedAttention}(\mathbf{Q}_d, \mathbf{K}_d, \mathbf{V}_d) = \text{softmax}\left(\frac{\mathbf{Q}_d \mathbf{K}_d^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}_d \quad (90)$$

其中, $\mathbf{M} \in \mathbb{R}^{F \times F}$ 是掩码矩阵, 通过将掩码矩阵中的未来时间步的对应位置设置为负无穷, 阻止解码器对这些位置的注意力计算, 避免模型看到未来的内容, 确保解码器在生成目标序列的每个时间步时, 只能依赖已生成的部分序列, 而无法访问未来时间步的信息。与公式 87 类似, 将多个掩码注意力头的输出拼接, 并通过线性变换生成最终的多头掩码注意力输出 $\mathbf{Z}_{mask}$ 。

随后, 通过残差连接和层归一化, 得到中间表示 $\mathbf{Z}_{mid2}$:

$$\mathbf{Z}_{mid2} = \text{LayerNorm}(\hat{\mathbf{X}}_{input} + \mathbf{Z}_{mask}) \quad (91)$$

接下来, 中间表示 $\mathbf{Z}_{mid2}$ 与编码器的输出 $\mathbf{Z}_{enc}$ 相结合, 通过编码器-解码器交叉注意力机制来捕获目标序列和输入序列之间的依赖关系, 对 $\mathbf{Z}_{mid2}$ 进行变换得到 $\mathbf{Q}_c$, 对 $\mathbf{Z}_{enc}$ 进行变换得到 $\mathbf{K}_c, \mathbf{V}_c$, 然后再进行注意力的计算, 并进行多头拼接, 得到多头交叉注意力输出 $\mathbf{Z}_{cross}$ 。

随后, 通过残差连接和层归一化, 生成新的中间表示 $\mathbf{Z}_{mid3}$:

$$\mathbf{Z}_{mid3} = \text{LayerNorm}(\mathbf{Z}_{mid2} + \mathbf{Z}_{cross}) \quad (92)$$

最后, 中间表示 $\mathbf{Z}_{mid3}$ 输入到前馈神经网络中, 经过残差连接与层归一化处理, 生成解码器的最终输出 $\mathbf{Z}_{dec}$:

$$\mathbf{Z}_{dec} = \text{LayerNorm}(\mathbf{Z}_{mid3} + \text{FFN}(\mathbf{Z}_{mid3})) \quad (93)$$

近年来, 涌现了许多基于 Transformer 的时间序列预测模型, 其关键技术如表 3.4 所示:

1. **时间序列特性 (Time Series Characteristics)** 指的是时间序列数据中固有的特性, 例如多尺度性质、非平稳性、变量相关性、季节性与趋势性等, 模型对这些特性的捕捉能力会直接影响预测性能, 以下将分别介绍针对不同特性的模型方法。

多尺度性质 是指时间序列数据中存在不同时间粒度下的特征。Triformer[10] 率先提出 **三角形堆叠结构** 来实现多尺度建模, 首次利用 **Patch 切分** 和 **Patch Attention** 机制, 仅将伪时间戳传递至更高层, 使得层间输入规模逐层缩减, 从而在不同时间粒度上高效提取和聚合特征。Pathformer[8] 从时间分辨率和时间距离两个角度进行建模, 利用多尺度自适应路径动态提取和聚合多尺度特征, 实现时间序列的 **自适应多尺度建模**; 而 Pyraformer[36] 则通过金字塔结构将时间序列数据分层处理, 每一层对应不同的时间粒度, 结合自注意力机制和 **时间卷积操作**, 综合全局和局部依赖关系, 提升时序预测的表现。Scaleformer[52] 通过结合 **下采样** 和 **上采样** 的多尺度处理, 使模型逐步从粗粒度到细粒度进行预测, 输入数据先下采样到低分辨率以捕捉长期趋势, 随后通过逐步上采样来细化预测, 在更高分辨率上提取短期波动特征, 并且引入 **跨尺度归一化**, 减少了分布偏移和误差积累。

表 3.4: 基于 Transformer 的时间序列预测模型总结 (H 为序列长度, N 为变量数)。

模型	年份	时序特性捕捉	计算效率	嵌入方式	嵌入方式
Transformer[58]	2017	-	$O(H^2)$	点嵌入	编码器-解码器
LogSparse[30]	2019	-	$O(H(\log H)^2)$	点嵌入	仅解码器
Autoformer[68]	2021	季节性与趋势性	$O(H \log H)$	点嵌入	编码器-解码器
Informer[81]	2021	-	$O(H \log H)$	点嵌入	编码器-解码器
FEDformer[82]	2022	季节性与趋势性	$O(H)$	点嵌入	编码器-解码器
Triformer[10]	2022	多尺度性质	$O(H)$	块嵌入	仅编码器
Pyraformer[36]	2022	多尺度性质	$O(H)$	点嵌入	仅编码器
Non-stationary Transformer[39]	2022	非平稳性	$O(H^2)$	点嵌入	编码器-解码器
ETSformer[66]	2022	季节性与趋势性	$O(H \log H)$	块嵌入	编码器-解码器
Crossformer[79]	2023	变量相关性	$O(H^2)$	块嵌入	编码器-解码器
Scaleformer[52]	2023	多尺度性质	$O(H^2)$	点嵌入	编码器-解码器
PatchTST[43]	2023	-	$O(H^2)$	块嵌入	仅编码器
PDF[13]	2024	周期性	$O(H^2)$	块嵌入	仅编码器
Pathformer[8]	2024	多尺度性质	$O(H^2)$	块嵌入	仅编码器
iTransformer[38]	2024	变量相关性	$O(N^2 + NH)$	变量嵌入	仅编码器

非平稳性 指时间序列的统计特性（如均值、方差）随时间动态变化，即统计特性随时间动态变化的现象，在现实数据中普遍存在，对具有非平稳性的时间序列进行预测是一项巨大挑战。传统的时序模型难以处理这种非平稳性，而 Non-stationary Transformers[39] 是一种针对非平稳时间序列数据设计的 Transformer 框架，它通过 Series Stationarization 模块对序列进行平稳化处理以提升预测的稳定性；同时通过 De-stationary Attention 模块在注意力机制中引入“**去平稳化因子**”，恢复原始非平稳数据中的特征，这种方法兼顾了平稳化的稳定性和非平稳特征的保留，有效避免了过度平稳化的问题，在处理复杂非平稳序列时显著提升了预测性能，成为非平稳时间序列预测领域中强有力的通用框架。时间序列除了具有非平稳性，还具有季节性、趋势性等特征。

变量相关性 指的是多变量时间序列中各变量之间的关联性，许多现有模型在多变量时间序列预测中往往忽视了对变量相关性的建模，而有效利用变量间的关联信息可以显著提升预测精度。iTransformer[38] 引入**维度倒置**设计，将每个变量的时间序列独立视为一个 token，使自注意力机制能够专注于捕捉不同变量之间的依赖关系，并通过前馈网络模块学习更优的全局序列表示；Crossformer[79] 则通过维度分段嵌入、Cross-Time Stage 和 Cross-Dimension Stage 双阶段注意力层，显式捕获时间和变量之间的双重依赖，从而有效利用变量间的关联信息，提升预测精度。Triformer[10] 提出变量特定建模方法，将投影矩阵分解为共享的变量无关部分与变量特定部分，并由每个变量的记忆向量生成其专属参数，从而在

保持参数高效的同时捕捉不同变量的独特时序模式，有效建模变量间的相关性。

季节性 通常指与自然或日历周期相关的固定重复模式，例如每日温度波动、季度销售旺季等。在更广义的意义上，这些规律性的重复模式可统称为周期性。因此，季节性可以视为周期性的一种特例：所有季节性模式都具有周期性，但并非所有周期性模式都与季节或日历时间直接相关。在时序预测研究中，二者有时会交替使用，但在强调日历规律时使用“季节性”更为恰当，而在频域建模或泛化到任意频率模式时则常用“周期性”。**趋势性** 则是指数据随时间推移表现出的长期增长、下降或平稳变化。Autoformer[68] 引入了**自相关机制 (Auto-Correlation)**，能够自动发现并利用时间序列内部的周期依赖关系。同时，Autoformer 采用渐进式分解架构，通过多层分解模块逐步分离时间序列的不同成分，从而在建模复杂模式的同时，能够分别对季节性（周期性）和趋势性进行建模。FEDformer[82] 提出了**频域增强机制**，利用傅里叶变换和小波变换从时间序列中提取趋势与周期性特征，并在频域中高效建模全局与局部依赖关系，从而在提升预测精度的同时显著降低计算复杂度。ETSformer[66] 则结合了**指数平滑注意力 (ESA)** 和**频率注意力 (FA)** 机制，其中 ESA 通过指数平滑公式强化对近期观测的关注以捕捉趋势成分，FA 则借助傅里叶变换选择主要频率分量以提取季节性模式。此外，ETSformer 采用**模块化分解**设计，将时间序列拆解为水平、增长和季节性等可解释成分，从而兼顾预测准确性与可解释性。周期性解耦框架 PDF (Periodicity Decoupling Framework) [13] 则更加突出广义的周期性建模。它通过**频域分析**识别并提取时间序列的关键周期，并提出“周期性解耦”机制，将短期变化和长期变化分开处理，即将一维序列解耦为短期和长期子序列，从而提升了模型对复杂多周期信号的预测能力。

此外，PatchTST[43] 通过引入**通道独立**和 **patching** 设计，显著增强了模型的效率和特征提取能力，**patching** 机制将时间序列划分为小的 **patch**，既保留局部语义信息，又减少输入 **token** 数量，降低注意力计算复杂度。

2. **计算效率 (Computational Efficiency)** 指的是模型在处理时间序列数据时的资源消耗，包括计算复杂度、内存使用，我们主要从计算复杂度方面衡量计算效率。传统 Transformer[58] 模型的自注意力机制计算复杂度为 $O(H^2)$ 。随着输入序列长度的增加，计算所需的时间和内存资源迅速增长，这在处理长序列时尤为明显。为了解决这一问题，许多研究致力于降低 Transformer 的计算成本。例如，LogSparse Transformer[30] 提出的**稀疏注意力机制**仅对选择的指数间隔位置进行注意力计算，从而避免存储和计算大量无关的权重矩阵元素，每一层的注意力计算都是稀疏的，但通过堆叠多个注意力层，每个位置最终可以从远距离位置获得信息，它可以将复杂度降低到 $O(H(\log H)^2)$ 。Autoformer[68] 则采用自相关机

制替代自注意力机制，通过发现序列周期性并聚合相似子序列，实现 $O(H \log H)$ 的复杂度。Informer[81] 模型通过引入 ProbSparse 自注意力机制，通过选择性关注最相关的时间步来减少计算量，将复杂度降低至 $O(H \log H)$ 。FEDformer[82] 引入频率增强机制，利用傅里叶变换或小波变换将输入信号从时间域转化为频率域，随机选择关键频率成分以捕捉全局和局部特征，保持时间序列的全局特征和重要信息可以将注意力计算复杂度降至 $O(H)$ 。Triformer[10] 模型通过引入 Patch Attention 机制和三角形堆叠结构来降低计算复杂度，它将输入时间序列划分为 Patch，并为每个 Patch 引入伪时间戳，减少每个 Patch 内注意力的计算，在堆叠多个 Patch Attention 层时，仅将伪时间戳传递至下一层，每一层的输入大小都会呈指数递减形成三角形结构，这能进一步保持线性复杂度，使复杂度从传统的平方复杂度降低为线性复杂度 $O(H)$ 。Pyraformer[36] 通过引入金字塔注意模块 (Pyramid Attention Module, PAM)，分层聚合时间序列特征，将序列划分为多级金字塔结构，并在每一级上采用稀疏注意力机制，通过层层聚合，全局信息得以在较低复杂度下被捕获，同时保持了高效的信息传递，可以将计算复杂度降低到 $O(H)$ 。

3. 嵌入方式 (Embedding Methods) 是指将时间序列数据表示为模型能够理解的特征输入方式，主要分为以下几种：

点嵌入 (Point Embedding) 是指对每个时间步的数据点独立进行嵌入，生成与时间步数量相等的嵌入向量序列。这种方式适用于单变量或简单多变量时间序列的建模，能够以最细粒度捕捉时间序列信息。例如，Transformer[58] 中使用位置编码直接对每个时间步嵌入。Informer[81] 和 FEDformer[82] 也都采用了点嵌入方式，将每个时间步的原始输入映射为高维向量，以便模型能够充分捕捉细粒度的时序特征。

块嵌入 (Patch Embedding) 是指将时间序列划分为若干时间块 (Patch)，并对每个时间块中的数据进行联合嵌入。该方式能够在保留局部信息的同时降低输入长度，从而提高计算效率。Triformer[10] 是首个在时间序列预测中引入 **Patch 切分** 思想的模型，并提出 **Patch Attention** 对块内时间点进行聚合建模，结合三角形堆叠结构，在层间仅传递伪时间戳以实现多尺度特征提取。这一设计开创了时间序列预测中基于 Patch 的高效注意力建模思路。PatchTST[43] 也采用了该思路，将时间序列划分为多个小块并生成嵌入向量，显著提升了长时间序列预测的性能和效率。

变量嵌入 (Variable Embedding) 是一种针对多变量时间序列数据的特征表示方法，其核心思想在于将多变量时间序列分解为多个单变量时间序列，其中每个单变量序列都被视为一个独立的序列单元。对于每个单变量序列，会为其生成独特

的嵌入表示，这种方法利用了自注意力机制的优势，能够有效捕捉不同变量之间的关系。例如，iTransformer[38] 采用了维度倒置设计，将时间序列中的每个变量视为一个 token，利用自注意力机制在变量维度上直接建模变量间的依赖关系，提升了对多变量时间序列中变量交互特性的捕捉能力。

4. 架构 (Framework) 是指模型在整体结构上的设计，主要包括以下三种：

编码器-解码器 (Encoder-Decoder) 是 Transformer 的经典结构，编码器用于提取输入时间序列的特征，解码器则根据编码结果生成预测序列。这种架构在处理复杂任务时展现出强大的适应性和优势，尤其是在处理具有长时依赖性和复杂结构的时间序列数据方面。例如，Autoformer[68] 和 Informer[81] 均采用了编码器-解码器架构，以提升长序列预测能力。

仅编码器架构 (Encoder-only) 仅使用编码器结构提取输入序列的特征，再通过轻量级的预测层生成输出。编码器充分发挥其特征提取能力，能够从输入序列中有效挖掘全局与局部信息：全局特征用于把握时间序列的整体趋势与长期模式，局部特征则有助于捕捉短期波动与细节变化。由于省去了复杂的解码器结构，模型的计算复杂度显著降低，使其在需快速处理大量数据或计算资源受限的场景中表现优异。例如，Pathformer[8] 和 PatchTST[43] 均采用仅编码器架构，在减少计算开销的同时，专注于高效特征提取。

仅解码器架构 (Decoder-only) 省略编码器部分，直接对输入时间序列进行解码生成预测结果。该架构更为轻量化，通过避免编码器的复杂计算，显著降低了计算开销。在处理短序列或需要高响应速度的在线预测任务时，此类架构优势明显，能够快速处理新数据并完成预测，同时维持较低的资源消耗。例如，LogSparse Transformer[30] 采用仅解码器架构并结合稀疏自注意力机制，在保证预测精度的同时有效提升计算效率。

3.4 时间序列通道关系

在多变量时间序列预测中，通道关系指的是不同通道（或变量）之间的相互依赖关系和信息传递机制。在实际应用中，许多时间序列数据不仅仅由单一的变量构成，而是由多个相互关联的变量组成。例如，在金融市场预测中，股票价格、交易量和市场情绪等不同变量之间是密切相关的。正确捕捉这些变量之间的依赖性是提高预测精度的关键。

3.4.1 通道关系的不同策略

通道策略代表着模型如何处理和融合不同通道之间的关系。在多变量时间序列数据中，不同通道之间可能存在或显著或虚假的相关性、或冗余或互补的历史信息以及影响预测的噪音。同时通道策略的选择不仅影响模型的表现，也决定了处理复杂多变量时序数据的效率和可扩展性。因此通道策略在多变量时间序列预测中扮演着至关重要的角色。在现有的研究中，通常在嵌入模块或专门的相关性模块中来实现通道间的信息交互，如图3.7所示。我们进一步将通道间的交互方式划分为三种类型：通道独立、通道完全依赖和通道部分依赖，从而更深入地探讨每种方式的优势与局限。

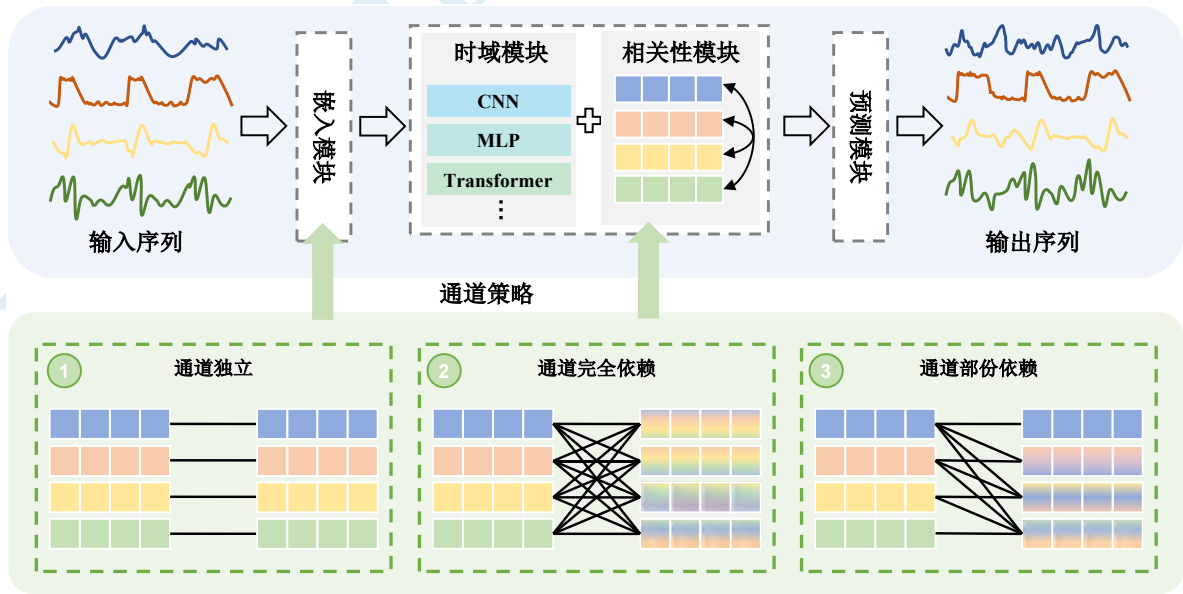


图 3.7: 不同的通道策略。

通道独立 通道独立策略在预测时，每个通道的预测仅使用当前通道的输入信息，不同通道间视为独立的时序样本，而不考虑通道之间的任何潜在交互或相关性，尽管模型权重在不同通道间是共享的，但是通道间相关信息没有直接的利用。代表性的方法如 PatchTST[43]、DLinear[77] 等，这种设计显著降低了模型复杂性，实现了更高效的模型推断，同时降低了通道间噪声或冗余相关性导致的过拟合风险。此外，通道独立策略提供了更多的灵活性，因为添加新通道不需要更改模型架构，从而使其能够无缝适应不断变化的数据集，保持预测性能。大部分基于通道独立策略的模型已在3.3节中有所介绍，本节主要聚焦于建模通道依赖的模型与方法。

通道完全依赖 通道完全依赖策略假设多变量时间序列中的所有通道都是内在相关和相互依赖的，在预测过程中将它们作为一个统一的实体来处理。基于学习通道间交互的阶段，现有的通道完全依赖方法可以分为两类：1) **嵌入融合**：这样的方法在获得其时间序列嵌入表示时融合来自不同通道的数据。例如 Informer[81]、Autoformer[68]

和 TimesNet[67] 使用 1D 或 2D 卷积来提取时间序列的嵌入表示。在卷积运算中，每个卷积核首先在每个输入通道内执行滑动卷积操作，以获得相应的特征图。然后，对所有通道的特征图进行加权合并，从而捕获通道之间的依赖关系。2) **显式建模**：这些模型通常设计专门的模块来显式地对通道相关性进行结构化的建模，从而促进依赖信息有效的在通道间传递。代表性的算法包括 iTransformer [38] 和 TSMixer[15]。iTransformer 在通道间采用了自注意力模块，将独立的序列通道作为令牌，并使用注意力机制捕获多元相关性。相比之下，TSMixer 在通道之间使用 MLP 模块来捕获通道之间复杂的相关性，这些相关性由通过完全连接的线性层提取的多级特征表示。完全依赖策略对于具有强相关的数据有更高的鲁棒性，但对噪声高度敏感。虚假相关性会显著降低其性能，并且该种策略的效率较低，因为它需要对所有通道间依赖关系进行建模。计算复杂度随着通道数量的增加而急剧增加，可扩展性较差。

通道部分依赖 通道部份依赖策略在通道独立策略和通道完全依赖策略之间取得平衡，允许每个通道在与其他相关通道互动的同时保持一定程度的独立性。这种方法强调了一种混合状态，其中通道选择性地相互作用并表现出部分相关性。现有基于通道部份依赖策略的方法可以分为两类：1) **固定通道交互**：这样的模型固定了每个通道的相关通道数，这意味着相关通道的集合随时间保持不变。例如，在 MTGNN[70] 中，通道关系被建模为 K 正则图，其中每个通道使用通道完全依赖策略与 K 个其他通道交互以建模相关依赖，而其余通道通过通道独立策略相互保持独立。类似地，在 MCformer[20] 中，每个通道仅与 K 个其他通道交互，与其余通道保持独立，以确保计算效率并防止过拟合。2) **动态通道交互**：这些模型允许每个通道的相关通道数量是动态的，随着时间的推移而变化，并提供更高的灵活性以适应不同的场景。例如，DUET[45] 利用频域中的度量学习方法来计算通道相似性，为每个通道生成不同的掩码，以屏蔽掉无关通道的影响。此外，CCM[7] 基于通道间的内在相似性，动态地对通道进行聚类。每个类内部通道完全依赖，而在类间保持独立。这种建模方式通过捕捉灵活和动态的关系表现出高鲁棒性以及最强的泛化能力，使其在存在噪声和数据分布复杂的场景中非常有效，但这种灵活的交互方式对模型的设计提出了更多的挑战。

3.4.2 基于 Transformer 的通道依赖建模

近年来，Transformer 已广泛应用于多变量时间序列预测任务，利用其强大的全局建模能力可以有效捕获时间依赖性和复杂的通道交互。如图3.8所示，现有的基于注意力机制的方法可以分为以下几类：

1. **标准注意力机制**：这类方法将序列通道单独的令牌，并直接应用注意力机制来对通道相关性进行建模。例如，CARD[62] 和 iTransformer。
2. **路由注意力机制**：当通道数 (N) 较大时，通道注意力的计算复杂度达到 $O(N^2)$ ，

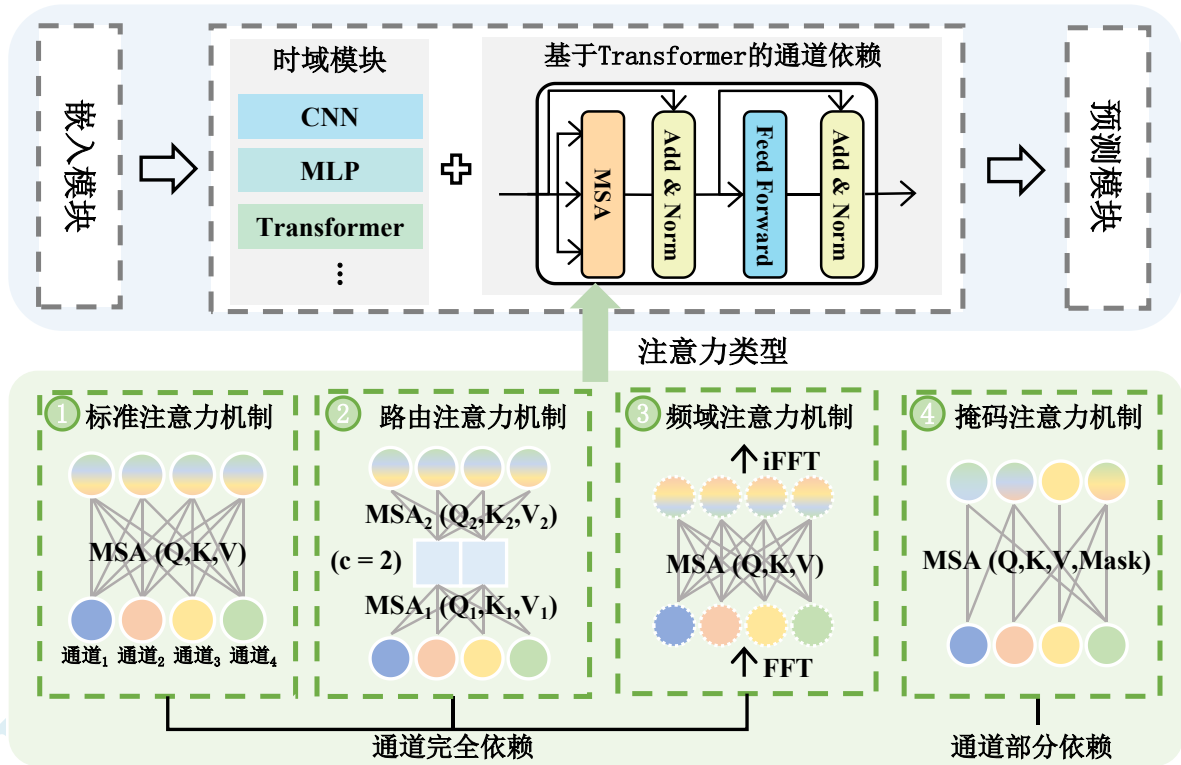


图 3.8: 基变换器的通道建模。

导致计算成本较高。为了解决这个问题，一些方法如 Crossformer[79] 在标准注意力的中加入了路由器机制, 通过少量固定数量的 c 个“路由器”从所有通道收集信息并重新分配。将复杂度降低到 $O(2cN) = O(N)$ ，有效地平衡了通道相关性建模和计算效率。

3. **频域注意力机制**：一些方法表明，频域信息比时域信息更有效地捕获通道间依赖性。基于此，FECAM[24] 将时间序列数据变换到频域后，再采用标准注意力进行建模。
4. **掩码注意力机制**：在标准注意力机制中，每个通道计算所有通道的注意力分数，这可能会受到不相关通道的负面影响。为了缓解这一问题，DUET 通过在频域构建可学习度量计算掩码矩阵，屏蔽部分通道间的注意力交互，减轻不相关通道的影响。

其中，标准注意力机制、路由注意力机制、频域注意力机制均属于完全依赖策略，而掩码注意力机制则属于部分依赖策略。

3.4.3 基于图神经网络的通道依赖建模

通过将时间序列沿着时间方向划分为不同的窗口，将窗口内的每个通道视为一个节点，通道间的相关性视为边，可以将多变量时间序列转换为基于图的数据。根据所构建的图的类型，可以将基于 GNN 的方法分为如图3.9中所示的 4 种：

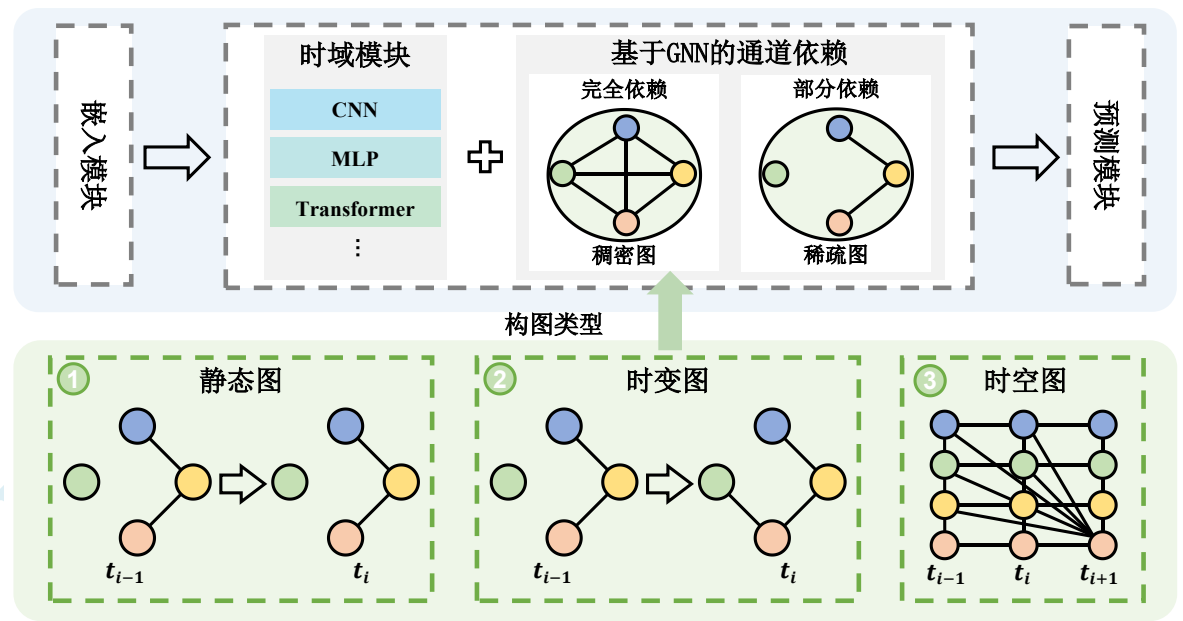


图 3.9: 基于图神经网络的通道建模。

- 静态图：** 该类方法，为通道学习稳定的图结构，并不同的时刻间共享该结构。现有方法使用了通道相似性度量（MTGNN、MSGNet[6]、CrossGNN[21]）、数据相似性度量（GTS[53]、WaveForM[73]）来学习通道之间的相关图结构。同时利用不同通道的时域（MTGNN，MSGNet，CrossGNN，GTS）或频域（WaveForM）信息来学习节点特征。在简单图中应用基于图卷积的消息传递以促进通道之间的依赖性信息的传输。
- 时变图：** 在真实的世界中，时间序列数据的相关性往往随着时间的推移而变化，形成动态的关系图。针对这一现象，MTSF-DG[80] 和 TPGNN[37] 分别使用动态图和多项式图来模拟这些相关性的变化模式。这类模型更侧重于捕获相邻时刻间相关性之间的关联，从而更好地拟合相关性关系。
- 时空图：** 与简单图不同，时空图将不同时间步长的多个通道合并到单个图中，进一步考虑不同时间步长的通道之间的关系。这种方法允许 GNN 同时对时间和通道依赖性进行建模，有效地解决了时间模块和 GNN 之间的潜在兼容性问题，但会

有更高的计算负担。基于时空图的模型如 FourierGNN[76] 以及 FCSTGNN[83]，在使用时空图建模的同时采用了不同的方法来进一步减少计算复杂度。

从图结构的角度来看，上述类型均可以构建稀疏图或稠密图。具体而言，稀疏图结构体现了通道间的部分依赖特性，而稠密图（或完全图）结构则表征了通道间的完全依赖关系。

3.4.4 基于多层感知机和卷积神经网络的通道依赖建模

多层感知器（MLP）是最基础的神经网络，根据通用逼近定理，具有强大的特征学习能力，同样也可以用于通道间交互。此外，卷积神经网络（CNN）是一种特殊的深度学习模型，可以利用卷积层的不同阶段从数据中提取局部特征。由于基于以上两种模型的通道建模方法，本质上都是在直接学习一组权重并对不同通道的信息进行加权合并，因此将二者一同说明。如图3.10所示，可以将这类方法分为以下 3 类：

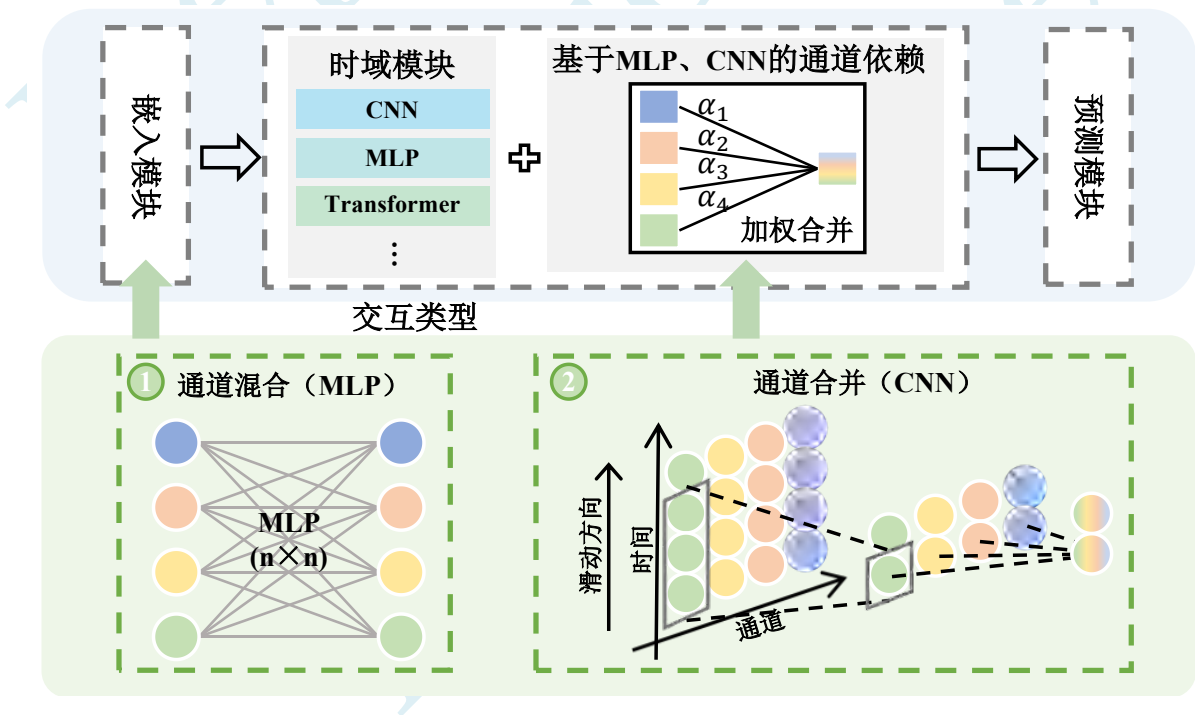


图 3.10: 基于多层感知机和卷积神经网络的通道建模。

- 1. **通道混合（MLP）**：将具有 N 个通道的时序数据表征，在通道维度上通过输入、输出都为 N 的 MLP 来进行混合。就可以使用简单的 MLP 模型来捕获通道之间的关系。例如 TS-Mixer[15] 与 TTM[14]。采用这种方法可以有效地融合通道之间的有效信息，以低计算成本实现强大的性能，属于通道完全依赖策略。

2. 通道合并 (CNN)：许多模型，如 Informer，Autoformer 和 FEDMormer，在初始特征提取层中使用具有沿时间维度沿着操作的 1D 卷积。这些方法将不同的时序通道视为卷积的不同输入通道，在卷积过程中对其特征进行加权和合并，从而实现通道间的交互。尽管 TimesNet 采用 2D 卷积，其将时间维度折叠成 2D 格式，其中的时序通道仍然用作卷积的不同输入通道，经由卷积的加权合并。

3.4.5 相关性的不同特征

为了更好地探索多变量时间序列中的通道相关性，通常有必要探讨通道之间相关性的不同特征。本节将解释当前方法中通常考虑的六个关键特征，如图3.11所示。

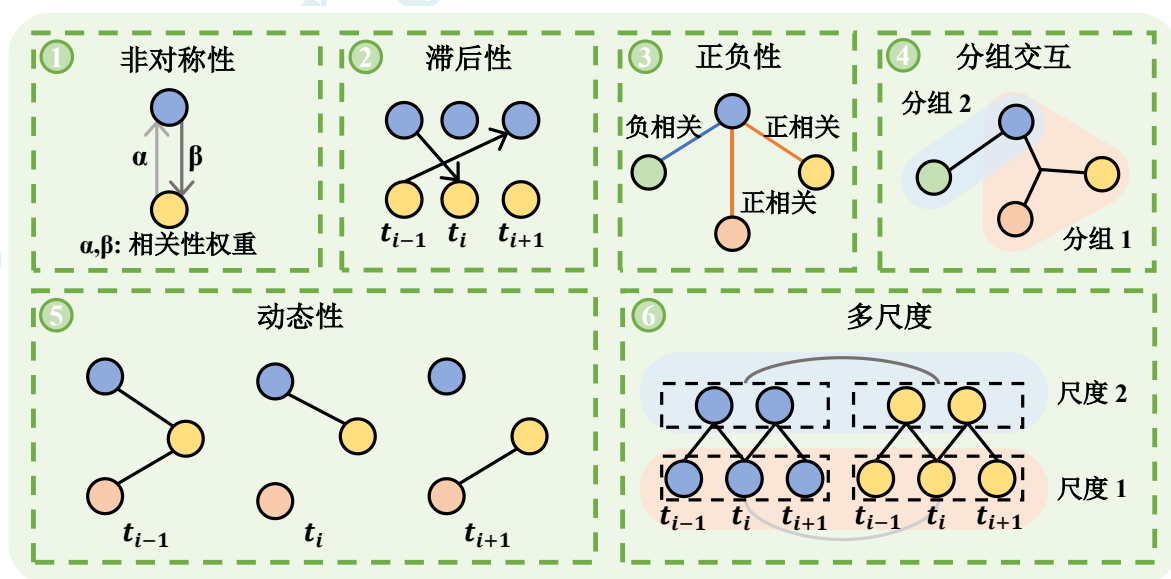


图 3.11: 相关性的不同特征。

非对称性: 不对称性是指多变量时间序列中通道之间具有不平等关系，在各个通道之间，相互影响的程度并不相同。基于 Transformer 和 MLP 的方法，由于其计算过程的特点，自然地具备不对称性，从而使它们能够有效捕获不对称的相关性。而基于 GNN 的方法通过非对称距离来建立有向图，从而使相互作用在不同的传输方向上具有不同的权重，如 MTGNN[70]，MSGNET[6]。

滞后性: 滞后性是指某个通道的当前状态不仅依赖于其他通道的当前状态，而且还可能受到它们的过去状态的影响。基于滞后性的特点，VCFormer[74] 融合通道间多时间步的滞后效应，共同来计算通道间的注意力分数。相比之下，ST-WA[12] 直接建立过去通道与未来通道间单向的因果注意力，来捕获滞后的通道关系。在基于 GNN 的方法中，FourierGNN[76] 和 FC-STGNN[63] 使用时空全连接图在来自不同通道、不同时间的节点之间进行消息传递，来细粒度地捕获具有滞后性通道关系。

正负性: 在通道之间的相互作用存在正相关和负相关的区别,前者是指不同通道的观测值同时增大或同时减小,后者指某一通道观测值增大(或减小)时,另一个通道的观测值随之减小(或增大)。**CrossGNN**[21]利用符号图方法,将相关性分类为正,负和中性关系。在消息传递期间,它整合了正面和负面信息交换,从而更有效地捕获了相关性的异质性。

分组交互: 通道之间的相关性表现出分组现象,其特征是同一组内的相关性强,不同组之间的相关性较弱,且不同组之间的相关性在强弱、作用变量个数等方面存在差异。**CCM**[7]和**DUET**[45]使用聚类方法来建模通道之间的分组交互,而**ReMo**[69]和则建立超图,通过超边连接部分通道,并在超边上进行消息传递以促进组间信息交互。此外,**CCM**和**REMO**在不同组内使用不同的**MLP**进行特征提取以促进不同分组之间差异性表达。

动态性: 多变量时间序列在不同的时间步中表现出不同的相关性,整体呈现动态变化。基于**MLP**的方法(如**TsMixer**[15],**TTM**[14])由于线性层权重在不同的时间步中保持恒定,无法表达动态性。而以序列为令牌的基于**Transformer**的方法如(**iTransformer**[38]、**DUET**[45])也不具备捕获动态相关性的能力。以patch为令牌的基于**Transformer**的方法则可以在不同时间步得到不同的注意力权重。在**GNN**的方法中,只有图结构随时间变化的方法才能捕获相关性的动态特征,如**MSGNet**。但是,上述考虑动态性的方法,仅是考虑了相关性在不同的时间步具有不同的数值,而**ESG**[75]、**MSTF-DG**[80]、**TPGNN**[37]则是进一步考虑了相邻时间步相关性的关联或是相关性随时间变化的规律。

多尺度: 多尺度是指时间序列在不同的时间尺度(例如小时,分钟或秒)上表现出不同的相关性。基于此,**MSGNet**、**Ada-MSHyper**在不同尺度内建立不同的结果的图,以描述不同尺度上不同的相关性,并最终融合不同尺度上的相关信息。考虑到在不同尺度上通道间存在不同的相关性有助于模型更好地了解时间序列数据的多尺度特征,从而产生更准确的预测。

3.4.6 总结

在本节中,我们从通道策略的角度对多变量时间序列预测的深度学习方法进行了总结。具体说明了如何基于不同的基础架构来有效捕获相关性关系,以及如何根据相关性的具体特征来建模更复杂的相关性关系。在此将以上提到的方法总结到表3.5中,以便直观的进行对比。

表 3.5: 通道依赖方法汇总。

通道策略	方法名称	年份	通道依赖机制	相关性特征					
				非对称性	滞后性	极性	分组交互	动态性	多尺度
通道完全依赖	Informer [81]	2021	CNN-based	✓	-	-	-	-	-
	Autoformer [68]	2021	CNN-based	✓	-	-	-	-	-
	FEDformer [82]	2022	CNN-based	✓	-	-	-	-	-
	TimesNet [67]	2023	CNN-based	✓	-	-	-	-	-
	TSMixer [15]	2023	MLP-based	✓	-	-	-	-	-
	TTM [14]	2024	MLP-based	✓	-	-	-	-	-
	iTransformer [38]	2024	Transformer-based	✓	-	-	-	✓	-
	Crossformer [79]	2023	Transformer-based	✓	-	-	-	✓	-
	VCformer [74]	2024	Transformer-based	✓	✓	-	-	-	-
	MOIRAI [65]	2024	Transformer-based	✓	✓	-	-	✓	-
	UniTS [17]	2024	Transformer-based	✓	-	-	-	✓	-
	GTS [53]	2021	GNN-based	-	-	-	-	✓	-
	MSGNet [6]	2024	GNN-based	✓	-	-	-	-	✓
	FourierGNN [76]	2023	GNN-based	-	✓	-	-	-	-
	FC-STGNN [63]	2024	GNN-based	-	✓	-	-	-	-
	TPGNN [37]	2022	GNN-based	✓	-	-	-	✓	-
	ESG [75]	2023	GNN-based	✓	-	-	-	✓	✓
	EnhanceNet [11]	2021	GNN-based	✓	-	-	-	✓	✓
通道部分依赖	ModernTCN [40]	2024	CNN-based	✓	-	-	-	-	-
	DUET [45]	2025	Transformer-based	✓	-	-	✓	-	-
	MCformer [20]	2024	Transformer-based	✓	-	-	-	✓	-
	DGCformer [35]	2024	Transformer-based	✓	-	-	✓	✓	-
	CM [29]	2024	Transformer-based	-	-	-	-	✓	-
	MTGNN [70]	2020	GNN-based	✓	-	-	-	-	-
	CrossGNN [21]	2023	GNN-based	✓	-	✓	-	-	-
	WaveForM [73]	2023	GNN-based	✓	-	-	-	✓	-
	MTSF-DG [80]	2023	GNN-based	-	-	-	-	✓	-
	ReMo [69]	2023	GNN-based	✓	-	-	✓	-	-

3.5 时间序列概率预测

在时间序列预测的实际应用之中，除了对于真值的预测之外，在很多情况下，需要模型能够对潜在风险、输出的变化范围以及多种可能结果的概率进行预测，因此需要对时间序列问题进行概率预测。为了实现对时间序列预测中不确定性的准确估计，模型需要刻画待预测时间点已在观测时间点条件下的后验分布。根据模型是否显式建模后验分布函数，可以将模型分类为显式模型与隐式模型。显式概率预测模型对后验分布函数或概率密度函数进行假设，而隐式概率预测模型并不直接进行对分布的假设。这两种模型的主要区别在于模型的输出不同，显式模型的输出可能为分布函数的参数或模型的分位数；而隐式模型则通过反复进行推理过程，从而在建模的分布之中进行多次采样，以获得模型对于分布的建模结果。

与时间序列的确定性预测类似，我们首先对数据集进行划分，分为训练集 $\mathbf{D}_{train} = \{\mathbf{x}_i^{train}\}_{i=1}^{T_{train}}$ 、验证集 $\mathbf{D}_{val} = \{\mathbf{x}_i^{val}\}_{i=1}^{T_{val}}$ 和测试集 $\mathbf{D}_{test} = \{\mathbf{x}_i^{test}\}_{i=1}^{T_{test}}$ 。以训练集为例， $\mathbf{x}_i^{train} = (\mathbf{x}_{t-H+1}^{train}, \mathbf{x}_{t-H+2}^{train}, \dots, \mathbf{x}_t^{train}, \mathbf{x}_{t+1}^{train}, \dots, \mathbf{x}_{t+F}^{train}) \in \mathbb{R}^{(H+F) \times N}$ 为训练集中第 i 个样本在 t 时刻的输入输出序列， H 、 F 分别是历史序列长度和未来序列长度， T_{train} 是整个

表 3.6: 不同的概率时间序列预测方法的总结。

模型	年份	概率建模方式	模型预测目标	损失函数
DeepAR [50]	2017	显式	分布参数	NLL
DeepState [46]	2018	显式	分布参数	NLL
MQ-RNN [64]	2018	显式	分位数	QL
CouplaCPTS [55]	2024	显式	分布参数	QL, NLL
MAF [48]	2021	隐式	待预测点真值	NLL
TimeGrad [47]	2021	隐式	待预测点真值	扩散噪声预测
mr-diff [54]	2024	隐式	待预测点真值	扩散噪声预测
HyVAE [5]	2023	隐式	待预测点真值	ELBO
ProFITiF [72]	2025	隐式	待预测点真值	NLL
EnCQR [23]	2024	集成	待预测点真值	-

测试集的样本数量。对于时间序列的概率预测，我们首先在训练集和验证集上完成对时间序列的概率预测训练，之后在测试集上对时间序列的概率预测进行评价。我们采用与确定性预测部分一致的符号表示。除此之外，我们使用 $\hat{\mathbf{x}}$ 指代网络的预测结果，并用 $\mathbb{1}_x$ 表示指示函数，即当条件 x 为真时，指示函数的值为 1，否则为 0。

时间序列概率预测模型训练与构建：显式概率预测模型和隐式概率预测模型在模型训练与构建上有较为明显的差异，如表3.6所示。

显式概率预测模型通常直接输出目标分布的参数或概率密度函数表达式，并通过最大似然损失、分位点损失等损失函数对模型进行训练。按照对于分布建模的方式，可以将显式概率预测模型分为：

1. 模型对概率分布函数进行假设，并对所假设的概率分布的参数进行预测。在这种情况下，模型的输出为所假设的概率分布的参数，例如所假设的分布均值 $\hat{\mu}_\theta(\mathbf{x})$ 和方差 $\hat{\sigma}_\theta(\mathbf{x})$ ；常见的假设为高斯分布或者负二项分布 [50, 46]。当模型直接对概率分布进行假设时，常见的损失函数为负对数似然（Negative Log Likelihood, NLL），即：

$$\mathcal{L}_\theta = - \sum_{i=1}^F \log p_\theta(\hat{\mathbf{x}}_{t+i} | \mathbf{x}_{t-H+1}^t) \quad (94)$$

当假定概率分布为高斯分布时：

$$p_\theta(\hat{\mathbf{x}}_{t+i} | \mathbf{x}_{t-H+1}^t) = \frac{1}{\sqrt{2\pi\hat{\sigma}_\theta^2(\mathbf{x}_i)}} \exp\left(-\frac{(\mathbf{x}_{t+i} - \hat{\mu}_\theta(\mathbf{x}_{t+i}))^2}{2\hat{\sigma}_\theta^2(\mathbf{x}_{t+i})}\right) \quad (95)$$

$$\mathcal{L}_\theta = \frac{1}{2} \sum_{i=1}^F \left[\log \hat{\sigma}_\theta^2(\mathbf{x}_{t+i}) + \frac{(\mathbf{x}_{t+i} - \hat{\mu}_\theta(\mathbf{x}_{t+i}))^2}{\hat{\sigma}_\theta^2(\mathbf{x}_{t+i})} \right] \quad (96)$$

2. 模型预测目标变量在不同分位点（如 10%、50%、90%）上的取值，从而得到完整的预测分布或置信区间 [64]。模型的输出是对于待预测区间的目标分位点的值 $\hat{q}_\theta^\tau(\mathbf{x}_{t-H+1}^t)$ ，如 50% 分位点的值（即中位数）。模型使用分位点损失（Quantile loss, QL），对于 τ 分位点：

$$\mathcal{L}_\theta^\tau(\mathbf{x}_{t+1}^{t+F}, \mathbf{x}_t^{t-H+1}) = \begin{cases} \tau \cdot (\mathbf{x}_{t+1}^{t+F} - \hat{q}_\theta^\tau(\mathbf{x}_{t-H+1}^t)) & \mathbf{x}_{t+1}^{t+F} \geq \hat{q}_\theta^\tau(\mathbf{x}_{t-H+1}^t) \\ (1 - \tau) \cdot (\hat{q}_\theta^\tau(\mathbf{x}_{t-H+1}^t) - \mathbf{x}_{t+1}^{t+F}) & \mathbf{x}_{t+1}^{t+F} < \hat{q}_\theta^\tau(\mathbf{x}_{t-H+1}^t) \end{cases} \quad (97)$$

在所有分位点上，模型的整体损失函数为：

$$\mathcal{L}_\theta = \sum_{k=1}^K \mathcal{L}_\theta^{\tau_k}(\mathbf{x}_{t+1}^{t+F}, \mathbf{x}_t^{t-H+1}) \quad (98)$$

3. 模型将边缘分布与联合分布解耦建模 [55]，模型的输出为边缘分布的分布参数以及联合结构的建模参数。一种常见的解耦方法是使用 Coupla 函数：

$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_F) = C(G_1(\mathbf{x}_1), G_2(\mathbf{x}_2), \dots, G_F(\mathbf{x}_F)) \quad (99)$$

其中 $p(\mathbf{x}_1, \dots, \mathbf{x}_F)$ 代表了 $\mathbf{x}_1, \dots, \mathbf{x}_F$ 的联合分布， $G_i(\mathbf{x}_i)$ 是第 i 个变量的边缘分布的累积分布函数（Cumulative Distribution Function, CDF）， $C: [0, 1]^T \rightarrow [0, 1]$ 是 Coupla 函数。在采用基于 Coupla 函数的方法时，其损失函数为最小化负对数似然：

$$\mathcal{L} = -\log C(G_1(\mathbf{x}_1), G_2(\mathbf{x}_2), \dots, G_F(\mathbf{x}_F)) - \sum_{i=1}^F \log p_i(\mathbf{x}_i) \quad (100)$$

隐式概率预测模型一般为生成模型，能够对数据的分布进行有效的建模。隐式概率预测模型并不直接输出待预测区间的值的概率密度函数，而是通过模型对分布进行隐式建模，并通过重复从模型所建模的分布之中进行采样（即反复生成预测区间值）以获得对于后验分布的描述。因此，隐式概率预测模型的输出通常与确定性的时间序列预测模型一致，即为待预测序列 $\mathbf{x}_{t+1}^{t+F}$ 。常见的隐式概率预测模型包括：

1. 基于扩散模型的概率时间序列预测 [47, 54]：将回看窗口或从回看窗口提取的特征作为条件值输入模型，通过不断向初始分布（通常为高斯分布）添加采样自高斯分布的独立微小噪声以实现对目标分布的模拟，并从训练完成的模型之中重复采样以实现对于预测分布以及置信区间的估计。基于扩散模型的概率时间序列预测模型的优化函数有多种，包含噪声估计、真值估计以及速度 (velocity) 估计等。

基于噪声估计的方法预测每一步骤之中所添加的独立微小噪声，噪声估计的损失函数为：

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \epsilon} [\|\epsilon - \hat{\epsilon}_\theta(\mathbf{x}_t, t)\|^2], \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad (101)$$

基于真值估计的方法直接输出对于原始数据 $\mathbf{x}_0$ 的预测，真值估计的损失函数为：

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \epsilon} [\|\mathbf{x}_0 - \hat{\mathbf{x}}_{0, \theta}(\mathbf{x}_t, t)\|^2], \hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}(\mathbf{x}_t, t) \right) \quad (102)$$

基于速度估计的方法利用单步噪声 ϵ 和原始数据 $\mathbf{x}_0$ 构造速度，扩散模型之中的速度反映了从噪声空间到数据空间的变换确实，速度估计的损失函数为：

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \epsilon} [\|\mathbf{v}_t - \hat{\mathbf{v}}_{\theta}(\mathbf{x}_t, t)\|^2], \mathbf{v}_t = \sqrt{\bar{\alpha}_t} \epsilon - \sqrt{1 - \bar{\alpha}_t} \mathbf{x}_0 \quad (103)$$

其中 $\mathbf{x}_0$ 代表数据的真值， $\mathbf{x}_t$ 代表在扩散步 t 中的数据值， $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ ， α_t 为扩散过程之中预定义好的扩散参数轨迹。

2. 基于变分自编码器 (Variational Autoencoder, VAE) 的概率时间序列预测 [5]：将初始分布映射到隐空间分布之中（通常可以设定为高斯分布），并从隐空间之中采样再映射回原始空间以获得预测结果，以最大化变分下界（Evidence Lower Bound, ELBO）为优化目标，对于观测数据 $\mathbf{x}$ 和隐变量 $\mathbf{z}$ ，ELBO 定义为：

$$\text{ELBO}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \quad (104)$$

对于损失函数 ELBO，包含了重建项与正则项，重建项为 $\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})]$ ，最小化了重建值与真实值之间的差异，以确保建模分布与真实分布的一致性。正则项为 $-D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$ ，在确保建模分布与真实分布的一致性的同时，也增加了生成的多样性。

3. 基于标准化流的概率时间序列预测：基于标准化流的概率时间序列预测方法 [48, 72] 通过多步可逆变换实现从标准高斯分布到目标分布的转换，其损失函数为：

$$\mathcal{L}_{\theta} = - \sum_{i=1}^F \log p_{\theta}(\mathbf{x}^{(i)} | \mathbf{h}^{(i)}) \quad (105)$$

$$\log p_{\theta}(\mathbf{x} | \mathbf{h}) = \log \mathcal{N}(g_{\theta}(\mathbf{x}; \mathbf{h}); 0, \mathbf{I}) + \sum_l \log \left| \det \frac{\partial g_{\theta}^l}{\partial \mathbf{x}^l} \right| \quad (106)$$

其中 $\mathbf{h}^{(i)}$ 为对应于 $\mathbf{x}^{(i)}$ 的条件输入，通常为回看窗口的真值 $\mathbf{x}_{t-H+1}^t$ ， $g_{\theta}(\mathbf{x}; \mathbf{h})$ 是对输入 $\mathbf{x}$ 通过标准化流变换后的对应隐空间变量。

除了上述描述的基于分布建模的方法之外，也可采用构建不同的确定性预测器，并将不同的预测器的结果集成 (ensemble) 以获得经验分布的方法 [23]。

时间序列概率预测模型评估：在对概率时间预测进行评价时，除了使用评价确定性时间序列预测的指标之外，还有一些专用于评价时间序列概率预测的指标，如对于分位数的评价指标、对于概率分布的评价指标、以及对预测区间的评价指标等：

1. 分位数损失 (Quantile Loss, QL): 分位数损失描述分布在不同分位数处的预测准确性。假设 $\hat{\mathbf{x}}_\tau$ 是模型预测的 τ 分位点, $\mathbf{x}$ 为真实值, $\tau \in (0, 1)$ 为分位等级, 则分位数指标为:

$$QL_\tau = (\tau - \mathbb{I}_{\mathbf{x} < \hat{\mathbf{x}}_\tau}) \cdot (\mathbf{x} - \hat{\mathbf{x}}_\tau) \quad (107)$$

假设共有 N 个样本, 模型对 M 个不同分位点 $\{\tau_1, \tau_2, \dots, \tau_M\}$ 进行预测, 则整个数据集上的分位数损失为:

$$QL = \frac{1}{M} \sum_{m=1}^M \left[\frac{1}{N} \sum_{i=1}^N QL_{\tau_m}(\mathbf{x}_i, \hat{\mathbf{x}}_{i, \tau_m}) \right] \quad (108)$$

2. 连续分级概率评分 (Continuous Ranked Probability Score, CRPS) 量化了连续概率分布与确定性观测样本之间的差异, 假设 $F(\mathbf{z})$ 是预测的累积分布函数 (Cumulative Distribution Function, CDF):

$$\text{CRPS}(F, \mathbf{x}) = \int_{-\infty}^{\infty} (F(\mathbf{z}) - \mathbb{I}_{\mathbf{x} \leq \mathbf{z}})^2 d\mathbf{z} \quad (109)$$

对于无法获得解析形式的分布, 使用观测样本 $\{\hat{\mathbf{x}}^{(j)}\}_{j=1}^S \sim F$ 对 CRPS 进行近似, S 代表从 F 采样的次数, 即执行推理过程的次数:

$$\text{CRPS}(F, \mathbf{x}) \approx \frac{1}{S} \sum_{j=1}^S |\hat{\mathbf{x}}^{(j)} - \mathbf{x}| - \frac{1}{2S^2} \sum_{j=1}^S \sum_{k=1}^S |\hat{\mathbf{x}}^{(j)} - \hat{\mathbf{x}}^{(k)}| \quad (110)$$

3. 负对数似然 (Negative log-likelihood, NLL) 描述了预测的概率与真实值之间的差异, 假设模型预测的条件概率密度为 $p(\hat{\mathbf{x}}_{t+1}^{t+F} | \mathbf{x}_{t-H+1}^t; \theta)$:

$$\text{NLL} = - \sum_{i=1}^F \log p(\hat{\mathbf{x}}_{t+i} | \mathbf{x}_{t-H+1}^t; \theta) \quad (111)$$

4. 区间覆盖概率 (Prediction Interval Coverage Probability, PICP) 能够对预测区间对真实值的覆盖程度进行评估, 表示在多次预测之中, 真实值落在预测区间之中的概率, 假设总计包含 N 个样本, $\mathbf{x}_i^{\text{test}}$ 表示测试集 $\mathbf{D}_{\text{test}}$ 之中第 i 个真实值, $\hat{\mathbf{x}}_i^L, \hat{\mathbf{x}}_i^U$ 代表模型在一次预测过程之中所输出的对 $\mathbf{x}_i^{\text{test}}$ 的预测的上界和下界, 那么 PICP 的计算方式如下:

$$\text{PICP} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}_{[\mathbf{x}_i \in [\hat{\mathbf{x}}_i^L, \hat{\mathbf{x}}_i^U]]} \quad (112)$$

参考文献

- [1] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [2] Shane Bergsma, Timothy Zeyl, Javad Rahimipour Anaraki, and Lei Guo. C2FAR: coarse-to-fine autoregressive networks for precise probabilistic forecasting. In *Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] George EP Box and David A Pierce. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *Journal of the American Statistical Association (JASA)*, 65(332):1509–1526, 1970.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [5] Borui Cai, Shuiqiao Yang, Longxiang Gao, and Yong Xiang. Hybrid variational autoencoder for time series forecasting. *Knowl. Based Syst.*, 281:111079, 2023.
- [6] Wanlin Cai, Yuxuan Liang, Xianggen Liu, Jianshuai Feng, and Yuankai Wu. Ms-gnet: Learning multi-scale inter-series correlations for multivariate time series forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, volume 38, pages 11141–11149, 2024.
- [7] Jialin Chen, Jan Eric Lenssen, Aosong Feng, Weihua Hu, Matthias Fey, Leandros Tassioulas, Jure Leskovec, and Rex Ying. From similarity to superiority: Channel clustering for time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 130635–130663, 2024.
- [8] Peng Chen, Yingying Zhang, Yunyao Cheng, Yang Shu, Yihang Wang, Qingsong Wen, Bin Yang, and Chenjuan Guo. Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [9] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, pages 785–794, 2016.
- [10] Razvan-Gabriel Cirstea, Chenjuan Guo, Bin Yang, Tung Kieu, Xuanyi Dong, and Shirui Pan. Triformer: Triangular, variable-specific attentions for long se-

- quence multivariate time series forecasting. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1994–2001, 2022.
- [11] Razvan-Gabriel Cirstea, Tung Kieu, Chenjuan Guo, Bin Yang, and Sinno Jialin Pan. Enhancenet: Plugin neural networks for enhancing correlated time series forecasting. In *37th IEEE International Conference on Data Engineering (ICDE)*, pages 1739–1750, 2021.
- [12] Razvan-Gabriel Cirstea, Bin Yang, Chenjuan Guo, Tung Kieu, and Shirui Pan. Towards spatio-temporal aware traffic time series forecasting. In *International Conference on Data Engineering*, pages 2900–2913, 2022.
- [13] Tao Dai, Beiliang Wu, Peiyuan Liu, Naiqi Li, Jigang Bao, Yong Jiang, and Shu-Tao Xia. Periodicity decoupling framework for long-term series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [14] Vijay Ekambaram, Arindam Jati, Pankaj Dayama, Sumanta Mukherjee, Nam H Nguyen, Wesley M. Gifford, Chandra Reddy, and Jayant Kalagnanam. Tiny time mixers (TTMs): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. In *Proceedings of The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*, pages 74147–74181, 2024.
- [15] Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2023.
- [16] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [17] Shanghua Gao, Teddy Koker, Owen Queen, Thomas Hartvigsen, Theodoros Tsiligkaridis, and Marinka Zitnik. Units: Building a unified time series model. *arXiv preprint arXiv:2403.00131*, 2024.
- [18] Federico Garza, Max Mergenthaler Canseco, Cristian Challú, and Kin G Olivares. Statsforecast: Lightning fast forecasting with statistical and econometric models. In *Proceedings of the PyCon*, page 66, 2022.

- [19] Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. In *Proceedings of the Conference on Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Track on Datasets and Benchmarks)*, 2021.
- [20] Wenyong Han, Tao Zhu, Liming Chen, Huansheng Ning, Yang Luo, and Yaping Wan. Mcformer: Multivariate time series forecasting with mixed-channels transformer. *IEEE Internet of Things Journal (IoTJ)*, 11(17):28320–28329, 2024.
- [21] Qihe Huang, Lei Shen, Ruixin Zhang, Shouhong Ding, Binwu Wang, Zhengyang Zhou, and Yang Wang. Crossgmn: Confronting noisy multivariate time series via cross interaction refinement. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 46885–46902, 2023.
- [22] Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with exponential smoothing: the state space approach*. Springer Science & Business Media, 2008.
- [23] Vilde Jensen, Filippo Maria Bianchi, and Stian Normann Anfinsen. Ensemble conformalized quantile regression for probabilistic time series forecasting. *IEEE Trans. Neural Networks Learn. Syst.*, 35(7):9014–9025, 2024.
- [24] Maowei Jiang, Pengyu Zeng, Kai Wang, Huan Liu, Wenbo Chen, and Haoran Liu. Fecan: Frequency enhanced channel attention mechanism for time series forecasting. *Advanced Engineering Informatics*, 58:102158, 2023.
- [25] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 3146–3154, 2017.
- [26] Yaxuan Kong. Unlocking the power of LSTM for long term time series forecasting. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, pages 11968–11976, 2025.
- [27] Guokun Lai. Modeling long- and short-term temporal patterns with deep neural networks. In *International ACM SIGIR Conference on Research & Development in Information Retrieval, SIGIR 2018, Ann Arbor, MI, USA, July 08-12, 2018*, pages 95–104, 2018.

- [28] Zhichen Lai, Dalin Zhang, Huan Li, Christian S. Jensen, Hua Lu, and Yan Zhao. LightCTS: A lightweight framework for correlated time series forecasting (pacmmode). *Proceedings of the ACM on Management of Data*, 1(2):125:1–125:26, 2023.
- [29] Seunghan Lee, Taeyoung Park, and Kibok Lee. Partial channel dependence with channel masks for time series foundation model. In *NeurIPS Workshop on Time Series in the Age of Large Models (NeurIPS Workshop)*, 2024.
- [30] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyu Zhou, Wenhui Chen, Yu-Xiang Wang, and Xifeng Yan. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 5244–5254, 2019.
- [31] Zhe Li, Shiyi Qi, Yiduo Li, and Zenglin Xu. Revisiting long-term time series forecasting: An investigation on linear mapping. *arXiv preprint arXiv:2305.10721*, 2023.
- [32] Shengsheng Lin. Segrnn: Segment recurrent neural network for long-term time series forecasting. *CoRR*, abs/2308.11200, 2023.
- [33] Shengsheng Lin, Weiwei Lin, Wentai Wu, Haojun Chen, and Junjie Yang. SparseTSF: Modeling long-term time series forecasting with 1k parameters. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 30211–30226, 2024.
- [34] Minhao Liu, Ailing Zeng, Muxi Chen, Zhijian Xu, Qiuxia Lai, Lingna Ma, and Qiang Xu. SCINet: Time series modeling and forecasting with sample convolution and interaction. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 5816–5828, 2022.
- [35] Qinshuo Liu, Yanwen Fang, Pengtao Jiang, and Guodong Li. Dgcformer: Deep graph clustering transformer for multivariate time series forecasting. *arXiv preprint arXiv:2405.08440*, 2024.
- [36] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiyao Lin, Alex X. Liu, and Schahram Dustdar. Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

- [37] Yijing Liu, Qinxian Liu, Jian-Wei Zhang, Haozhe Feng, Zhongwei Wang, Zihan Zhou, and Wei Chen. Multivariate time-series forecasting with temporal polynomial graph neural networks. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 19414–19426, 2022.
- [38] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [39] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, volume 35, pages 9881–9893, 2022.
- [40] Donghao Luo and Xue Wang. ModernTCN: A modern pure convolution structure for general time series analysis. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2024.
- [41] Jie Mei, Dawei He, Ronald Harley, Thomas Habetler, and Guannan Qu. A random forest method for real-time price forecasting in new york electricity market. In *Proceedings of the IEEE Power & Energy Society General Meeting | Conference & Exposition (PES)*, pages 1–5, 2014.
- [42] Mohammadshirazi. Piad-srnn: Physics-informed adaptive decomposition in state-space rnn. In *ICML 2025 CO-BUILD Workshop on Computational Optimization of Buildings*, 2025.
- [43] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [44] Yao Qin. A dual-stage attention-based recurrent neural network for time series prediction. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2627–2633, 2017.
- [45] Xiangfei Qiu, Xingjian Wu, Yan Lin, Chenjuan Guo, Jilin Hu, and Bin Yang. Duet: Dual clustering enhanced multivariate time series forecasting. In *ACM SIGKDD*

- International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2025.
- [46] Syama Sundar Rangapuram, Matthias W. Seeger, Jan Gasthaus, Lorenzo Stella, Yuyang Wang, and Tim Januschowski. Deep state space models for time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 7796–7805, 2018.
 - [47] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *Proceedings of the 38th International Conference on Machine Learning, ICML*, pages 8857–8868, 2021.
 - [48] Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs M. Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. In *9th International Conference on Learning Representations, ICLR*, 2021.
 - [49] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
 - [50] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. Deepar: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.
 - [51] Rajat Sen, Hsiang-Fu Yu, and Inderjit S. Dhillon. Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 4838–4847, 2019.
 - [52] Mohammad Amin Shabani, Amir H. Abdi, Lili Meng, and Tristan Sylvain. Scale-former: Iterative multi-scale refining transformers for time series forecasting. In *Proceedings of The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
 - [53] Chao Shang, Jie Chen, and Jinbo Bi. Discrete graph structure learning for forecasting multiple time series. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2021.

- [54] Lifeng Shen, Weiyu Chen, and James T. Kwok. Multi-resolution diffusion models for time series forecasting. In *The Twelfth International Conference on Learning Representations, ICLR*, 2024.
- [55] Sophia Huiwen Sun and Rose Yu. Copula conformal prediction for multi-step time series prediction. In *The Twelfth International Conference on Learning Representations, ICLR*, 2024.
- [56] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015.
- [57] William Toner and Luke Nicholas Darlow. An analysis of linear time series forecasting models. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 48404–48427, 2024.
- [58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [59] Huiqiang Wang, Jian Peng, Feihu Huang, Jince Wang, Junhui Chen, and Yifei Xiao. MICN: multi-scale local and global context modeling for long-term series forecasting. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2023.
- [60] Shiyu Wang, Jiawei Li, Xiaoming Shi, Zhou Ye, Baichuan Mo, Wenze Lin, Shengtong Ju, Zhixuan Chu, and Ming Jin. TimeMixer++: A general time series pattern machine for universal predictive analysis. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2025.
- [61] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. TimeMixer: Decomposable multiscale mixing for time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [62] Xue Wang, Tian Zhou, Qingsong Wen, Jinyang Gao, Bolin Ding, and Rong Jin. Card: Channel aligned robust blend transformer for time series forecasting.

- In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2024.
- [63] Yucheng Wang, Yuecong Xu, Jianfei Yang, Min Wu, Xiaoli Li, Lihua Xie, and Zhenghua Chen. Fully-connected spatial-temporal graph for multivariate time-series data. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 15715–15724, 2024.
- [64] Ruofeng Wen, Kari Torkkola, Balakrishnan Narayanaswamy, and Dhruv Madeka. A multi-horizon quantile recurrent forecaster. *CoRR*, abs/1711.11053, 2018.
- [65] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 53140–53164, 2024.
- [66] Gerald Woo, Chenghao Liu, Doyen Sahoo, Akshat Kumar, and Steven Hoi. Etsformer: Exponential smoothing transformers for time-series forecasting. *arXiv preprint arXiv:2202.01381*, 2022.
- [67] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. TimesNet: Temporal 2D-variation modeling for general time series analysis. In *Proceedings of The International Conference on Learning Representations (ICLR)*, 2023.
- [68] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 22419–22430, 2021.
- [69] Jinming Wu, Qi Qi, Jingyu Wang, Haifeng Sun, Zhikang Wu, Zirui Zhuang, and Jianxin Liao. Not only pairwise relationships: Fine-grained relational modeling for multivariate time series forecasting. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4416–4423, 2023.
- [70] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. Connecting the dots: Multivariate time series forecasting with graph neural networks. In *SIGKDD*, pages 753–763, 2020.

- [71] Zhijian Xu, Ailing Zeng, and Qiang Xu. FITS: Modeling time series with 10k parameters. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [72] Vijaya Krishna Yalavarthi, Randolph Scholz, Stefan Born, and Lars Schmidt-Thieme. Probabilistic forecasting of irregularly sampled time series with missing values via conditional normalizing flows. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 21877–21885, 2025.
- [73] Fuhao Yang, Xin Li, Min Wang, Hongyu Zang, Wei Pang, and Mingzhong Wang. Waveform: Graph enhanced wavelet learning for long sequence forecasting of multivariate time series. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, volume 37, pages 10754–10761, 2023.
- [74] Yingnan Yang, Qingling Zhu, and Jianyong Chen. Vcformer: Variable correlation transformer with inherent lagged correlation for multivariate time series forecasting. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 5335–5343, 2024.
- [75] Junchen Ye, Zihan Liu, Bowen Du, Leilei Sun, Weimiao Li, Yanjie Fu, and Hui Xiong. Learning the evolutionary and multi-scale graph structure for multivariate time series forecasting. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*, page 2296–2306, 2022.
- [76] Kun Yi, Qi Zhang, Wei Fan, Hui He, Liang Hu, Pengyang Wang, Ning An, Longbing Cao, and Zhendong Niu. Fouriergnn: Rethinking multivariate time series forecasting from a pure graph perspective. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 69638–69660, 2023.
- [77] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, volume 37, pages 11121–11128, 2023.
- [78] Lingyu Zhang, Wenjie Bian, Wenyi Qu, Liheng Tuo, and Yunhai Wang. Time series forecast of sales volume based on xgboost. *Journal of Physics: Conference Series (JPCS)*, 1873(1):012067, 2021.
- [79] Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

- [80] Kai Zhao, Chenjuan Guo, Yunyao Cheng, Peng Han, Miao Zhang, and Bin Yang. Multiple time series forecasting with dynamic graph modeling. *Proceedings of the VLDB Endowment (PVLDB)*, 17(4):753–765, 2023.
- [81] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, pages 11106–11115, 2021.
- [82] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *Proceedings of International Conference on Machine Learning (ICML)*, pages 27268–27286, 2022.
- [83] Xuanbing Zhu, Dunbin Shen, Hongyu Wang, and Yingguang Hao. Fcnet: Fully complex network for time series forecasting. *IEEE Internet of Things Journal (IOTJ)*, 11(24):40127–40139, 2024.

4 时间序列异常检测

4.1-4.3 节作者为：金健新、严思雨、郭晨娟、杨彬；

4.4 节作者为：金健新、严思雨、李北步、申屠琦超、胡诗彦、郭晨娟、杨彬。

本章主编为郭晨娟。

本章主要介绍时间序列异常检测任务的相关知识。时间序列异常检测旨在识别时间序列中偏离正常模式的时刻或片段，这些异常往往对应潜在的故障、风险或关键事件。根据是否依赖已标注的异常样本，时间序列异常检测可分为两类：**有监督异常检测 (Supervised Anomaly Detection)** 和 **无监督异常检测 (Unsupervised Anomaly Detection)**。有监督方法利用包含正常与异常标签的历史数据进行训练，能够直接学习区分两者的判别边界，适用于异常样本充足且分布稳定的场景（如工业质量检测中已知类型的缺陷识别）；无监督方法则无需异常标签，通过建模正常模式并检测偏离程度来识别异常，更适用于异常样本稀少或未知类型的场景（如设备早期故障预警或网络安全入侵检测）。

4.1 时间序列异常检测定义及流程

时间序列异常检测 (Time Series Anomaly Detection) 是指通过分析时间序列数据来判断哪个时间段或时刻发生异常。随着各种传感器的广泛应用，时间序列数据在许多生产、生活场景中变得无处不在。有效地检测这些时间序列中的异常现象对于及时发现潜在问题、采取必要措施以保证系统的正常运行至关重要，从而避免可能的经济损失和安全威胁。

考虑一条长度为 T ，维度为 N 的时间序列： $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times N}$ ，每个数据点标注为 $\mathbf{x}_t \in \mathbb{R}^{1 \times N}$ 。时间序列异常检测任务可以定义为得到一个检测结果 $\mathbf{Y} = (\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_T)$ 来判断每个时刻是否发生异常，其中 $\mathbf{y}_t \in \{0, 1\}$ ，0 表示正常数据点，1 表示异常数据点。可以用公式表示为：

$$\mathbf{Y} = f_{\theta}(\mathbf{X}) \quad (113)$$

其中， $f(\cdot)$ 是某种用于异常检测的映射函数， θ 是可学习参数。

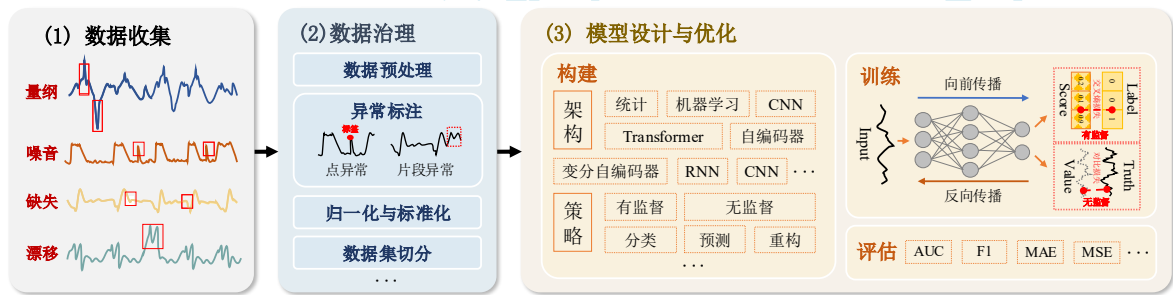


图 4.1: 异常检测流程图。

时间序列异常检测任务可以根据不同的检测方法进行划分。首先，可以根据是否需要标注数据分为有监督异常检测和无监督异常检测。有监督检测依赖于标注的异常数据，通过学习正常和异常模式来识别异常，例如使用过去已识别的机器故障数据来检测新的故障事件；无监督检测则无需标注数据，直接通过数据的内在结构识别异常，

例如通过分析网络流量的统计特征来检测异常访问模式。此外，还可以根据检测方法的原理分为基于预测的方法、基于重构的方法和基于分类的方法。其中基于重构的方法和基于预测的方法通常属于无监督学习，如Anomaly Transformer [66]、STOC [35]等；基于分类的方法通常是有监督学习，如TCQSA [41]等。基于预测的方法通过预测未来的时间序列值，并比较实际值与预测值的差异来检测异常，适用于具有较强时间依赖关系的序列；基于重构的方法则通过重构现有数据并分析重构误差来识别异常，适用于复杂模式的数据；基于分类的方法直接输出数据标签，通过将数据点分为正常或异常类别来检测异常，适用于拥有明确分类边界的场景。

图 4.1展示了时间序列异常检测任务的流程，通常包括以下几个步骤：1) 数据收集，从传感器等设备获取原始数据；2) 数据预处理，执行去噪、归一化、填补缺失值等操作；3) 模型设计与优化，选择合适的模型架构（如MLP、RNN、CNN等），通过前向传播和反向传播优化模型参数，学习训练数据集的变化规律，在模型训练完成后，使用测试集进行性能评估，常用的评估指标包括均方误差损失（MSE）、交叉熵损失（Cross Entropy）；4) 将模型部署到服务器进行实际应用，得到检测结果；5) 根据检测结果采取相应措施，如发出警报或启动故障处理程序。

4.2 时间序列异常类型

异常定义为相对于正常数据的偏差。如图 4.2所示，根据异常的表现形式，可以将时间序列异常划分为两大类：点异常（Point Anomaly）和子序列异常（Subsequence Anomaly）。

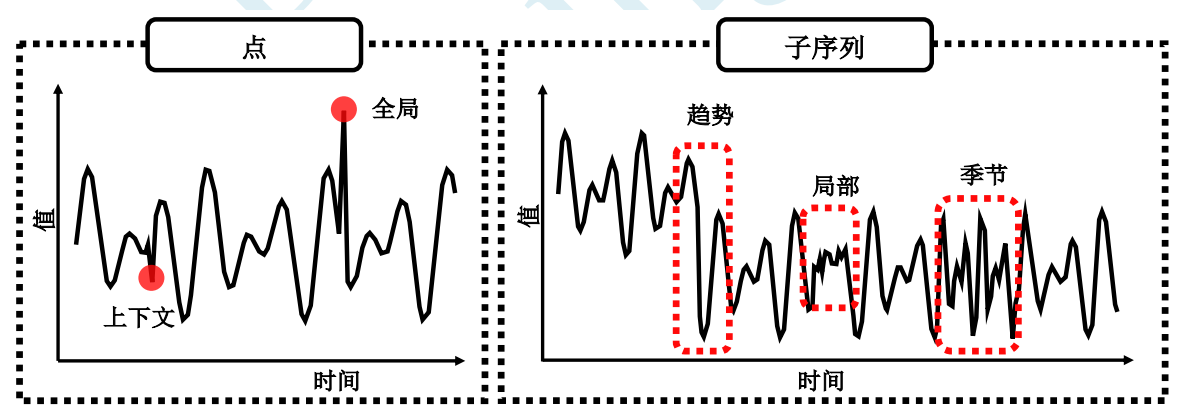


图 4.2: 时间序列异常类型。

点异常关注的是单个时间点上的不寻常行为，这又可细分为全局异常（Global Anomaly）与上下文异常（Contextual Anomaly）两种情况。全局异常指的是在整体时间序列中显著偏离正常范围的单一值，它们通常以尖峰或者突变的形式出现；上下文异常更加隐蔽，这类异常表现为在一个特定的时间窗口内与其他相邻数据点相比具

有明显差异，但在整个时间序列范围内可能并不显眼，需要通过对比局部环境才能有效识别出来。

子序列异常是指在一定长度的时间段内发生的变化，这些变化不能简单归结为个别数据点的问题，它进一步细分为三类 [68]：局部异常 (Shapelet Anomaly)、季节性异常 (Seasonal Anomaly) 以及趋势异常 (Trend Anomaly)。其中，局部异常指的是与时间序列中频繁出现的标准形状存在较大差异的连续片段；季节性异常是指那些表现出不同于常规季节性模式的行为的子序列，即使它们保持了相似的整体形状和趋势；当某个子序列导致了整条时间序列长期发展趋势的重大改变时，则被认为是趋势异常，在这种情况下，虽然原有的季节性和形态特征得以保留，但是趋势线的斜率出现了显著变动，从而引发了数据均值的持续偏移现象。

4.3 时间序列异常检测模型训练与评估

由于异常数据在实际应用中非常罕见，这使得异常时序数据的收集和标注变得极其困难。考虑到异常情况的多样性和复杂性，几乎不可能穷尽地标注所有潜在的异常模式，因此限制了有监督异常检测技术的适用场景，而无监督异常检测技术不需要异常数据的标注，成为一种实用且有效的解决方案被广泛研究和应用。无监督异常检测方法不需要依赖大量标注的异常数据，而是通过学习正常数据的模式来识别与之显著不同的数据点或模式作为异常。

与此同时，为了克服有监督异常检测场景下的异常数据标注不足的问题，一些研究工作尝试采用异常注入的方法来增强训练数据集。这种方法通过人为地向数据集中添加各种类型的异常样本，从而为监督学习模型提供更多的训练数据。然而，这种策略的成功很大程度上取决于所注入异常的真实性和多样性，以及如何平衡正常数据与异常数据的比例，以避免模型过拟合到特定的人造异常模式上去。

本小节将探讨时间序列异常检测的方法。首先，我们介绍有监督和无监督的检测方式。有监督方法的优点在于其高准确性，因为它可以从标注数据中提取具体的异常特征。然而，其缺点在于需要大量的异常标注数据，异常标注过程可能昂贵且耗时，并且模型可能会过拟合于特定的异常模式 [52]。无监督方法的优点在于不需要异常标注数据，这降低了数据准备的成本和时间。损失函数的选择和误差阈值的设定对检测结果有重要影响，需要仔细设计和调节。如图 4.3 所示，按照检测机制的不同，我们分别讨论基于分类 (4.3.1 节)、基于重构 (4.3.2 节) 和基于预测 (4.3.3 节) 的异常检测方法。最后，简要概述其他创新性方法的应用与发展。我们以点异常为例展示了时间序列异常检测的方法，而子序列异常是由多个点异常在时间上聚集形成的。

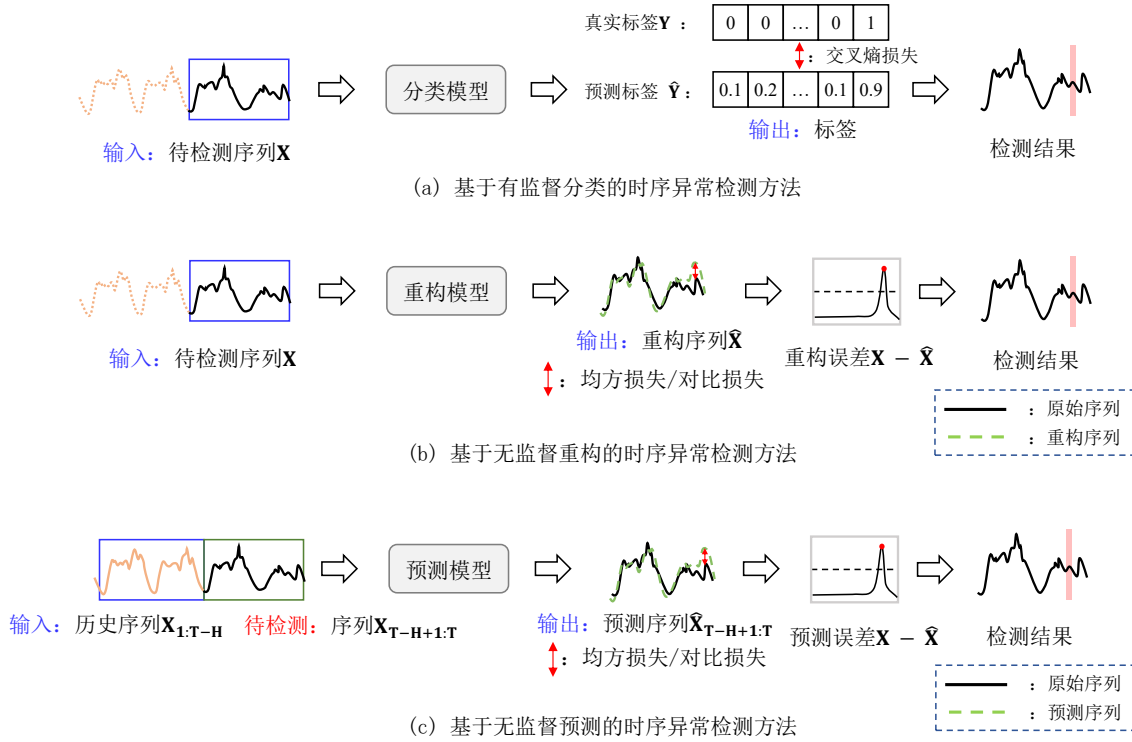


图 4.3: 异常检测方法总览。

4.3.1 基于有监督分类的异常检测方法

有监督的时间序列异常检测是一种依赖标注数据集来训练模型的方法。用于训练的标注数据集中包括正常和异常样本，通过使用分类算法（如支持向量机、决策树）或神经网络（如 CNN、RNN），学习这些样本的特征，从而具有区分正常和异常数据的能力。

具体来说，需要以一定的比例进行数据集划分，准备带有标签的训练集数据 $\mathbf{D}_{train} = \{\mathbf{X}_i^{train}, \mathbf{Y}_i^{train}\}_{i=1}^{T_{train}}$ ，验证集数据 $\mathbf{D}_{val} = \{\mathbf{X}_i^{val}, \mathbf{Y}_i^{val}\}_{i=1}^{T_{val}}$ 和测试集数据 $\mathbf{D}_{test} = \{\mathbf{X}_i^{test}, \mathbf{Y}_i^{test}\}_{i=1}^{T_{test}}$ 。以训练集为例， $\mathbf{X}_i^{train} = (\mathbf{x}_{t-C+1}^{train}, \mathbf{x}_{t-C+2}^{train}, \dots, \mathbf{x}_t^{train}) \in \mathbb{R}^{C \times N}$ 为训练集中第 i 个样本的输入序列， $\mathbf{Y}_i^{train} = (\mathbf{y}_{t-C+1}^{train}, \mathbf{y}_{t-C+2}^{train}, \dots, \mathbf{y}_t^{train}) \in \mathbb{R}^{C \times N}$ 为其对应的标签， C 是输入序列长度， T_{train} 是整个训练集的样本数量。数据集中每个时间点或子序列都被标注为正常或异常。这些标签通常由专家标注或通过规则识别。每个数据点被表示为 $\mathbf{x}_t^{train}$ ，以及其对应的标签 $\mathbf{y}_t^{train}$ ，其中 $\mathbf{y}_t^{train}$ 为 0 表示正常，1 表示异常。如图 4.3(a) 所示，随后，将在 $\mathbf{D}_{train}$ 、 $\mathbf{D}_{val}$ 和 $\mathbf{D}_{test}$ 上顺序执行以下两个阶段：

1. **训练阶段**：训练阶段的目的是通过学习时间序列数据中的正常与异常的特征和规律，以建立一个能够区分正常与异常数据的模型 f_θ 。具体来说，首先是**数据分析**，模型通过对待分类序列 $\{\mathbf{x}_i^{train}\}_{i=t-C+1}^t \in \mathbb{R}^{C \times N}$ 进行分析，从中提取关键特征信息，如序列的趋势性与季节性变化规律、多变量序列间的相互依赖关系、多尺

度的复杂时间模式等。

异常点分类模型训练：利用准备好的数据和标签训练点异常模型 f_θ ，模型学习时间序列中的规律性，并区别正常和异常样本的特征，输出分类标签 $\{\hat{y}_i^{train}\}_{i=t-C}^t \in \mathbb{R}^{C \times N}$ 。训练过程中采用交叉熵损失来优化模型，损失函数公式如下：

$$\mathcal{L} = - \sum_{i=t-C+1}^t \left[y_i^{train} \log(\hat{y}_i^{train}) + (1 - y_i^{train}) \log(1 - \hat{y}_i^{train}) \right] \quad (114)$$

其中 $\hat{y}_i^{train}$ 是模型输出序列的异常概率。经过优化，训练完成的模型可以准确分类正常和异常数据。

需要注意的是，在模型训练过程中，出于避免模型 f_θ 在 $\mathbf{D}_{train}$ 过拟合或者筛选最优超参数等目的，通常使用验证集 $\mathbf{D}_{val}$ 判断模型 f_θ 性能，获得**最优模型** f_{θ^*} ，详见公式 (164)。

2. **检测阶段：**检测阶段的目的是使用训练好的最优模型 f_{θ^*} 去识别和分类测试集数据 $\mathbf{D}_{test}$ 中的序列 $\{\mathbf{x}_i^{test}\}_{i=t-C+1}^t \in \mathbb{R}^{C \times N}$ 并输出测试结果 $\{\hat{y}_i^{test}\}_{i=t-C+1}^t \in \mathbb{R}^{C \times N}$ 以实现实时或批处理的异常检测。

判断异常：根据分类模型的输出结果，如果某个数据点的标签 $\hat{y}_i^{test}$ 大于预设的阈值，则认为该数据点是异常的；否则 $\hat{y}_i^{test} = 0$ 认为该数据点是正常的。

评价指标：通常，**F1 分数**等指标被用于评估检测效果，其公式如下所示：

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (115)$$

还有常用的指标包括：

- (a) **准确率 (accuracy)：**预测对的总次数占所有预测次数的比例

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (116)$$

- (b) **精确率 (precision)：**预测为异常中，真正异常的比例

$$\text{Precision} = \frac{TP}{TP + FP} \quad (117)$$

- (c) **召回率 (recall)：**实际异常中，被预测出来的比例

$$\text{Recall} = \frac{TP}{TP + FN} \quad (118)$$

等。其中，在二分类任务中，模型的预测结果可以分为以下四种情况：

- **真正例 (True Positive, TP)**: 实际标签 $y_i^{\text{test}} = 1$, 且模型预测为 $\hat{y}_i^{\text{test}} = 1$ 的样本数量。
- **真负例 (True Negative, TN)**: 实际标签 $y_i^{\text{test}} = 0$, 且模型预测为 $\hat{y}_i^{\text{test}} = 0$ 的样本数量。
- **假正例 (False Positive, FP)**: 实际标签 $y_i^{\text{test}} = 0$, 但模型预测为 $\hat{y}_i^{\text{test}} = 1$ 的样本数量 (即 I 类错误)。
- **假负例 (False Negative, FN)**: 实际标签 $y_i^{\text{test}} = 1$, 但模型预测为 $\hat{y}_i^{\text{test}} = 0$ 的样本数量 (即 II 类错误)。

因此, 检测结果中的 TP、TN、FP、FN 均是通过对比模型预测标签 $\hat{y}_i^{\text{test}}$ 与真实标签 y_i^{test} , 按照上述定义统计得到的。基于这些统计量即可进一步计算 F1 分数、准确率、精确率与召回率等指标, 从而全面评估模型的检测性能。

4.3.2 基于无监督重构的异常检测方法

无监督的时间序列异常检测是一种无需标注数据的方法。该方法通过探索时间序列的内在特性 (如规律性、关联性) 或基于重构误差、分布偏差等标准, 自动识别异常点或模式的偏离。相比有监督方法, 无监督方法无需大量标注数据, 适用于大规模时间序列异常检测任务, 但在检测精度上可能受模型泛化能力的影响, 从而导致部分异常难以准确识别。基于重构的时间序列异常检测方法利用深度学习模型通过重构输入数据来学习时间序列数据的正常模式。此方法依赖于通过模型学习正常序列的特征, 从而在检测阶段能够识别与正常模式不符的异常序列。

具体来说, 需要以一定的比例进行**数据集划分**, 准备带有标签的训练集数据 $\mathbf{D}_{\text{train}} = \{\mathbf{X}_i^{\text{train}}\}_{i=1}^{T_{\text{train}}}$, 验证集数据 $\mathbf{D}_{\text{val}} = \{\mathbf{X}_i^{\text{val}}\}_{i=1}^{T_{\text{val}}}$ 和测试集数据 $\mathbf{D}_{\text{test}} = \{\mathbf{X}_i^{\text{test}}, \mathbf{Y}_i^{\text{test}}\}_{i=1}^{T_{\text{test}}}$ 。以训练集为例, $\mathbf{X}_i^{\text{train}} = (\mathbf{x}_{t-R+1}^{\text{train}}, \mathbf{x}_{t-R+2}^{\text{train}}, \dots, \mathbf{x}_t^{\text{train}}) \in \mathbb{R}^{R \times N}$ 为训练集中第 i 个样本的输入序列, R 是待重构训练序列长度, T_{train} 是整个训练集的样本数量。每个数据点被表示为 $\mathbf{x}_t^{\text{train}}$ 。如图 4.3(b) 所示, 随后, 将在 $\mathbf{D}_{\text{train}}$ 、 $\mathbf{D}_{\text{val}}$ 和 $\mathbf{D}_{\text{test}}$ 上顺序执行以下两个阶段:

1. **训练阶段**: 训练阶段的目的是通过学习时间序列数据中的特征和规律, 以建立一个能够区分正常与异常数据的模型 f_θ 。具体来说, 首先是**数据分析**, 模型通过对待分类序列 $\{\mathbf{x}_i^{\text{train}}\}_{i=t-R+1}^t \in \mathbb{R}^{R \times N}$ 进行分析, 从中提取关键特征信息, 如序列的趋势性与季节性变化规律、多变量序列间的相互依赖关系、多尺度的复杂时间模式等。

异常点重构模型训练: 使用训练集 $\mathbf{D}_{\text{train}}$ 训练一个重构模型 f_θ 。模型通过内部的非线性变换学习到正常时间序列的规律, 尝试生成与输入序列尽可能相似的重构序列 $\{\hat{\mathbf{x}}_i^{\text{train}}\}_{i=t-R+1}^t \in \mathbb{R}^{R \times N}$ 。

训练过程中的损失函数: 在训练过程中, 常见的损失函数包括均方误差 (MSE) 或平均绝对误差 (MAE)。

(a) **均方误差 (MSE)** 衡量预测值与真实值的平方误差均值, 对大误差敏感:

$$\mathcal{L} = \frac{1}{R} \sum_{i=t-R+1}^t (\mathbf{x}_i^{train} - \hat{\mathbf{x}}_i^{train})^2 \quad (119)$$

(b) **平均绝对误差 (MAE)** 反映绝对误差的平均值, 对异常值鲁棒:

$$\mathcal{L} = \frac{1}{R} \sum_{i=t-R+1}^t |\mathbf{x}_i^{train} - \hat{\mathbf{x}}_i^{train}| \quad (120)$$

通过优化该损失函数, 模型学习到如何生成更准确的正常模式表示。

同样, 在模型训练过程中, 出于避免模型 f_θ 在 $\mathbf{D}_{train}$ 过拟合或者筛选最优超参数等目的, 通常使用验证集 $\mathbf{D}_{val}$ 判断模型 f_θ 性能, 获得**最优模型** f_{θ^*}

2. **检测阶段:** 检测阶段的目的是使用训练好的最优模型 f_{θ^*} , 根据测试集数据 $\mathbf{D}_{test}$ 中的历史序列 $\{\mathbf{x}_i^{test}\}_{i=t-R+1}^t \in \mathbb{R}^{R \times N}$, 预测未来结果 $\{\hat{\mathbf{x}}_i^{test}\}_{i=t-R+1}^t \in \mathbb{R}^{R \times N}$ 以实现实时或批处理的异常检测, 通过计算重构值与真实值之间的误差来量化模型的异常检测性能。

检测阶段的异常得分: 在检测阶段, 模型计算重构误差, 即原始输入序列 $\{\mathbf{x}_i^{test}\}_{i=1}^{T_{test}}$ 与重构序列 $\{\hat{\mathbf{x}}_i^{test}\}_{i=1}^{T_{test}}$ 之间的差异。这个差异即为重构误差, 可以通过均方误差 (MSE)、平均绝对误差 (MAE) 或其他相似度指标来衡量。

(a) **均方误差 (MSE)** 衡量预测值与真实值的平方误差均值, 对大误差敏感:

$$\text{Score} = \frac{1}{R} \sum_{i=t-R+1}^t (\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test})^2 \quad (121)$$

(b) **平均绝对误差 (MAE)** 反映绝对误差的平均值, 对异常值鲁棒:

$$\text{Score} = \frac{1}{R} \sum_{i=t-R+1}^t |\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test}| \quad (122)$$

在实际应用中, 通常计算每个时间点或时间窗口的误差, 然后根据这些误差评估异常得分。

异常点判定: 在获得每个时间点的异常得分后, 可通过设定阈值对各个时间点进行异常判定。异常得分高于阈值的时间点被视为异常点, 而低于阈值的则被认为是正常点。阈值的设定方式灵活多样, 可以是根据经验预先定义的固定数值, 也可以是基于数据分布设定的比例阈值 (例如将得分最高的前 1% 样本识别为异常点), 还可以通过自动化方法 (如 SPOT 算法 [58]) 根据异常得分分布进行自适应计算。

评价指标: 详见 4.3.1 小节的评价指标。

4.3.3 基于无监督预测的异常检测方法

与基于重构的方法不同，基于预测的异常检测方法通过使用历史数据来预测未来的时间点，并通过比较预测值与实际观测值的差异来训练检测模型。其核心理念是，如果模型一般预测未来的数据点的发展趋势，那么预测数据点与实际观测点的偏差越大，就说明该点越可能是异常点，

具体来说，需要以一定的比例进行**数据集划分**，得到训练集 $\mathbf{D}_{train} = \{\mathbf{x}_i^{train}\}_{i=1}^{T_{train}}$ 、验证集 $\mathbf{D}_{val} = \{\mathbf{x}_i^{val}\}_{i=1}^{T_{val}}$ 和测试集 $\mathbf{D}_{test} = \{\mathbf{x}_i^{test}, \mathbf{y}_i^{test}\}_{i=1}^{T_{test}}$ 。以训练集为例， $\mathbf{x}_i^{train} = (\mathbf{x}_{t-H+1}^{train}, \mathbf{x}_{t-H+2}^{train}, \dots, \mathbf{x}_t^{train}, \mathbf{x}_{t+1}^{train}, \dots, \mathbf{x}_{t+F}^{train}) \in \mathbb{R}^{(H+F) \times N}$ 为训练集中第 i 个样本的输入序列， H 、 F 分别是历史序列长度和未来序列长度， T_{train} 是整个测试集的样本数量。如图 4.3(c) 所示，随后，将在 $\mathbf{D}_{train}$ 、 $\mathbf{D}_{val}$ 和 $\mathbf{D}_{test}$ 上顺序执行以下两个阶段：

1. **训练阶段**：训练阶段的目的是通过学习时间序列数据中的特征和规律，以建立一个能够区分正常与异常数据的模型 f_θ 。具体来说，首先是**数据分析**，模型通过对待预测序列 $\{\mathbf{x}_i^{train}\}_{i=t-H+1}^t \in \mathbb{R}^{H \times N}$ 进行分析，从中提取关键特征信息，如序列的趋势性与季节性变化规律、多变量序列间的相互依赖关系、多尺度的复杂时间模式等。**异常点预测模型训练**：使用训练集 $\mathbf{D}_{train}$ 训练一个预测模型 f_θ 。模型通过内部的非线性变换学习到正常时间序列的规律，尝试生成与待预测序列尽可能相似的输出序列 $\{\hat{\mathbf{x}}_i^{train}\}_{i=t+1}^{t+F} \in \mathbb{R}^{F \times N}$ 。**训练过程中的损失函数**：与基于无监督重构方法类似，基于无监督预测的方法在训练过程中，常见的损失函数也包括均方误差（MSE）或平均绝对误差（MAE），只不过，在基于无监督预测的异常检测方法中，损失函数被运用在模型预测的未来序列和真实的未来序列。

(a) **均方误差（MSE）** 衡量预测值与真实值的平方误差均值，对大误差敏感：

$$\mathcal{L} = \frac{1}{F} \sum_{i=t+1}^{t+F} (\mathbf{x}_i^{train} - \hat{\mathbf{x}}_i^{train})^2 \quad (123)$$

(b) **平均绝对误差（MAE）** 反映绝对误差的平均值，对异常值鲁棒：

$$\mathcal{L} = \frac{1}{F} \sum_{i=t+1}^{t+F} |\mathbf{x}_i^{train} - \hat{\mathbf{x}}_i^{train}| \quad (124)$$

同样，在模型训练过程中，出于避免模型 f_θ 在 $\mathbf{D}_{train}$ 过拟合或者筛选最优超参数等目的，通常使用验证集 $\mathbf{D}_{val}$ 判断模型 f_θ 性能，获得**最优模型** f_{θ^*} ，详见公式 (164)。

2. **检测阶段**：检测阶段的目的是使用训练好的最优模型 f_{θ^*} 去识别和预测测试集数据 $\mathbf{D}_{test}$ 中的序列 $\{\mathbf{x}_i^{test}\}_{i=t-H+1}^t \in \mathbb{R}^{H \times N}$ 并输出测试结果 $\{\hat{\mathbf{x}}_i^{test}\}_{i=t+1}^{t+F} \in \mathbb{R}^{F \times N}$ 以实现实时或批处理的异常检测，通过计算预测值与真实值之间的误差来量化模型的预测性能。

计算异常得分：计算实际观测序列 $\{\mathbf{x}_i^{test}\}_{i=1}^{T_{test}}$ 与模型预测的序列 $\{\hat{\mathbf{x}}_i^{test}\}_{i=1}^{T_{test}}$ 之间的差异，与无监督重构类似，常见的差异度量包括均方误差（MSE）或平均绝对误差（MAE）。

(a) **均方误差（MSE）** 衡量预测值与真实值的平方误差均值，对大误差敏感：

$$\text{Score} = \frac{1}{F} \sum_{i=t+1}^{t+F} (\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test})^2 \quad (125)$$

(b) **平均绝对误差（MAE）** 反映绝对误差的平均值，对异常值鲁棒：

$$\text{Score} = \frac{1}{F} \sum_{i=t+1}^{t+F} |\mathbf{x}_i^{test} - \hat{\mathbf{x}}_i^{test}| \quad (126)$$

异常点判定：与基于无监督重构的异常检测方法类似。

评价指标：详见 4.3.1 小节的评价指标。

4.3.4 其他异常检测方法

除了基于分类、基于重构、基于预测的时间序列异常检测方法外，还有其他创新的技术，如 Yang 等人 [68] 使用的对比学习方法，它利用对比学习框架，通过最大化正常样本间的相似度和最小化正常样本与异常样本之间的差异，来识别异常。在这一过程中，模型通过学习时间序列的对比特征，将正常和异常数据映射到一个潜在空间中，从而有效区分异常点。对比学习方法不依赖于传统的标签数据，而是通过设计合适的损失函数和训练策略，使得模型能够在无标签的情况下进行有效的异常检测。这种方法对于处理复杂的、没有明确标注的时间序列异常检测任务非常有效。

4.4 时间序列异常检测模型

4.4.1 统计方法

统计方法通常通过构建模型来刻画时间序列的正常行为模式。这类方法依赖于时间序列自身的特性，例如趋势性、周期性和随机性。经典的统计方法 [67] 包括移动平均、指数平滑 [54] 和自回归积分滑动平均模型（ARIMA）[4]。这些方法适合处理单变量时间序列，通过预测模型的残差或显著偏离正常模式的数据点来识别异常。统计方法的优势在于其理论基础扎实，易于解释，但它们在多变量或非线性时间序列中的表现有限。

4.4.2 经典机器学习方法

经典机器学习方法从样本的特征分布或相似性中挖掘异常模式，广泛应用于多种场景。例如，聚类 [32, 48] 算法（如 k-means）将时间序列窗口分为不同的聚类，距离正常聚类中心较远或属于小型聚类的点可能被视为异常。基于密度 [5] 的方法（如局部异常因子 LOF）评估样本周围的密度，低密度点被检测为异常点。基于分区的方法 [42, 33]（如隔离森林 ISF）通过递归分割空间来区分异常观测值。基于分类的方法则利用分类器 [46]（如支持向量机 SVM）将偏离正常范围的样本标注为异常。基于相似性度量的方法 [69]（如 Matrix Profile）计算时间序列子序列之间的全局最近邻距离，距离较大的子序列（discords）被视为异常。这些方法在处理复杂模式和高维数据方面表现优越，但通常需要较多的超参数调整和训练数据。

4.4.3 卷积神经网络

卷积神经网络（Convolutional Neural Networks, CNN）最初在计算机视觉领域取得了显著成功，随后，越来越多的工作将 CNN 引入到时间序列异常检测工作当中，CNN 在异常检测中的核心优势在于其通过卷积操作能够高效地建模时间序列数据中的局部特征。如图 4.4 所示，基于 CNN 的异常检测方法主要可分为无监督和有监督两大类。具体而言，图 4.4(a) 展示了无监督学习方法，这类方法通常基于重构任务实现异常检测；图 4.4(c) 呈现了有监督学习方法，其核心思想是通过分类任务进行异常识别；此外，图 4.4(b) 重点展示了利用 CNN 提取时序特征的典型网络架构，其中包括一维卷积和二维卷积两种主要结构。

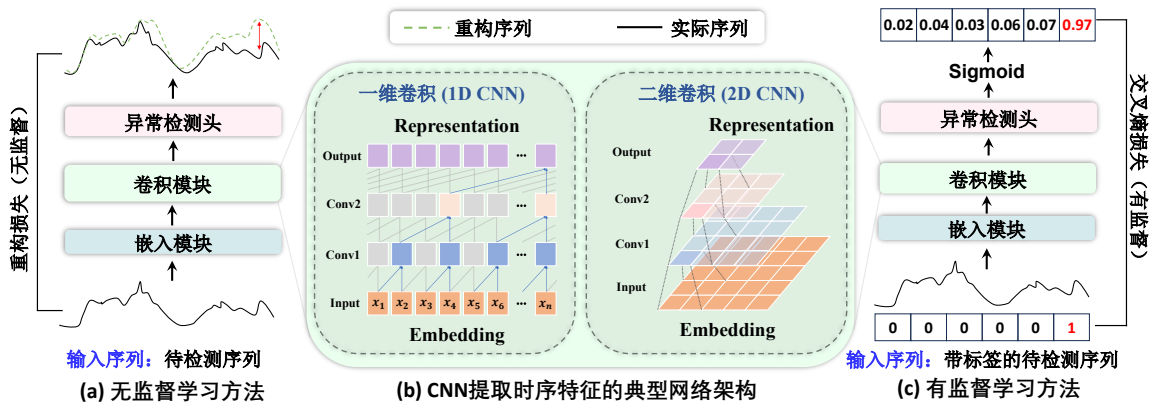


图 4.4: CNN 异常检测

基于一维卷积时间序列异常检测模型 1D-CNN 通过滑动窗口的方式高效提取时间依赖特征，并能利用并行计算提高推理速度。一维卷积直接在时间维度上进行卷积操作，适用于处理单变量时间序列，简单来说，给定一段时间序列 $\mathbf{X} = (x_1, x_2, \dots, x_T)$ ，(其中

$\mathbf{x}_t \in \mathbb{R}^{T \times N}$, N 是变量个数), 首先通过嵌入模块将其映射到隐藏空间, 随后, 将嵌入模块输出的向量表示传递到卷积层, 具体卷积过程可参考公式 (80) 和公式 (81), 其操作本质上是用固定大小的卷积核在时间序列上滑动, 每次计算一个局部特征。通过多个卷积核, 一维卷积能够学习不同时间模式, 如周期性波动、趋势变化等。此外, 多层卷积可以提取更高层次的特征。例如, 第一层可能捕获短时模式, 而更深层的卷积可以整合多个短时模式, 形成长期依赖特征。

因果卷积是在一维卷积中的一种变体, 它在时间序列处理中保持了因果性, 即卷积核仅使用当前及之前的时间步进行计算, 避免信息从未来泄漏。这在实时检测场景中特别重要, 因为它确保模型的预测仅基于过去和当前的状态。此外, 因果卷积可以与扩展卷积结合, 扩大感知野以捕捉长时间依赖关系, 提升模型识别长时间跨度异常模式的能力。

不同模型的一维卷积的卷积核可能有所不同, 如除了基于因果卷积的 TCN [2] 外, MACE [8] 提出一种在频域中处理多模式正常性并高效进行时间序列异常检测的方法。为提高模型对短期异常的敏感性, MACE 将双重卷积 (dualistic convolution) 机制引入时间域, 通过设置不同的 得到峰值卷积 (peak convolution) 和谷值卷积 (valley convolution):

$$\text{DualisticConv}(\mathbf{X}) = \sqrt[\gamma]{\text{Conv}_{1 \times L}\left(\frac{\mathbf{X}^\gamma}{\sigma}, s\right)} \quad (127)$$

其中 γ 是一个超参数, 使卷积更加关注偏差, σ 是一个缩放超参数, $\text{Conv}_{1 \times L}(\mathbf{X}, s)$ 表示对步幅 s 和核长度为 L 的 $\mathbf{X}$ 应用标准卷积。

这是由于标准卷积会将频谱压缩到频谱分布主干周围的低维空间, 而双重卷积机制在卷积核滑动过程中, 总是选取接近被卷积核覆盖的子序列中频率值最大的值作为结果。这使其能够放大偏差, 凸显出异常, 扩大正常与异常之间的重建误差差异。

基于二维卷积时间序列异常检测模型 在时间序列异常检测任务中, 除了多通道特征的共同影响, 时间序列通常还具有复杂的多尺度和多周期结构, 这些因素交织在一起, 影响着异常模式的表达。相比于一维卷积仅沿时间维度进行特征提取, 二维卷积能够同时在时间和通道维度上进行信息聚合, 从而捕捉不同变量之间的关联模式以及跨时间步的局部依赖关系。这种方式有助于模型 [64] 更好地识别由多个特征协同作用产生的异常模式, 尤其适用于多变量时间序列中的复杂异常检测任务。二维卷积则通常在时间和特征维度上进行卷积, 适用于处理多变量时间序列。具体卷积过程可参考公式 (83)。

基于空洞卷积时间序列异常检测模型 在时间序列异常检测中, 传统的一维卷积只能捕捉有限的历史信息, 无法有效处理长时间依赖。为了克服这一限制, 可以采用空洞卷积 [2] (dilated convolution)。空洞卷积通过引入膨胀因子 (dilation factor) 来扩

展感受野，从而在较少的层数下捕捉更广泛的历史信息。

空洞卷积的核心思想是在卷积核的每两个相邻元素之间引入固定步长，从而使得卷积操作能够覆盖更大的时间范围。对于序列输入 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ ，(其中 $\mathbf{x}_t \in \mathbb{R}^{T \times N}$ ， T 是时间戳个数， N 是变量个数) 和滤波器 f ，空洞卷积操作 F 定义为：

$$F(s) = \sum_{i=0}^{k-1} f(i) \cdot x(s - d \cdot i) \quad (128)$$

其中， d 为膨胀因子， k 为滤波器大小， s 是序列中的位置， $d \cdot i$ 表示卷积核元素之间的间隔。当 $d=1$ 时，空洞卷积退化为常规卷积。

基于 CNN 的时间序列异常检测模型关键技术 近年来，涌现了许多基于 CNN 的时间序列异常检测模型，表 4.1 总结了常见的方法：

表 4.1: 基于 CNN 的异常检测方法总结。

模型	数据域	学习策略	局部特征	全局特征	模型架构	训练方法
TCN [2]	时域	重构	因果卷积	空洞卷积	仅编码器	无监督
SR-CNN [55]	时域	分类	一维卷积	大尺度卷积核	-	有监督
CAE-Ensemble [6]	时域	重构	卷积+GLU 单元	注意力机制	编码器-解码器	无监督
TS2Vec [71]	时域	表征	因果卷积	空洞卷积+层次化对比学习	仅编码器	自监督
TimesNet [64]	时域	预测	二维卷积	拼接	仅编码器	无监督
ModernTCN [43]	时域	重构	深度卷积和组卷积	大尺度卷积核	仅编码器	无监督
MACE [8]	时域	重构	双重卷积	大尺度卷积核	-	无监督
DACAD [15]	时空域	表征	因果卷积	空洞卷积	-	自监督
CutAddPaste [62]	时域	分类	因果卷积+局部数据增强	空洞卷积+全局数据增强	仅编码器	自监督

- 1. 数据域 (Data Domain)** 模型学习时间依赖的方式通常可以分为三种主要的形式：**时域**、**时频域**和**时空域**。**时域**是指数据随时间变化的表示方式。在时域中进行异常检测，主要关注数据在时间轴上的变化规律，直接利用时间序列的数值特征，适合处理单变量或多变量时间序列中的异常检测任务。在时域中，CNN（卷积神经网络）可以通过一维卷积 [2, 15, 43, 62] 操作直接处理时间序列数据。一维卷积能够捕捉时间序列中的局部模式和趋势，适用于检测时间维度上的异常 [2, 43]。**时频域**同时考虑时间和频率信息，能够揭示信号在时间和频率上的动态变化特性。常用的时频分析方法包括短时傅里叶变换（STFT）和小波变换，它们可以将信号分解为不同频率成分并分析其随时间的变化 [14]。在时频域中，CNN 通常用于处理时频变换后的二维图像数据（如短时傅里叶变换或小波变换生成的时频谱图）。时频谱图将时间序列转换为时间和频率的二维表示，CNN 可以通过二维卷积操作提取时频域中的空间特征 [14]。**时空域**结合了空间和时间信息，适用于处理多维时间序列数据（如传感器网络、视频数据等）。如 DACAD [15] 在时间域通过对比学习和异常注入机制，使模型能够适应目标域的异常分布，提高检测未

见异常的能力；在空间（特征）域，通过对比损失和中心熵分类器，学习稳定的领域不变特征，增强跨域泛化能力。在时空域中进行异常检测，需要同时考虑数据在空间上的分布和时间上的变化规律。在时空域中，CNN 可以通过三维卷积或二维卷积结合时间维度来处理时空数据。对于视频或传感器网络数据，三维卷积可以直接捕捉空间和时间的联合特征，而二维卷积则可以逐帧处理空间信息，再通过时间维度整合（如使用 3D CNN 或 CNN+LSTM）[15]。

2. **特征提取 (Feature Extraction)** 作为时序异常检测的核心环节，通过数学变换与模式挖掘将原始信号转化为具有判别性的表征，其核心目标是构建对异常敏感的特征空间。该过程需平衡信息压缩与特征保持的悖论，主要包含**局部特征提取**和**全局特征提取**两个维度。

(a) **局部特征 (Local Features)** 聚焦于短期时序模式的捕捉，其关键技术包括卷积架构创新和分层下采样。在卷积架构创新方面，因果卷积确保了时序因果关系，而空洞卷积通过指数扩展率实现了多尺度特征的捕获，从而兼顾了长程依赖与计算效率 [2, 43]。在分层下采样方面，采用了最大池化（保留显著特征）与平均池化（抑制噪声）的混合策略 [6]。例如，CAE-Ensemble [6] 通过多个卷积自编码器（CAE）来捕捉数据的多样性和复杂模式，最终通过重构误差来识别异常。而 SR-CNN [55] 在光谱残差（SR）模型的输出上应用 CNN，通过训练分类器来检测异常。CutAddPaste [62] 则通过数据增强策略生成伪局部异常样本进行训练，进一步提升了局部特征的提取能力。

(b) **全局特征 (Global Features)** 描述时间序列的整体趋势、周期性和长期规律。基于 CNN 的异常检测模型建模序列全局特征的常规方式是**卷积层堆叠**。在低层卷积层中，网络关注短时间窗口内的局部变化；在高层卷积层中，网络将局部特征结合起来，形成更长时间跨度的模式 [6, 55]。卷积层的堆叠以一种简单而有效的方式捕捉序列的全局特征。然而，如果仅将最高层次的特征传递到后续网络，可能会丢失序列中的局部细节信息；而若将所有卷积层的输出都传播下去（如 DACAD [15] 采用拼接操作聚合不同层次输出），则会导致计算复杂度的显著增加。针对这一挑战，TS2Vec [71] 提出了“层次化对比学习”，通过递归地对时间戳表示进行最大池化并在各尺度上进行对比，自然地在顶层形成包含整体序列信息的全局特征。由于不断堆叠卷积层会造成模型的复杂度的增加，一些研究认为**大尺度卷积核**是扩大感受野、捕捉全局特征的更有效方法 [43]。其中，MACE [8] 通过双重卷积机制来放大异常与正常之间的差异，捕获了序列中的全局特征。

3. **模型架构 (Model Architecture)** 在模型架构中，**Encoder-Only (仅编码器)** 的模型架构主要由一个编码器模块组成，用于对输入数据进行特征提取和表示学习。

编码器通过堆叠多个层（如自注意力、卷积或递归层）来捕获输入的模式、依赖关系和结构信息，生成高度抽象的表征。**Encoder-Decoder（编码器-解码器）**的模型架构包含两个模块：**编码器（Encoder）**负责提取输入的高层次特征，生成紧凑的表征（通常是固定长度的向量）。**解码器（Decoder）**负责基于编码器生成的表征，通过一系列反向操作将信息映射回原始数据空间或目标输出，例如生成文本、图像或其他结构化输出。

4.4.4 Transformer

Transformers 在自然语言处理（NLP）和计算机视觉（CV）等领域展现出了卓越的性能，尤其是在处理复杂任务时表现出了强大的能力。这一成功引发了时间序列分析领域的广泛关注和深入探索。时间序列数据通常具有复杂的时序依赖关系，而Transformer的自注意力机制不仅能建模局部的信息，还能够有效地学习这些长期依赖，捕捉到远距离的相互作用。如图 4.5 所示，与基于 CNN 的异常检测方法类似，基于 Transformers 的异常检测方法分为无监督和有监督两类，图 4.5(a) 展示了无监督的方法，其通常基于重构或预测，为了简明起见，我们以重构为例；图 4.5(c) 展示了有监督的方法，其通常基于分类；图 4.5(b) 展示了利用 Transformers 捕捉时序特征的最常见的 backbone。

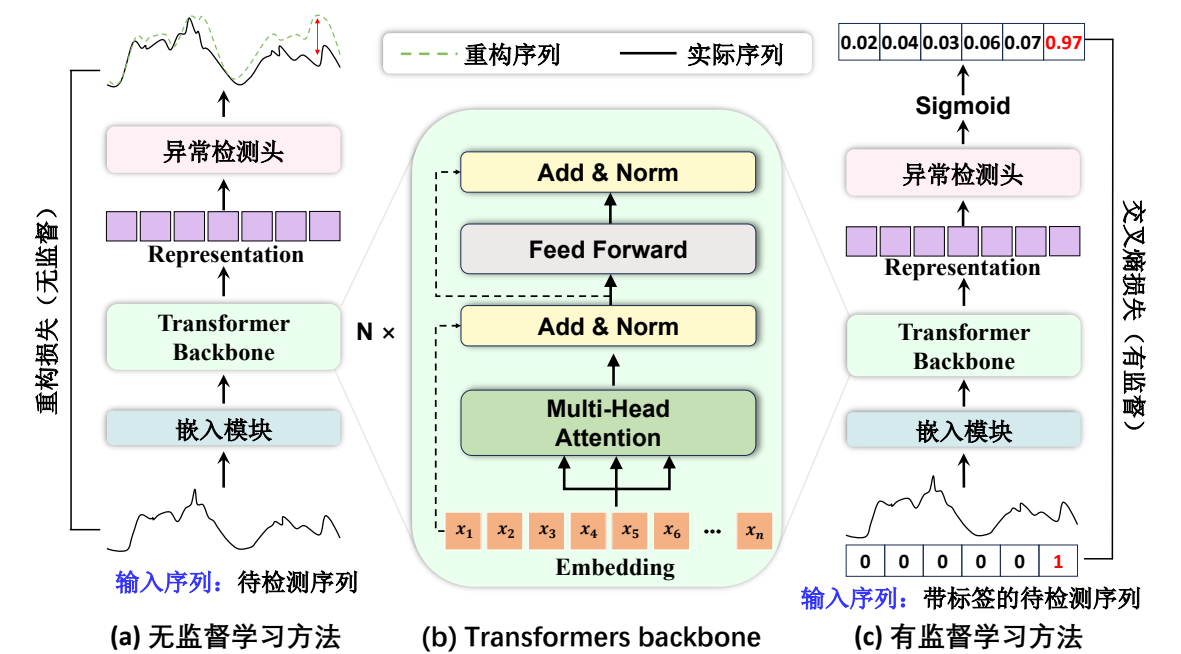


图 4.5: TRANSFORMER 异常检测

基于 **Transformer** 的时间序列异常检测模型的通用架构 以基于重构的方法为例，对

于给定的观测时间序列 $\mathbf{X} = (\mathbf{x}_{t-C+1}, \mathbf{x}_{t-C+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{C \times N}$ (图中以 $N = 1$ 的单变量时间序列为例进行展示)，首先运用包含线性变换、卷积或位置编码等操作的嵌入模块，将输入的序列映射到指定的隐藏空间维度，生成初始表示 $\mathbf{H}_0 \in \mathbb{R}^{C \times D}$ ，其中 C 表示待重构序列的时间步数， D 表示嵌入维度。随后， $\mathbf{H}_0$ 作为输入传递至主干网络 (Transformer Backbone) 中，利用自注意力机制捕获时间序列中的全局依赖关系与复杂关联模式，得到表示序列 $\mathbf{H}_L \in \mathbb{R}^{C \times D}$ 。在异常检测任务中，通常采用两种方式进行异常评分 (Anomaly Scoring)：一是基于重构的方法，设计重构头 (Reconstruction Head)，根据 $\mathbf{H}_L$ 重构原始序列 $\hat{\mathbf{X}} \in \mathbb{R}^{C \times N}$ ，通过计算重构误差作为异常分数；二是基于表征的方法，直接利用 Transformer 中计算得到的注意力矩阵或隐空间表示，构建异常评分函数，衡量某一时刻与其他时刻之间的相关性异常程度，例如 Anomaly Transformer [66] 中通过计算先验关联分布与实际关联分布之间的 KL 散度作为异常分数。模型最终根据异常分数与设定阈值，判断对应时间步是否存在异常，并通过优化重构误差或异常分布提升检测性能，从而增强对复杂时间序列异常模式的捕捉能力。

具体来说，Transformer 通过自注意力机制和多头注意力机制，能够在时间序列建模中捕捉到长距离的全局依赖关系，而不受传统 RNN 在处理长序列时常见的梯度消失和爆炸问题的限制。自注意力机制允许模型在序列的各个位置之间灵活地建立直接联系，从而更有效地识别跨时间跨度的异常模式。与传统的序列建模方法不同，Transformer 不依赖于时间步的顺序处理，而是通过同时关注序列中所有位置的信息，在显著提高了处理复杂时间序列任务的能力的同时也大幅提升大规模时间序列数据的处理速度。通过多层自注意力和前馈网络的堆叠，Transformer 能够逐步提取时间序列中的多尺度、多层次特征，精准捕捉不同时间尺度上的异常行为。

关于 Transformer Backbone 模块的内部结构，在本书的 3.3.4 小节已经介绍的非常详细，本小节不再赘述。

基于 Transformer 的时间序列异常检测模型关键技术 近年来，涌现了许多基于 Transformer 的时间序列异常检测模型，其关键技术如表 4.2 所示，主要体现在时序特征捕捉方法、序列嵌入方式、模型架构等，关于表中列名的名词解释已在本书 3.3.4 小节有非常详细的介绍，本小节主要介绍在时序异常检测领域关于时序特征捕捉的方法，因为在异常检测领域，一些模型或策略在传统 Transformer 的基础上，结合先验知识或实验观察，对自注意力机制进行了有针对性的修改，取得了出乎意料的优异效果。这些改进通常是为了更好地适应异常检测的需求，使得模型能够在异常检测场景中展现出更强的表达能力：

1. **时间序列特性 (Time Series Characteristics)** 指的是时间序列数据中固有的特性，例如多尺度性质、非平稳性、变量相关性、季节性与趋势性、周期性等，模型对这些特性的捕捉能力会直接影响预测性能，以下将分别介绍针对不同特性的

表 4.2: 基于 Transformer 的异常检测方法总结。

模型	数据域	学习策略	时序特征捕捉	嵌入方式	模型架构	训练方法
GTA [12]	时空域	预测	多尺度性质 + 时空耦合	点嵌入	编码器-解码器	半监督
TranAD [61]	时域	重构	长程 + 短程序列关联	窗口嵌入	编码器-解码器	无监督
Anomaly Transformer [66]	时域	表征	序列关联 + 先验关联	点嵌入	仅编码器	无监督
MT-RVAE [63]	时空域	重构	多尺度性质 + 周期性	点嵌入	编码器-解码器	无监督
DCT-GAN [39]	时域	重构	多尺度性质	点嵌入	仅编码器	无监督
SBT [22]	时域	重构	高效序列关联	点嵌入	编码器-解码器	无监督
STOC [35]	时域	预测	多尺度性质 + 序列关联	点嵌入	编码器-解码器	无监督
SAT [70]	时域	重构	次邻近区域序列关联	点嵌入	编码器-解码器	无监督
Dual-TF [49]	时频域	重构	序列关联 + 先验关联	点嵌入	仅编码器	无监督

模型方法。

(a) **多尺度性质** 是指时间序列数据中存在不同时间粒度下的特征, DCT-GAN [39] 使用扩张卷积 (Dilated CNN) 用于在固定检测窗口内提取多尺度特征, 使用基于 Transformer 的网络组织多个检测窗口, 捕捉时间序列中的全局依赖关系, 其多尺度生成器 (Multi-scale Generators) 从不同尺度和视角生成伪造数据, 以提升模型的泛化能力。在将时间序列数据输入 Transformer 模型之前, 通常需要通过全连接层或一维卷积层将其映射到与模型输入维度相匹配的高维向量空间。然而, 单独依靠全连接层或一维卷积层难以拟合精确的映射关系, 导致在上采样过程中丢失大量细节信息, 因此, MT-RVAE [63] 引入了特征金字塔结构 (Feature Pyramid Structure), 仅在时间维度上对数据进行卷积和上采样操作, 弥补了上采样过程中特征细节的缺失问题。不仅如此, MT-RVAE 还提出了一种结合周期信息与时间序列信息的全局时间编码方法。全局时间编码与局部时间编码共同构成本文的位置编码模块。其中, 全局时间编码包含时间序列编码与周期性编码两部分。时间序列编码通过对时间戳信息进行分解, 将其编码为向量形式。考虑到卫星等复杂设备的周期性运行特性, 周期性编码利用傅里叶变换对数据的周期信息进行分析, 并将其编码成向量。如此, MT-RVAE 实现了时序上的多尺度性质和周期性信息的获取。STOC 模型 [35] 同样致力于捕捉时间序列中的多尺度特征, 但其实现路径与前述方法不同。STOC 的核心思想是堆叠 Transformer 编码器中所有层的输出表示, 而非仅使用最终的顶层表示。在 Transformer 的深层, 特征更偏向于抽象的全局信息; 而在其浅层, 则保留了更多局部细节。通过将不同层的表示进行堆叠, STOC 自然地融合了从局部到全局的多层次信息, 从而能够同时感知时间序列的局部变异性和整体趋势。随后, 模型使用一个轻

量级的 1D 卷积神经网络 (CNN) 作为解码器，来融合这些堆叠的、富含多尺度信息的表示，并最终进行预测。这种“堆叠表示 + 1D-CNN 解码”的架构，使 STOC 在基于预测的异常检测任务中，能够更精准地学习正常数据的分布，并对偏离该分布的异常点更为敏感。

- (b) **时空耦合** 在许多物联网场景 (如水资源管理、电力系统) 中，多变量时间序列的各个传感器节点在物理空间上存在复杂的、未知的拓扑连接关系。GTA [12] (Graph-learning Transformer for Anomaly detection) 模型的核心创新就在于能够自动学习这种空间拓扑结构，并与时间依赖关系进行耦合建模。GTA 没有依赖预定义的图结构，而是通过一个基于 Gumbel-Softmax 的可微连接学习策略，直接从数据中推断出传感器之间的有向依赖关系。随后，它提出了一种影响传播图卷积，将学习到的图结构用于在传感器节点间传播信息，模拟异常在物理系统中的传播过程。在时间维度，GTA 采用了分层扩张卷积来捕获多尺度的时间上下文。最终，这个增强了空间结构信息的时序表征被送入一个经过优化的多分支 Transformer 中进行预测。GTA 的这种将自动学习的空间图结构与多层次时间建模紧密结合的方法，使其在 SWaT、WADI 等具有强物理依赖关系的真实物联网数据集上取得了卓越的异常检测性能。

除了上述时间序列数据中固有的特性，在异常检测领域，人们通过对传统 Transformer 主干网络 (Transformer Backbone) 进行改进，使模型能够更加关注异常点所处的上下文信息、全局依赖关系及局部异常特征，从而提升异常检测的准确性与鲁棒性，接下来我们来详细介绍这些方法。

- (c) **关联差异** Long 等人通过实验发现：由于异常点的稀有性，从异常点到整个序列构建非平凡的关联极为困难，因此，异常点的关联主要集中在其相邻的时间点上。而主导的正常时间点则能够发现与整个序列的相关性，不仅仅局限于相邻区域，这一区别在异常点和正常点之间可能会产生显著的差异。由此，他们提出了具有异常注意力机制的 Anomaly Transformer [66]，能够同时建模先验关联和序列关联，从而体现关联差异 (Association Discrepancy)，并认为异常点相比于正常点会体现出更小的关联差异。Anomaly Transformer 利用 Transformer 固有的自注意力机制计算序列关联 (series-association)，并添加了一个模块计算先验关联 (prior-association)，这使得 Anomaly Transformer 的自注意力机制模块能同时建模先验关联和序列关联，其计算过程如下：

$$p^l = \text{Rescale} \left(\left[\frac{1}{\sqrt{2\pi}\sigma_i} \exp \left(-\frac{|j-i|^2}{2\sigma_i^2} \right) \right]_{i,j \in \{1, \dots, N\}} \right) \quad (129)$$

$$s^l = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_{\text{model}}}} \right) \quad (130)$$

其中， p^l 表示先验关联， s^l 表示序列关联， $K, V \in R^{N \times d_{\text{model}}}$ ， $\sigma \in R^{N \times 1}$ 分别表示自注意力中的键 (key)、值 (value) 和学习到的高斯核尺度。先验关联和序列关联的计算能够体现关联差异 (Association Discrepancy)，其表现为先验关联和序列关联之间的对称 KL 散度。

与 Anomaly Transformer 通过显式建模两种关联来捕捉差异的思路不同，TranAD [61] (Deep Transformer Networks for Anomaly Detection) 则采用了一种基于“自条件” (Self-Conditioning) 与“对抗训练” (Adversarial Training) 的动态注意力引导机制。其核心思想是通过一个两阶段的推理过程来放大异常信号。在第一阶段，模型以零矩阵作为“焦点分数” (Focus Score)，对输入窗口进行初步重构。第一阶段的重构误差（即输出与输入的偏差）被计算出来，作为第二阶段的“焦点分数”。在第二阶段，这个焦点分数被拼接到输入中，再次通过 Transformer 编码器-解码器进行重构。这个设计使得模型在第二次重构时，能够将更多的注意力（即更高的注意力权重）分配到那些在第一阶段难以重构的、可能是异常的子序列上。这种“自条件”机制，本质上是一种动态的、数据驱动的注意力调制策略，它不需要像先验关联那样的固定假设，而是让模型自己发现并聚焦于潜在的异常上下文，从而实现了对序列关联更富表达力的捕捉，特别擅长于放大那些微小的、容易被忽略的异常偏差。

与 Anomaly Transformer 类似，SAT (Sub-Adjacent Transformer) [70] 也从基于时间点之间的关联的假设入手。SAT 的基本假设是，与正常点相比，异常点与其非直接邻域的相关性较小。因此，仅关注这些非直接邻域可能会导致异常的较大重建误差。此外，SAT 还认为标准自注意力中使用的传统 Softmax 操作阻碍了期望注意力矩阵的形成。SAT 定义次邻域为距离目标点在 $K1$ 和 $K2$ 之间的区域，通过在注意力矩阵中引入次邻域的概念，设 A_{ij} 为注意力矩阵 A 中第 i 行第 j 列的元素。 A_{ij} 的值反映了点 i 对点 j 的贡献， A_{ij} 越大，贡献越显著。SAT 定义每个点的次邻域注意力贡献为该列中预定义次邻域范围内的值的总和，公式如下：

$$\text{SACon}(\mathbf{A}) = [\text{SACon}(\mathbf{A}_{:,i})]_{i=0, \dots, \text{win_size}-1} \quad (131)$$

$$\text{SACon}(\mathbf{A}_{:,i}) = \sum_{|j-i| \geq K_1}^{|j-i| \leq K_2} \mathbf{A}_{ji}, \quad 0 \leq j < \text{win_size} \quad (132)$$

$$\Phi(\cdot) = \text{Softmax}_{\text{row}}(\cdot/\tau) \quad (133)$$

其中，下标 $(:, i)$ 表示对应矩阵的第 i 列， winsize 表示窗口大小。定义 $[j] = j + n \cdot \text{winsize}, n \in \{0, \pm 1\}$, 使得 $0 \leq [j] \leq \text{winsize}$ 。

- (d) **模型压缩** 与上述通过引入复杂假设来显式引导模型关注特定时序模式的方法不同，Sparse Binary Transformer (SBT) [22] 从模型压缩和效率的角度对时序特征捕捉进行了探索。SBT 的核心思想是验证极致的模型压缩（包括权重二值化和连接稀疏化）是否会损害 Transformer 捕捉时序依赖关系的基本能力。

SBT 在时序特征捕捉上采用了两种简化的策略：

- **固定稀疏投影**：在模型初始化时，对 Query、Key 和 Value 的投影矩阵施加固定的随机掩码，使激活值稀疏化。这种“朴素”的稀疏化并不依赖于任何时序领域的先验知识，其成功表明标准 Transformer 的注意力机制本身具有很强的鲁棒性。即使随机丢弃一部分信息，剩余的连接仍足以捕捉到对异常检测任务关键的全局依赖关系。
- **Step-T 注意力掩码**：针对异常检测任务（通常重构当前时间点），该掩码强制模型仅计算当前时间步的 Query 与所有历史时间步的 Key 和 Value 之间的注意力。这种方法并非为了更精细地建模关联，而是基于任务特性进行的重大计算优化。它使得模型能够高效地利用整个历史窗口的信息来重构当前点，本质上是一种定向的、高效的全局依赖捕捉方式，同时避免了在无关时间步上进行冗余计算。

SBT 的工作表明，在时间序列异常检测任务中，即使不设计复杂的注意力变体，通过对标准 Transformer 进行激进的压缩和面向任务的简化，其固有的自注意力机制依然能够有效地捕捉序列关联，完成时序建模。这为在资源受限环境下部署轻量级异常检测模型提供了新的可能性。

4.4.5 自编码器

自编码器 (Autoencoder, AE) 是一种无监督学习模型，在数据降维、特征提取和去噪方面表现卓越。如图 4.6 所示，其独特之处在于对称的结构设计，自编码器包含一个编码器 (Encoder) 和一个解码器 (Decoder)，编码器将输入数据压缩到低维潜在空间，而解码器则负责从这个潜在表示中重建原始数据。通过最小化输入与重建数据之间的误差，自编码器能够自动学习数据的非线性特征。

基于自编码器的时间序列异常检测模型的通用架构 如图 4.6 所示，基于自编码器 (Autoencoder, AE) 的时间序列异常检测方法通常由编码器 (Encoder) 和解码器 (De-

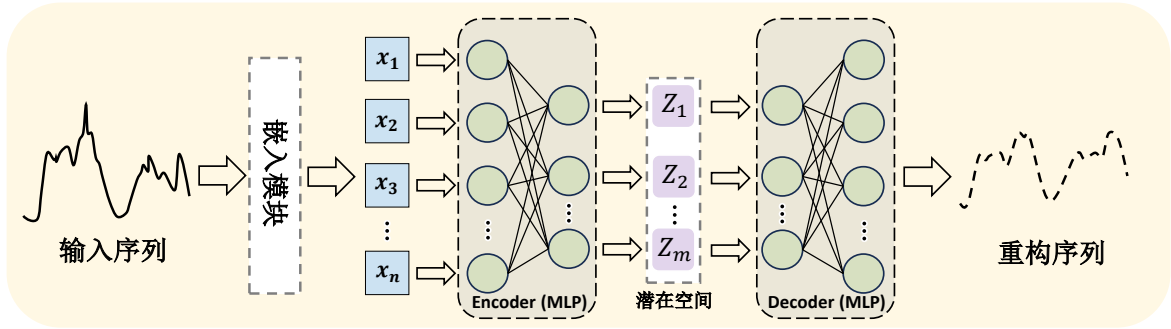


图 4.6: AE 异常检测

coder) 两部分组成。对于给定的观测时间序列 $\mathbf{X} = (\mathbf{x}_{t-C+1}, \mathbf{x}_{t-C+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{C \times N}$ (图中以 $N=1$ 的单变量时间序列为例进行展示), 首先将时间序列划分为长度为 C 的滑动窗口片段, 作为模型的输入。随后, 编码器部分通常由多层全连接神经网络 (MLP) 或卷积神经网络 (CNN) 构成, 负责将输入的时间序列映射至低维潜在空间, 生成潜在表示 $\mathbf{Z} \in \mathbb{R}^{C \times D}$, 其中 D 表示潜在空间维度。潜在空间中的表示 $\mathbf{Z}$ 尽可能保留原始序列中的关键特征信息, 同时去除冗余或噪声成分。接着, 解码器模块将潜在表示 $\mathbf{Z}$ 作为输入, 利用对称结构的神经网络将其映射回原始输入空间, 生成重构序列 $\hat{\mathbf{X}} \in \mathbb{R}^{C \times N}$ 。在训练阶段, 模型通过最小化输入序列 $\mathbf{X}$ 与重构序列 $\hat{\mathbf{X}}$ 之间的重构误差 (如均方误差, MSE) 来更新模型参数, 使得对正常数据的重构尽可能精确。在异常检测阶段, 由于模型在训练过程中主要学习了正常数据的特征分布, 因此对于正常数据, 其重构误差较小; 而异常数据由于偏离训练数据分布, 模型难以有效重构, 通常表现为较大的重构误差。通过设定重构误差阈值, 便可据此划分正常与异常数据, 从而实现异常检测任务。此外, 部分方法还可基于潜在空间中的表示分布特性, 结合聚类、密度估计等方式辅助判断异常样本, 从而提升检测性能。

在时间序列异常检测中, 自编码器的这些特性被巧妙地利用: 它通过学习正常时间序列的数据模式, 能够在检测阶段通过计算重构误差来识别异常数据。正常数据因符合模型学习的模式而呈现低重构误差, 而异常数据则因偏离正常模式而导致高误差。通过设定一个重构误差的阈值, 自编码器可以有效区分正常与异常的数据点。这种方法不仅充分发挥了自编码器的特征学习能力, 还为高维和复杂时间序列数据的异常检测提供了一种高效的解决方案。

基于 AE 的时间序列异常检测模型关键技术 近年来, 基于自编码器 (Autoencoder, AE) 的时间序列异常检测方法也得到了广泛关注, 其典型模型及特性如表 4.3 所示。基于 AE 的方法大多围绕模型结构设计、潜在空间约束方式以及重构策略展开, 以提升异常检测性能。这些方法通常遵循“编码器-解码器”框架, 通过重构误差大小判断样本是否异常, 但在编码方式、潜在空间建模及训练范式上存在一定差异。具体来看, 部分方法 (如 LSTM-AE [29] 和 S-RNNAE [34]) 在编码器和解码器结构中引入循环神

神经网络 (RNN) 单元, 以增强对时间序列动态变化特性的建模能力; 而 DAGMM [80]、USAD [1] 等方法则在潜在空间中融合聚类、判别器等辅助机制, 增强模型对异常样本的敏感性。此外, 近年来还有方法如 AMSL [77] 和 ContextDA [36], 采用对抗性训练策略或多子空间学习机制, 提升模型在复杂异常场景下的泛化能力。综上, 这些基于 AE 的方法通常通过改进编码方式、重构目标或训练范式, 有效提升了异常检测的表现, 并为后续研究提供了丰富的模型设计思路。

表 4.3: 基于 AE 的异常检测方法总结。

模型	数据域	学习策略	神经网络类型	潜在空间约束	模型架构	训练方法
AE/DAE [56]	时域	重构	多层感知机	无	编码器-解码器	自监督
EncDec-AD [44]	时域	重构	循环神经网络	无	编码器-解码器	自监督
SPREAD [24]	时空域	重构	循环神经网络	稀疏性约束	编码器-解码器	有监督
DAGMM [80]	时域	重构	高斯混合模型	概率分布约束	编码器-解码器	无监督
MSCRED [74]	时空域	重构	卷积 + 循环神经网络	无	编码器-解码器	有监督
S-RNNAE [34]	时域	重构	循环神经网络	无	编码器-解码器	有监督
LSTM-AE [29]	时域	重构	循环神经网络	无	编码器-解码器	无监督
USAD [1]	时域	重构	多层感知机	对抗性约束	编码器-解码器	无监督
AMSL [77]	时域	重构	卷积神经网络	记忆约束	编码器-解码器	自监督
ContextDA [36]	-	重构	循环神经网络	域对齐约束	编码器-解码器	无监督

1. **神经网络类型 (Type of neural network)** 指自编码器 (AE) 模型中编码器和解码器所采用的具体神经网络结构形式, 如多层感知机、卷积神经网络、循环神经网络或 Transformer 等。不同类型的神经网络具备不同的特性, 决定了模型对时间序列中周期性、趋势性、多尺度性等特征的捕捉能力。

(a) **多层感知机** 是最基础的前馈神经网络结构, 结构简单, 易于训练。将其作为 AE 的编码器和解码器, 能够快速学习时间序列的整体映射关系, 适用于特征维度固定、结构单一的场景。但由于缺乏对时序依赖和局部结构的建模能力, 通常仅作为基础 AE 或其他结构的辅助模块。Sakurada 等人提出基于 MLP 的自编码器方法 [56], 将其用于时间序列异常检测任务, 通过隐藏层实现对数据的非线性降维。相比于线性方法如 PCA 和 KPCA, 该方法无需复杂的核函数设计, 便能有效捕捉数据中的非线性结构。此外, 作者还扩展了去噪自编码器, 将部分输入随机置零, 从而提升模型对潜在模式的鲁棒性和异常检测能力。此外, Audibert 等人提出了 USAD (UnSupervised Anomaly Detection) 方法 [1], 将 MLP 结构的自编码器与对抗训练机制相结合, 实现多元时间序列的无监督异常检测。该方法采用共享编码器和双解码器架构, 通过对抗性训练提升模型区分正常与异常数据的能力, 尤其在异常与正常样本相似时, 能有效放大重构误差差异, 提高检测性能。USAD 利用自编码器

自动学习正常数据的特征分布，无需依赖专家经验，适合高维复杂 IT 系统环境下的异常检测需求。

- (b) **卷积神经网络** 擅长捕捉时间序列中的局部模式与局部依赖特性。将 CNN 与 AE 结合，能够有效提取时间序列中的局部特征、周期性与多尺度结构，使模型对异常点在局部上下文中的异常模式更为敏感，适合多尺度、多周期异常检测任务。AMSL [77] 结合了自监督学习与局部及全局记忆模块，针对无监督异常检测任务设计。该模型通过多种数据增强生成伪标签，提升对正常模式的表达能力，并利用局部记忆模块捕捉细节特征，结合全局记忆模块学习整体时序结构。通过动态融合这两类记忆，AMSL 能有效提升模型对异常的识别能力和泛化性能。
- (c) **循环神经网络** 通过递归结构建模时间序列中的顺序性和长期依赖性。作为 AE 的编码器和解码器，RNN 能逐步捕捉时间序列中的趋势性、非平稳性和复杂依赖关系，特别适用于异常点依赖于长时间上下文的检测场景。常见变种如 LSTM 和 GRU 还能缓解梯度消失问题，提升长序列建模效果。Malhotra 等人提出的 EncDec-AD 方法 [44]，采用 LSTM 结构作为编码器与解码器，对多传感器时间序列进行无监督异常检测。该方法将固定长度的正常子序列输入 LSTM 编码器，将其压缩为上下文向量，再由 LSTM 解码器基于该向量重构原始序列。通过最大化正常序列的重构准确性，模型能够捕捉多维时间序列中的非线性动态变化及复杂依赖模式。在检测阶段，通过计算重构误差衡量每个时刻的异常程度，实现对不可预测或复杂相关性的异常点识别。**LSTM-AE** 方法 [29] 是基于 LSTM 的自编码器在智能制造领域的成功应用。该方法针对工业产线多变量传感数据中异常模式有限且不规则的特点，构建了无监督的在线检测框架。其核心是利用 LSTM 自编码器学习正常序列的模式，通过重构误差识别异常。为满足实时性要求，采用滑动窗口处理流数据，并结合多数投票机制提升检测稳定性。该工作还展示了迁移学习的有效性，通过将在 Chamber A 上训练的模型迁移至 Chamber B，显著提升了在小样本场景下的检测性能。实验结果表明，LSTM-AE 在真实工业数据集上的准确率超过 90%。此外，ContextDA 方法 [36] 针对跨领域时间序列异常检测问题，设计基于 LSTM 的上下文感知领域自适应模型。该方法在编码阶段，利用双向 LSTM 捕捉序列的前后上下文信息，提取领域无关的共享特征，同时保留源领域和目标领域的领域特异性表示。通过对源、目标领域样本的特征分布执行自监督对齐，缩小跨域分布差异，增强模型在新领域下的异常检测能力。检测阶段依据重构误差与上下文一致性判断异常点，有效应对多变量时序数据中的领域偏移与复杂依赖特性。针对高维时间序列，Gugulothu 等人提出了 SPREAD 方法 [24]。该方法在标准的循环自编码器基础上，引入

了一个稀疏连接的前馈降维层。该层位于编码器之前，通过 L1 正则化使其权重矩阵稀疏化，从而让每个隐藏单元仅连接到输入维度的一个小子集。这种结构创新使得 SPREAD 能够有效处理高维输入，稀疏层充当了强大的正则化器，并隐式地执行了无监督特征选择，同时 RNN 部分继续负责捕捉时序依赖。实验证明，该混合结构在高维时间序列异常检测任务上显著优于标准的循环自编码器及其变体。为提升 RNN 自编码器的鲁棒性，Kieu 等人 [34] 提出了 S-RNNAE 方法。该方法通过稀疏连接的循环跳跃网络构建多个结构差异化的自编码器，形成集成系统。研究提出了两种框架：独立框架 (IF) 独立训练各自编码器，共享框架 (SF) 通过共享层联合训练以增强模型协同。异常评分采用多个自编码器重构误差的中位数，有效降低单一模型过拟合的影响。S-RNNAE 将集成学习成功应用于时间序列异常检测，显著提升了检测的准确性和稳定性。

- (d) **高斯混合模型** 作为一种概率生成模型，本身即可用于对数据分布进行密度估计，并基于低概率识别异常。将其与自编码器结合，可以克服高维空间中直接进行密度估计的困难，并实现联合优化。DAGMM [80] 是该方向的典型代表，其创新之处在于它将一个由 MLP 构成的自编码器（称为“压缩网络”）与一个高斯混合模型 (GMM) 通过另一个 MLP（称为“估计网络”）相连，形成了一个强大的端到端联合学习框架。该模型首先使用自编码器将高维输入数据压缩成一个低维表示，该表示不仅包含编码后的潜在特征，还融入了重构误差特征。这种“双视角”的潜在表示能够更全面地描述一个样本：正常样本通常在潜在空间中聚集且易于重构，而异常样本则可能在潜在空间中偏离或在重构上表现不佳。DAGMM 通过端到端训练同时优化重构损失和样本在 GMM 下的能量（负对数似然），使得自编码器学习到的降维表示能够直接有利于后续的密度估计任务，从而显著提升了异常检测的性能。
- (e) **混合神经网络** 通过结合多种神经网络结构（如 CNN 与 RNN），以同时捕捉时间序列中的局部空间模式与长期时间依赖。这类模型结构复杂，但能更全面地表征多元时间序列的复杂特性。Zhang 等人提出的 MSCRED 模型 [74] 是混合神经网络的典型代表。该模型首先构建多尺度的系统特征矩阵，以刻画不同时间片段内多元时间序列变量间的相互关系。随后，采用一个卷积编码器来编码特征矩阵中的空间相关性（即传感器间的相互关联），并利用一个基于注意力的卷积长短期记忆网络 (ConvLSTM) 来捕获复杂的时间模式。最后，使用一个卷积解码器基于融合了时空信息的特征映射来重构原始特征矩阵。MSCRED 的创新之处在于它将异常检测和诊断（包括根本原因识别和异常严重性评估）统一在一个无监督的编码器-解码器框架内。模型基于重构残差进行异常判断：异常时期的状态由于与训练所见的正常状态不同，会导

致更高的重构误差。该模型在合成数据集和真实电厂数据集上均展现出优于传统方法的性能。

2. **潜在空间约束 (Latent spatial constraints)** 指在自编码器模型中对编码得到的潜在空间表示施加的分布性或结构性限制，以增强编码特征的可分性和对异常点的辨识能力。常见方法包括概率分布约束、聚类性约束或距离约束，目的是提升模型在异常检测任务中的判别效果。

(a) **对抗性约束** 通过引入对抗训练机制，使潜在空间中的正常样本和异常样本特征分布尽可能可分，通常通过判别器或双重重构策略对编码特征施加约束，从而提升模型对异常点的区分能力。比如，在 USAD 方法 [1] 中，通过两个共享编码器的自编码器 (AE1 和 AE2) 构建对抗结构，AE1 试图生成与真实数据难以区分的重构结果，而 AE2 则作为判别器，努力区分真实数据与来自 AE1 重构的“伪数据”。这种双重重构和对抗训练策略，实质上对潜在空间中的编码特征施加了隐式的分布性约束，使得异常样本在编码后的分布与正常样本显著不同，从而提高异常检测性能。

(b) **记忆约束** 借助记忆模块 (如 memory bank 或 prototype memory) 保存典型的正常特征表示，编码后的潜在特征需与记忆中的正常 prototype 保持接近或偏离，从而帮助检测异常样本在 latent space 中的偏离程度。例如，ASML 模型 [77] 在潜在空间中引入自适应记忆单元，动态存储多样化的正常特征 prototype，通过最小化编码特征与最近邻 memory item 的距离，保证正常样本编码后的表示靠近记忆单元。同时，异常样本由于特征分布差异，难以匹配记忆单元，从而在潜在空间中自然偏离，便于后续检测。此外，ASML 结合自监督任务增强潜在特征的判别性，进一步提升了基于记忆约束的异常检测效果。

(c) **稀疏性约束** 通过对编码器或潜在表示施加稀疏性限制，可以迫使模型学习更紧凑、更具判别性的特征。这种约束可以视为一种结构性的潜在空间约束。例如，SPREAD 方法 [24] 通过在输入层和降维层之间施加 L1 稀疏约束，使得前馈降维层的连接权重高度稀疏。这实质上对模型学习到的潜在表示施加了一种结构性约束：每个潜在维度只能由少数几个输入维度激活。这种稀疏性迫使模型专注于学习输入变量间最关键的依赖关系，从而提高了潜在空间的质量。当异常发生时，其模式通常不符合学习到的稀疏依赖结构，导致其在潜在空间中的表示和后续的重构出现较大误差，进而被有效检测。

(d) **域对齐约束** 在跨域异常检测场景中，通过对潜在空间特征分布进行对齐，使源域和目标域编码后的特征分布相似，从而提升模型在目标域上的检测性能，常结合上下文采样或对抗式特征对齐方法实现。例如，ContextDA 模型 [36]

在潜在空间中引入上下文感知的对齐策略，通过上下文片段采样（context window sampling）和上下文对齐损失，确保源域和目标域在相似上下文条件下编码后的潜在特征分布尽可能一致。具体而言，ContextDA 利用对抗性训练方式，迫使域判别器无法区分潜在空间中的源域与目标域特征，同时结合上下文采样缓解时间序列上下文变化对对齐效果的影响，从而在保持异常检测能力的同时，实现跨域迁移。

- (e) **概率分布约束** 旨在迫使潜在空间的表示符合一个特定的先验概率分布，如高斯分布或高斯混合分布。通过最大化正常样本在该分布下的似然概率（或最小化其能量），可以使得异常样本因其低概率而被识别。DAGMM 是这种方法的一个典型代表。它通过其估计网络预测每个低维表示的样本属于高斯混合模型中各分量的后验概率（软成员关系），并基于此直接估计 GMM 的参数。其目标函数显式地包含了样本能量项，即负的 GMM 对数似然。这一设计对潜在空间施加了强烈的高斯混合分布约束，使得正常样本会聚集在各个高斯分量中心附近（低能量），而异常样本则因偏离这些中心而具有高能量。此外，DAGMM 还通过一个正则化项防止 GMM 的协方差矩阵退化，进一步稳定了潜在空间的结构。

4.4.6 变分自编码器

变分自编码器（Variational Autoencoder, VAE）是一种生成模型，结合了概率图模型和深度学习的优势，最初被用于图像生成和图像表示学习方面。VAE 能够学习数据的隐含表示，并在此基础上生成与输入数据相似的输出。基于 VAE 的时间序列异常检测通过学习正常时序数据的分布来捕捉正常序列的特征，利用其生成能力和概率建模特性来识别异常模式。

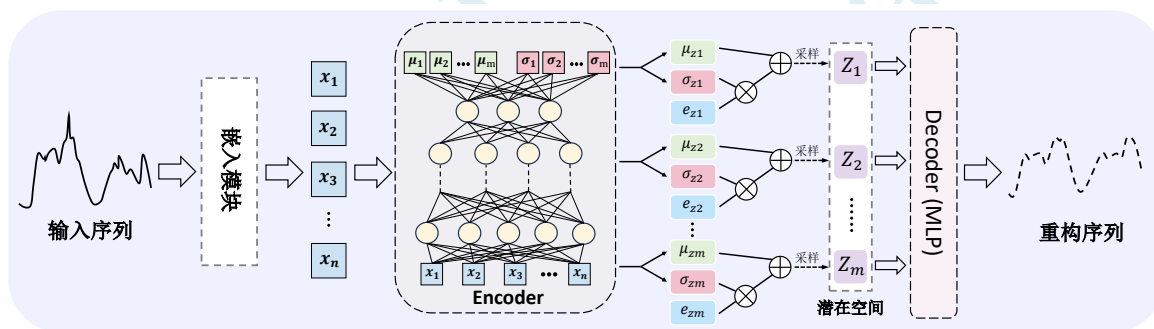


图 4.7: VAE 异常检测

基于变分自编码器的时间序列异常检测模型的通用架构 如图 4.7 所示，基于变分自编码器（Variational Autoencoder, VAE）的时间序列异常检测方法同样由编码

器 (Encoder) 和解码器 (Decoder) 两部分组成。对于给定的观测时间序列 $\mathbf{X} = (\mathbf{x}_{t-C+1}, \mathbf{x}_{t-C+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{C \times N}$ (图中同样以 $N = 1$ 的单变量时间序列为例进行展示), 首先将时间序列划分为长度为 C 的滑动窗口片段, 作为模型输入。随后, 编码器部分通常由多层全连接神经网络 (MLP)、卷积神经网络 (CNN) 或循环神经网络 (RNN) 构成, 负责将输入的时间序列映射至潜在空间, 学习潜在变量的分布参数, 包括均值向量 $\boldsymbol{\mu} \in \mathbb{R}^{C \times D}$ 和方差向量 $\boldsymbol{\sigma}^2 \in \mathbb{R}^{C \times D}$, 其中 D 表示潜在空间维度。为了实现潜在空间中的可控分布建模, VAE 在编码阶段对潜在表示 $\mathbf{Z}$ 引入概率分布假设, 通常假定潜在变量服从标准正态分布, 即 $p(\mathbf{Z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ 。通过重参数化技巧 (reparameterization trick), 将潜在变量表示为 $\mathbf{Z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, 其中 $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, 从而实现可微分的随机采样。解码器模块接收潜在变量 $\mathbf{Z}$ 作为输入, 利用对称结构的神经网络将其映射回原始输入空间, 生成重构序列 $\hat{\mathbf{X}} \in \mathbb{R}^{C \times N}$ 。在训练阶段, 模型通过联合最小化输入序列 $\mathbf{X}$ 与重构序列 $\hat{\mathbf{X}}$ 之间的重构误差 (如均方误差, MSE) 以及潜在分布与先验分布之间的 KL 散度, 得到变分下界 (Evidence Lower Bound, ELBO) 损失函数, 形式如下:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(\mathbf{Z}|\mathbf{X})} [\|\mathbf{X} - \hat{\mathbf{X}}\|_2^2] + \text{KL}(q(\mathbf{Z}|\mathbf{X}) \parallel p(\mathbf{Z})) \quad (134)$$

其中, 第一项表示重构误差, 第二项表示编码器输出的潜在分布 $q(\mathbf{Z}|\mathbf{X})$ 与先验分布 $p(\mathbf{Z})$ 之间的 KL 散度。

在异常检测阶段, 由于模型在训练过程中主要学习了正常数据的潜在分布特性与重构模式, 因此对于正常数据, 其重构误差与潜在分布 KL 项较小; 而异常数据由于偏离训练数据分布, 模型难以有效重构, 通常表现为较大的重构误差和潜在分布偏离。通过设定联合重构误差与 KL 散度的阈值, 便可据此划分正常与异常数据, 实现异常检测任务。此外, 部分方法还可基于潜在空间中的分布特性, 结合聚类、密度估计或潜在空间可视化等方式辅助判别异常样本, 从而进一步提升检测性能。

基于 VAE 的时间序列异常检测模型关键技术 近年来, 基于变分自编码器 (Variational Autoencoder, VAE) 的时间序列异常检测方法同样得到了广泛关注, 其典型模型及特性如表 4.4 所示。相较于传统 AE 方法, VAE 通过对潜在空间中的隐变量分布进行建模, 有效提升了模型对数据分布特性的捕捉能力, 在生成建模与泛化能力上也更为出色。这类方法通常基于“编码器-解码器”框架, 联合最小化重构误差与潜在分布与先验分布之间的 KL 散度, 从而实现对时间序列的有效重构与异常检测。

具体来看, 部分方法 (如 OmniAnomaly、STORN 和 LSTM-VAE) 在编码器和解码器结构中采用循环神经网络 (RNN) 单元, 增强对时间序列动态依赖特性的建模能力; 而 GMM-VAE、SISVAE 等方法则在潜在空间中引入高斯混合建模、子空间独立性约束等机制, 提升对异常样本的识别能力。此外, 近年来如 VAE-GAN 和 TopoMAD 等方法, 通过引入对抗性训练策略、拓扑流形约束等方式, 进一步增强模型在复杂分布异常场景下的检测性能。总体而言, 这些基于 VAE 的方法围绕潜在空间分布建模、重构策略以及训练范式设计, 有效提升了异常检测表现, 并为多样化异常检测场景提

供了丰富的技术路径。

表 4.4: 基于 VAE 的异常检测方法总结。

模型	数据域	学习策略	神经网络类型	潜在空间分布假设	模型架构	训练方法
STORN [59]	时空域	重构	循环神经网络	条件高斯分布	编码器-解码器	无监督
GMM-VAE [25]	时域	重构	循环神经网络	高斯混合分布	编码器-解码器	无监督
LSTM-VAE [53]	时域	重构	循环神经网络	条件高斯分布	编码器-解码器	无监督
Buzz [11]	时域	重构	卷积神经网络	标准正态分布	编码器-解码器	无监督
OmniAnomaly [60]	时域	重构	循环神经网络	条件高斯分布	编码器-解码器	无监督
VELC [73]	时域	重构	循环神经网络	标准正态分布	编码器-解码器	无监督
SISVAE [37]	时域	重构	循环神经网络	条件高斯分布	编码器-解码器	无监督
VAE-GAN [51]	时域	重构	循环神经网络	标准正态分布	编码器-解码器	半监督
TopoMAD [28]	时空域	重构	图神经网络 + 循环神经网络	标准正态分布	编码器-解码器	无监督
PAD [9]	时域	重构	多层感知机	标准正态分布	编码器-解码器	无监督
InterFusion [40]	时空域	重构	卷积神经网络 + 循环神经网络	标准正态分布 + 条件高斯分布	编码器-解码器	无监督
MT-RVAE [63]	时空域	重构	Transformer	标准正态分布	编码器-解码器	无监督
RDSMM [38]	时域	重构	循环神经网络	Student's t 分布	编码器-解码器	无监督

1. **神经网络类型 (Type of neural network)** 与基于 AE 的方法类似，基于 VAE 的时间序列异常检测方法通常依赖神经网络结构对时间序列片段进行特征编码与重构。常用的神经网络类型包括：

- (a) **多层感知机 (Multilayer Perceptron, MLP)**：通过堆叠全连接层实现对时间序列片段的特征提取，适用于低维、无明显时序依赖的数据场景。PAD 方法基于变分自编码器 (VAE) 与多层感知机 (Multilayer Perceptron, MLP) 结构，专门面向 IT 运维场景下的时间序列预测与异常检测联合任务。该方法在编码器与解码器中均采用 MLP 结构，通过堆叠多层全连接层对输入时间序列片段进行特征提取，捕捉变量间静态关系与全局趋势特征，适用于依赖性较弱或变量关系较稳定的多维监控数据。编码器根据当前时刻及过去窗口内的观测值，提取潜在空间分布参数，解码器依据潜在变量重构未来序列，实现预测与异常检测功能统一建模。
- (b) **卷积神经网络 (Convolutional Neural Network, CNN)**：通过局部卷积操作捕捉时间序列中的局部模式，提升模型对局部异常和短期趋势的建模能力。InterFusion 方法将卷积神经网络 (CNN) 与循环神经网络 (RNN) 相结合，设计层次化的嵌入机制，专门针对多变量时间序列中的复杂依赖关系与异常模式检测问题。该方法首先通过一维卷积神经网络 (1D-CNN) 对时间序列片段进行局部卷积操作，提取短期趋势与局部模式特征，增强模型对

突发性异常和短周期扰动的感知能力。随后，利用门控循环单元（GRU）对局部卷积特征进行递归建模，捕捉多变量时间序列中的长期依赖关系与跨变量间动态交互模式。编码器部分基于 CNN 和 GRU 联合提取的时序上下文特征，估计潜在变量分布参数，实现潜在空间表征。解码器同样采用 GRU 结构，依据潜在变量重构原始多变量时间序列，保持时间维度与变量间依赖一致性。InterFusion 在此基础上设计了层次化嵌入机制，分别对变量间依赖关系（Inter-Metric Embedding）和时间维度动态特性（Temporal Embedding）进行建模。Buzz 方法同样采用 CNN 作为其核心组件，但其网络结构更为复杂，由三个子网络构成：变分网络、生成网络和判别网络。其变分网络（编码器）将一维时间序列窗口重塑为二维矩阵后，使用四个卷积层提取高层次特征，进而输出潜在空间分布的参数（均值和方差）。其生成网络（解码器）则采用对称结构，通过全连接层和四个转置卷积层从潜在变量中重构出原始时间序列窗口。此外，Buzz 引入的判别网络也使用卷积层来提取输入窗口的特征，以计算 Wasserstein 距离，这是其对抗训练机制的关键部分。通过 CNN 强大的局部特征提取能力，Buzz 能够有效捕捉复杂 KPI 中常见的剧烈抖动和局部模式。

- (c) **循环神经网络 (Recurrent Neural Network, RNN)**: 包括 vanilla RNN、长短期记忆网络 (LSTM) 和门控循环单元 (GRU) 等，能够有效建模时间序列中的长期依赖关系，是多数基于 VAE 的时间序列异常检测方法中的主流结构之一。STORN 通过在时间步之间显式建模潜在变量的递归依赖性，将潜在空间动态化，使得模型能够捕捉复杂的时间相关性和隐含结构。OmniAnomaly 则在 STORN 的基础上进一步优化，提出了一种基于变分递归结构的异常检测框架，专门针对多变量时间序列数据中的复杂依赖关系与异常模式。该方法在编码器和解码器中均采用双向 GRU 网络，以捕捉时间序列的前后双向上下文信息，提升对复杂动态相关性的建模能力。OmniAnomaly 引入了变分贝叶斯递归过程（Variational Recurrent Process），通过对潜在变量序列施加条件依赖关系，使每个时刻的潜在变量不仅依赖于当前观测数据，还依赖于前一时刻潜在状态，从而实现潜在空间的时序动态建模。此外，VELC 方法同样采用了基于循环神经网络的变分自编码器架构。为了增强对时间序列的建模能力，VELC 在编码器、解码器以及其特有的再编码器中均使用了双向 LSTM 网络。这种设计旨在充分捕捉时间序列中前后时刻的长期依赖关系与双向上下文信息。与 OmniAnomaly 等模型在潜在变量间建立时序依赖不同，VELC 的核心创新在于其约束网络，该网络作为一个可训练的记忆矩阵，通过对潜在向量施加稀疏相似性约束，限制模型对异常样本的重构能力，从而提升异常检测的区分度。GGM-VAE 采用门控循环单元（GRU）构建编码

器-解码器结构，以捕捉多维时间序列的时序依赖关系。其核心创新在于用高斯混合模型（GMM）先验替代标准 VAE 中的正态分布先验。编码器 GRU 提取时序特征后，输出 GMM 后验分布参数（各组件均值、方差及权重），将多模态时序数据映射至表达能力更强的混合潜在空间，从而精细刻画不同运行模式。解码器 GRU 对从混合分布中采样的潜在变量进行重构。异常检测时，若样本在所有混合组件下均呈现低重构概率，则判定为异常。该方法通过融合 GRU 的时序建模与 GMM 的多模态表征，显著提升了复杂多维时间序列的异常检测精度。SISVAE 同样采用 GRU 作为其主干循环神经网络，构建编码器-解码器结构。其核心创新在于为生成模型的输出引入了平滑性诱导先验。具体而言，SISVAE 在标准序列 VAE 的目标函数中增加了一个平滑性正则项，该正则项使用相邻时间步重构分布之间的 KL 散度，旨在惩罚生成模型输出的非平滑过渡，从而鼓励模型学习到更能反映时间序列内在连续性的“正常”模式，提升在异常污染数据下的鲁棒性。针对机器人交互任务中的异常检测，LSTM-VAE 采用了基于 LSTM 的变分自编码器架构。该模型利用 LSTM 在编码器和解码器中直接对多模态时序信号进行端到端建模，有效捕捉长程依赖关系。其核心优势在于能够理解任务执行进度的动态变化，从而无需依赖固定时间窗即可精准检测偏离正常模式的异常行为，如突发性碰撞或用户误操作。VAE-GAN 方法在编码器与解码器中均采用 LSTM 结构，利用递归机制捕捉时间序列中的长期依赖关系和复杂动态特性。不同于传统 VAE 仅依赖重构误差判别异常，LSTM-based VAE-GAN 引入判别器，对解码器生成的重构序列与真实序列进行对抗判别，判别器同样采用 LSTM 结构，保证对时间序列数据时序特性的感知能力。此外，RDSMM 方法将深度状态空间模型（Deep State Space Model, DSSM）与变分推断框架相结合，在状态转移模型与观测模型中均采用门控循环单元（GRU）结构，不同于 STORN 和 OmniAnomaly 中对潜在变量的标准递归建模方式，RDSMM 利用异常污染下的似然损失下界优化方法，有效提升潜在空间在异常干扰条件下的表征能力与稳定性，RDSMM 还设计了状态转移模型和观测生成模型双 GRU 结构，使得潜在状态不仅递归依赖于前一时刻潜在状态，同时考虑观测噪声扰动对状态演化过程的影响。

- (d) **Transformer**: 基于自注意力机制，通过动态分配注意力权重，捕捉时间序列中的全局依赖关系，具备良好的并行计算能力和长期序列建模优势，逐渐成为复杂时序异常检测任务中的主流方法。MT-RVAE 方法基于变分自编码器（VAE）与 Transformer 结构，面向多变量时间序列异常检测任务，充分利用自注意力机制捕捉全局依赖关系与复杂多变量交互模式。该方法在编码器和解码器中均采用多层 Transformer 模块，通过自注意力机制动态分配序

列中不同时间步和变量间的注意力权重，建模长期依赖性与变量间的潜在耦合关系。编码器基于多变量时间序列片段提取上下文相关的潜在表示，并估计潜在变量分布参数，解码器依据采样潜在变量重构原始序列。与传统 RNN 或 CNN 结构相比，MT-RVAE 借助 Transformer 的并行计算与全局建模优势，提升了对复杂时序异常模式的检测能力。

(e) **图神经网络 (Graph Neural Network, GNN)**：通过构建时间序列变量间或片段间的图结构，利用邻接关系和节点特征传播机制建模变量间的复杂依赖关系，适用于多变量时序或存在隐含拓扑结构的数据场景，近年来广泛应用于多变量异常检测与因果关系建模任务。TopoMAD 方法基于图神经网络 (Graph Neural Network, GNN) 与递归结构相结合，专为云系统环境中多变量时空依赖异常检测任务设计。TopoMAD 首先利用图卷积网络 (GCN) 建模云系统各组件或监控指标间的静态或动态邻接关系，捕捉多变量时间序列数据中的空间拓扑依赖模式。在时间建模方面，TopoMAD 采用门控循环单元 (GRU) 对图卷积提取的节点特征进行递归编码，捕捉时序维度上的动态变化与长期依赖特性。编码器部分基于 GCN 和 GRU 提取的时空联合特征，估计潜在变量分布参数，解码器则依据采样潜在变量重构原始图结构时间序列，实现异常模式的重构对齐。

2. **潜在分布假设 (Latent distribution assumption)** 相较于传统 AE 仅将输入映射至潜在空间，VAE 在编码阶段对潜在变量空间引入概率分布假设，通常假定潜在变量服从标准正态分布 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ ，并通过重参数化技巧实现可微分采样。部分方法如 GMM-VAE 和 SISVAE 进一步扩展了潜在空间分布形式，如高斯混合分布或子空间独立性假设，以增强模型对复杂异常模式的建模能力。总体而言，VAE 的潜在空间建模方式与 AE 相似，但在分布假设与正则化机制上具备更强的理论约束性与生成建模能力。

(a) **标准正态分布 (Standard Normal Distribution)**：最常用的潜在空间分布假设，设定潜在变量服从均值为 0、协方差为单位矩阵的正态分布。该设定简化了后验分布的建模与采样过程，有助于提升模型的稳定性与训练效率，是多数 VAE 及其变体默认采用的分布形式。在多数基于 VAE 的时间序列异常检测方法中，编码器将输入序列映射至潜在空间，假设潜在变量服从多维标准正态分布，即：

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (135)$$

其中， $\mathbf{z} \in \mathbb{R}^d$ 表示潜在变量， $\mathbf{I}$ 为 d 维单位矩阵。

以 VAE-GAN [51] 为例，编码器基于 LSTM 单元提取输入时间序列的时序依赖特征，并估计潜在变量的分布参数 μ_t 和 σ_t 。通过重参数化技巧 (reparam-

eterization trick) 将潜在变量表示为：

$$\mathbf{z}_t = \boldsymbol{\mu}_t + \boldsymbol{\sigma}_t \odot \boldsymbol{\epsilon}, \quad \text{其中 } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (136)$$

解码器依据采样得到的 $\mathbf{z}_t$ 重构时间序列片段，判别器对原始序列与重构序列执行对抗判别，提升异常检测中的重构精度与异常点敏感性。这种标准正态分布假设具备简洁的数学形式，便于优化，同时通过 KL 散度正则化项将潜在分布约束向标准正态靠拢：

$$D_{\text{KL}}(q_{\phi}(\mathbf{z}_t | \mathbf{x}_{1:T}) \parallel p(\mathbf{z}_t)) \quad (137)$$

从而确保潜在空间结构合理，提升模型对异常点分布的可区分性。

同样，VELC 方法也遵循了标准正态分布作为其潜在空间的基本先验分布假设。其编码器（双向 LSTM）将输入时间序列映射为潜在分布的参数（均值与方差），并通过重参数化技巧进行采样。VELC 的独特之处在于，它并未改变这个基本的先验假设，而是通过在采样得到的潜在变量 $\mathbf{z}$ 之后引入一个约束网络，对该变量进行“修正”。这个网络将 $\mathbf{z}$ 线性组合为一个新的、与正常数据潜在模式更相似的变量 $\hat{\mathbf{z}}$ ，再送入解码器进行重构。这一设计在保留标准 VAE 训练稳定性的同时，巧妙地约束了潜在空间的表达，使其更专注于正常样本的特征。

Buzz 方法在其潜在分布假设上同样采用了标准的多元高斯分布。其先验分布 $p(\mathbf{z})$ 设为 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ ，其后验分布 $q_{\phi}(\mathbf{z} | \mathbf{x})$ 也是一个由编码器输出参数确定的多元高斯分布。然而，Buzz 的核心创新在于其通过对抗训练与分区分析，巧妙地优化了基于该标准先验的模型。它通过理论推导，将其训练目标与一个观测分布 $p_{\theta}(\mathbf{x} | \mathbf{z})$ 为拉普拉斯分布 $\left(p_{\theta}(\mathbf{x} | \mathbf{z}) = \frac{1}{Z(\lambda)} \exp\{-\lambda \|\mathbf{x} - G(\mathbf{z})\|\}\right)$ 的特殊 VAE 变体联系起来。这种拉普拉斯观测似然比标准的高斯假设对异常值更不敏感，其重尾特性使其更能容忍和建模复杂 KPI 中存在的非高斯噪声和剧烈波动。因此，Buzz 可以被视为一个在标准正态潜在先验下，通过改进训练机制和观测似然来显著提升模型对复杂时间序列建模能力的杰出代表。

- (b) **高斯混合分布 (Gaussian Mixture Distribution)**：为了捕捉时间序列数据中固有的多模态特性（例如系统可能处于多种不同的“正常”运行模式），部分方法采用高斯混合模型作为潜在空间的先验分布。该假设认为潜在变量是由多个高斯分布以一定权重混合而成，能够更精细地刻画复杂的数据分布，从而更好地区分不同模式下的正常行为与异常行为。

以 GMM-VAE 为代表，该方法假设潜在变量 $\mathbf{z}$ 的先验分布 $p(\mathbf{z})$ 是一个由 K 个高斯分布组成的混合模型：

$$p(\mathbf{z}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{z} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (138)$$

其中, π_k 是第 k 个混合组件的权重 ($\sum_{k=1}^K \pi_k = 1$), $\boldsymbol{\mu}_k$ 和 $\boldsymbol{\Sigma}_k$ 分别是该组件的均值向量和协方差矩阵。

在模型训练过程中, 编码器不仅需要学习潜在变量的后验分布参数, 还需要推断输入数据属于哪个混合组件的后验概率 $q(w|\mathbf{x})$ 。其变分下界相较于标准 VAE 更为复杂:

$$\mathcal{L}_{\text{VAE}}^* = \mathbb{E}_{q(\mathbf{z}, w|\mathbf{x})} [\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q(\mathbf{z}, w|\mathbf{x}) \| p(\mathbf{z}, w)) \quad (139)$$

通过引入这种多模态的先验假设, GMM-VAE 能够学习到更具表现力的潜在空间。在异常检测时, 一个在所有混合组件下都具有低重构概率的样本, 更可能被判定为异常, 这提升了模型在复杂多模态数据上的检测精度。

- (c) **Student's t 分布 (Student's t Distribution)**: 为应对异常值敏感性和尾部分布厚度问题, 部分方法引入自由度参数可调的 Student's t 分布作为潜在空间假设, 其重尾性质使模型在面对噪声污染或异常点干扰时具备更强的鲁棒性。常见于异常检测与鲁棒生成模型场景, 能够有效缓解标准正态分布在异常数据建模中的局限。

以 RDSMM [38] 为代表, 针对异常污染严重的时间序列数据, 模型在潜在空间中引入重尾性质的 Student's t 分布假设。假设潜在变量 $\mathbf{z}_t$ 在每个时间步 t 满足:

$$p(\mathbf{z}_t) = \text{Student-}t(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t, \nu) \quad (140)$$

其中, $\boldsymbol{\mu}_t$ 和 $\boldsymbol{\Sigma}_t$ 分别为均值和协方差矩阵, ν 为自由度参数, 控制分布尾部厚度, 自由度越小, 分布尾部越厚, 对异常值越敏感。

RDSMM 中编码器基于深度状态空间模型 (Deep State Space Model, DSSM), 通过递归方式估计潜在状态的条件分布参数, 并采用重参数化技巧对 Student's t 分布进行采样。为保证可微分性, 采样采用标准化方法, 将 Student's t 分布的采样过程重写为:

$$\mathbf{z}_t = \boldsymbol{\mu}_t + \sigma_t \cdot \frac{\mathbf{u}}{\sqrt{v/\nu}} \quad (141)$$

其中, $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $v \sim \chi^2(\nu)$, 从而实现对异常污染时间序列的稳健建模。该重尾潜在分布假设显著提升了模型在异常点干扰、噪声污染和复杂依赖环境下的异常检测能力。

- (d) **条件高斯分布 (Conditional Gaussian Distribution)**: 在标准正态分布基础上, 进一步根据特定条件变量对潜在变量分布进行条件化建模, 形式为 $p(\mathbf{z}|\mathbf{c}) \sim \mathcal{N}(\boldsymbol{\mu}(\mathbf{c}), \boldsymbol{\Sigma}(\mathbf{c}))$ 。该方式能够增强模型对复杂上下文依赖或动态场景下潜在空间分布的适应性。根据条件变量的不同性质, 主要体现为以下两种形式:

- i. **时间条件先验 (Time-Conditioned Prior)**: 当条件变量是任务进度或时间本身时, 先验分布的参数是一个确定的函数。在机器人执行序列化任务 (如喂食) 时, 整个过程的正常模式会随着任务进度 (从开始到结束) 而发生系统性变化。为建模这种特性, LSTM-VAE 方法摒弃了静态的标准正态先验, 提出了一种动态的、基于进度的先验分布。该假设认为潜在变量的先验分布 $p(\mathbf{z}_t)$ 是一个高斯分布 $\mathcal{N}(\boldsymbol{\mu}_{p_t}, \mathbf{I})$, 但其均值 $\boldsymbol{\mu}_{p_t}$ 不是一个固定的零点, 而是随着任务执行的进度从初始点 p_1 线性地变化到终点 p_T 。

$$p(\mathbf{z}_t) = \mathcal{N}(\mathbf{z}_t; \boldsymbol{\mu}_{p_t}, \mathbf{I}) \quad (142)$$

这种先验为模型提供了关于“当前阶段潜在编码应该在哪里”的强时序线索。在训练过程中, VAE 的 KL 散度正则化项会迫使每个时间步的近似后验分布 $q_\phi(\mathbf{z}_t|\mathbf{x}_t)$ 靠近该时刻动态变化的先验, 而非一个全局固定的原点。这使得潜在空间能够根据任务进度自然地组织起来, 从而更清晰地区分正常执行过程中的预期变化与真正的异常偏差, 在机器人交互任务中实现了更敏感和更低误报的异常检测。

- ii. **状态条件先验 (State-Conditioned Prior)**: 当条件变量是序列中之前的潜在状态时, 先验分布成为一个随机递归过程。以 OmniAnomaly 为代表, 模型基于随机循环神经网络 (Stochastic RNN) 结构, 将多变量时间序列的潜在变量分布条件化为历史状态的函数。该方法假设潜在变量 $\mathbf{z}_t$ 在每个时间步 t 的分布依赖于前一时刻的潜在状态 $\mathbf{z}_{t-1}$, 即 $p(\mathbf{z}_t|\mathbf{z}_{t-1}) = \mathcal{N}(f(\mathbf{z}_{t-1}), \Sigma)$ 。类似地, STORN 等方法也采用了这种递归先验。编码器根据当前时间步的观测值及隐状态, 估计变分后验分布:

$$q_\phi(\mathbf{z}_t|\mathbf{x}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t^{(q)}, \boldsymbol{\Sigma}_t^{(q)}) \quad (143)$$

再通过重参数化技巧采样潜在变量:

$$\mathbf{z}_t = \boldsymbol{\mu}_t^{(q)} + \boldsymbol{\sigma}_t^{(q)} \odot \boldsymbol{\epsilon}, \quad \text{其中 } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (144)$$

解码器则根据当前时间步的 $\mathbf{z}_t$ 重构观测值 $\mathbf{x}_t$ 。这种状态条件先验假设潜在变量序列自身构成一个动态系统, 能捕捉复杂的时间演化模式, 其参数变化依赖于前一状态的随机结果, 增强了模型对复杂时序依赖的建模能力, 从而提升了在异常污染、上下文依赖性强的多变量时序数据场景中的异常检测鲁棒性。SISVAE 同样采用了这种状态条件先验。其生成模型中, 潜在变量 $\mathbf{z}_t$ 的先验 $p_\theta(\mathbf{z}_t|\mathbf{x}_{<t}, \mathbf{z}_{<t})$ 是一个高斯分布, 其参数 $(\boldsymbol{\mu}_{0,t}, \boldsymbol{\sigma}_{0,t})$ 由先验网络根据前一时刻的 RNN 隐藏状态 $\mathbf{h}_{t-1}$ 计算得出。这使得潜在空间的演化能够捕获序列的时序动态。SISVAE 的独特之处在于, 它

不仅对潜在变量的演化进行建模，还通过其提出的平滑性正则项，对生成模型输出的观测分布的平滑性进行显式约束。该正则项衡量相邻时间步重构分布 $p_{\theta}(\hat{\mathbf{x}}_{t-1}|\cdots)$ 与 $p_{\theta}(\hat{\mathbf{x}}_t|\cdots)$ 之间的 KL 散度，作为一种对观测空间平滑性的分布级约束，进一步强化了模型对正常时序连续性的学习。

总体而言，条件高斯分布通过引入时间或状态等条件变量，使潜在先验能够动态变化，更好地契合了时间序列数据的非平稳特性，是提升模型对复杂动态场景适应性的重要途径。

4.4.7 循环神经网络

循环神经网络（Recurrent Neural Network, RNN）是一种专为处理序列数据设计的神经网络模型。与传统的前馈神经网络不同，RNN 通过循环连接允许信息在不同时间步之间传递，从而捕捉序列数据中的时序依赖关系。在每个时间步，RNN 都有一个隐藏状态，该状态会根据当前输入和前一时间步的隐藏状态进行更新，使其能够记忆之前输入的信息，并将其应用于当前的预测任务。这种特性使 RNN 非常适合处理时间序列数据等序列相关的任务。

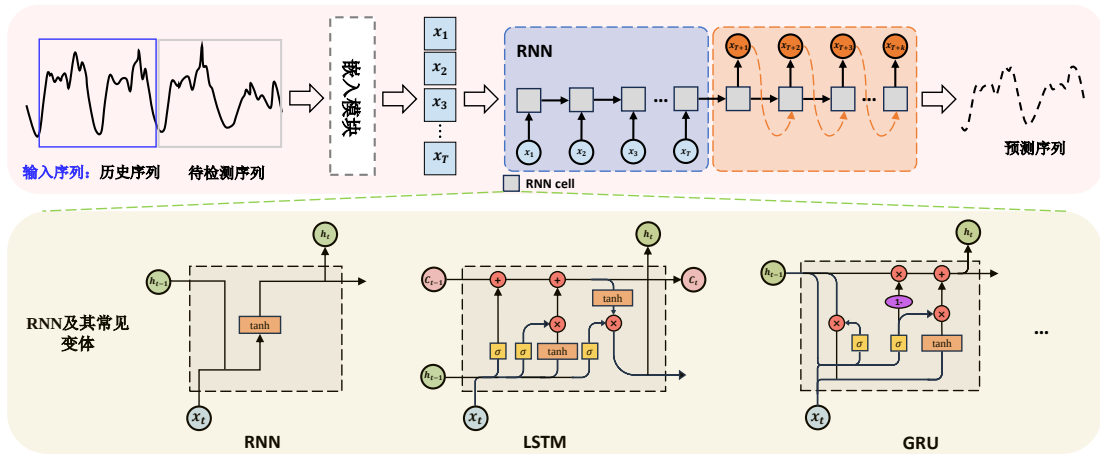


图 4.8: RNN 异常检测

基于循环神经网络的时间序列异常检测模型的通用架构 如图 4.8 所示，基于循环神经网络（Recurrent Neural Network, RNN）的时间序列异常检测方法主要依赖于 RNN 模型在建模序列数据中的时序依赖特性。对于给定的观测时间序列

$$\mathbf{X} = (\mathbf{x}_{t-C+1}, \mathbf{x}_{t-C+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{C \times N} \quad (145)$$

（图中以 $N = 1$ 的单变量时间序列为例展示），通常将时间序列划分为长度为 C 的滑动窗口片段，作为模型输入。随后，通过数据标准化或归一化等预处理步骤，将输入序列映射至统一尺度空间。

在模型预测阶段，输入的时间序列片段依次送入 RNN 网络单元，模型根据当前时间步的输入特征 $\mathbf{x}_t$ 和前一时刻的隐藏状态 $\mathbf{h}_{t-1}$ ，递归计算当前隐藏状态 $\mathbf{h}_t$ 。该过程沿时间轴展开，逐步捕捉序列中的时序依赖特性。RNN 的输出通常为下一时刻的预测值 $\hat{\mathbf{x}}_{t+1}$ 或滑动窗口内所有时间步的预测序列。模型通过比较预测值与真实观测值之间的残差，进而判断异常点。

图 4.8 下半部分展示了常见的 RNN 单元类型，包括标准 RNN、长短期记忆网络 (Long Short-Term Memory, LSTM) 和门控循环单元 (Gated Recurrent Unit, GRU)：
基于循环神经网络 (RNN) 的时间序列异常检测模型 循环神经网络 (RNN) 是一类适合处理序列数据的神经网络，它通过在时间步之间共享隐藏状态来建模序列中的时间依赖关系。对于输入序列 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ ，RNN 会递归更新隐藏状态 $\mathbf{h}_t$ 并用于预测或重构序列。在异常检测中，RNN 可以捕捉短期依赖，但在处理长时间序列时容易出现梯度消失或梯度爆炸问题，从而对跨长时间步的异常不敏感。实际上，在时间序列异常检测任务中，纯 RNN 的应用并不多，大多数方法更倾向于使用其改进变体，如 LSTM 或 GRU，以更好地捕捉长期依赖并提高检测性能。

基于长短期记忆网络 (LSTM) 的时间序列异常检测模型 LSTM 是 RNN 的一种改进，它引入了遗忘门、输入门和输出门的门控机制，有效缓解了梯度消失问题。门控机制允许网络选择性地保留或丢弃信息，从而捕捉长期依赖。其核心思想可以简化表示为：

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad (146)$$

其中 $\mathbf{c}_t$ 为细胞状态， $\mathbf{h}_t$ 为隐藏状态， $\mathbf{f}_t$ 、 $\mathbf{i}_t$ 、 $\mathbf{o}_t$ 分别表示遗忘门、输入门和输出门。LSTM 能够捕捉长时间跨度的异常模式，因此在复杂时间序列异常检测任务中应用广泛。

基于门控循环单元网络 (GRU) 的时间序列异常检测模型 GRU 是 LSTM 的简化版本，它将遗忘门和输入门合并为更新门，并引入重置门，从而减少模型参数。GRU 的隐藏状态更新可以表示为：

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t, \quad (147)$$

其中 $\mathbf{z}_t$ 为更新门， $\tilde{\mathbf{h}}_t$ 为候选隐藏状态。GRU 结构更简单，计算效率高，在中短期异常检测上表现良好，同时仍能保留一定的长期依赖信息。

不同类型的 RNN 单元在序列记忆机制与信息流控制策略上各具特点，研究者可根据具体的时间序列异常检测任务需求，选择适配的递归单元结构，以在复杂动态环境下实现对异常模式的有效捕捉与识别。

在后续的工作中，RNN 常常被用作异常检测系统中的骨干网络，与其他方法结合，以增强其在异常检测任务中的性能。

基于 RNN 的时间序列异常检测模型关键技术 近年来，基于循环神经网络 (Recurrent Neural Network, RNN) 的时间序列异常检测方法持续受到学术界与工业界的广泛关

注。依托 RNN 模型对序列数据时序依赖建模能力，该类方法能够有效捕捉时间序列中的动态模式变化，从而提升异常检测的准确性与适应性。表 4.5 总结了近年来典型基于 RNN 的时间序列异常检测方法及其主要特性。

表 4.5: 基于 RNN 的异常检测方法总结。

模型	数据域	学习策略	特征提取设计	模型架构	训练方法
LSTM-AD [45]	时域	预测	双层 LSTM	仅编码器	无监督
DeepLSTM [7]	时域	预测	多个周期性 LSTM 层	仅编码器	半监督
LSTM-PRED [21]	时域	预测	LSTM RNN	-	无监督
LGMAD [17]	时域	预测	LSTM+GMM	仅编码器	半监督
GGM-VAED [26]	时域	预测	GRU+VAE	编码器-解码器	无监督
SLMR [47]	时域	预测 + 重构	GRU+ 注意力机制	编码器-解码器	自监督
TCQSA [41]	时域	预测	LSTM+CNN+ 注意力机制	-	有监督
THOC [57]	时域	分类	扩张 RNN+ 层次聚类	仅编码器	无监督
AD-LTI [65]	时域	预测	时间序列分解 +GRU	-	无监督
TAnoGAN [3]	时域	重构	LSTM+GAN	编码器-解码器	无监督
VLSTM [20]	时域	分类	多 LSTM 并行	编码器-解码器	有监督

具体来看，现有方法主要围绕**学习策略**、**特征提取设计**、**模型架构**以及**训练范式**等维度展开设计。其中，学习策略方面可分为基于**预测误差**的异常检测方法与基于**分类判别**的策略，分别利用预测残差或分类结果作为异常性指标。在 RNN 单元类型方面，当前方法多采用标准 LSTM 与 GRU 结构，部分方法融合多种递归单元或引入改进型结构，以提升对复杂异常模式的表征能力。

此外，为增强模型对复杂时间依赖关系的建模能力，部分方法在网络结构中集成了**时间特性建模机制**，如跳跃连接、时间门控或连续时间建模策略，缓解长序列依赖与非均匀采样问题。

1. **特征提取 (Feature Extraction)** 的目的是从原始时序数据中构建一个能够精确表征其正常行为模式的特征空间，从而使得异常数据因偏离该空间而被有效识别。与预测或分类任务不同，异常检测通常缺乏异常样本进行监督学习，因此，构建一个全面、鲁棒的正常模式表征至关重要。这要求特征提取模块不仅能感知局部点状的剧烈变化，更能捕获隐藏在长期上下文中的集体异常和趋势性异常。围绕这一目标，现有的特征提取设计主要可分为基于单一网络与基于复合架构两种思路。

(a) **基于单一网络的特征提取** 侧重于模型的简洁与高效。这类方法假定一个强大的序列模型足以学习到正常的时序动态。例如，通过堆叠长短时记忆网络 (LSTM) 来建模序列的长期依赖，并将重构误差或预测误差作为异常分数。

然而，单一模型（尤其是普通 RNN 或 LSTM）往往受限于其固定的时间感受野和单一的层次结构，难以同时捕捉不同时间尺度下的正常模式，对于在宏观趋势上才显现的细微异常不够敏感。例如，LSTM-AD 方法通过堆叠 LSTM 网络对时间序列进行建模，学习序列的长期依赖结构。具体来说，LSTM-AD 使用训练集中的正常序列对网络进行训练，并通过预测下一步或多步的序列值来计算重构误差。测试阶段，重构误差或预测误差被用作异常分数：当观测值与模型预测值偏离较大时，即被判定为异常。该方法充分利用 LSTM 的长期依赖能力，能够识别跨多个时间步的异常模式，但仍受限于单一网络的表达能力，对于多时间尺度或复杂多变量序列的异常可能不够敏感。

- (b) **基于复合架构的特征提取** 则成为当前研究的主流，其核心思想是通过融合不同神经网络的优势，构建一个多尺度的特征空间，以更全面地覆盖正常模式的各个方面。例如，CNN-LSTM 架构利用卷积神经网络（CNN）提取局部子序列的形态特征，再由 LSTM 对这些局部特征序列进行时间建模，从而结合了局部敏感性与中期依赖性。

在此基础上的一个重要演进是 TCQSA，其核心在于协同利用 LSTM 与 CNN 进行特征提取，并引入注意力机制进行精细调制。其流程可精确描述为：

- i. **整体趋势提取**：首先，一个堆叠双向 LSTM（SB-LSTM）网络处理整个准周期片段，旨在捕获其全面的、序列级的整体时间依赖与宏观变化趋势。
- ii. **局部特征挖掘**：然后，LSTM 所有时间步的精细化输出与原始输入序列被共同组织成一个特征图，送入一个二维 CNN（TD-CNN）中。CNN 利用其卷积核在该特征图上进行扫描，以专注于挖掘序列中关键的局部形态特征（如特定波形、尖锐峰值等）。

为了进一步提升特征质量，TCQSA 嵌入了三种注意力机制：趋势注意力门（TAG）被嵌入 LSTM，用于根据输入点的位置和上下文重要性来调制其输出的趋势信息；特征注意力机制（FAM）与位置注意力机制（LAM）被嵌入 CNN，分别用于增强判别性强的特征通道以及关键时间位置上的特征响应。这种 LSTM 与 CNN 的协同，辅以注意力机制的精细化调制，使得 TCQSA 能够为后续分类器提供对正常波动模式极其鲁棒且区分性强的特征表示。

另一条值得关注的技术路线以时序层次一类网络（THOC）为代表，它通过一种结构化的设计来实现多尺度特征学习。THOC 的核心在于统一使用扩张循环神经网络（Dilated RNN）并结合可微分的层次聚类，从而系统地捕捉不同时间尺度的模式。其设计思路如下：

- i. **多尺度特征提取**：THOC 使用一个 L 层的扩张 RNN 作为骨干网络。其层与层之间采用指数增长的跳跃连接，这使得底层神经元感受野小，专注

于短期模式；而高层神经元感受野大，专注于长期趋势，从而结构化地生成了多尺度时序特征 $\{f_t^1, f_t^2, \dots, f_t^L\}$ 。

- ii. **层次化特征融合**：与仅使用最后一层特征或简单拼接不同，THOC 引入了一个可微分层次聚类过程。该过程从最底层开始，将特征软分配到多个聚类中心；随后，将聚类得到的表征与下一层（更长期尺度）的原始特征进行拼接和变换，再输入到下一层聚类中。这种自底向上的融合机制，使得宏观趋势信息能够与微观波动模式进行交互，最终形成一个层次化的多分辨率特征表示。

除此之外，SLMR 方法也同样值得关注，其核心思想在于同时捕获短期突发异常和长期趋势性异常，通过掩码机制对不同时间尺度的特征进行选择性地编码，其流程可以描述为：

- i. **短期特征掩码**：针对序列中局部时间窗口，SLMR 学习一个短期掩码矩阵，该掩码能够突出突发变化或尖锐异常点，从而增强模型对短期异常的敏感性。
- ii. **长期趋势掩码**：同时，SLMR 使用一个长期掩码对序列的全局模式进行建模，使模型能够捕获跨较长时间步的趋势性异常。
- iii. **掩码融合与特征表示**：短期和长期掩码生成的特征表示被融合成最终的多尺度表征，送入下游异常评分模块。该设计在无监督场景下无需异常标签，即可同时感知多种时间尺度下的异常行为。

SLMR 方法通过短期与长期掩码的协同作用，实现了对多变量时间序列中局部突发异常和全局趋势异常的统一建模，从而在多尺度异常检测任务中表现出优异的鲁棒性和判别力。

4.4.8 图神经网络

得益于 GNN 的成功，越来越多的工作将 GNN 引入到时间序列异常检测工作中 [31]。GNN 能进行异常检测的核心在于通过将通道建模为图中的节点，有效地对时间序列数据中不同通道间的复杂依赖关系进行建模。在检测期间，无法符合低差异预期的模型行为被判断为异常。其中，常见的诊断异常方式包括重构和预测，一些新颖的方法探索了通过分析 GNN 图结构随时间的变化来检测异常的可能性。

基本流程 以重构为例，基于 GNN 的异常检测方法流程如图 4.9 所示。给定一条待检测序列 $\mathbf{X} = (\mathbf{x}_{t-C+1}, \mathbf{x}_{t-C+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{C \times N}$ （图中以 $N=3$ 的多变量时间序列为例进行展示），首先运用包含线性变换、卷积等操作的嵌入模块，将输入的序列从原始维度变换到指定的隐藏空间维度，生成初始表示 $\mathbf{C}_0 \in \mathbb{R}^{N \times D}$ ，其中 N 表示时间序列维度， D 表示嵌入维度。随后， $\mathbf{H}_0$ 作为输入传递到主干网络中，通过空域模块的 GNN 捕获序列

空间维度的依赖，同时通过时域模块的注意力机制、卷积等捕获时间维度的依赖（部分方法不建模时间维度依赖），并生成表征（Representation）。同时进入空域模块的还有由先验知识、全连接或嵌入向量相似度等方式得到的邻接矩阵 $A \in \mathbb{R}^{N \times N}$ ，其中 A_{ij} 表示节点 i 与 j 的关联度。最终表征由重构模块通过线性变换将其映射到目标维度，生成重构结果 $\hat{\mathbf{X}} = (\mathbf{x}_{t-C+1}, \mathbf{x}_{t-C+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{C \times N}$ 。

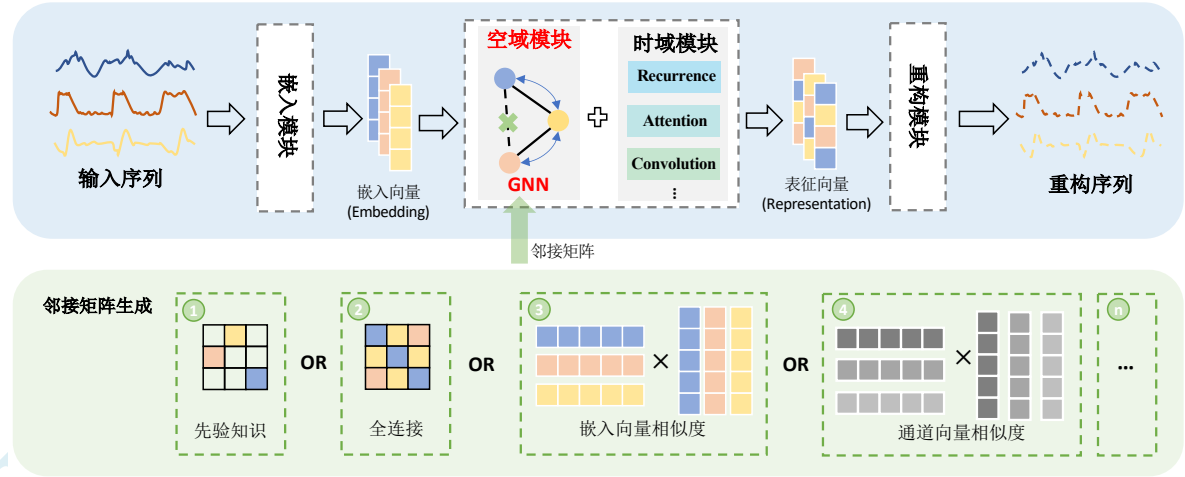


图 4.9: 基于 GNN 的无监督异常检测方法。

基于 GNN 的时间序列异常检测模型 图神经网络（GNN）是一种专为处理具有图结构的数据而设计的深度学习模型。其主要思想是通过迭代地聚合邻近节点的信息来更新每个节点的表示，从而有效地捕捉图中复杂的依赖关系和模式。在形式上，一个图可被定义为 $\mathcal{G} = (A, H)$ ，其中 A 代表邻接矩阵， h_i 表示节点 i 的特征向量。在这一框架下，每个节点通过聚集其邻居节点的特征信息，并经过特定的非线性变换来生成更新后的特征向量。GNN 的学习过程基于两个关键步骤：消息传递（Aggregate($\cdot$ ））和消息聚合（Combine($\cdot$ ））。消息传递负责收集来自邻接节点的信息，消息聚合则结合这些信息对当前节点的状态进行更新，以形成更丰富的节点表示 [31]。这两个步骤的协同作用使得 GNN 能够有效地学习图中的复杂模式和关系。用 W 表示参数矩阵， $\mathcal{N}_i$ 表示节点 i 的邻接节点，则节点 i 第 k 层的表征 $h_i^{(k)}$ 的更新可以建模为以下过程：

$$w_i^{(k)} = \text{Aggregate}(\{h_j^{(k)}\}), j \in \mathcal{N}_i \quad (148)$$

$$h_i^{(k)} = \text{Combine}(h_i^{(k-1)}, w_i^{(k)}) \quad (149)$$

基于 GNN 的时间序列异常检测模型关键技术 表 4.6 中总结了基于 GNN 的异常检测方法的关键技术，并对相关方法进行分类总结：

1. **预定义图结构** 从预定义图结构来看，现有方法可以分为两类：一类需要预定义的先验图结构 [23, 72]，这类方法通常需要有特定领域专家经验构建的领域知识来

表 4.6: 基于 GNN 的异常检测方法总结。

模型	提出年份	学习策略	模型学习时间依赖的方式	预定义图结构	图结构类型
CCM-CDT [23]	2019	重构	时域 循环	需要	-
MTAD-GAT [78]	2020	预测 重构	时域 注意力	不需要	-
GDN [16]	2021	预测	-	不需要	静态
GANF [13]	2020	重构 关联差异	时域 循环	不需要	静态
Grelen [75]	2022	重构 关联差异	时域 混合	不需要	动态
FuSAGNet [27]	2022	预测 重构	时域 循环	不需要	静态
VGCRN [10]	2022	预测 重构	时域 循环	不需要	静态
GIF [72]	2022	重构	-	需要	静态
GST-GL [79]	2023	预测	时域 卷积	不需要	静态

提前确定邻接矩阵节点之间的依赖；另一类方法不需要预定义图结构 [79, 10, 13]，这类方法通常假设图结构是全连接的，或者在模型训练的过程中逐渐构建图结构。

在很多在实际应用情况下，图结构的先验信息可能是不完整或缺失的。MTAD-GAT 通过假设多元时间序列中的所有通道形成一个全连接图来简化这一问题。尽管这种方法提供了一个简单的解决方案，但它可能导致将一些实际上无关或弱相关的通道错误地连接起来，从而引入不必要的噪声，进而可能削弱模型的整体性能。为了解决上述挑战，GDN 提出了一种更加灵活和自适应的方法来处理多传感器数据中的复杂关联性。考虑到不同传感器可能展现出极为不同的特性，并且这些特性之间的相互作用往往十分复杂，GDN 采取了更为灵活的方式来表示每个传感器。具体来说，它为每一个传感器分配了一个随机初始化的向量 v_i ，这个向量旨在捕捉该传感器的独特特征。随着模型训练过程的推进，这些向量会根据传感器间的真实交互情况动态调整，以更好地反映它们之间的真实关系。

$$v_i \in \mathbb{R}^d, \text{for } i \in \{1, 2, \dots, N\} \quad (150)$$

将这些嵌入之间的相似性来表示行为的相似性，因此，具有相似嵌入值的传感器应该具有较高的相互关联倾向。作者认为传感器之间的关系是非对称的，因此这里使用了有向图进行建模。构建稀疏的邻接矩阵 A 时，首先计算传感器 i 的嵌入向量与候选关系 j 的归一化点积 e_{ij} ，然后挑选最大的 k 个值：

$$e_{ji} = \frac{v_i^\top v_j}{\|v_i\| \cdot \|v_j\|} \text{for } j \in C_i \quad (151)$$

$$A_{ji} = \mathbb{1}\{j \in \text{TopK}(\{e_{ki} : k \in C_i\})\} \quad (152)$$

其中, C_i 是根据先验知识为每个传感器构建的候选关联者集合, 当没有先验知识时, 候选关联者集合就是除了本身以外的所有节点。

在相似的框架下, FuSAGNet 的作者认为传感器在不同过程中会存在一定的相关性甚至因果关系, 给定传感器相互作用的关系可能是顺序循环排列的, 而 GDN 缺乏了对通道间动态关联的考虑以及通道内的时域依赖关系。因此, 作者在不同过程为传感器分别进行嵌入, 并选择使用循环单元捕获过程中一组传感器的时间依赖性。具体来说, 作者为每个通道每个过程随机初始化一个向量, 并使用循环网络处理他们, 最后将他们串联 (concatenate) 起来作为最终的传感器特征表示。

2. **图结构类型** 从图结构类型来看, 现有方法可以分为两类: 一类方法学习到的图结构是静态的 [16, 27], 这类方法假设节点间的交互模式在时间维度上保持稳定; 另一类方法学习到的图结构是动态的 [75], 这类方法依据当前时序的特征构建图结构, 且图结构会随着特征的变化而动态变化。GReLeN 是第一个通过学习到的动态图从关联差异的角度进行异常检测的方法。GReLeN 采用 VAE 结构对输入进行重构, 并动态地构建图结构。在进行异常得分计算时, GReLeN 的作者认为传感器之间的特定依赖关系也揭示了异常, 因此将通道节点出度与入度和值的变化作为一个异常得分。GANF 假设异常发生在分布的低密度区域, 提出了一种图增强的归一化流模型来计算数据的分布密度, 其中密度越低表明异常的可能性越大。

4.4.9 对比学习

近年来, 对比学习 (Contrastive Learning) 在计算机视觉 (CV) 和自然语言处理 (NLP) 等领域取得了显著成果, 其核心思想是通过构建相似与不相似样本对来学习判别性表示, 从而在无监督条件下获得具有泛化能力的特征表示。受此启发, 越来越多的研究将对比学习范式引入到时间序列异常检测任务中, 以期在缺乏标注数据的情况下, 自动捕获正常模式的判别性特征。如图 4.10 所示, 基于对比学习的时间序列异常检测方法通常属于无监督学习框架, 其主要思想是通过数据增强 (Data Augmentation) 生成时间序列的不同视图, 经过编码器 (Encoder) 提取特征后, 利用对比损失函数 (Contrastive Loss) 最大化正样本对之间的相似性并最小化负样本对之间的相似性, 从而获得鲁棒且可分的表征空间。图 4.10 展示了模型的核心结构, 即由双分支编码器和相似度比较模块组成的对比学习框架; 此外, 部分研究还在对比学习阶段引入预测任务或重构任务作为辅助约束, 以进一步提升表征学习的质量。

基于对比学习的时间序列异常检测模型的通用架构 基于对比学习 (Contrastive Learning) 的时间序列异常检测模型通过构建判别性的表征空间, 使模型能够在无监督条件

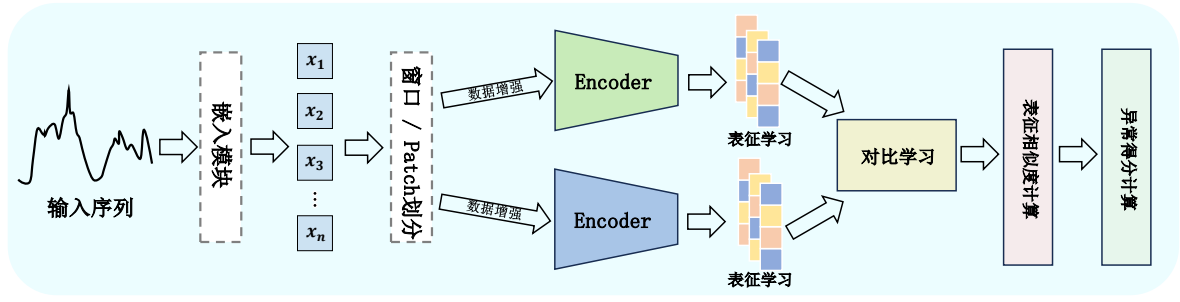


图 4.10: 基于对比的异常检测

下区分正常与异常模式。如图 4.10 所示，给定输入时间序列 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times N}$ ，模型首先对原始序列进行滑动窗口划分或片段切分（Segmentation），得到多个长度为 L 的子序列。随后，通过时间抖动、加入噪声、时间掩蔽等数据增强（Data Augmentation）操作，为每个子序列生成两种不同的视图 $\mathbf{X}_a$ 和 $\mathbf{X}_b$ 。

两种视图分别输入到共享参数的编码器（Encoder）中，提取其潜在表示：

$$\mathbf{h}_a = f_\theta(\mathbf{X}_a), \quad \mathbf{h}_b = f_\theta(\mathbf{X}_b), \quad (153)$$

其中 $f_\theta(\cdot)$ 表示编码网络，可以是 CNN、RNN 或 Transformer 结构等。模型通过最小化对比损失（Contrastive Loss），使同一序列的不同视图（正样本对）在表征空间中相似，而不同序列（负样本对）尽可能远离，典型形式如：

$$\mathcal{L}_{CL} = -\log \frac{\exp(\text{sim}(\mathbf{h}_a, \mathbf{h}_b)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{h}_a, \mathbf{h}_j)/\tau)}, \quad (154)$$

其中 $\text{sim}(\cdot)$ 表示相似度函数（如余弦相似度）， τ 为温度系数。

在推理阶段，模型利用已学习的编码器生成输入序列的表征，通过计算不同时间片或视图之间的相似度来进行异常评分（Anomaly Scoring）。若相似度显著低于阈值，则认为该时间段存在异常。该类模型通常不依赖显式的重构过程，而是通过对比目标隐式地学习时间序列的正常模式，从而在检测未知异常时具有更强的泛化能力。

基于对比学习的时间序列异常检测模型关键技术 近年来，涌现了许多基于对比学习的时间序列异常检测模型，表 4.7 总结了常见的方法：

1. 数据增强（Data Augmentation）

在对比学习（Contrastive Learning, CL）框架下，数据增强是整个表示学习过程的起点与核心。其目标是构造出在语义上等价但在形式上不同的样本对，使模型能够学习到“变化不变性”（Invariance to Transformation）特征。与计算机视觉中常见的图像旋转、裁剪或颜色扰动不同，时间序列数据的增强需要同时保持时间依赖性与序列模式的语义一致性，因而设计更加复杂和具有针对性。总体来

表 4.7: 基于对比学习的时间序列异常检测方法总结。

模型	数据域	学习策略	数据增强	对比范式	模型架构	训练方式
TimeAutoAD [30]	时域	表征学习	时频扰动	正负样本对比	仅编码器	自监督
TFMAE [19]	时频域	重构 + 对比	掩蔽增强	视图对比	仅编码器	无监督
TS-TCC [18]	时域	表征学习	多视图增强	视图对比	仅编码器	自监督
DCdetector [68]	时域	表征学习	多视图增强	视图对比	仅编码器	无监督
CL-TAD [50]	时域	重构 + 对比	掩蔽增强	重构对比	编码器-解码器	无监督
DConAD [76]	时域	表征学习	差分增强	视图对比	仅编码器	自监督

看，基于对比学习的时间序列异常检测方法通常围绕**时序扰动增强**、**掩蔽增强**与**多视图/差分增强**三类策略展开。

(a) **时序扰动增强**: 该类增强方法直接作用于原始时间轴, 通过对序列进行轻量级扰动生成正样本视图, 从而在不同时间尺度下提升表征鲁棒性。TS-TCC [18] 是代表性工作之一。它通过弱增强 (如时间缩放、随机裁剪、平滑扰动) 与强增强 (如时间抖动、顺序打乱) 构建多视图样本, 模型同时最小化增强视图间的时间依赖差异。形式化地, 设时间序列 $x = [x_1, x_2, \dots, x_T]$, 增强后的两种视图分别为 $x_w = \mathcal{T}_w(x)$ 与 $x_s = \mathcal{T}_s(x)$, 则对比目标可定义为:

$$\mathcal{L}_{\text{temporal}} = - \sum_{t=1}^T \log \frac{\exp(\text{sim}(z_t^w, z_t^s)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(z_t^w, z_j^s)/\tau)} \quad (155)$$

其中 $\text{sim}(\cdot)$ 表示余弦相似度, τ 为温度系数。通过跨视图的一致性约束, 模型不仅能捕捉局部动态变化, 还能学习到跨时间步的稳定依赖结构。

除了时序扰动, TimeAutoAD [30] 引入了**噪声增强**思想。它基于时间自编码器生成伪负样本, 通过向原始序列添加轻微高斯噪声或随机漂移来模拟异常:

$$x_{\text{neg}} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (156)$$

这种方法在保持时序结构的同时, 引导模型学习异常方向上的微小差异, 使其在潜在空间中自然形成异常区分边界。

(b) **掩蔽增强 (Mask-based Augmentation)**: 掩蔽策略最早来源于视觉领域的 Masked Autoencoder (MAE), 但在时序任务中具有新的语义。TFMAE [19] 将掩蔽思想迁移到时频域, 利用时间和频率的双重扰动构建不同视图。具体而言, 它首先计算滑动窗口下的波动性指标:

$$v_t = \frac{\sqrt{\frac{1}{W-1} \sum_{k=t-W}^t (x_k - \mu_t)^2}}{\mu_t} \quad (157)$$

其中 μ_t 为窗口内均值, v_t 越大表示波动越剧烈。模型依据 v_t 选择性掩蔽高变异段, 从而保留平稳区域用于上下文建模。同时, 在频域中随机屏蔽部分频带, 迫使模型在时频双空间下学习完备的信号重建能力。这种基于掩蔽的增强机制等价于对模型施加信息约束, 促使其捕获全局相关性并具备重构缺失模式的能力, 对检测异常尖峰和突变尤为有效。

- (c) **多视图与差分增强**: 随着对比学习在时间序列中的扩展, 多视图增强成为趋势。DCdetector [68] 提出了**双重注意力对比结构**: Patch 间注意力用于捕获长期依赖, Patch 内注意力用于捕捉局部动态变化。两个分支分别生成全局与局部视图, 并在投影空间进行对比学习, 从而强化多尺度一致性。DConAD [76] 进一步引入了**差分增强**机制, 通过对输入序列进行时间差分变换, 强化模型对突变型异常的感知能力。差分操作定义为:

$$\Delta x_t = x_t - x_{t-1} \quad (158)$$

在此基础上构建 $(x_t, \Delta x_t)$ 的视图对, 使模型能够学习变化率层面的时序特征。这种基于导数的增强方式在检测趋势突变和结构性异常时表现突出。

总体而言, 基于对比学习的时间序列增强策略正从单一扰动向**多维融合**方向发展: 时域增强关注时间依赖保持, 频域增强提升模式鲁棒性, 差分与掩蔽增强强化变化建模。通过组合多种增强策略, 模型能够在不依赖标签的情况下获得丰富的语义一致性约束, 从而学习出稳健且异常敏感的时序表征。

2. 对比范式 (Contrastive Paradigm)

对比范式决定了模型如何在潜在空间中度量“相似”与“不相似”。它定义了正负样本对的生成方式及优化目标, 是对比学习能否成功的关键。在时间序列异常检测中, 由于缺乏显式标签, 模型通常通过不同视图的构造、重构误差的约束或自监督判别任务来学习表示。根据范式差异, 可分为三种主要类别: **视图对比**、**正负对比**与**重构对比**。

- (a) **视图对比 (View-level Contrast)**: 该类方法通过不同增强视图间的对比优化, 使模型学习到跨扰动不变的时间表示。典型代表 TS-TCC [18] 采用跨视图预测 (Cross-view Prediction) 结构, 使模型在强弱增强视图间保持一致性。DCdetector [68] 则在局部与全局视图之间构建 InfoNCE 损失:

$$\mathcal{L}_{\text{view}} = -\log \frac{\exp(\text{sim}(z_g, z_l)/\tau)}{\sum_n \exp(\text{sim}(z_g, z_n)/\tau)} \quad (159)$$

其中 z_g 表示全局特征, z_l 表示局部特征。通过对齐不同尺度的特征空间, 模型在时序多层次上均能保持一致的表征稳定性。

- (b) **正负样本对比 (Positive-Negative Contrast)**: 与传统分类任务类似, 正负对比范式通过最大化正样本相似性、最小化负样本相似性来优化潜空间分布。TimeAutoAD [30] 提出了自监督正负样本生成机制, 利用噪声扰动产生伪异常样本 (x_{neg}), 并以二元交叉熵作为对比目标:

$$\mathcal{L}_{PN} = f_{BCE}(o_{pos}, 0) + f_{BCE}(o_{neg}, 1) \quad (160)$$

其中 o_{pos} 和 o_{neg} 分别为模型对正负样本的输出。通过区分正常与伪异常样本, 模型能在无监督环境下形成隐式异常边界, 从而实现无标签异常检测。

- (c) **重构对比 (Reconstruction Contrast)**: 此类方法将自编码器的重构能力与对比学习结合, 通过约束不同视图的重构一致性来提升异常判别力。TF-MAE [19] 在时域与频域重构后, 利用 KL 散度对齐两种重构表征分布:

$$\mathcal{L}_{contrast} = D_{KL}(P^{(L)} || F^{(L)}) + D_{KL}(F^{(L)} || P^{(L)}) \quad (161)$$

这种双向约束方式既保留了重构模型的生成性, 又融入了对比学习的判别性, 使模型在检测局部扰动与整体偏移异常时均具鲁棒性。CL-TAD [50] 进一步将掩蔽增强与重构对比结合, 通过在输入序列上随机遮蔽部分时间片段, 使模型必须在保持一致性的同时恢复缺失信息。实验表明, 该方式能显著提升模型对局部异常的敏感度。

总体而言, **视图对比**注重跨扰动一致性, 适合捕捉正常模式的时序不变性; **正负对比**强调异常方向建模, 增强模型区分能力; 而**重构对比**融合生成与判别学习优势, 更适合捕获潜在分布差异。三种范式在不同任务场景中各具优势, 当前趋势是通过多范式融合 (如视图 + 重构) 来实现更加全面的时序表征学习。这些机制共同构建了对比学习在时间序列异常检测中的理论基础, 使模型在**无监督条件下具备自我辨识能力**, 并在面对复杂动态环境时展现出良好的泛化性与异常捕获能力。

参考文献

- [1] Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A Zuluaga. Usad: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3395–3404, 2020.
- [2] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [3] Md Abul Bashar and Richi Nayak. Tanogan: Time series anomaly detection with generative adversarial networks. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1778–1785. IEEE, 2020.
- [4] George EP Box and David A Pierce. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *Journal of the American statistical Association*, 65(332):1509–1526, 1970.
- [5] Markus M Breunig, Hans-Peter Kriegel, Raymond T Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 93–104, 2000.
- [6] David Campos, Tung Kieu, Chenjuan Guo, Feiteng Huang, Kai Zheng, Bin Yang, and Christian S Jensen. Unsupervised time series outlier detection with diversity-driven convolutional ensembles–extended version. *arXiv preprint arXiv:2111.11108*, 2021.
- [7] Sucheta Chauhan and Lovekesh Vig. Anomaly detection in ecg time signals via deep long short-term memory networks. In *2015 IEEE international conference on data science and advanced analytics (DSAA)*, pages 1–7. IEEE, 2015.
- [8] Feiyi Chen, Yingying Zhang, Zhen Qin, Lunting Fan, Renhe Jiang, Yuxuan Liang, Qingsong Wen, and Shuiguang Deng. Learning multi-pattern normalities in the frequency domain for efficient time series anomaly detection. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 747–760. IEEE, 2024.
- [9] Run-Qing Chen, Guang-Hui Shi, Wan-Lei Zhao, and Chang-Hui Liang. A joint model for it operation series prediction and anomaly detection. *Neurocomputing*, 448:130–139, 2021.

- [10] Wenchao Chen, Long Tian, Bo Chen, Liang Dai, Zhibin Duan, and Mingyuan Zhou. Deep variational graph convolutional recurrent network for multivariate time series anomaly detection. In *ICML*, 2022.
- [11] Wenxiao Chen, Haowen Xu, Zeyan Li, Dan Pei, Jie Chen, Honglin Qiao, Yang Feng, and Zhaogang Wang. Unsupervised anomaly detection for intricate kpis via adversarial training of vae. In *IEEE INFOCOM 2019-IEEE conference on computer communications*, pages 1891–1899. IEEE, 2019.
- [12] Zekai Chen, Dingshuo Chen, Xiao Zhang, Zixuan Yuan, and Xiuzhen Cheng. Learning graph structures with transformer for multivariate time-series anomaly detection in iot. *IEEE Internet of Things Journal*, 9(12):9179–9189, 2021.
- [13] Enyan Dai and Jie Chen. Graph-augmented normalizing flows for anomaly detection of multiple time series. *arXiv preprint arXiv:2202.07857*, 2022.
- [14] Zahra Zamanzadeh Darban, Geoffrey I Webb, Shirui Pan, and Mahsa Salehi. Carla: A self-supervised contrastive representation learning approach for time series anomaly detection. *arXiv preprint arXiv:2308.09296*, 2023.
- [15] Zahra Zamanzadeh Darban, Geoffrey I Webb, and Mahsa Salehi. Dacad: Domain adaptation contrastive learning for anomaly detection in multivariate time series. *arXiv preprint arXiv:2404.11269*, 2024.
- [16] Ailin Deng and Bryan Hooi. Graph neural network-based anomaly detection in multivariate time series. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 4027–4035, 2021.
- [17] Nan Ding, HaoXuan Ma, Huanbo Gao, YanHua Ma, and GuoZhen Tan. Real-time anomaly detection based on long short-term memory and gaussian mixture model. *Computers & Electrical Engineering*, 79:106458, 2019.
- [18] Emadeldeen Eldele, Mohamed Ragab, Zhenghua Chen, Min Wu, Chee Keong Kwoh, Xiaoli Li, and Cuntai Guan. Time-series representation learning via temporal and contextual contrasting. *arXiv preprint arXiv:2106.14112*, 2021.
- [19] Yuchen Fang, Jiandong Xie, Yan Zhao, Lu Chen, Yunjun Gao, and Kai Zheng. Temporal-frequency masked autoencoders for time series anomaly detection. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 1228–1241. IEEE, 2024.

- [20] Prakhar Ganesh and Puneet Rakheja. Vlstm: Very long short-term memory networks for high-frequency trading, 2020.
- [21] Jonathan Goh, Sridhar Adepu, Marcus Tan, and Zi Shan Lee. Anomaly detection in cyber physical systems using recurrent neural networks. In *2017 IEEE 18th International Symposium on High Assurance Systems Engineering (HASE)*, pages 140–145. IEEE, 2017.
- [22] Matt Gorbett, Hossein Shirazi, and Indrakshi Ray. Sparse binary transformers for multivariate time series modeling. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 544–556, 2023.
- [23] Daniele Grattarola, Daniele Zambon, Lorenzo Livi, and Cesare Alippi. Change detection in graph streams by learning graph embeddings on constant-curvature manifolds. *IEEE Transactions on neural networks and learning systems*, 31(6):1856–1869, 2019.
- [24] Narendhar Gugulothu, Pankaj Malhotra, Lovekesh Vig, Gautam Shroff, et al. Sparse neural networks for anomaly detection in high-dimensional time series. In *AI4IOT workshop in conjunction with ICML, IJCAI and ECAI*, pages 1551–3203, 2018.
- [25] Yifan Guo, Weixian Liao, Qianlong Wang, Lixing Yu, Tianxi Ji, and Pan Li. Multidimensional time series anomaly detection: A gru-based gaussian mixture variational autoencoder approach. In *Asian Conference on Machine Learning*, pages 97–112. PMLR, 2018.
- [26] Yifan Guo, Weixian Liao, Qianlong Wang, Lixing Yu, Tianxi Ji, and Pan Li. Multidimensional time series anomaly detection: A gru-based gaussian mixture variational autoencoder approach. In *Asian conference on machine learning*, pages 97–112. PMLR, 2018.
- [27] Siho Han and Simon S Woo. Learning sparse latent graph representations for anomaly detection in multivariate time series. In *SIGKDD*, 2022.
- [28] Zilong He, Pengfei Chen, Xiaoyun Li, Yongfeng Wang, Guangba Yu, Cailin Chen, Xinrui Li, and Zibin Zheng. A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems. *IEEE Transactions on Neural Networks and Learning Systems*, 34(4):1705–1719, 2020.

- [29] Ruei-Jie Hsieh, Jerry Chou, and Chih-Hsiang Ho. Unsupervised online anomaly detection on multivariate sensing time series data for smart manufacturing. In *2019 IEEE 12th conference on service-oriented computing and applications (SOCA)*, pages 90–97. IEEE, 2019.
- [30] Yang Jiao, Kai Yang, Dongjing Song, and Dacheng Tao. Timeautoad: Autonomous anomaly detection with self-supervised contrastive loss for multivariate time series. *IEEE Transactions on Network Science and Engineering*, 9(3):1604–1619, 2022.
- [31] Ming Jin, Huan Yee Koh, Qingsong Wen, Daniele Zambon, Cesare Alippi, Geoffrey I Webb, Irwin King, and Shirui Pan. A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [32] Neha Kant and Manish Mahajan. Time-series outlier detection using enhanced k-means in combination with pso algorithm. In *Engineering Vibration, Communication and Information Processing: ICoEVCI 2018, India*, pages 363–373. Springer, 2019.
- [33] Paweł Karczmarek, Adam Kiersztyn, Witold Pedrycz, and Ebru Al. K-means-based isolation forest. *Knowledge-based systems*, 195:105659, 2020.
- [34] Tung Kieu, Bin Yang, Chenjuan Guo, and Christian S Jensen. Outlier detection for time series with recurrent autoencoder ensembles. In *IJCAI*, pages 2725–2732, 2019.
- [35] Jina Kim, Hyeongwon Kang, and Pilsung Kang. Time-series anomaly detection with stacked transformer representations and 1d convolutional network. *Engineering Applications of Artificial Intelligence*, 120:105964, 2023.
- [36] Kwei-Herng Lai, Lan Wang, Huiyuan Chen, Kaixiong Zhou, Fei Wang, Hao Yang, and Xia Hu. Context-aware domain adaptation for time series anomaly detection. In *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*, pages 676–684. SIAM, 2023.
- [37] Longyuan Li, Junchi Yan, Haiyang Wang, and Yaohui Jin. Anomaly detection of time series with smoothness-inducing sequential variational auto-encoder. *IEEE transactions on neural networks and learning systems*, 32(3):1177–1191, 2020.

- [38] Longyuan Li, Junchi Yan, Qingsong Wen, Yaohui Jin, and Xiaokang Yang. Learning robust deep state space for unsupervised anomaly detection in contaminated time-series. *IEEE Transactions on Knowledge and Data Engineering*, 35(6):6058–6072, 2022.
- [39] Yifan Li, Xiaoyan Peng, Jia Zhang, Zhiyong Li, and Ming Wen. Dct-gan: dilated convolutional transformer-based gan for time series anomaly detection. *IEEE Transactions on Knowledge and Data Engineering*, 35(4):3632–3644, 2021.
- [40] Zhihan Li, Youjian Zhao, Jiaqi Han, Ya Su, Rui Jiao, Xidao Wen, and Dan Pei. Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 3220–3230, 2021.
- [41] Fan Liu, Xingshe Zhou, Jinli Cao, Zhu Wang, Tianben Wang, Hua Wang, and Yanchun Zhang. Anomaly detection in quasi-periodic time series based on automatic data segmentation and attentional lstm-cnn. *IEEE Transactions on Knowledge and Data Engineering*, 34(6):2626–2640, 2020.
- [42] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 eighth IEEE international conference on data mining*, pages 413–422. IEEE, 2008.
- [43] Donghao Luo and Xue Wang. ModernTCN: A modern pure convolution structure for general time series analysis. In *The Twelfth International Conference on Learning Representations*, 2024.
- [44] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. Lstm-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*, 2016.
- [45] Pankaj Malhotra, Lovekesh Vig, Gautam Shroff, Puneet Agarwal, et al. Long short term memory networks for anomaly detection in time series. In *Esann*, volume 2015, page 89, 2015.
- [46] Larry M Manevitz and Malik Yousef. One-class svms for document classification. *Journal of machine Learning research*, 2(Dec):139–154, 2001.
- [47] Qiucheng Miao, Chuanfu Xu, Jun Zhan, Dong Zhu, and Chengkun Wu. An unsupervised short-and long-term mask representation for multivariate time

- series anomaly detection. In *International Conference on Neural Information Processing*, pages 504–516. Springer, 2022.
- [48] Masud Moshtaghi, James C Bezdek, Christopher Leckie, Shanika Karunasekera, and Marimuthu Palaniswami. Evolving fuzzy rules for anomaly detection in data streams. *IEEE Transactions on Fuzzy Systems*, 23(3):688–700, 2014.
- [49] Youngeun Nam, Susik Yoon, Yooju Shin, Minyoung Bae, Hwanjun Song, Jae-Gil Lee, and Byung Suk Lee. Breaking the time-frequency granularity discrepancy in time-series anomaly detection. In *Proceedings of the ACM on Web Conference 2024*, pages 4204–4215, 2024.
- [50] Huynh Cong Viet Ngu and Keon Myung Lee. Cl-tad: A contrastive-learning-based method for time series anomaly detection. *Applied Sciences*, 13(21):11938, 2023.
- [51] Zijian Niu, Ke Yu, and Xiaofei Wu. Lstm-based vae-gan for time-series anomaly detection. *Sensors*, 20(13):3738, 2020.
- [52] Guansong Pang, Anton van den Hengel, Chunhua Shen, and Longbing Cao. Toward deep supervised anomaly detection: Reinforcement learning from partially labeled anomaly data. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 1298–1308, 2021.
- [53] Daehyung Park, Yuuna Hoshi, and Charles C Kemp. A multimodal anomaly detector for robot-assisted feeding using an lstm-based variational autoencoder. *IEEE Robotics and Automation Letters*, 3(3):1544–1551, 2018.
- [54] Peter CB Phillips and Sainan Jin. Business cycles, trend elimination, and the hp filter. *International Economic Review*, 62(2):469–520, 2021.
- [55] Hansheng Ren, Bixiong Xu, Yujing Wang, Chao Yi, Congrui Huang, Xiaoyu Kou, Tony Xing, Mao Yang, Jie Tong, and Qi Zhang. Time-series anomaly detection service at microsoft. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3009–3017, 2019.
- [56] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*, pages 4–11, 2014.

- [57] Lifeng Shen, Zhuocong Li, and James Kwok. Timeseries anomaly detection using temporal hierarchical one-class network. *Advances in Neural Information Processing Systems*, 33:13016–13026, 2020.
- [58] Alban Siffer, Pierre-Alain Fouque, Alexandre Termier, and Christine Largouet. Anomaly detection in streams with extreme value theory. pages 1067–1075, 2017.
- [59] Maximilian Sölch, Justin Bayer, Marvin Ludersdorfer, and Patrick van der Smagt. Variational inference for on-line anomaly detection in high-dimensional time series. *arXiv preprint arXiv:1602.07109*, 2016.
- [60] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2828–2837, 2019.
- [61] Shreshth Tuli, Giuliano Casale, and Nicholas R Jennings. Tranad: Deep transformer networks for anomaly detection in multivariate time series data. *arXiv preprint arXiv:2201.07284*, 2022.
- [62] Rui Wang, Xudong Mou, Renyu Yang, Kai Gao, Pin Liu, Chongwei Liu, Tianyu Wo, and Xudong Liu. Cutaddpaste: Time series anomaly detection by exploiting abnormal knowledge. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3176–3187, 2024.
- [63] Xixuan Wang, Dechang Pi, Xiangyan Zhang, Hao Liu, and Chang Guo. Variational transformer-based anomaly detection approach for multivariate time series. *Measurement*, 191:110791, 2022.
- [64] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186*, 2022.
- [65] Wentai Wu, Ligang He, Weiwei Lin, Yi Su, Yuhua Cui, Carsten Maple, and Stephen Jarvis. Developing an unsupervised real-time anomaly detection scheme for time series with multi-seasonality. *IEEE Transactions on Knowledge and Data Engineering*, 34(9):4147–4160, 2020.
- [66] Jiehui Xu. Anomaly transformer: Time series anomaly detection with association discrepancy. *arXiv preprint arXiv:2110.02642*, 2021.

- [67] Kenji Yamanishi and Jun-ichi Takeuchi. A unifying framework for detecting outliers and change points from non-stationary time series data. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 676–681, 2002.
- [68] Yiyuan Yang, Chaoli Zhang, Tian Zhou, Qingsong Wen, and Liang Sun. Dcdetector: Dual attention contrastive representation learning for time series anomaly detection. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3033–3045, 2023.
- [69] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. Matrix profile i: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets. In *2016 IEEE 16th international conference on data mining (ICDM)*, pages 1317–1322. Ieee, 2016.
- [70] Wenzhen Yue, Xianghua Ying, Ruohao Guo, DongDong Chen, Ji Shi, Bowei Xing, Yuqing Zhu, and Taiyan Chen. Sub-adjacent transformer: Improving time series anomaly detection with reconstruction error from sub-adjacent neighborhoods. *arXiv preprint arXiv:2404.18948*, 2024.
- [71] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. Ts2vec: Towards universal representation of time series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8980–8987, 2022.
- [72] Daniele Zambon, Lorenzo Livi, and Cesare Alippi. Graph iforest: Isolation of anomalous and outlier graphs. In *IJCNN*, pages 1–8, 2022.
- [73] Chunkai Zhang, Shaocong Li, Hongye Zhang, and Yingyang Chen. Velc: A new variational autoencoder based model for time series anomaly detection. *arXiv preprint arXiv:1907.01702*, 2019.
- [74] Chuxu Zhang, Dongjin Song, Yuncong Chen, Xinyang Feng, Cristian Lumezanu, Wei Cheng, Jingchao Ni, Bo Zong, Haifeng Chen, and Nitesh V Chawla. A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 1409–1416, 2019.

- [75] Weiqi Zhang, Chen Zhang, and Fugee Tsung. Grelen: Multivariate time series anomaly detection from the perspective of graph relational learning. In *IJCAI*, pages 2390–2397, 2022.
- [76] Wenxin Zhang, Xiaojian Lin, Wenjun Yu, Guangzhen Yao, Yu Li, Renda Han, Songcheng Xu, Hao Shi, Cuicui Luo, et al. Dconad: A differencing-based contrastive representation learning framework for time series anomaly detection. *arXiv preprint arXiv:2504.14204*, 2025.
- [77] Yuxin Zhang, Jindong Wang, Yiqiang Chen, Han Yu, and Tao Qin. Adaptive memory networks with self-supervised learning for unsupervised anomaly detection. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12068–12080, 2022.
- [78] Hang Zhao, Yujing Wang, Juanyong Duan, Congrui Huang, Defu Cao, Yunhai Tong, Bixiong Xu, Jing Bai, Jie Tong, and Qi Zhang. Multivariate time-series anomaly detection via graph attention network. In *2020 IEEE international conference on data mining (ICDM)*, pages 841–850. IEEE, 2020.
- [79] Yu Zheng, Huan Yee Koh, Ming Jin, Lianhua Chi, Khoa T Phan, Shirui Pan, Yi-Ping Phoebe Chen, and Wei Xiang. Correlation-aware spatial-temporal graph learning for multivariate time-series anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems*, 35(9):11802–11816, 2023.
- [80] Bo Zong, Qi Song, Martin Renqiang Min, Wei Cheng, Cristian Lumezanu, Daeki Cho, and Haifeng Chen. Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In *International conference on learning representations*, 2018.

5 时间序列分类

5.1 节作者为：陈宇轩、郭晨娟；

5.2 节作者为：黄姗姗、郭晨娟；

5.3-5.4 节作者为：黄姗姗、陈宇轩、郭晨娟。

本章主编为郭晨娟。

本章主要介绍时间序列分类任务的相关知识。时间序列分类旨在根据整条时间序列的波形、统计特征或隐含动态，将其划归到预先定义类别标签之一。时间序列分类可划分为两大范式：基于传统机器学习的分类与基于深度学习的端到端分类。前者基于传统的机器学习方法，具有较为良好的可解释性；后者则在舍弃部分可解释性的情况下，有效利用了深度模型庞大参数量的特点，取得了更好的分类准确率。5.1 章节给出了时序分类任务的背景和工作流程。5.2 章介绍了时序分类认为的模型训练与评估过程，同时给出了一些常用的评价指标。而 5.3 章节介绍了传统机器学习方法和深度学习方法的时序分类策略。5.4 章则介绍了一些自监督训练方法，常用于提取时序特征供分类使用。

5.1 时间序列分类定义及流程

时间序列分类（Time Series Classification, TSC）是时间序列分析中的一项关键任务，是通过给定的时间序列片段判断其属于什么类别的任务。分类任务广泛应用于生产生活中的方方面面，包括人类活动感知、基于电信号的健康监测以及系统监控问题等。给定一个时间序列数据集 $D = \{(X_1, y_1), (X_2, y_2), \dots, (X_n, y_n)\}$ ，其中： $X_i = [x_1, x_2, \dots, x_M] \in \mathbb{R}^{M \times T}$ 表示第 i 个时间序列实例， M 是变量维度的数量， T 是时间步的数量。当 $M=1$ 时，序列称为单变量；当 $M>1$ 时，序列称为多变量。 $y_i \in C$ 是时间序列 X_i 对应的标签， $C = \{c_1, c_2, \dots, c_k\}$ 是类别集合， k 是类别数。时间序列分类任务是学习一个分类函数 $f: \mathbb{R}^{M \times T} \rightarrow y$ ，通过该函数，能够将新的时间序列 X_{new} 映射到正确的类别标签 y_{new} 。可以将其定义为如下优化问题：

$$f^* = \underset{f}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(X_i), y_i) \quad (162)$$

其中， f^* 是学到的最佳分类模型， $\mathcal{L}$ 是分类损失函数（如交叉熵损失），用来衡量预测的标签 $f(X_i)$ 与真实标签 y_i 之间的差异。

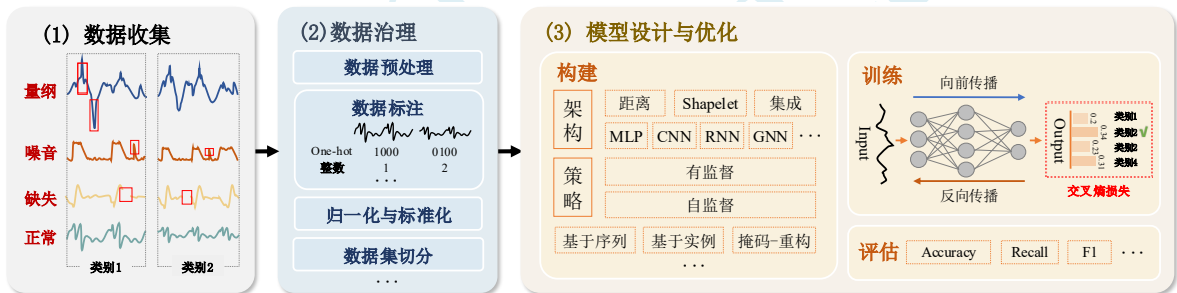


图 5.1: 时间序列分类任务流程。

图5.1展示了时间序列分类任务的训练过程，通常包括以下几个步骤：1) 数据收集，获取各个领域的时序数据；2) 数据预处理，进行数据清洗、归一化和标准化等一

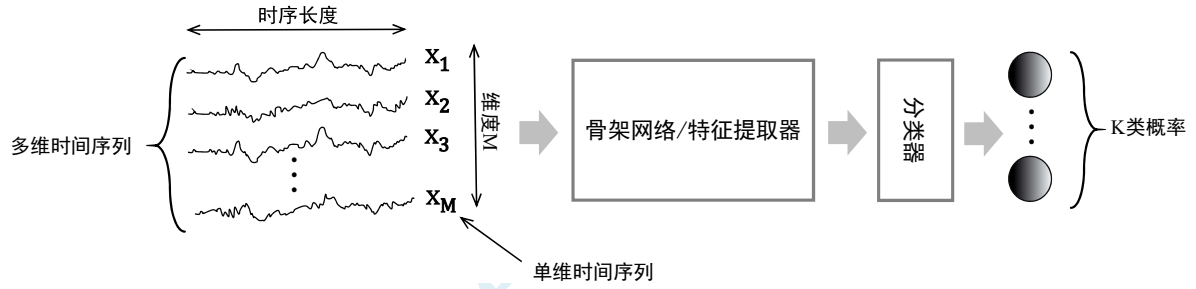


图 5.2: 训练框架。

系列工作，防止低质量数据误导模型；3) 模型设计和优化，包括选择合适的模型框架，优化模型参数，常用的损失函数为交叉熵损失（Cross-Entropy）114；4) 在训练完成后使用测试集进行评估，常用的评估指标为分类准确率等；5) 将模型部署到实际的应用场景中，监控其性能，必要时进行模型更新和再训练。

5.2 时间序列分类模型训练与评估

对时间序列分类的具体应用领域而言，标签的类型是多种多样的，但是监督任务的训练方式本身遵循一套统一的流程。时序分类模型的目标是通过给定的时间序列的模式判断其属于什么类别。需要以一定比例划分数数据集。对于给定的数据集，划分为训练集 $D_{\text{train}} = \{(X_i^{\text{train}}, y_i^{\text{train}})\}_{i=1}^{N_{\text{train}}}$ ，验证集 $D_{\text{val}} = \{(X_i^{\text{val}}, y_i^{\text{val}})\}_{i=1}^{N_{\text{val}}}$ 和测试集 $D_{\text{test}} = \{(X_i^{\text{test}}, y_i^{\text{test}})\}_{i=1}^{N_{\text{test}}}$ 。其中 $y_i \in C$ 是时间序列 X_i 对应的标签， $C = \{c_1, c_2, \dots, c_k\}$ 是类别集合， k 是类别数。随后，需要对数据集的三个部分进行以下两个阶段：

1. **训练阶段**：训练阶段的目标是最小化模型的分类误差。具体而言，首先是**数据分析**，对于给定的训练集 D_{train} ，通过学习输入序列 X_i^{train} 的各种特征来捕捉时序的具体模式和全局结构。其次是**模型优化**，通常根据模型预测标签 $\hat{y}_i^{\text{train}}$ 与真实标签 y_i^{train} 之间的误差来指导模型的优化方向，误差越小，表明模型学习到的序列信息越多。常见的**损失函数**为交叉熵损失：

$$\mathcal{L} = -\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} y_i^{\text{train}} \log(\hat{y}_i^{\text{train}}) \quad (163)$$

需要注意的是，在模型训练过程中，出于避免模型 f_{θ} 在 D_{train} 过拟合或者筛选最优超参数等目的，通常使用验证集 D_{val} 判断模型 f_{θ} 性能，获得**最优模型** f_{θ^*} ：

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \mathcal{L}_{\text{val}}(\theta) \quad (164)$$

2. **评估阶段**：在模型评估阶段，使用测试集 D_{test} 检测模型性能。即使用训练阶段得到的最优模型 f_{θ^*} 建模历史序列 $\{X_i^{\text{test}}\}_{i=1}^{N_{\text{test}}}$ 并输出预测值，将预测概率最大的类别作为最终预测类别 $\hat{y}_i^{\text{test}}$ 。

(a) **准确率 (Accuracy)** 衡量模型预测正确的样本比例：

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (165)$$

对于多分类任务，也可定义为：

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(y_i^{\text{test}} = \hat{y}_i^{\text{test}}) \quad (166)$$

其中： y_i^{test} ：样本 i 的真实类别； $\hat{y}_i^{\text{test}}$ ：样本 i 的预测类别； N ：总样本数； $\mathbf{1}(\cdot)$ ：指示函数，若为真则返回 1，否则为 0。

(b) **精确率 (Precision)** 衡量被预测为正类的样本中，有多少是真正的正类：

$$\text{Precision} = \frac{TP}{TP + FP} \quad (167)$$

在多分类情形中，常使用宏平均：

$$\text{Macro-Precision} = \frac{1}{C} \sum_{c=1}^C \frac{TP_c}{TP_c + FP_c} \quad (168)$$

(c) **召回率 (Recall)** 衡量所有真实正类中，有多少被正确预测为正类：

$$\text{Recall} = \frac{TP}{TP + FN} \quad (169)$$

对应的宏平均形式为：

$$\text{Macro-Recall} = \frac{1}{C} \sum_{c=1}^C \frac{TP_c}{TP_c + FN_c} \quad (170)$$

(d) **F1 分数 (F1-Score)** F1 分数是精确率与召回率的调和平均：

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (171)$$

多分类下的宏平均为：

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (172)$$

(e) **AUC-ROC (仅适用于二分类)** AUC 是 ROC 曲线下的面积，反映了模型对正负样本的排序能力：

$$\text{AUC} = \int_0^1 \text{TPR}(FPR) d(FPR) \quad (173)$$

其中：

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN} \quad (174)$$

- (f) **平均排名 (Average Rank)** 在比较多个分类器在多个数据集上的性能时，使用平均排名 (Average Rank) 作为指标具有鲁棒性。

假设有 K 个分类器，在 D 个数据集上进行比较。对于第 d 个数据集，所有分类器根据某个性能指标（如准确率）进行排序，获得排名 $r_k^{(d)}$ ，则分类器 k 的平均排名定义为：

$$\bar{r}_k = \frac{1}{D} \sum_{d=1}^D r_k^{(d)} \quad (175)$$

通常，排名越低表示模型性能越好。

- (g) **临界差值图 (Critical Difference Diagram)** 为了可视化多个分类器的平均排名差异，使用 Nemenyi 检验进行显著性分析，并通过临界差值图 (Critical Difference Diagram, CD 图) 表示。当两个分类器的平均排名之差大于临界差值 (Critical Difference, CD) 时，说明它们在统计意义上存在显著差异。CD 的计算方式为：

$$CD = q_\alpha \cdot \sqrt{\frac{K(K+1)}{6D}} \quad (176)$$

其中： q_α 是根据显著性水平 α 和分类器个数 K 由 Studentized range 分布表得到的常数； D 是数据集数量； K 是参与比较的分类器数量。

CD 图以一条线表示平均排名，从左（性能好）到右（性能差）排布。所有平均排名之间的差值小于 CD 的分类器，将用线段连接，表示差异不显著。

5.3 时间序列分类模型

5.3.1 经典机器学习方法

经典机器学习方法，一般是指代除了基于神经网络的深度学习方法的其他方法。本节将介绍四种非深度学习方法，分别是基于距离的分类方法、基于 Shapelet 的分类方法、基于树和森林的分类方法和基于集成学习的分类方法。

基于距离的分类 基于距离的分类器使用距离函数来衡量整个时间序列之间的相似性。历史上，这些距离函数主要与最近邻 (Nearest Neighbour, NN) 分类器一起使用。关于时间序列距离的其他应用方式可参见 [1]。基于 DTW (动态时间规整) 的 1-NN 曾

经被认为是时间序列分类 (TSC) 的最新进展 [42]。除了 DTW，还提出了多种替代的弹性距离度量 (elastic distance measures, 用于补偿序列之间可能的错位)。这些方法结合了序列本身和其导数的规整与编辑操作 [19]。此前的研究 [32] 显示，不同弹性距离下的 1-NN 分类器性能差异很小。需要注意的是，后续算法并不一定优于前者，但往往是前者的扩展或受到其启发。

弹性集成 (Elastic Ensemble, EE) 第一个在 UCR 数据上显著超越 1-NN DTW 的算法是 [32]。EE 是由 11 种不同弹性距离的 1-NN 分类器构成的加权集成方法。由于弹性距离计算开销较大，EE 需要通过交叉验证来确定每个分类器的权重。Tan 等人 [45] 提出了缓存机制以加速拟合过程。

近邻森林 (Proximity Forest, PF) PF [38] 是由多个 Proximity Tree 分类器组成的集成方法。PF 使用与 EE 相同的 11 种距离函数，但其准确率更高、可扩展性更好。在树的每个节点，会随机选取一个距离函数并固定超参数值，每个类随机选择一个代表序列。随后，从多个组合 (距离函数、参数值和类代表序列) 中选择 Gini 指数最高的组合进行分裂。每个序列根据与代表序列的最小距离选择分支，树结构递归生长直至叶节点纯净。

形状 DTW (ShapeDTW) ShapeDTW [58] 通过滑动窗口提取序列的形状特征描述符，如斜率、小波变换和分段近似。论文使用原始子序列和导数子序列作为特征，之后再用 1-NN + DTW 进行分类。

通用表示学习 (Generic Representation Learning, GRAIL) GRAIL [41] 专注于在定制距离约束下高效学习时间序列表示。GRAIL 借助核方法，尤其是 Nyström 方法，在这些约束下学习精确表示。其过程是将每个时间序列表示为一组表达性“地标” (通过聚类中心获得) 的线性组合。该方法引入了 Shift-Invariant Kernel (SINK) 核函数，使用快速傅里叶变换 (FFT) 实现时间序列在移位不变性下的比较。在分类任务中，GRAIL 和 SINK 核函数结合线性 SVM 使用。

基于 Shapelet 的分类 Shapelet 是时间序列中的一个独特概念，表示一条时间序列中“最具代表性和特异性”的子序列片段。其概念最早提出是为了解决时间序列分类任务的开销问题，通过区分一个子片段以区分整条时间序列的思路是非常符合直觉的做法。

Shapelet 被定义为能最大程度区分不同类别的子序列。设原始序列集合为 $\mathcal{D} = \{T^{(i)}\}_{i=1}^n$ ，每条序列长度为 m 。候选 Shapelet 为任意长度 l 的子序列：

$$s = T_{t:t+l-1}^{(i)}, \quad 1 \leq t \leq m - l + 1. \quad (177)$$

对任意序列 T ，定义 Shapelet 到 T 的距离为

$$d(s, T) = \min_{1 \leq j \leq m-l+1} \|s - T_{j:j+l-1}\|_2. \quad (178)$$

将每个候选 s 在整个数据集上的距离分布映射为二分类问题，使用信息增益等指标评估其判别力。

围绕 Shapelet 的研究主要体现在两个主题：第一个主题侧重于寻找生成或选择 Shapelet 的最佳策略，以减小枚举整条时间序列生成候选 Shapelet 或计算距离的时间复杂度；第二个主题致力于探索如何使用 Shapelet 进行有效的分类。

生成候选 Shapelet 和选择最优的 Shapelet 方法主要有两种：搜索型方法和学习型方法。搜索型方法通过穷举或启发式搜索，在所有可能子序列或部分候选子序列中，计算每个候选 Shapelet 的判别性指标（如信息增益、F-score），从而筛选出最优 Shapelet。学习型方法将 Shapelet 参数化为可学习向量，将 Shapelet 与分类器共同嵌入损失函数（如交叉熵）中，利用梯度下降等优化方法联合训练，直接学习出能最优分类的 Shapelet。

1. **搜索方法**：最早 Shapelet 的研究 [53] 提出了蛮力算法来通过搜索时间序列数据的所有子序列来找到候选形状。通过对所有可能的子序列进行遍历，计算每个候选 Shapelet 与所有序列的距离，并以信息增益等指标评估其判别力，时间复杂度高达 $O(n^2 m^4)$ 。

给定数据集 D ，类别集合 $\mathcal{Y} = \{A, B\}$ ，其中

$$p(A) = \frac{|\{i : y_i = A\}|}{|D|}, \quad p(B) = \frac{|\{i : y_i = B\}|}{|D|}, \quad (179)$$

则划分前的熵定义为

$$I(D) = -p(A)\log p(A) - p(B)\log p(B). \quad (180)$$

令候选 Shapelet S 与阈值 d_{th} 将 D 划分为

$$D_1 = \{T \in D \mid d(S, T) \leq d_{th}\}, \quad D_2 = D \setminus D_1, \quad (181)$$

其中 $d(S, T)$ 表示 Shapelet 到序列 T 的最小欧氏距离。记

$$f(D_1) = \frac{|D_1|}{|D|}, \quad f(D_2) = \frac{|D_2|}{|D|}, \quad (182)$$

则划分后的加权平均熵为

$$\tilde{I}(D) = f(D_1)I(D_1) + f(D_2)I(D_2). \quad (183)$$

信息增益 (Gain) 定义为划分前后熵的减少量：

$$\text{Gain}(S, d_{th}) = I(D) - \tilde{I}(D) = I(D) - [f(D_1)I(D_1) + f(D_2)I(D_2)]. \quad (184)$$

对于每个候选 Shapelet S ，找到**最优分割点**

$$d_{\text{OSP}}(D, S) = \arg\max_{d_{\text{th}}} \text{Gain}(S, d_{\text{th}}). \quad (185)$$

该点即在所有可能的 d_{th} 上令信息增益最大的分割阈值。

为了解决此过程的耗时性质，作者接着引入了两种技术：早期放弃，当部分距离超过当前最小值时，它会停止距离计算，而早期的熵修剪，当上限无法超过最佳信息增益时，它会停止计算。尽管有用，但这些技术仍然效率低下，促使开发更快的技术，例如搜索空间修剪，时间序列抽样，候选过滤，随机抽样，智能缓存和有效的距离计算 [33, 27, 21]。

Perceptually Important Points (PIPs) 方法 [5] 是一种通过识别时间序列中最重要拐点（关键点）的方法。PIPs 最初用于时间序列降维和可视化，后续被应用于时间序列分类任务 [26, 25]，作为高效的 Shapelet 提取方法。该方法通过保留对原始时间序列重构误差最大的点，筛选出一组能够表征局部结构特性的点，再基于这些点之间的片段作为候选 Shapelet。

设原始时间序列为：

$$T = \{t_1, t_2, \dots, t_n\} \quad (186)$$

初始时，选取序列首尾两个点作为初始 PIPs 集合：

$$P = \{p_1, p_2\} = \{(1, t_1), (n, t_n)\} \quad (187)$$

然后，在剩余点中，计算每个点到其相邻两个 PIP 所在直线的垂直距离，选取距离最大的点加入 PIPs 集合。

设待计算点为 (x_i, y_i) ，相邻两个 PIP 为 $(x_a, y_a), (x_b, y_b)$ ，则该点到直线 $y = ax + c$ 的垂直距离公式为：

$$d_i = \frac{|ax_i - y_i + c|}{\sqrt{a^2 + 1}} \quad (188)$$

其中：

$$a = \frac{y_b - y_a}{x_b - x_a}, \quad c = y_a - ax_a \quad (189)$$

PIPs-Based Shapelet 提取流程

- (a) 对每条时间序列，使用 PIPs 方法提取固定数量的关键点（如 10%、20% 点数）。
- (b) 将相邻 PIPs 之间的子序列作为候选 Shapelet。
- (c) 计算这些候选 Shapelet 与所有序列的距离，评估它们的分类能力（如信息增益、F 值）。

(d) 筛选出最佳 Shapelet，构建分类模型。

评估每个候选 Shapelet 的区分能力除了使用上面提到的信息增益，还有一些研究基于统计检验方法进行选择。

F 检验 (ANOVA F-test) F 检验是一种方差分析方法，适用于多个类别下的均值差异检验。其核心思想是：

$$F = \frac{\text{类间方差}}{\text{类内方差}} \quad (190)$$

设将某个 Shapelet S 与所有时间序列计算距离，得到距离向量 $D = \{d_1, d_2, \dots, d_n\}$ ，对应标签为 $Y = \{y_1, y_2, \dots, y_n\}$ 。根据类别对 D 进行分组，组间均值方差越大，说明不同类间差异越明显；组内方差越小，说明同类样本对 Shapelet 的响应越一致。

Kruskal-Wallis H 检验 (非参数方法) Kruskal-Wallis H 检验是一种非参数检验方法，可看作是 F 检验在无正态分布假设下的替代。它基于秩而非均值，对异常值更鲁棒。其检验统计量为：

$$H = \frac{12}{N(N+1)} \sum_{i=1}^k n_i (R_i - \bar{R})^2 \quad (191)$$

其中： N 是样本总数， k 是类别数， n_i 是第 i 类别的样本数量， R_i 是第 i 类别的秩均值， $\bar{R}$ 是所有样本的秩均值。H 值越大表示类间差异越显著，Shapelet 越有区分力。

2. **可学习方法**：在传统 Shapelet 方法中，计算时间序列 T 与 Shapelet S 的距离是通过滑动窗口计算子序列与 S 之间的最小距离：

$$d(T, S) = \min_{t=1}^{n-l+1} D(T_{t:t+l-1}, S) \quad (192)$$

其中 $D(\cdot, \cdot)$ 通常为欧氏距离。然而，最小值函数不可导，无法应用梯度下降优化。因此，**Soft-Minimum Function** 被提出作为光滑可导的替代。Soft-Minimum 的形式如下：

$$\text{SoftMin}(x_1, x_2, \dots, x_n) = -\frac{1}{\alpha} \log \left(\sum_{i=1}^n e^{-\alpha x_i} \right) \quad (193)$$

其中 $\alpha > 0$ 控制近似程度， α 越大，Soft-Min 越接近真实的最小值：

$$\lim_{\alpha \rightarrow +\infty} \text{SoftMin}(x_1, \dots, x_n) = \min(x_1, \dots, x_n) \quad (194)$$

Soft-Min 保持了最小值的特性，同时是处处可微，方便在神经网络中端到端训练。基于 Soft-Minimum Function，**Learnable Shapelet 方法** [14] 将 Shapelet 视为可训练参数，配合 Soft-Min 计算距离，从而实现全可微、可学习的 Shapelet 分类器。方法流程如下：

(a) 定义 m 个可学习的 Shapelet 参数 $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ 。

(b) 对每个时间序列 T ，计算其与所有 Shapelet 的 Soft-Min 距离：

$$d(T, S_j) = \text{SoftMin}\left(\left\{D(T_{t:t+l_j-1}, S_j) \mid t = 1, \dots, n - l_j + 1\right\}\right) \quad (195)$$

(c) 将所有距离 $\{d(T, S_j)\}$ 作为输入特征，连接至分类器，如 Logistic Regression 或 MLP。

(d) 定义交叉熵损失函数，使用反向传播和梯度下降同时更新 Shapelet 参数 $\mathcal{S}$ 和分类器参数。

根据训练集选择出最优 Shapelet 之后，还会基于输入序列和 Shapelet 的距离对输入序列的类别进行划分，常见的划分方式有两种：

1. **基于距离阈值的划分方法** 在基于 Shapelet 的分类方法中，一种常见的策略是通过计算样本与 Shapelet 之间的距离，并根据预定的阈值将样本划分为不同类别 [53]。这种方法通常与决策树等分类模型结合使用，以进行类别划分。

假设我们有一个时间序列样本集 $\mathcal{D}$ 和一组 Shapelet $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ 。对于每个样本 T_i ，我们计算其与所有 Shapelet 的距离，并将样本与某个 Shapelet 的最小距离作为其特征：

$$d(T_i, S_j) = \min_{t=1}^{n-l_j+1} D(T_i(t:t+l_j-1), S_j) \quad (196)$$

其中 $D(\cdot, \cdot)$ 表示样本子序列与 Shapelet 之间的距离， l_j 是 Shapelet 的长度， n 是时间序列的长度。接下来，样本与 Shapelet 的距离 $d(T_i, S_j)$ 会作为特征输入到一个分类模型中，通常使用决策树来根据距离阈值进行划分。

在决策树模型中，对于某个节点的分裂，计算数据集 $\mathcal{D}$ 中所有样本与 Shapelet 的距离，并选择一个最佳阈值 θ_j ，将样本按以下规则划分：

$$\mathcal{D}_L = \{T_i \mid d(T_i, S_j) \leq \theta_j\}, \quad \mathcal{D}_R = \{T_i \mid d(T_i, S_j) > \theta_j\} \quad (197)$$

这种基于距离阈值的划分方法的核心在于选择最佳的阈值 θ_j ，通常可以通过信息增益或 Gini 指数来评估。

2. **基于距离特征的判别方式** 已有工作 [14, 28, 25, 37] 通过计算与 Shapelet 的距离作为特征来进行分类。每个时间序列样本 T_i 与多个 Shapelet 之间的距离被计算，并作为输入特征提供给分类器（如 Logistic Regression、支持向量机 (SVM)、神经网络等）。

具体而言，假设有 m 个 Shapelet $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ ，对于每个时间序列 T_i ，计算其与每个 Shapelet S_j 的距离 $d(T_i, S_j)$ 。这些距离特征组成一个特征向量：

$$\mathbf{d}(T_i) = [d(T_i, S_1), d(T_i, S_2), \dots, d(T_i, S_m)] \quad (198)$$

然后，这些特征被输入到一个分类器中，分类器的目标是通过最小化损失函数来学习一个最优的参数组合。假设使用的是 Logistic Regression，模型的预测概率为：

$$P(y = c | T_i) = \frac{e^{\mathbf{w}_c^\top \mathbf{d}(T_i) + b_c}}{\sum_{j=1}^C e^{\mathbf{w}_j^\top \mathbf{d}(T_i) + b_j}} \quad (199)$$

其中 $\mathbf{d}(T_i)$ 是与 Shapelet 的距离特征， $\mathbf{w}_c$ 是每个类别的权重， b_c 是偏置项。

在训练过程中，如果 Shapelet 是通过可学习方法生成的，则 Shapelet 的参数和分类器的参数会同时进行优化。如果 Shapelet 是通过离线方法生成的，则只会优化分类器的参数，通常通过反向传播和梯度下降算法来进行端到端训练。

基于集成方法的分类 集成学习一直是分类问题中的常用方法，在时序分类中也很受欢迎，基于森林的方法本身也包含了集成学习的思想，是一种同构分类器的集成学习。本节将主要介绍使用异构分类器的集成学习方法。

Elastic Ensemble (EE) 是一种组合十一种基于度量方法的集成学习方法。由于一系列根据时间扭曲 (Time Warp) 和编辑距离 (Edit Distance) 的弹性度量方法的提出，研究人员开始积极探索这类方法在分类器中的实用价值。虽然这些方法在单独使用的过程中并没有比基于 DTW 的方法有明显提升，但是通过结合使用这些距离度量构建的 1-NN 分类器的预测，并使用根据交叉验证训练集准确性加权的投票方案，结果明显优于了单一的基于 DTW 的方法。

Collective of Transformation Ensembles (COTE) 选择集成不同分类策略的分类器，这是一种在时间域、自相关域、功率谱域和形状元 (shapelet) 域中组合分类器的方法。EE (弹性集成) 和 ST (形状元变换) 的组件与基于自相关变换 (ACF) 和功率谱变换 (PS) 构建的分类器一起进行汇总。EE 使用了上述描述的 11 个分类器。ACF 和 PS 使用了与形状元变换结合使用的相同 8 个分类器。我们在 [2] 中使用了名为 flat-COTE 的分类器。这涉及将所有 35 个分类器汇总到一个集成中，并根据训练集交叉验证的准确率对投票进行加权。

传统机器学习方法往往在理论上更加严谨，可解释性上更符合直觉，然而，伴随着硬件技术的蓬勃发展，基于神经网络的深度学习的方法逐步成为了时序分类问题的主流，大规模的数据和参数让深度学习的性能全面超越了传统机器学习方法，以至于近年来的工作绝大部分都是基于深度学习思路。除了性能上的卓越，深度学习的思想使得时序分类问题变得更加简单、更加模块化。一个基于深度学习的时序分类问题可以被简化为如下图的框架，绝大部分的方法也是在下面的框架基础上进行局部地调整和改进。

时序分类问题的深度学习框架往往可以分为输入时的数据处理，骨架网络和特征提取器，分类器和分类方法三个部分，本节将围绕这三个部分介绍一些经典的时序分类方法以及其局限性和创新性。

5.3.2 多层感知机

多层感知机（MLP）是一种前向结构的人工神经网络，是结构最为简单的深度模型，包含输入层、输出层及多个隐藏层。多层感知机用于时间序列分类任务时，通常对时间序列进行简单的预处理，例如归一化，对缺失值的处理。此外，通常不区分骨架网络和分类器，多层感知机具有特征提取和分类器的双重作用。

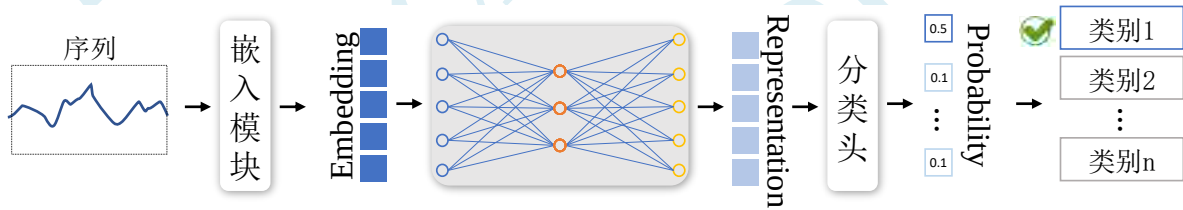


图 5.3: MLP 架构。

基本分类流程 整体上来看，多层感知机在时序分类中的使用方式和在时序预测中并没有什么不同，如图 5.3 所示。Felipe Arias del Campo 等人 [8] 首次提出了用于单变量时间分类的多层感知机。该方法无法手工为不同的数据集选择超参数，而是根据时间序列数据的性质自动确定批次大小和隐藏层中神经元数量相关的超参数。对于 MLP 本身的模型结构，文章并没有做过多的创新，模型结构依旧可以形式化为公式 76。而时序分类和时序预测方法在深度学习方法中，最大的区别在于最终的输出被映射到了类别的数量，可以形式化为：

$$\mathbf{Z}^{(l)} = g^{(l)}(\mathbf{w}^{(l)} \mathbf{Z}^{(l-1)} + \mathbf{b}^{(l)}), l = 1, \dots, L \quad (200)$$

其中， $\mathbf{Z}^{(l-1)} \in \mathbb{R}^{H \times D}$ 是第 $l-1$ 层时间序列的特征表示， D 是其维度， $\mathbf{w}^{(l)} \in \mathbb{R}^{F \times H}$ 是权重矩阵， $\mathbf{b}^{(l)} \in \mathbb{R}^{F \times 1}$ 是偏置项， $g^{(l)}(\cdot)$ 是非线性激活函数。在经过 L 个隐藏层后得到

$\mathbf{Z}^{(L)} = (\mathbf{z}_1^{(L)}, \mathbf{z}_2^{(L)}, \dots, \mathbf{z}_F^{(L)}) \in \mathbb{R}^{F \times D}$ ，最终通过分类头输出预测的时间序列分类标签 $\hat{\mathbf{Y}}$ ：

$$\hat{\mathbf{Y}} = \text{ClassificationHead}(\mathbf{Z}^{(L)}) \quad (201)$$

基于 MLP 的时间序列分类模型关键技术 作为结构简单的高效模型，研究人员对 MLP 在一定程度的探索，主要在于如何通过 MLP 捕捉序列的时间依赖。Felipe Arias del Campo 等人 [8] 首次提出了用于单变量时间分类的多层感知机。该方法无法手工为不同的数据集选择超参数，而是根据时间序列数据的性质自动确定批次大小和隐藏层中神经元数量相关的超参数。

1. **动态时间规整** 从数据角度直接进行处理是研究者最初的尝试。BK Iwana 等人 [20] 增加了对时间序列的预处理，从而提高前馈神经网络的性能。具体而言，它利用动态时间规整（Dynamic Time Warping, DTW）的弹性匹配能力将层的输入与权重动态对齐，避免使用预先确定的固定输入到权重的映射。通过这种方式，DTW-NN 在捕捉时间依赖性的同时解决了前馈架构中的时间失真和模式长度可变的问题。
2. **多尺度建模** 尽管上述研究试图解决 MLP 捕获时间依赖性的不足，但仍然存在其他限制。通常，时间序列具有不同尺度的模式和结构，例如长期趋势和短期波动，而 MLP 要求输入数据具有固定的长度，导致其难以通过分层或多尺度的方式处理输入数据，这就使得多尺度信息的整合难以在 MLP 架构上实现。MLP 简单的架构不仅使得多尺度信息的捕捉十分困难，也由于其过于简单的架构缺乏创新和调整空间，因此其他深度学习模型迅速被应用于时序分类问题，例如 CNN、RNN 和 transformer，这些模型在捕获时间序列数据中的时间依赖关系和模式方面具备更强大的建模能力，且其模块化的特性为各种应用场景提供了定制空间。

5.3.3 卷积神经网络

卷积神经网络（Convolutional Neural Network, CNN）在各个领域上的成功已经经历了时间的检验，深度卷积框架自然被引入到了时序分类的任务场景下。不同于 MLP，卷积结构天然地为捕获依赖时间的信息提供了便利，而在其他领域的长时间应用过程中也已经提出了许多实用且稳定的 CNN 架构，这些条件为 CNN 应用于时序分类铺平了道路。本节将首先阐述基于时序模态的 CNN 模型，即不对输入数据进行调整，仅在骨架网络进行设计的工作。之后本节将探讨一些对时序数据进行变化之后再使用基于 CNN 的方法，这类方法还比较基础，不会作为本节的重点。

基本分类流程 在时序分类中，卷积神经网络往往作为特征提取器，使用思路与前文中的时序预测有很多相似之处。与预测最大的不同在于，分类问题在输入特征提取器的选择上更加丰富，常用于预测任务的嵌入模块并不一定应用在分类中，但仍然以如图 5.4 所示的整体流程为主。

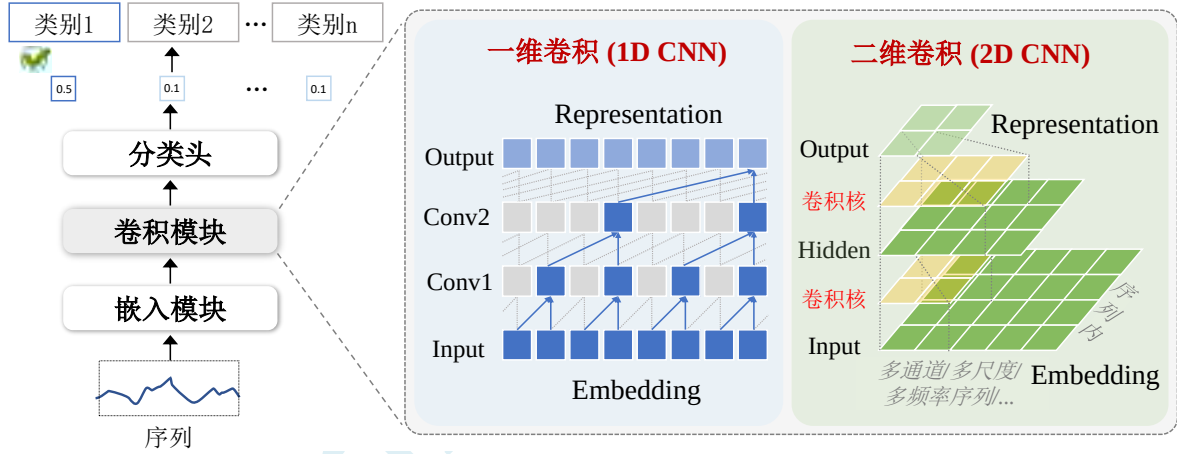


图 5.4: CNN 架构。

CNN（卷积神经网络）通过卷积操作提取时间序列的局部时序特征。与 MLP 不同，CNN 能够在时间维度上共享权重，从而捕捉相邻时间点之间的模式依赖。首先，将时间序列嵌入到高维表示：

$$\mathbf{Z}^{(0)} = (\mathbf{z}_1^{(0)}, \mathbf{z}_2^{(0)}, \dots, \mathbf{z}_H^{(0)}) \in \mathbb{R}^{H \times D} \quad (202)$$

其中， H 表示时间步长， D 表示输入特征维度。嵌入表示 $\mathbf{Z}^{(0)}$ 输入到一维卷积层，通过滑动卷积核提取局部时间片段的特征：

$$\mathbf{Z}^{(l)} = g^{(l)}(\text{Conv1D}(\mathbf{Z}^{(l-1)}; \mathbf{w}^{(l)}, \mathbf{b}^{(l)})), \quad l = 1, \dots, L \quad (203)$$

其中， $\mathbf{w}^{(l)}$ 表示卷积核参数， $\mathbf{b}^{(l)}$ 为偏置项， $g^{(l)}(\cdot)$ 为非线性激活函数（如 ReLU）。通过堆叠多层卷积和池化操作，模型能够逐步扩大感受野，捕获多尺度的时间依赖关系。

在经过 L 层卷积后，得到高层时序特征表示：

$$\mathbf{Z}^{(L)} = (\mathbf{z}_1^{(L)}, \mathbf{z}_2^{(L)}, \dots, \mathbf{z}_F^{(L)}) \in \mathbb{R}^{F \times D'} \quad (204)$$

随后，通过全局平均池化（Global Average Pooling, GAP）或最大池化（Global Max Pooling, GMP）将时间维度聚合为固定长度的全局特征向量：

$$\mathbf{h} = \text{Pooling}(\mathbf{Z}^{(L)}) \quad (205)$$

最后，同样通过一个分类头预测类别分布。

基于 CNN 的时间序列分类模型关键技术 卷积神经网络作为最经典的深度模型，得到了研究人员最大程度的开发。在时序分类问题中，一部分研究人员将视角聚焦于通道关系的捕捉，这得益于卷积神经网络的模型特性。而更多的研究者选择设计和调整卷积结构，在研究过程中创造出了诸多新颖且有效的模型结构。本章主要介绍的工作见表 5.1。

表 5.1: 基于 CNN 的时间序列分类模型关键技术总结

模型	通道处理策略	特征提取设计
MC-DCNN [60]	通道独立	一维卷积 (2 层)
MC-CNN [52]	通道依赖	一维卷积 (3 层)
FCN [50]	通道依赖	全卷积网络
ResNet [50]	通道依赖	残差网络 (9 层)
DCNNs [30]	通道依赖	空洞卷积 (4 层)
Wang and Oates [49]	通道依赖	Tiled CNN

- 1. 通道处理策略**是指对多维时间序列变量相互关系的处理方式。其中，通道独立 (Channel Independence) 在时间序列分类应用不多。最早将 CNN 引入时序分类的工作 MC-DCNN [60] 就使用了通道独立策略建模，具体而言，MC-DCNN 对每个输入通道独立地应用卷积，以此来处理多通道时间序列数据，对于每一个输入的变量，都经历了两次卷积和一次 ReLU 激活，最后进行最大池化操作，将每个维度的输出拼接后交由 softmax 分类器进行分类。通道依赖 (Channel Dependence) 指的是在多通道时间序列 (multi-channel time series) 数据中，不同通道之间的相互关系被建模，即在学习过程中考虑通道之间的信息交互。大部分的基于 CNN 的方法都是使用通道依赖策略，同时对所有输入变量应用卷积 [52, 50, 30] 等。
- 2. 特征提取设计**是指通过调整模型结构实现对于时序原始数据的特征提取。不同的模型设计侧重于提取时序数据的不同特性，比如有的侧重于时序的时间依赖性，而有些则聚焦于多尺度建模。以下列出了一些经典的基于卷积神经网络设计的模型结构。

一维卷积 通过滑动卷积核在时间维度上提取局部特征 [60, 52]，通过堆叠多层卷积层，可以逐步提取更加抽象的时序特征。每一层卷积都会通过卷积核对输入数据进行处理，提取特定的局部模式或高阶特征，最终形成一个丰富的特征表示用于分类任务。以下是用于分类的卷积的主要公式。

- (a) 第一层卷积：使用卷积核 $\mathbf{w}^{(1)} = [w_1^{(1)}, w_2^{(1)}, \dots, w_k^{(1)}]$ (卷积核大小为 k_1)，对输入时间序列进行卷积操作：

$$z_t^{(1)} = \sum_{i=1}^{k_1} w_i^{(1)} \cdot x_{t+i-1} + b^{(1)} \quad (206)$$

加入激活函数 (如 ReLU) 作用于卷积输出：

$$a_t^{(1)} = \text{ReLU}(z_t^{(1)}) \quad (207)$$

得到第一层的输出特征图 $\mathbf{a}^{(1)} = [a_1^{(1)}, a_2^{(1)}, \dots, a_{T-k_1+1}^{(1)}]$ 。

- (b) 第 m 层卷积：输入第 $m-1$ 层的输出 $\mathbf{a}^{(m-1)}$ ，应用第 m 个卷积核 $\mathbf{w}^{(m)}$ （卷积核大小为 k_m ）：

$$z_t^{(m)} = \sum_{i=1}^{k_m} w_i^{(m)} \cdot a_{t+i-1}^{(m-1)} + b^{(m)} \quad (208)$$

加入激活函数作用于第二层卷积输出：

$$a_t^{(m)} = \text{ReLU}(z_t^{(m)}) \quad (209)$$

得到第二层的输出特征 $\mathbf{a}^{(m)} = [a_1^{(m)}, a_2^{(m)}, \dots, a_{T-k_2+1}^{(m)}]$ 。

- (c) 池化层（可选）：最大池化（Max Pooling）或平均池化（Average Pooling），通过减少特征图的时间维度，保留最重要的信息。假设使用最大池化，池化窗口大小为 p ，则池化操作为：

$$p_j = \max\{a_j^{(m)}, a_{j+1}^{(m)}, \dots, a_{j+p-1}^{(m)}\} \quad (210)$$

- (d) 全连接层（Fully Connected Layer）：经过多层卷积和池化后，我们将通过一个全连接层（FC layer）来连接所有特征。设定全连接层的权重为 $W^{(fc)}$ 和偏置为 $b^{(fc)}$ ，该层的输出为：

$$z^{(fc)} = W^{(fc)} \cdot \mathbf{a}^{(m)} + b^{(fc)} \quad (211)$$

然后，激活函数（例如 ReLU 或 Sigmoid）作用于全连接层的输出：

$$h = \text{ReLU}(z^{(fc)}) \quad \text{或} \quad h = \text{Sigmoid}(z^{(fc)}) \quad (212)$$

- (e) 分类层（Softmax）：全连接层对特征向量 $\mathbf{h}$ 进行处理，最终通过 Softmax 层得到分类概率：

$$P(y = j | \mathbf{x}) = \frac{\exp(h_j)}{\sum_{c=1}^C \exp(h_c)} \quad (213)$$

其中， C 是类别数， h_j 是第 j 类的得分。

全卷积网络（Fully Convolutional Networks, FCN） 是传统 CNN 的变体，与 CNN 不同的核心在于，FCN 使用全局平均池化（Global Average Pooling, GAP）替代了传统 CNN 中常用的全连接层，这使得输入数据长度可变。GAP 层的存在保留部分通道信息的同时对通道信息进行了压缩，这使得其可以与类激活图（class activation map, CAM）[61] 一同使用，以突出显示输入中对预测类最重要的区域。**残差网络（ResNet）** 通过残差连接来减小影响深度学习模型的梯度消失效应，每个残差块类似 FCN 架构，这使得 ResNet 同样可以对变长数据进行建

模。在 [50] 中成功的将这两种模型应用到了端到端的时序分类任务中。**全局平均池化 (GAP)** 用于对每个特征通道取平均，得到一个固定长度的特征向量 $\mathbf{h}$ ：

$$h_c = \frac{1}{T'} \sum_{t=1}^{T'} a_t^{(L,c)} \quad (214)$$

其中 T' 是最后一层卷积的时间维度， $a_t^{(L,c)}$ 是第 L 层第 c 个通道的激活值。

空洞卷积 (Dilated convolution neural networks, DCNNs) [30] 最早被应用在计算机视觉相关的领域中，其设计之初的目的在于扩大卷积对于图片的感受野 (receptive field) 的同时保证参数不增加。空洞卷积利用一个不断扩张的卷积核进行卷积，这种有“空隙”的卷积核正是其名字“空洞”卷积的由来，这样的设计能够覆盖更大的输入区域，帮助网络捕获更远距离的依赖关系。而时间序列数据的周期和趋势信息往往被认为是关键特征，长距离依赖是反映这两个特征的重要体现，这正和空洞卷积的优点一拍即合，因此在时序任务中使用空洞卷积的思路应运而生，文章 [13] 率先在时序分类任务中使用了空洞卷积作为特征提取器，引领了一系列后续工作的设计思路。假设输入为一维时间序列 $\mathbf{x} = [x_1, x_2, \dots, x_T]$ ，卷积核大小为 k ，空洞率 (dilation rate) 为 d ，卷积的输出为 y_t 。空洞卷积的核心公式可以表示为：

$$y_t = \sum_{i=1}^k w_i \cdot x_{t+(i-1) \cdot d} + b \quad (215)$$

当 $d = 1$ 时，空洞卷积退化为普通卷积，卷积核在输入序列上逐一滑动，没有间隔；当 $d > 1$ 时，卷积核中的每个元素与输入序列的元素之间会有间隔。

时序图像化 (Time Series Imaging) 是将一维时间序列转换为二维图像，以利用深度学习 (如 CNN) 进行特征提取和分类。常见的时间序列图像化方法包括递归绘图 (RP)、Gram 角谱 (GAF)、Markov 转移字段 (MTF)、可视感知变换 (ViT 相关等)。[49] 将时序图像化并通过 CNN 进行图像建模。第一个方法名为格拉姆角场 (Gramian Angular Field, GAF)，GAF 在极坐标下表示时间序列，使用多种操作将这些角度转换为对称矩阵；第二个方法名为马尔可夫过渡场 (Markov Transition Field, MTF)，MTF 使用时序数据观测点间的转移概率对矩阵继续编码。这两种方法都会导致生成的图像过大，因此他们提出了一种缩小图片尺寸的策略，并将这两种图像组合起来形成一个双通道图像，最终使用 CNN 对这个图像进行分类，取得了良好的效果。自此产生了一些相关的时序图像化策略，直到现在人们也一直在探索时序图像化的方法和图像对于时序的影响。

这类方法往往被作为一种可供参考的思路，但始终没有成为时序分类策略的主流框架，其中的重要原因在于如何在图像中保留时序原本的关系和模式，哪部分信

息应该被压缩而在图像生成过程中添加的冗余信息又是否有必要，这些问题仍然悬而未决。

5.3.4 循环神经网络

循环神经网络是一种专门用于处理序列数据的神经网络模型。与传统的前馈神经网络不同，这种网络具有循环连接，可以记住输入序列中的上下文信息。循环神经网络的核心思想在于，判定过去的隐变量是否会对当前的样本产生影响，这样的考虑赋予了其处理依赖性较高的数据的能力，因此该模型在时间序列分析、自然语言处理和语音识别等领域表现出色。

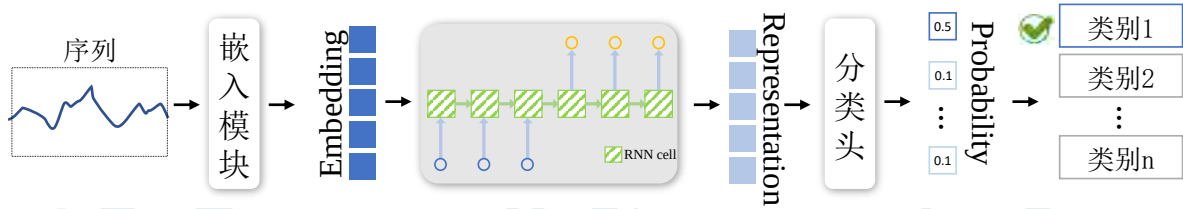


图 5.5: RNN 架构。

基本分类流程 基于循环神经网络的时序分类流程以图 5.5 中所示的内容为主，其中，RNN 状态更新可以形式化为以下形式：

$$\mathbf{h}_t = \sigma(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (216)$$

其中， $\mathbf{h}_t$ 表示时间步 t 的隐藏状态向量， $\sigma(\cdot)$ 为激活函数，通常为 $\tanh$ 或者 ReLU 。而输出层可以形式化为：

$$\mathbf{y}_t = \phi(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y) \quad (217)$$

循环神经网络侧重于处理序列数据，而时序的通道间关系往往是段和段或者点和点之间的关系，因此目前很少有工作依靠循环神经网络捕捉通道间联系，而是直接使用卷积或者注意力机制。所以在本章的模型介绍中将不特别介绍其对于通道间的设计。

特征提取 (Feature Extraction) 的目的是从时序数据中提取能够用来表达其语义的高维特征来将时序片段进行分类。循环神经网络本身对于序列数据的时间关系比较敏感，因此适合于建模时序数据。然而时序分类任务本身不同于预测和异常检测任务，分类任务需要将一整段时序视为一个整体进行建模，因此许多工作认为不能仅仅依靠上下文的关系，更需要整段序列“概括性”信息，这就需要卷积等建模方式进行辅助。本节将分别介绍仅使用循环神经网络建模的工作以及结合卷积的工作。

基于 RNN 的时间序列分类模型关键技术 使用循环神经网络进行时间序列分类的探索经历了几个阶段，最初的工作中，研究者直接使用 RNN 等网络进行特征提取，这样

表 5.2: 基于 RNN 的时间序列分类模型关键技术总结

模型	特征提取设计	其他设计
Vanilla RNN [18]	单层 RNN	无
Shallow RNNs [9]	双层 RNN	Brick 划分
GCRNN [31]	CNN + RNN	带组 Lasso 惩罚的全连接层
CNN-LSTM [40]	CNN + LSTM	无
LSTM-FCN [23]	LSTM + FCN	注意力机制
TimeNet [39]	GRU	无

的做法取得了一定的成效，现在也有一些工作沿用了这样的思路。而另一些工作选择使用其他模型进行融合设计，这也是目前主流的设计思路。其他工作则侧重在这些融合模型的基础上进行一些其他的辅助设计。一些经典的工作如表 5.2 所示。

- 模型架构** 仅使用循环网络的方法是研究人员的初步思路。在 Vanilla RNN 被直接使用在时序分类任务中之后，Don Kurian Dennis 等人 [9] 提出一种新型浅层 RNN 架构 Shallow RNNs，以同时实现对长依赖的建模和推理过程的并行化。因为对于输出的循环神经网络的顺序性质使得处理长序列时推理成本较高，这样的特性使得循环神经网络效率较低。因此，Shallow RNNs 采用两层设计，保持基线 RNN 的感受野长度 (T) 和模型规模，同时实现了可并行性和有限递归性。除了 RNN，也有工作基于 GRU 的序列自编码器 (SAE) 来处理时间序列分类 (TSC) 问题 [39]。该模型使用 GRU 作为编码器和解码器，能够处理不同长度的输入序列，并生成固定长度的输出表示。
- 特征提取设计** 侧重于将 RNN 与其它模型进行融合来提取时序特征。在 GCRNN 中 [31]，循环神经网络被用于增强时间序列数据的时间特征建模。其中，CNN 模块学习任务特定的特征，RNN 模块接收重新分布的 CNN 输出，进一步增强时间序列的时间特性建模，通过新架构实现 CNN 与 RNN 的无缝连接。这种设计显著提升了时间序列分类的效果。基于 LSTM，研究者在之前的 RNN 工作基础上进行了一系列的改进。通过结合 CNN 的空间特征提取能力和 LSTM 的时间建模能力，CNN-LSTM [40] 提高了人体活动识别的鲁棒性。LSTM-FCN [23] 是一种结合 LSTM 和全卷积网络 (FCN) 的深度学习模型，设计用于高效处理时间序列数据，尤其是在分类任务中的表现尤为突出。通过将 LSTM 和 FCN 的优点相结合，既能够捕获时间序列中的长期依赖关系，又能够提取序列中的局部时空特征。
- 其它设计** 的目的是为了使工作的内容更加完整，也可以辅助循环神经网络的特征捕捉。在 Don Kurian Dennis 等人 [9] 的工作中，在第一层将每个序列数据点 (感受窗) 划分为独立部分，称为砖块 (brick)，每块大小为 k，并将它们分别输

入到多个独立的 RNN。第二层在每个砖块上独立运行共享的浅层 RNN，通过该 RNN 整合第一层的输出，在捕获长依赖的同时，显著缩短递归长度并减小模型规模。而在 GCRNN [31] 中作者额外使用了带组 lasso 惩罚的全连接模块来辅助建模。在 LSTM-FCN [23] 中，作者探讨了使用注意力机制来进一步提升时间序列分类性能，提出了 Attention LSTM-FCN (ALSTM-FCN) 模型。

5.3.5 图神经网络

在时间序列分类任务中，一些深度学习方法（如 RNN、LSTM、CNN）通常忽略跨维度的隐藏依赖关系，难以直接建模时间序列中变量之间的复杂关系。而图神经网络（Graph Neural Networks, GNNs）通过显式建模图结构，能够有效捕捉变量或通道之间的依赖性，从而为时间序列分类提供一种强大的工具。

基本分类流程 使用图神经网络进行时序分类与其他方式有所不同，其中的核心区别在于图关系构建。不同于其他模型，使用图神经网络进行时序分类时，输入神经网络的并不是时序数据或时序嵌入，而是基于关系构建成的图，因此在流程上相比其他方法在输入模型前增加了图关系构建过程。

GNN 模型在每一层通过邻居聚合机制更新节点特征表示。对于第 l 层，每个节点 v_i 的表示更新为：

$$\mathbf{h}_i^{(l)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}^{(l)} \mathbf{h}_j^{(l-1)} + \mathbf{b}^{(l)} \right) \quad (218)$$

其中， $\mathbf{h}_i^{(l)}$ 表示第 l 层节点 i 的表示， $\mathcal{N}(i)$ 为其邻居节点集合， $\mathbf{W}^{(l)}$ 为可学习的权重矩阵， α_{ij} 为邻接权重（可为固定或由注意力机制学习）， $\sigma(\cdot)$ 为非线性激活函数。通过多层消息传递，GNN 能够捕捉节点间的高阶依赖关系，得到最终节点嵌入表示 $\mathbf{h}_i^{(L)}$ 。

在完成 L 层图卷积后，模型通过聚合函数（如平均、加权和或注意力加权池化）将节点级表示整合为全局图表示：

$$\mathbf{h}_{\text{graph}} = \text{Readout}(\{\mathbf{h}_1^{(L)}, \mathbf{h}_2^{(L)}, \dots, \mathbf{h}_H^{(L)}\}) \quad (219)$$

随后，通过分类头预测类别分布。通过图结构的信息传播与全局聚合，GNN 模型能够有效捕捉时间序列中变量间的结构依赖与全局动态特征，实现结构化的时间序列分类。

基于图神经网络的时间序列分类模型关键技术 基于图神经网络的时间序列分类问题研究往往集中在两个地方，一大部分的工作将侧重点放在了如何有效构造邻接矩阵的问题上，也就是如何建图，这也是图神经网络与其他模型在使用过程中的重要不同点，而另一些工作将重心放在了对于图神经网络模型结构的调整上。相关的工作如表 5.3 所示。

1. **图神经网络模型** 的是特征提取设计重要的研究工作，包括以下几类：

表 5.3: 基于 GNN 的时间序列分类模型关键技术总结

模型	邻接矩阵	特征提取设计	其他设计
Covert et al. [6]	人工定义	GCN	一维卷积
DGCNN [44]	动态学习	GCN	1×1 卷积
GraphSleepNet [22]	动态学习	GCN	时域注意力、二维卷积
MRF-GCN [29]	动态学习	GCN	FFT、阈值高斯核权重函数
Tang et al. [46]	动态学习	GCN	阈值高斯核权重函数、扩散模型
Cheng et al. [3]	动态学习	GAT	Shapelet
TodyNet [34]	动态学习	GCN	时间图池化层、一维卷积

- (a) **图卷积网络 (Graph Convolutional Network, GCN)** 通过邻接矩阵传播节点特征，并进行线性变换和激活函数映射 [6, 44, 22]。GCN 聚合邻居节点特征的方式为：

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right) \quad (220)$$

其中， $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ 为加上自环的邻接矩阵， $\tilde{\mathbf{D}}$ 是对应的度矩阵， $\mathbf{W}^{(l)}$ 是第 l 层的可学习权重， σ 为激活函数（如 ReLU）。

- (b) **图注意力网络 (Graph Attention Network, GAT)** [3] 在邻居特征聚合时引入注意力权重，自适应调整邻居对中心节点的影响：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))} \quad (221)$$

节点更新公式：

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j^{(l)} \right) \quad (222)$$

GAT 能动态调整邻居重要性，适合捕捉时间点间的非均匀依赖，常用于长短期时序依赖建模，多变量时间序列内部变量间相互关系建模。

2. **邻接矩阵** 在图神经网络用于时间序列分类任务中，首先需要构建时间序列之间或时间点之间的关系图。图结构通常通过邻接矩阵 (Adjacency Matrix) $\mathbf{A}$ 表示，决定了节点之间的信息传递方式。邻接矩阵的构建方法主要分为两类：

- (a) **人工定义邻接矩阵** [6] 是指根据领域知识或预设规则确定节点之间的连接关系。其优点是直观易解释，但缺点是缺乏灵活性，难以适应复杂动态数据。常见的定义方法包括：

- **阈值相似度法**：根据时间序列之间的相似度（如欧氏距离、DTW、余弦相似度）是否超过预设阈值，决定是否连边。

- 固定 K 邻居法：每个节点与相似度最高的 K 个邻居节点相连，形成 K -近邻图。
- 空间/结构先验法：如传感器网络中，节点物理位置接近的互相连接。

例如，基于余弦相似度构建邻接矩阵：

$$A_{ij} = \begin{cases} 1, & \text{if } \cos_sim(T_i, T_j) \geq \theta \\ 0, & \text{otherwise} \end{cases} \quad (223)$$

或者基于 K -近邻：

$$A_{ij} = \begin{cases} 1, & \text{if } T_j \in \text{Top-K-Neighbors}(T_i) \\ 0, & \text{otherwise} \end{cases} \quad (224)$$

人工定义方法适用于有先验知识的应用场景，但固定结构可能无法捕捉数据分布的动态变化。

- (b) **动态学习邻接矩阵** 是为了克服人工定义邻接矩阵的局限，近年来提出的邻接矩阵构造方法 [44, 22, 29, 46, 3, 34]。该方法在训练过程中根据节点特征自适应地学习邻接矩阵，使模型能够挖掘隐藏的时空依赖关系。一种常用方式是通过可学习函数 $\mathcal{F}$ 动态计算邻接矩阵：

$$A_{ij} = \mathcal{F}(\mathbf{h}_i, \mathbf{h}_j) \quad (225)$$

其中 $\mathbf{h}_i, \mathbf{h}_j$ 为节点 i 和 j 的特征向量， $\mathcal{F}$ 可以是：

- 内积相似度：

$$A_{ij} = \sigma(\mathbf{h}_i^\top \mathbf{h}_j) \quad (226)$$

- 神经网络：

$$A_{ij} = \sigma(\text{MLP}([\mathbf{h}_i, \mathbf{h}_j])) \quad (227)$$

- 注意力机制 (Graph Attention)：

$$A_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))} \quad (228)$$

动态学习邻接矩阵使模型能够根据特征自适应捕捉节点之间的隐含依赖，特别适用于无先验或高动态性的数据场景。

3. **其他技术** 除了构建邻接矩阵并通过图神经网络建模时间序列之间的关系，一些研究针对不同的数据会有专门的处理方式。在获得图结构后，Covert et al. 设计了一个时空卷积层 [6]，通过类似 2D 卷积的操作捕捉电极之间的交互关系（空间结构）及时间维度上的动态变化，即特征提取在时间和空间上是共享的。在不同

的时空卷积层中，对于下一层某个节点是由上一层的几个邻居聚合而来是作为一个参数 k 来设置的，即利用了一个 k 步可达性矩阵来合并来自 k 步内可达的附近节点的信息。

GraphSleepNet [22] 除了动态构建表示节点之间的成对关系的邻接矩阵，还分别使用空间图卷积和时间卷积来提取空间和时间特征。具体而言，GraphSleepNet 使用基于谱图理论的图卷积来提取空间维度上的空间特征。对于要识别的每个睡眠阶段，自适应睡眠图学习模块提供用于图卷积的邻接矩阵 A 。在图卷积运算充分提取每个睡眠阶段网络的空间特征后，再利用 2D 卷积层来提取当前睡眠阶段的时间上下文信息。

大部分的 GCN 方法只是聚合本地邻居的信息，类似于 CNN 中的固定内核，这意味着它只能聚合某个节点周围固定数量的节点信息。此外，这些 GCN 方法中邻接矩阵中的边权重均为 1，所有邻居被赋予相同的重要性。在实际应用中，不同邻居节点对目标节点的信息贡献可能不一致，例如社交网络中，朋友之间的影响力存在差异；分子图中，某些化学键的重要性高于其他键。因此，在另一个研究 [29] 中，Tianfu Li 等人提出了一种加权图，通过阈值高斯核权重函数计算每两个节点之间的权重。此外，在每一层图卷积中，设置了具有不同感受野或核大小的多个图卷积来聚合来自不同数量邻居的信息，即多感受野图卷积网络 (MRF-GCN) 来进行有效的智能故障诊断。用于分类的输出特征由这些不同卷积的输出融合而成。相似地，Siyi Tang 等人 [46] 也应用阈值高斯核来计算两条边的权重。

此外，一些模型通过引入其他方法，并与图神经有机结合实现了对图神经方法的改进。例如，Tang et al[46] 将脑电图信号的空间依赖性建模为扩散过程，将扩散模型引入时序分类中，在有向图 G 上进行双向随机游走，正向随机游走用于聚合下游信息，反向随机游走则关注上游信息，增强图的全局表示能力。[3] 引入将时间序列数据转换 shapelet 的思想，将每个时间序列数据转换为唯一的未加权 shapelet 图，并使用图注意力网络 (GAT) 自动捕获 shapelet 之间的相关性。

图神经网络在时间序列分类中显示出显著优势，尤其在处理复杂的高维、多通道和动态变化数据时。通过结合图的结构化信息与时间序列的动态特性，GNN 成为探索时间序列分类新方向的重要工具。

5.3.6 注意力机制

注意力机制 (Attention) 作为深度学习的一项重要技术革新，极大地提升了模型在序列数据处理中的效率和表现能力。在时间序列分类任务中，通过在 CNN、LSTM 等神经网络中嵌入注意力机，动态分配特征或者表征向量的权重，可以帮助模型聚焦于关键时间点或特征维度，从而有效捕捉全局依赖和局部模式。这种灵活性使其成为

解决高维、复杂时间序列问题的强大工具。本章将首先介绍注意力机制的核心组成部分 **Self-Attention**，阐述其在捕捉时间序列内在依赖关系方面的优势；随后，我们将进一步讨论 **Multi-Head Attention**，展示其如何通过多角度的特征表示增强模型的表达能力。

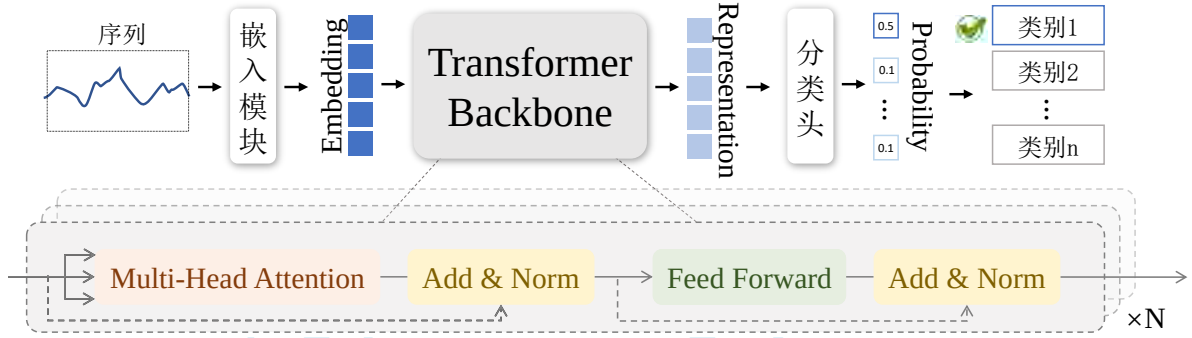


图 5.6: Transformer 架构。

基本分类流程 使用注意力机制进行时间序列分类的核心流程如图 5.6 所示，其核心流程依然可以抽象为对原始数据进行嵌入提取后，将嵌入输入到特征提取器中，最后使用一个分类头映射到对应类别。具体过程如下：

首先，通过线性层或嵌入层将每个时间步映射到 d_{model} 维的表示空间，并加上位置编码以保留时间顺序信息：

$$\mathbf{Z}^{(0)} = \text{Embedding}(\mathbf{X}) + \text{PositionalEncoding}(H, d_{\text{model}}) \quad (229)$$

注意力层通过“查询-键-值”（Query-Key-Value）机制对序列内的元素进行加权建模。对于多头自注意力层（Multi-Head Self-Attention），其计算过程为：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \quad (230)$$

其中， $\mathbf{Q} = \mathbf{Z}^{(l-1)}\mathbf{W}_Q$ ， $\mathbf{K} = \mathbf{Z}^{(l-1)}\mathbf{W}_K$ ， $\mathbf{V} = \mathbf{Z}^{(l-1)}\mathbf{W}_V$ 分别为查询、键、值矩阵， $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ 为可学习参数， d_k 为键向量维度。多头机制允许模型在不同的子空间中独立学习序列的相关性：

$$\text{MultiHead}(\mathbf{Z}^{(l-1)}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}_O \quad (231)$$

经过多层自注意力和前馈网络后，模型获得包含全局时序依赖的表示：

$$\mathbf{Z}^{(L)} = [\mathbf{z}_1^{(L)}, \mathbf{z}_2^{(L)}, \dots, \mathbf{z}_H^{(L)}] \quad (232)$$

在分类任务中，通常对所有时间步的输出进行聚合，以获得全局时间序列表示。常见的做法包括取序列首位的特殊标记（如 [CLS] token）或通过注意力加权池化：

$$\mathbf{h} = \sum_{t=1}^H \alpha_t \mathbf{z}_t^{(L)}, \quad \text{其中 } \alpha_t = \frac{\exp(\mathbf{w}^T \mathbf{z}_t^{(L)})}{\sum_{j=1}^H \exp(\mathbf{w}^T \mathbf{z}_j^{(L)})} \quad (233)$$

最终，通过分类头预测类别分布：

$$\hat{\mathbf{y}} = \text{Softmax}(\text{ClassificationHead}(\mathbf{h})) \tag{234}$$

其中， $\hat{\mathbf{y}} \in \mathbb{R}^C$ 为模型对 C 个类别的预测概率。

基于注意力机制的时间序列分类模型关键技术 在时序分类问题中，多种注意力机制都有广泛的应用与尝试。最初的工作集中在使用基本的自注意力机制进行适合于时间序列的修改，而后续的工作为了更好的分类效果，将视角转向了特征提取效果更好的多头注意力机制，对于多头注意力机制的调整和探索也是目前时序分类问题研究的主要方向。另外，还有一些工作将注意力机制与其他模型进行融合，或为注意力机制添加一些辅助策略满足特定需求，产生了许多有价值的成果。一些经典且重要的工作如表 5.4 中所示。

表 5.4: 基于 Attention 的时间序列分类模型关键技术总结

模型	注意力机制	特征提取设计	其他设计
CA-SFCN [15]	时间注意力、变量注意力	FCN	无
LAXCAT [16]	变量注意力	CNN	无
RTFN [51]	变量注意力	CNN	LSTM 获取查询、键和值
MACNN [2]	变量注意力	多尺度 CNN	全局池化
WHEN [48]	时间注意力	CNN + BiLSTM	自适应小波函数
SAnD [43]	多头注意力	线性嵌入 (Linear Embedding)	无
GTN [35]	时间注意力	线性嵌入 (Linear Embedding)	门控机制
FMLA [57]	多头注意力	可变形卷积 (Deformable CNN)	掩模机制
ConvTran [12]	多头注意力	Disjoint-CNN	时间绝对与相对位置编码

1. 注意力机制 (Attention Mechanism) 不同设计模式对时序分类效果也有所区别。

自注意力是注意力机制的一种核心形式，广泛应用于处理序列数据，特别是时间序列分类任务。与传统的注意力机制依赖外部信息源不同，**Self-Attention** 完全基于输入序列本身，计算序列中不同位置之间的相关性。通过这种方式，**Self-Attention** 能够捕捉时间序列中长距离依赖关系，灵活建模全局上下文信息。对于时间序列分类任务，这种能力尤为重要，因为时间序列中的远程依赖或跨维度交互可能对分类结果至关重要。**Self-Attention** 不仅能够提升模型的表达能力，还能提高对复杂时间序列结构的建模效果。

在 RTFN 模型 [51] 中，自注意力机制被创新性地应用于连接两个多层卷积特征提取器。该模型在两层多头卷积层之间加入了自注意力模块，专门用于关联第一个卷积层提取的局部特征在序列中的相对位置关系。这种设计使得第二个卷积层能够接收到经过全局上下文信息增强的特征输入，从而获得更丰富的特征表示。特

别值得注意的是，RTFN 采用 LSTM 而非传统的卷积或线性变换来生成注意力机制中的查询、键和值矩阵，充分利用了 LSTM 在序列建模方面的优势，为注意力计算提供了更具时序感知的特征表示。

多头注意力机制是注意力机制的重要扩展,通过并行化多个独立的 Self-Attention 计算来提升模型的表达能力。在时间序列分类任务中，单一的注意力头可能难以全面捕捉序列的多样性特征，而 **Multi-Head Attention** 则通过多头机制学习不同的特征表示，使模型能够从多个角度关注关键时间点或特征维度。这种机制不仅增强了对全局依赖的捕捉能力，还提高了时间序列中局部和全局模式的建模效果。此外，多头的并行计算还具备较强的扩展性，为复杂时间序列分类任务提供了更丰富的特征表示能力。

SAnD 模型 [43] 是第一个直接将多头注意力机制作为主要特征提取器用于时间序列分类的工作。与之前将注意力机制作为辅助模块的设计不同，**SAnD** 完全基于注意力机制构建，通过堆叠多头注意力层来构建深层的序列建模架构。这种纯粹基于注意力的设计避免了循环神经网络的序列依赖性，使得模型能够充分利用现代硬件的并行计算能力，同时保持了强大的序列建模能力。**ConvTran** 模型 [12] 和 **FMLA** 模型 [57] 进一步发展了多头注意力机制的应用。**ConvTran** 通过引入专门设计的位置编码来增强多头注意力对时间序列顺序信息的感知能力，而 **FMLA** 则创新性地将可变形卷积与多头注意力相结合，通过可变形机制使模型能够自适应地调整感受野大小，更好地捕捉不同尺度的时间模式。**FMLA** 还引入了掩模机制，在协作线性注意力块的末端添加随机和规则掩模层，有效减少了噪声对注意力权重的干扰，提高了模型的鲁棒性。

时间注意力专门针对时间维度设计，用于识别序列中不同时间点的重要性差异。在时间序列分类任务中，并非所有时间点对分类决策都具有同等重要性。某些关键事件或模式可能只出现在序列的特定时间段，而其他时间段可能包含冗余或噪声信息。时间注意力机制通过学习动态权重，使模型能够重点关注那些包含判别性信息的时间区域。

CA-SFCN 模型 [15] 采用了精细化的时间注意力设计。在该模型中，时间注意力模块被放置在 FCN 特征提取器之后，专门用于捕获每个变量对应时间序列中的关键历史点。这种设计使得模型能够同时考虑到长期和短期的时序依赖关系，有效捕捉时间序列中的多尺度时间模式。模型通过时间注意力权重的计算，实现了对重要历史时刻的强调和对无关时刻的抑制，从而提高了分类性能。**WHEN** 模型 [48] 在时间注意力机制的应用上展现了创新性。该模型将经典的动态时间规整 (DTW) 技术转化为一种新型的时间注意力机制 (DTWAtt)。传统的 DTW 通过寻找两个序列之间的最优对齐路径来度量它们的相似性，而 **DTWAtt** 则将这种对

齐过程重新表述为注意力权重的计算。这种设计使得模型能够克服多序列间的时间扭曲问题，实现对非对齐序列的有效分类。同时，WHEN 还通过注意力机制动态计算小波函数中的参数，为每个时间序列输入的每个时间步自适应设置适合数据特征的频带参数，进一步增强了模型对非平稳时间序列的建模能力。

变量注意力（也称为通道注意力或特征注意力）专门为多元时间序列设计，用于评估不同变量在分类任务中的相对重要性。在多元时间序列中，不同变量可能对最终的分类决策贡献不同，某些关键变量可能包含更强的判别信息，而其他变量可能包含较多噪声或冗余信息。变量注意力机制通过为每个变量分配自适应的权重，使模型能够重点关注那些信息量丰富的变量。

CA-SFCN 模型 [15] 采用了双阶段的注意力设计，在时间注意力之后引入了变量注意力模块。该模块专门用于计算每个时间点上不同变量特征之间的注意力权重，从而捕捉多元时间序列中变量间的动态相互作用。这种时序感知的变量注意力使得模型能够理解变量重要性随时间变化的关系，为分类决策提供了更丰富的上下文信息。LAXCAT 模型 [16] 将变量注意力机制与可解释性分析紧密结合。该模型在卷积网络提取特征后，通过变量注意力模块计算每个变量的全局重要性权重。这些权重不仅用于改进分类性能，还作为模型决策的解释依据，帮助用户理解哪些变量对最终的分类结果产生了关键影响。这种设计在医疗诊断、金融风控等需要模型透明度的应用场景中具有重要价值。MACNN 模型 [2] 创新地将多尺度卷积与变量注意力相结合。该模型首先通过多尺度卷积块提取时间序列的短期、中期和长期时间依赖性，然后在不同尺度特征上分别应用变量注意力机制。这种多尺度变量注意力使得模型能够识别变量重要性在不同时间尺度上的变化，为复杂时间序列模式的理解提供了更细致的视角。

2. **特征提取设计 (Feature Extraction Design)** 特征提取是时间序列分类模型的基础模块，负责从原始序列中提取有判别性的特征表示。优秀特征提取设计能够有效捕获时间序列中的关键模式，为后续的注意力机制和分类器提供高质量的特征输入。

线性嵌入在基于 Transformer 的架构中扮演着重要角色，负责将原始时间序列数据映射到适合注意力机制处理的高维特征空间。SAnD [43] 和 GTN [35] 均采用线性嵌入作为特征提取的初步步骤。这种设计的优势在于其简单高效，同时保留了原始序列的大部分信息，为后续的注意力计算提供了良好的基础。

卷积神经网络 (CNN) 及其变体因其强大的局部特征提取能力而成为时间序列分析的首选。基础 CNN 通过滑动卷积核在时间维度上进行局部感知，能够有效捕捉时间序列中的局部模式和短期依赖。LAXCAT [16] 和 RTFN [51] 均采用这种设计，通过一维卷积层从原始序列中提取具有时间局部性的特征表示。

多尺度 CNN 进一步扩展了基础 CNN 的能力，通过并行使用不同大小的卷积核来捕获时间序列中的多尺度时间模式。MACNN [2] 设计了专门的多尺度卷积块，分别针对短期、中期和长期的时间依赖性进行优化。这种设计使得模型能够同时感知不同时间尺度上的模式，从而更好地理解复杂的时间序列结构。

可变形卷积为 CNN 提供了自适应调整感受野的能力。FMLA [57] 集成了可变形卷积网络，通过可学习的偏移量使卷积核能够根据输入数据的特性动态调整其采样位置。这种机制使得模型能够更灵活地适应不同形状和时间尺度的时间模式，特别是在处理非平稳或具有复杂形态的时间序列时表现出色。

混合特征提取器通过结合不同类型网络的优点来获得更全面的特征表示。WHEN [48] 采用了 CNN 与双向 LSTM (BiLSTM) 的混合架构，其中 CNN 负责提取局部时间模式，BiLSTM 则捕捉序列的前后依赖关系。这种组合充分利用了 CNN 在局部特征提取和 BiLSTM 在序列建模方面的各自优势，为模型提供了丰富而互补的特征表示。

3. **其他设计**除了核心的注意力机制和特征提取模块外，各种**辅助性设计**在提升模型性能和实用性方面发挥着重要作用。这些设计通常针对时间序列数据的特定挑战或实际应用需求而开发。

位置编码是弥补自注意力机制缺失位置感知能力的关键技术。由于自注意力机制本身不具备感知序列顺序的能力，位置编码通过为每个时间步添加位置信息来注入序列的顺序特性。ConvTran [12] 提出了专门针对时间序列设计的时间绝对位置编码 (tAPE) 和高效相对位置编码 (eRPE)。tAPE 通过将序列长度和输入嵌入维度融入编码过程，创造了更适合时间序列特性的位置表示；eRPE 则通过优化实现方式，提高了相对位置编码在长序列情况下的计算效率。

门控机制提供了自适应融合不同特征源的有效方法。GTN [35] 使用门控机制来学习分别对变量维度和时间维度进行建模的两个 Transformer 输出的融合权重。这种设计使得模型能够根据具体输入数据的特性，动态调整不同特征源的贡献程度，实现了更灵活和自适应的特征融合策略。

掩模机制通过有选择地抑制部分特征或注意力权重来提高模型的鲁棒性。FMLA [57] 在协作线性注意力块末端添加随机和规则掩模层，通过随机丢弃部分注意力权重或根据预定规则抑制特定权重来减少噪声干扰。这种设计类似于计算机视觉中的 Dropout 技术，但专门针对注意力机制进行了优化，有效防止了模型对特定注意力模式的过拟合。

通过对上述技术维度的系统分析可以看出，现代基于注意力机制的时间序列分类模型正朝着更加精细化、自适应和可解释的方向发展。各种注意力变体的出现使得模型能够针对时间序列的不同特性进行专门优化，而多样化的特征提取设计和辅助技术

的引入则进一步扩展了模型的应用范围和实用价值。未来，随着对时间序列本质特性的深入理解和计算技术的持续进步，基于注意力机制的时序分类技术有望在更多复杂场景中发挥重要作用。

5.4 时间序列分类自监督方法

时间序列预测任务天然的存在用以监督的真实值，即回看窗口的后续值，然而时序分类任务的监督用的标签来自于各领域专家的手工标注，这就导致有标签的时序分类数据十分稀有。为了应对这样的现实难题，研究人员希望通过挖掘时序数据的内部特征来进行分类，这样可以减少标注数据稀少带来的困难，基于此类方法产生的不依赖标签的任务就是无监督或自监督方法。自监督方法通常更加关注通过深度模型提取后的表征，在这些表征之间设计的代理任务是自监督学习的核心设计点，因此本节在介绍时会淡化骨架网络的描述，重点阐述方法对于代理任务的设计，表 5.5 是一部分经典的自监督工作。

表 5.5: 基于自监督学习的时间序列分类模型关键技术总结

模型	自监督策略	特征提取设计	其他设计
TCL [17]	对比学习	MLP	基于序列的对比
TNC [47]	对比学习	双向 RNN	基于序列的对比
TS-TCC [10]	对比学习	CNN + Transformer	基于序列的对比
TS2Vec [54]	对比学习	空洞卷积 (Dilated CNN)	基于实例/序列的对比
TF-C [56]	对比学习	ResNet	基于实例的对比
BENDR [24]	重构学习	CNN + Transformer	序列掩码
TST [55]	重构学习	Transformer	双向掩码
TARNet [4]	重构学习	Transformer	双向掩码

5.4.1 基于序列的对比学习

如何区分两种数据的语义信息，一个最直接的想法就是“找不同”，在一些规则的约束下，将应该相似的数据投影到表征空间中相似的位置，而不应该相似的数据尽量远离，这样就能自然地分开不同的样本。虽然这样的想法十分简单，但在哪些数据“应该”相似，哪些数据“不该”相似这个问题上不同的工作有不同的看法，这也是对比学习中最为重要的研究点——如何选取正负样本。在对比学习中，我们常称一个待比较的样本为锚点，而与其相似、希望在表征空间中拉近的样本称为正例，反之则为负例。

最早的工作中，人们普遍认为时序的正例和负例应该来自与同一条序列的不同部分，Time-Contrastive Learning(TCL) [17] 首次将对比学习应用到了时序表征学习当中，其选择正负例的想法十分朴素——与锚点相邻的窗口中的时序理应是相似的，因为时序数据有时间上的关联性，而距离较远的数据则与锚点数据并没有太多联系，所以被选为负例。这样的思想虽然乍一看很有道理，但是与时序数据的性质有些相悖，时序数据通常会呈现出一定形式的周期性，因此远端的数据有可能其实反而是相似的正例，基于这样的反思，TNC [47] 应运而生。TNC 根据时序的平稳性检验方法，给定了一个更具可解释性的邻域确定方法，在平稳的窗口内的样本被视为正例，而为了应对上述的远距离正例的问题，TNC 通过相似度度量的方法判定邻域外的样本是否应该属于负例，并使用加权的方法调整其作为正例或负例的权重。

这类基于序列选择正负例的做法虽然不无道理，但是如何在一段序列中选取紧邻窗口，如何在序列中判定某部分是正例还是负例，都是悬而未决的难题。因此，受到图像分类工作的启发，一些研究人员决定换一个视角来审视时序分类问题，是否可以将每一个样本直接作为一个实例，然后基于实例进行对比学习？自然的，获得或者选取正例就成为了一个值得深究的问题，研究者通过借鉴视觉领域的方法和分析时间序列的自身性质，提出了一系列的数据增强策略。

数据增强 数据增强是指对样本进行一定程度的变化获得新的样本，最早在计算机视觉领域广泛使用。数据增强有许多方面的应用，而关于其起作用的原因也有许多解释的角度。最直接的原因在于，数据增强扩充了原本的样本数量，对于数据驱动的机器学习、尤其是深度学习方法而言，更多的数据意味着更好的拟合效果。还有一些其他的观点认为，数据增强是一种正则化的手段，避免模型产生过拟合。而从鲁棒性的角度而言，数据增强一定程度上模拟了在获得数据的过程中可能产生的客观问题，比如传感器暂时失灵等，如果这些产生微小扰动的数据不会影响模型建模，则证明模型有良好的鲁棒性。因此，数据增强的研究是必要且有意义的。

数据增强有一个非常重要的假设——变换不变性，即认为增强后的样本仅仅产生了细微的变化，这样的变化不会带来语义或类别的改变。在视觉领域，数据增强通常是指对图像的各种变化，比如裁切、旋转、翻转等。视觉领域大量的工作验证了这些增强方式满足变换不变性，在大部分情况下不会产生影响。然而，这些适合于图像的方法并不能完全照搬到时间序列的增强中，比如旋转操作会严重破坏时序的时间依赖性，导致样本不再是时间序列。本节将介绍一些常用的时序增强方法，以及增广数据在模型中的使用方法。

1. **随机变换** 这是最为常用的增强策略，这类方法重点关注时间序列的幅值信息，具体体现为对时序数据的具体数值进行改变。抖动 (Jittering) 向时序数据添加了随机噪声；翻转 (Flipping) 字面意义的将时序进行了反转；缩放 (Scaling) 将时序数据乘以一个符合高斯分布的系数；扭曲 (Warping) 是一系列方法的总称，

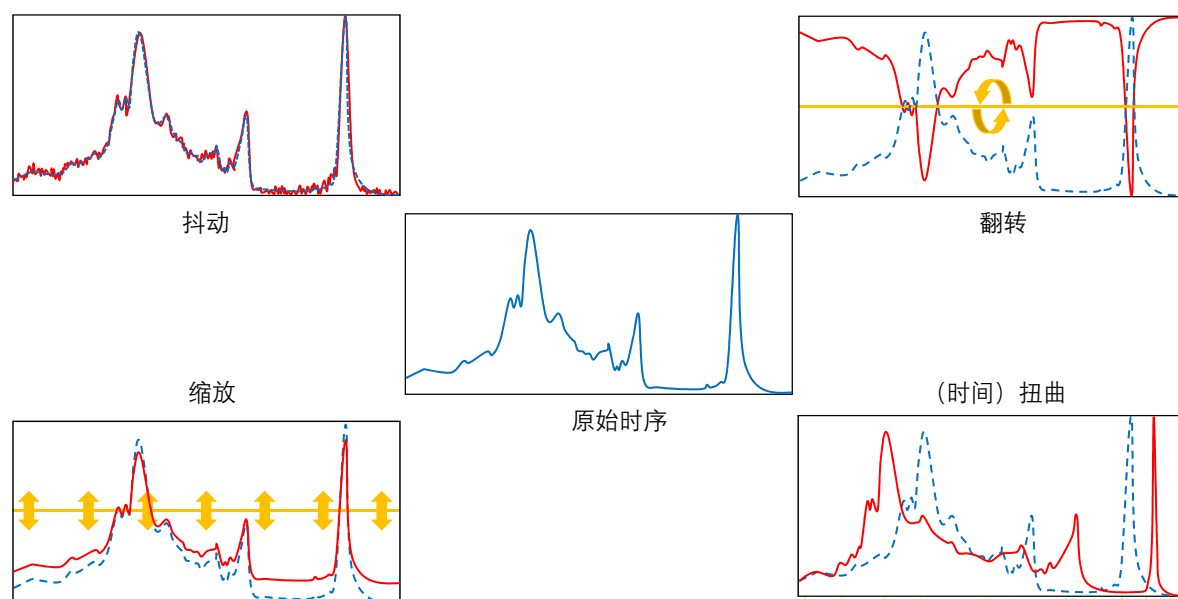


图 5.7: 数据增强方法。

比如随即扭曲、时间扭曲、时间拉伸等，这些方法会根据一些规则对时序数据进行扭曲。类似的，通过使用傅里叶变换或小波变换等方式将时间序列从时域转换为频域同样可以利用这些基于幅值变换的增强方式，而将频域信息和时域信息进行组合又可以产生许多能够描述随时间变换的频率信息的时频增强策略，本节仅抛砖引玉进行简单基础的介绍。这些方法不会影响时序内部的性质，能最低程度的保护时序的内部性质，但是即使这些做法也不能完全保证时序语义信息不会在增强的过程中丢失或改变，这也是数据增强可以获得持续且广泛关注的原因。

2. **组合样本** 如果一个类别中包含四个样本，那么将这四个样本进行两两组合，得到的十二个样本是不是也应该属于这个类别呢？基于组合样本的增强方式认为这个问题的答案是肯定的。具体到方法，这种思想可以产生各种各样的变种，最为普通的方法被称为 **Mixup**，这种方法通过一个采样自贝塔分布的参数为两个样本加权得到新的样本；另外，Cui 等人在其工作中 [7] 将一个样本内进行了多次划分，基于集成学习的思路将这些分段样本进行分类并组合投票，以获得整段样本的类别；考虑到直接进行叠加可能产生的语义扭曲问题，加权 **DTW** 重心平均方法 (**wDBA**) [11] 应运而生。这些通过组合已知样本进行增广的思路是一种启发式的方法，虽然符合直觉，但理论上缺少足够的依据，而由于其增广样本的产生方式也导致这些不够丰富，相对于随机变换缺乏多样性。
3. **选择合适的增强方式** 虽然本节提到了许多时序中常用的增强策略，这些策略在通常情况下是满足变换不变性的，但是由于时序数据涵盖了非常多的领域，获取时序数据的方法也非常多样，这使得完全找到一种适合于所有时序数据的增强

方法变成了一种奢望，如何针对数据选择合适的增强方法一直以来都是时序建模的难题。为了探索这类问题，自适应增强策略变成了数据增强的一个重要课题。目前已有的自适应增强策略保持了一个统一的思路，就是将多种增强的样本进行数据自适应的累加，依靠权重控制不同增强在不同数据集上的影响。比如 InfoTS [36] 就提出了一种基于高保真度和多样性的信息感知定义来选择有效增强的标准；AutoTCL [59] 提出了一个基于引子分解的自适应框架，用于搜索数据增强的策略。目前，时序领域的数据自适应增强策略还不够完善，良好的增强策略依赖于对时序性质的充分了解，在数据增强方式上依然有许多可以探索的方向和空间。

5.4.2 基于实例的对比学习

通过使用数据增强，TS-TCC [10] 率先做出了尝试，在这篇工作中，每段时序对应一个实例，每一个实例的“强弱”增强互为正例，而同一个批次内的其他实例则作为负例。这样的做法隐含了将每一个样本作为一个整体看待的思路，凸显了样本整体信息对于分类任务的重要性。而 TS2Vec [54] 兼顾了上述的两种思想，通过两个的视角得到两种损失，相加得到了自己的训练任务。由于时间序列可以通过频域进行描述，基于频域的对比学习也是一种合理的思路，TF-C [56] 将时序从时域转化为了频域并进行了基于频域的对比和时频之间的对比。

5.4.3 掩码-重构学习

除了对比学习还有许多其他的自监督任务可以在未标注数据上进行训练。最为常见的自监督任务是基于自预测的重构任务，这类方法通常通过一些规则将时序数据进行部分掩码，训练模型通过已知数据重构掩码数据的能力。这样的方法在时序预测和异常检测任务中十分自然且贴合任务需求，将时序尾部掩码并重构本质是一种时序预测的能力，而通过重构正常模式判断异常序列则是异常检测的任务目标。虽然这样的做法对于分类任务的帮助比较间接，但仍有相当的工作基于重构思想进行了适合时序分类的模型设计。BErt-inspired Neural Data Representations (BENDR) [24] 使用 Transformer 架构重构心电图 (ECG) 数据，证明其可以有效处理不同硬件记录的大量 ECG 数据，而 TST [55] 和 TARNet [4] 同样使用 Transformer 架构，使用基于自预测的预训练方法对多变量时序数据进行分类。这些工作证明了自预测类方法对于分类任务的价值。

参考文献

- [1] Anthony Bagnall, Jason Lines, Jon Hills, and Aaron Bostrom. Time-series classification with cote: The collective of transformation-based ensembles. *IEEE Transactions on Knowledge and Data Engineering*, 27(9):2522–2535, 2015.
- [2] Wei Chen and Ke Shi. Multi-scale attention convolutional neural network for time series classification. *Neural Networks*, 136:126–140, 2021.
- [3] Ziqiang Cheng, Yang Yang, Shuo Jiang, Wenjie Hu, Zhangchi Ying, Ziwei Chai, and Chunping Wang. Time2graph+: Bridging time series and graph representation learning via multiple attentions. *IEEE Transactions on Knowledge and Data Engineering*, 35(2):2078–2090, 2021.
- [4] Ranak Roy Chowdhury, Xiyuan Zhang, Jingbo Shang, Rajesh K. Gupta, and Dezhi Hong. Tarnet: Task-aware reconstruction for time-series transformer. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '22*, page 212–220, New York, NY, USA, 2022. Association for Computing Machinery.
- [5] Fu Lai Korris Chung, Tak-Chung Fu, Wing Pong Robert Luk, and Vincent To Yee Ng. Flexible time series pattern matching based on perceptually important points. In *Workshop on learning from temporal and spatial data in international joint conference on artificial intelligence*, 2001.
- [6] Ian C Covert, Balu Krishnan, Imad Najm, Jiening Zhan, Matthew Shore, John Hixson, and Ming Jack Po. Temporal graph convolutional networks for automatic seizure detection. In *Machine learning for healthcare conference*, pages 160–180. PMLR, 2019.
- [7] Zhicheng Cui, Wenlin Chen, and Yixin Chen. Multi-scale convolutional neural networks for time series classification, 2016.
- [8] Felipe Arias Del Campo, María Cristina Guevara Neri, Osslán Osiris Vergara Villegas, Vianey Guadalupe Cruz Sánchez, Humberto de Jesús Ochoa Domínguez, and Vicente García Jiménez. Auto-adaptive multilayer perceptron for univariate time series classification. *Expert Systems with Applications*, 181:115147, 2021.

- [9] Don Dennis, Durmus Alp Emre Acar, Vikram Mandikal, Vinu Sankar Sadasivan, Venkatesh Saligrama, Harsha Vardhan Simhadri, and Prateek Jain. Shallow rnn: accurate time-series classification on resource constrained devices. *Advances in neural information processing systems*, 32, 2019.
- [10] Emadeldeen Eldele, Mohamed Ragab, Zhenghua Chen, Min Wu, Chee Keong Kwoh, Xiaoli Li, and Cuntai Guan. Time-series representation learning via temporal and contextual contrasting, 2021.
- [11] Germain Forestier, François Petitjean, Hoang Anh Dau, Geoffrey I. Webb, and Eamonn J. Keogh. Generating synthetic time series to augment sparse datasets. *2017 IEEE International Conference on Data Mining (ICDM)*, pages 865–870, 2017.
- [12] Navid Mohammadi Foumani, Chang Wei Tan, Geoffrey I Webb, and Mahsa Salehi. Improving position encoding of transformers for multivariate time series classification. *Data Mining and Knowledge Discovery*, 38(1):22–48, 2024.
- [13] Jean-Yves Franceschi, Aymeric Dieuleveut, and Martin Jaggi. Unsupervised scalable representation learning for multivariate time series, 2020.
- [14] Josif Grabocka, Nicolas Schilling, Martin Wistuba, and Lars Schmidt-Thieme. Learning time-series shapelets. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 392–401, 2014.
- [15] Yifan Hao and Huiping Cao. A new attention mechanism to classify multivariate time series. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, 2020.
- [16] Tsung-Yu Hsieh, Suhang Wang, Yiwei Sun, and Vasant Honavar. Explainable multivariate time series classification: a deep neural network which learns to attend to important variables as well as time intervals. In *Proceedings of the 14th ACM international conference on web search and data mining*, pages 607–615, 2021.
- [17] Aapo Hyvarinen and Hiroshi Morioka. Unsupervised feature extraction by time-contrastive learning and nonlinear ica, 2016.
- [18] Michael Hüsken and Peter Stagge. Recurrent neural networks for time series classification. *Neurocomputing*, 50(2):223–235, 2003.

- [19] Ali Ismail-Fawaz, Maxime Devanne, Jonathan Weber, and Germain Forestier. Deep learning for time series classification using new hand-crafted convolution filters. In *Proceedings of the 2022 IEEE International Conference on Big Data (Big Data)*, pages 972–981. IEEE, 2022.
- [20] Brian Kenji Iwana, Volkmar Frinken, and Seiichi Uchida. Dtw-nn: A novel neural network for time series recognition using dynamic alignment between inputs and weights. *Knowledge-Based Systems*, 188:104971, 2020.
- [21] Cun Ji, Chao Zhao, Shijun Liu, Chenglei Yang, Li Pan, Lei Wu, and Xiangxu Meng. A fast shapelet selection algorithm for time series classification. *Computer networks*, 148:231–240, 2019.
- [22] Ziyu Jia, Youfang Lin, Jing Wang, Ronghao Zhou, Xiaojun Ning, Yuanlai He, and Yaoshuai Zhao. Graphsleepnet: Adaptive spatial-temporal graph convolutional networks for sleep stage classification. In *Ijcai*, volume 2021, pages 1324–1330, 2020.
- [23] Fazle Karim, Somshubra Majumdar, Houshang Darabi, and Shun Chen. Lstm fully convolutional networks for time series classification. *IEEE access*, 6:1662–1669, 2017.
- [24] Demetres Kostas, Stephane Aroca-Ouellette, and Frank Rudzicz. Bendr: using transformers and a contrastive self-supervised learning task to learn from massive amounts of eeg data, 2021.
- [25] Xuan-May Le, Ling Luo, Uwe Aickelin, Minh-Tuan Tran, David Berlowitz, and Mark Howard. Ship: A shapelet-based approach for interpretable patient-ventilator asynchrony detection. *arXiv preprint arXiv:2503.06571*, 2025.
- [26] Xuan-May Le, Minh-Tuan Tran, and Van-Nam Huynh. Learning perceptual position-aware shapelets for time series classification. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 53–69. Springer, 2022.
- [27] Guozhong Li, Byron Choi, Jianliang Xu, Sourav S Bhowmick, Kwok-Pan Chun, and Grace Lai-Hung Wong. Efficient shapelet discovery for time series classification. *IEEE transactions on knowledge and data engineering*, 34(3):1149–1163, 2020.

- [28] Guozhong Li, Byron Choi, Jianliang Xu, Sourav S Bhowmick, Kwok-Pan Chun, and Grace Lai-Hung Wong. Shapenet: A shapelet-neural network approach for multivariate time series classification. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 8375–8383, 2021.
- [29] Tianfu Li, Zhibin Zhao, Chuang Sun, Ruqiang Yan, and Xuefeng Chen. Multireceptive field graph convolutional networks for machine fault diagnosis. *IEEE Transactions on Industrial Electronics*, 68(12):12739–12749, 2020.
- [30] Yuhong Li, Xiaofan Zhang, and Deming Chen. Csrnet: Dilated convolutional neural networks for understanding the highly congested scenes, 2018.
- [31] Sangdi Lin and George C Runger. Gcrnn: Group-constrained convolutional recurrent neural network. *IEEE transactions on neural networks and learning systems*, 29(10):4709–4718, 2017.
- [32] Jason Lines and Anthony Bagnall. Time series classification with ensembles of elastic distance measures. *Data Mining and Knowledge Discovery*, 29(3):565–592, 2015.
- [33] Jason Lines, Luke M Davis, Jon Hills, and Anthony Bagnall. A shapelet transform for time series classification. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 289–297, 2012.
- [34] Huaiyuan Liu, Donghua Yang, Xianzhang Liu, Xinglei Chen, Zhiyu Liang, Hongzhi Wang, Yong Cui, and Jun Gu. Todynet: temporal dynamic graph neural network for multivariate time series classification. *Information Sciences*, page 120914, 2024.
- [35] Minghao Liu, Shengqi Ren, Siyuan Ma, Jiahui Jiao, Yizhou Chen, Zhiguang Wang, and Wei Song. Gated transformer networks for multivariate time series classification. *arXiv preprint arXiv:2103.14438*, 2021.
- [36] Dongsheng Luo, Wei Cheng, Yingheng Wang, Dongkuan Xu, Jingchao Ni, Wenchao Yu, Xuchao Zhang, Yanchi Liu, Yuncong Chen, Haifeng Chen, et al. Time series contrastive learning with information-aware augmentations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 4534–4542, 2023.

- [37] Qianli Ma, Wanqing Zhuang, Sen Li, Desen Huang, and Garrison Cottrell. Adversarial dynamic shapelet networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 5069–5076, 2020.
- [38] Vivek Mahato, Muhannad Ahmed Obeidi, Dermot Brabazon, and Pádraig Cunningham. Detecting voids in 3d printing using melt pool time series data. *Journal of Intelligent Manufacturing*, 33(3):845–852, 2022.
- [39] Pankaj Malhotra, Vishnu TV, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. Timenet: Pre-trained deep recurrent neural network for time series classification. *arXiv preprint arXiv:1706.08838*, 2017.
- [40] Ronald Mutegeki and Dong Seog Han. A cnn-lstm approach to human activity recognition. In *2020 international conference on artificial intelligence in information and communication (ICAIIIC)*, pages 362–366. IEEE, 2020.
- [41] John Paparrizos and Michael J. Franklin. Grail: Efficient time-series representation learning. *Proceedings of the VLDB Endowment*, 12(11):1762–1777, 2019.
- [42] Juan J. Rodríguez, Ludmila I. Kuncheva, and Carlos J. Alonso. Rotation forest: A new classifier ensemble method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(10):1619–1630, 2006.
- [43] Huan Song, Deepta Rajan, Jayaraman Thiagarajan, and Andreas Spanias. Attend and diagnose: Clinical time series analysis using attention models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [44] Tengfei Song, Wenming Zheng, Peng Song, and Zhen Cui. Eeg emotion recognition using dynamical graph convolutional neural networks. *IEEE Transactions on Affective Computing*, 11(3):532–541, 2018.
- [45] Chang Wei Tan, Christoph Bergmeir, François Petitjean, and Geoffrey I. Webb. Time series extrinsic regression: Predicting numeric values from time series data. *Data Mining and Knowledge Discovery*, 35(3):1032–1060, 2021.
- [46] Siyi Tang, Jared A Dunnmon, Khaled Saab, Xuan Zhang, Qianying Huang, Florian Dubost, Daniel L Rubin, and Christopher Lee-Messer. Self-supervised graph neural networks for improved electroencephalographic seizure analysis. *arXiv preprint arXiv:2104.08336*, 2021.
- [47] Sana Tonekaboni, Danny Eytan, and Anna Goldenberg. Unsupervised representation learning for time series with temporal neighborhood coding, 2021.

- [48] Jingyuan Wang, Chen Yang, Xiaohan Jiang, and Junjie Wu. When: A wavelet-dtw hybrid attention network for heterogeneous time series analysis. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2361–2373, 2023.
- [49] Zhiguang Wang and Tim Oates. Encoding time series as images for visual inspection and classification using tiled convolutional neural networks. 2014.
- [50] Zhiguang Wang, Weizhong Yan, and Tim Oates. Time series classification from scratch with deep neural networks: A strong baseline. *2017 International Joint Conference on Neural Networks (IJCNN)*, pages 1578–1585, 2016.
- [51] Zhiwen Xiao, Xin Xu, Huanlai Xing, Shouxi Luo, Penglin Dai, and Dawei Zhan. Rtfm: A robust temporal feature network for time series classification. *Information sciences*, 571:65–86, 2021.
- [52] Jianbo Yang, Minh Nhut Nguyen, Phyo Phyo San, Xiaoli Li, and Shonali Krishnaswamy. Deep convolutional neural networks on multichannel time series for human activity recognition. In *Ijcai*, volume 15, pages 3995–4001. Buenos Aires, Argentina, 2015.
- [53] Lexiang Ye and Eamonn Keogh. Time series shapelets: a new primitive for data mining. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 947–956, 2009.
- [54] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. Ts2vec: Towards universal representation of time series, 2022.
- [55] George Zerveas, Srideepika Jayaraman, Dhaval Patel, Anuradha Bhamidipaty, and Carsten Eickhoff. A transformer-based framework for multivariate time series representation learning, 2020.
- [56] Xiang Zhang, Ziyuan Zhao, Theodoros Tsiligkaridis, and Marinka Zitnik. Self-supervised contrastive pre-training for time series via time-frequency consistency. *Advances in Neural Information Processing Systems*, 35:3988–4003, 2022.
- [57] Bowen Zhao, Huanlai Xing, Xinhan Wang, Fuhong Song, and Zhiwen Xiao. Rethinking attention mechanism in time series classification. *Information Sciences*, 627:97–114, 2023.

- [58] Jiaping Zhao and Laurent Itti. shapedtw: shape dynamic time warping. *Pattern Recognition*, 74:171–184, 2019.
- [59] Xu Zheng, Tianchun Wang, Wei Cheng, Aitian Ma, Haifeng Chen, Mo Sha, and Dongsheng Luo. Parametric augmentation for time series contrastive learning. *arXiv preprint arXiv:2402.10434*, 2024.
- [60] Yi Zheng, Qi Liu, Enhong Chen, Yong Ge, and J. Zhao. Time series classification using multi-channels deep convolutional neural networks. In *International Conference on Web-Age Information Management*, 2014.
- [61] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. Learning deep features for discriminative localization, 2015.

Preprint
Decision
Intelligence
Lab

6 自动化时间序列分析

6.1 节作者为：吴行健、李正宇、杨彬；

6.2-6.3 节作者为：李正宇、杨彬；

6.4 节作者为：李正宇、吴行健、杨彬。

本章主编为杨彬。

不同行业（如能源、交通、金融、医疗等）产生的时间序列数据具有不同特性，对时序分析模型的需求也各不相同。通常情况下，人工智能专家为不同特性的数据设计不同的模型。这种传统的依赖人工专家的方式成本高，设计出的不同模型的适用场景和参数设置差异较大，调参过程耗时，通用性和适应性较差，难以在多样化的数据特征和预测任务中快速应用。如图 6.1 所示，其核心步骤通常依赖专家人工迭代设计，资源消耗大，需要投入大量人力、时间和资金，且人工设计的模型并不一定能达到最优效果。相比之下，自动化设计通过算法自动选择最合适的模型，或生成最优的模型架构及其超参数，不仅能显著提升预测精度，缩短设计周期，降低成本与人力投入，还能减少对 AI 专家的依赖，使行业专家与领域科学家也能轻松使用，从而推动人工智能大众化。

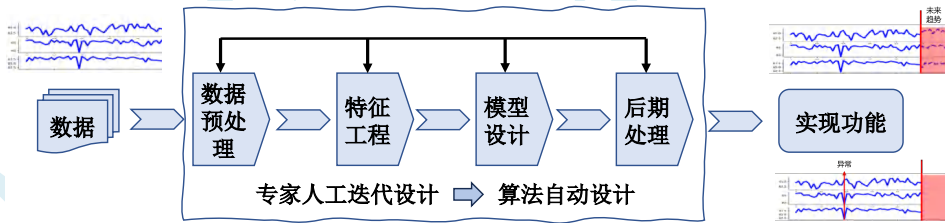


图 6.1: 时间序列分析流程：传统人工设计 vs. 自动化设计

本章介绍自动化时间序列分析的常见方法。6.1 节介绍时序分析自动化的定义并对其进行分类。模型自动选择、模型自动集成和模型自动设计是三种主流的自动化策略：我们在 6.2 节介绍模型自动选择，在 6.3 节介绍模型自动集成，在 6.4 节介绍模型自动设计。

6.1 时间序列分析自动化简介

自动化时间序列分析（Automated Time Series Analytics）是自动机器学习（AutoML）的一个重要分支。目的是以自动化的方式，实现针对不同时间序列数据集，不同分析场景（比如近期 vs. 远期预测），自动选择或构建针对这个数据集和场景的专用高效模型，降低各行业使用时序分析的门槛和成本（比如省去雇佣大量机器学习专家和数据科学家的人力成本，降低获得高效时序分析模型的时间和人力成本），推动时序人工智能的大众化。

如图 6.2 所示，给定用户提供的数据集和分析场景（比如近期 vs. 远期预测），主流的自动化方法大致可以分为三类：

- **模型自动选择**：从已有模型池 M 里选最优的一个模型 $m_k \in M$ 。
- **模型自动集成**：从已有模型池 M 里选出若干个基模型 $\{m_i, m_j, \dots, m_k\}$ ，并赋予其合适的权重 $\{\alpha_i, \alpha_j, \dots, \alpha_k\}$ ，进而组合出一个最优的集成模型（ensemble）。

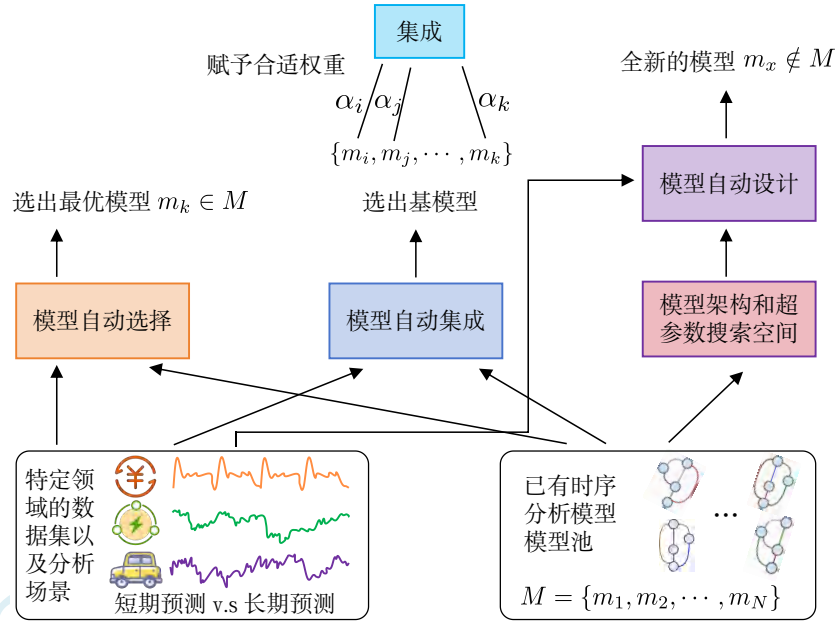


图 6.2: 自动化时间序列分析方法论分类

- **模型自动设计**：从已有模型池包含的模型中抽象出模型架构和超参数的联合搜索空间，进而构建一个不存在于当前模型池 M 中的全新模型 $m_x \notin M$ 。在深度学习的语境下，又称为神经架构搜索（Neural Architecture Search）和神经架构-超参数联合搜索（Joint Neural Architecture and Hyperparameter Search）[19, 20]。

6.2 时间序列分析模型自动选择

6.2.1 模型自动选择的定义及流程

模型自动选择，是在面对时间序列分析任务时，依据用户提供的数据集特征和特定的分析场景需求，从预先设定的模型池中，自动筛选出最适配该任务的模型。

具体而言，给定一个模型池 M ，其包含一组候选模型集合 $M = \{m_i, m_j, \dots, m_k\}$ 。其中，每个模型 $m_i \in M$ 都被定义为一个元组：

$$m_i = (f_i, h_i)$$

这里 f_i 代表预测方法，例如可以是 ARIMA、LSTM 等不同的时序预测模型或算法； h_i 则是对应该预测模型的超参数，其不同的设置会影响该预测算法的预测性能。对于一个时间序列 x_j ，模型 m_i 在该时间序列上的性能表示为：

$$P_j^i = P(m_i, x_j)$$

其中 P 是性能评估指标，通常会采用平均绝对误差（MAE）、均方误差（MSE）或平均绝对百分比误差（MAPE）等误差度量方式。模型选择的目标，就是从候选模型池

M 中挑选出最合适的模型 $m_k \in M$ ，使其在时间序列 x_j 上达到最佳性能（当性能评估指标为误差时，即希望误差最小化），这一过程可以用数学公式形式化地表达为：

$$m_k = \arg \min_{m_i \in M} P(m_i, x_j) \quad (235)$$

即对候选模型池 M 中的每一个模型 m_i ，评测其在时间序列 x_j 上的性能指标 $P(m_i, x_j)$ ，最终选择使该指标最小的模型 m_k 作为针对此时间序列的最优模型。

6.2.2 模型自动选择策略

如图 6.3 所示，模型自动选择策略可分为两类：基于搜索的模型自动选择和基于元学习的模型自动选择。前者通过系统性搜索模型池内候选模型的不同参数配置，选取在目标数据集的验证集上性能最优的模型；后者依托历史数据构建“数据特征 - 模型性能”的元学习映射关系，通过训练得到元学习器，可根据数据特征预测不同模型的性能，在推理阶段直接预测在目标数据集的数据特征下可能性能表现最优的模型。基于搜索的方法以较高的计算成本换取结果的稳健性；而基于元学习的方法则通过知识迁移实现高效推断。

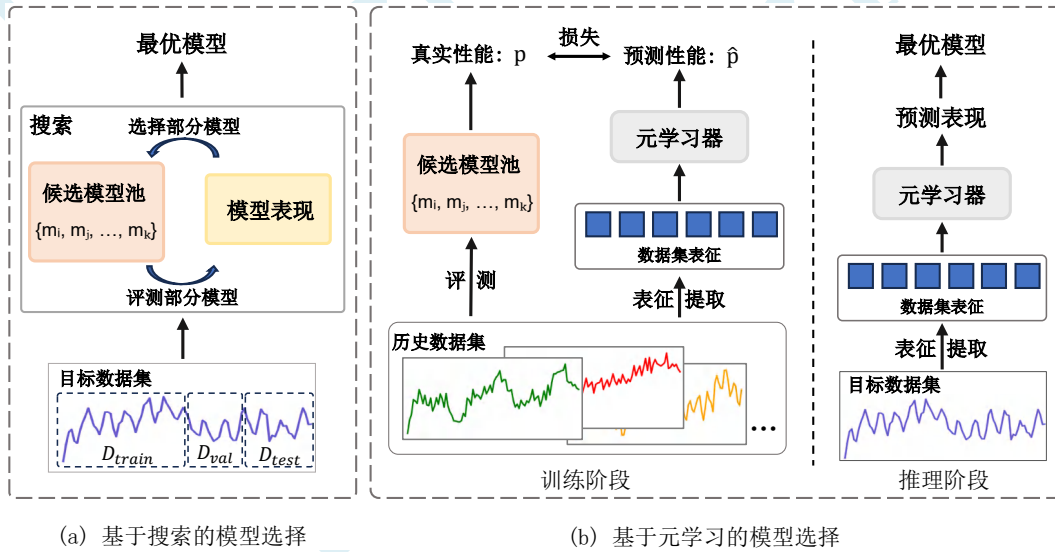


图 6.3: 模型自动选择策略

基于搜索的方法无需依赖历史任务数据，选择结果具有较高的可解释性，但搜索过程通常耗时且资源消耗大，且面向不同数据集时泛化能力有限。基于元学习的方法能够利用大量历史任务数据快速进行模型选择，并具有良好的泛化性，但结果可解释性较低。表 6.1 对模型自动选择策略的优势、劣势及适用场景进行了总结和对比。表 6.2 对现有的模型自动选择方法进行了总结。

表 6.1: 模型自动选择策略对比

策略	优势	劣势	适用场景
基于搜索	无需历史任务数据进行预训练； 选择结果可解释性高；	搜索过程时间、资源消耗较大； 泛化性差，面向特定目标数据集；	没有历史训练数据； 需要高可解释性；
基于元学习	可直接利用历史经验，高效且节省计算资源； 泛化性强，可迁移到新数据集；	需要大量历史任务数据进行预训练； 结果可解释性较差；	拥有大量历史任务数据； 对模型选择速度要求高；

表 6.2: 时间序列预测自动化模型选择策略总结

策略名称	选择方式	模型池	关键设计
Merlion [2]	搜索	SL/ML/DL	优化的网格搜索策略
AutoAI-TS [15]	搜索	SL/ML/DL	启发式搜索策略
TSPO [4]	搜索	ML/DL	基于遗传算法的搜索策略
AutoXPCR [6]	元学习	DL	多目标优化、注重可解释性
AutoForecast [1]	元学习	SL/ML/DL	基于 LSTM 的序列元学习
SimpleTS [24]	元学习	SL/ML/DL	数据集分类，模型聚类
AutoGluon [16]	元学习 + 搜索	SL/ML/DL	堆叠集成、超参搜索结合

注：SL 表示传统统计模型（Statistical Learning），ML 表示机器学习模型（Machine Learning），DL 表示深度学习模型（Deep Learning）。

6.2.3 基于搜索的模型自动选择

基于搜索的模型自动选择通过在候选模型池中进行策略性搜索，能够在目标数据上直接完成模型选择。基于搜索的模型自动选择主要分为两个阶段：**搜索阶段**和**评估阶段**。

为便于后续的讨论，假设将目标数据集以一定的比例进行**数据集划分**，得到训练集 $\mathbf{D}_{train} = \{\mathbf{X}_i^{train}\}_{i=1}^{T_{train}}$ 、验证集 $\mathbf{D}_{val} = \{\mathbf{X}_i^{val}\}_{i=1}^{T_{val}}$ 和测试集 $\mathbf{D}_{test} = \{\mathbf{X}_i^{test}\}_{i=1}^{T_{test}}$ 。以训练集为例， $\mathbf{X}_i^{train} = (\mathbf{x}_{t-H+1}^{train}, \mathbf{x}_{t-H+2}^{train}, \dots, \mathbf{x}_t^{train}, \mathbf{x}_{t+1}^{train}, \dots, \mathbf{x}_{t+F}^{train}) \in \mathbb{R}^{(H+F) \times N}$ 为训练集中第 i 个样本在 t 时刻的输入输出序列， H 、 F 分别是历史序列长度和未来序列长度， N 为变量的个数， T_{train} 是整个训练集的样本数量。

1. **搜索阶段**：具体而言，在处理一个目标时间序列数据集时，首先按一定比例将数据划分为训练集 $\mathbf{D}_{train}$ 与验证集 $\mathbf{D}_{val}$ ，模型池内的候选模型在训练集上进行训

练，并在验证集上评估其性能，以判断其优劣。最直接的策略是对模型池内的所有候选模型遍历，进行完整训练与评估，选取验证集上性能最好的模型。虽然该方法具有最高的准确率（其结果就是模型自动选择的 **ground truth**），但其计算资源消耗较大，难以实际应用。因此，各类搜索策略的核心目标是降低计算成本与时间开销。多数策略在初始阶段仅使用部分训练数据对模型进行训练和评估，提前淘汰性能较差的模型；随着筛选过程推进，逐步增加训练数据量，持续剔除性能表现不佳的模型，最终选出验证集上性能最优的模型。

2. **评估阶段**：评估阶段主要关注策略是否能够选出在目标数据集上性能最优的模型。常用的评估指标包括：

- **Top-k 命中率 (Hit@k)**：表示真实性能最优的模型是否包含在搜索策略所选出的前 k 个候选模型中。计算公式如下：

$$\text{Hit@k} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[m_i^* \in \text{Top-}k(\hat{M}_i)], \quad (236)$$

其中， N 表示目标数据集的数量， m_i^* 表示在第 i 个目标数据集上实际性能最优的模型， $\hat{M}_i$ 表示搜索策略在第 i 个目标数据集上返回的模型列表， $\text{Top-}k(\cdot)$ 表示取前 k 个性能最优的模型， $\mathbb{I}[\cdot]$ 是指示函数，若括号内条件成立则为 1，否则为 0。

- **平均排名 (Average Rank)**：表示搜索策略所选模型在真实性能排名中的位置，数值越低代表选择质量越高。计算公式如下：

$$\text{Average Rank} = \frac{1}{N} \sum_{i=1}^N \text{Rank}(\hat{m}_i) \quad (237)$$

其中， $\hat{m}_i$ 表示搜索策略在第 i 个数据集上选择的一个最优模型， $\text{Rank}(\hat{m}_i)$ 表示在第 i 个数据集上，模型 $\hat{m}_i$ 的实际排名。

基于搜索的模型自动选择的核心目标是降低计算成本与时间开销。主要关注两个研究重点：其一是**搜索空间的设计**，即如何合理界定模型类型及其超参数的取值范围，以在保证覆盖潜在优良解的同时控制搜索规模；其二是**搜索方法的效率**，即如何在有限计算资源下尽可能快速逼近最优解。该类方法的发展路径体现了从穷举式全局搜索（如网格搜索）、到引入启发式机制以提升效率，再到更复杂的启发式搜索（如遗传算法）的演进趋势。总体而言，当计算资源相对充足或任务特征未知时，基于搜索的策略通常能够取得较为稳健的性能表现。

- **搜索空间的设计**：搜索空间的设计旨在在保证覆盖潜在优良解的前提下，有效控制搜索规模。总体上，需要明确候选模型类型、超参数范围及其依赖关系。

- Bhatnagar 等人 [2] 提出的基于网格搜索的模型自动选择方法，对于模型池中的每类候选方法，会先通过对数据集的特征分析来确定部分关键参数，从而有效缩小搜索空间，图 6.4 展示了这个流程。举例来说 SARIMA 算法有 7 个超参数，其中的季节周期 m 、差分阶数 d 、季节性差分阶数 D 均可以通过对时间序列进行特征分析获取，从而避免无效的组合被纳入搜索，显著缩小搜索空间；随后，对剩余超参数组合采用基于 AIC（Akaike Information Criterion）的信息准则进行评估，以实现拟合效果与模型复杂度的平衡。

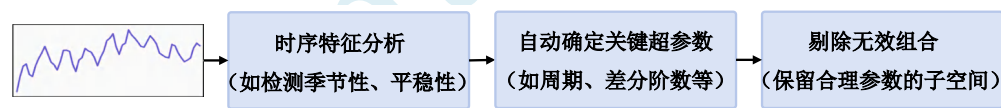


图 6.4: 搜索空间缩减

- Dahl 等人 [4] 提出的基于遗传算法（genetic algorithm）的搜索空间设计方法，将数据集特征、模型及超参数字典整合为进化个体，并通过随机采样生成初始种群，从而实现对候选解空间的全面覆盖。同时也利用了 Bhatnagar 等人 [2] 类似的做法，用数据集特征预先锁定部分超参数，从而约束搜索空间，提高潜在优良解的占比。

- **搜索方法的设计：**搜索方法的设计主要关注如何在有限计算资源下尽可能高效地逼近最优解。总体策略包括优化搜索顺序、训练量安排以及评估方式。
 - Shah 等人 [15] 提出基于增量训练的方法较为有代表性，如图 6.5 所示通过阶段性增加训练数据量，并利用线性回归预测模型在完整数据集上的性能，实现了在无需完整训练所有模型的情况下快速筛选最优模型。
 - Bhatnagar 等人 [2] 结合逐步式搜索与候选模型预筛选策略，提高了网格搜索在高维超参数空间中的效率：首先快速生成一组性能较优的候选模型，再对这些候选模型进行完整训练与评估，从而兼顾搜索的全面性与计算可行性。
 - Dahl 等人 [4] 在遗传算法框架中，通过选择、交叉和变异操作对种群进行迭代优化，每代个体的适应度由训练后的预测指标计算得出，从而在搜索过程中高效逼近最优的模型-特征组合。

6.2.4 基于元学习的模型自动选择

基于元学习的模型自动选择通过模型池内的候选模型在不同历史数据集上的表现训练一个元学习器，用于通过数据集的特征预测模型的性能，从而实现对新数据任务中最优模型的快速预测与选择。具体来说，会按照一定的比例对历史数据集进行划分，需要注意的是这里与第 6.2.3 节中提到的数据划分不同，如时序的预测，异常检测

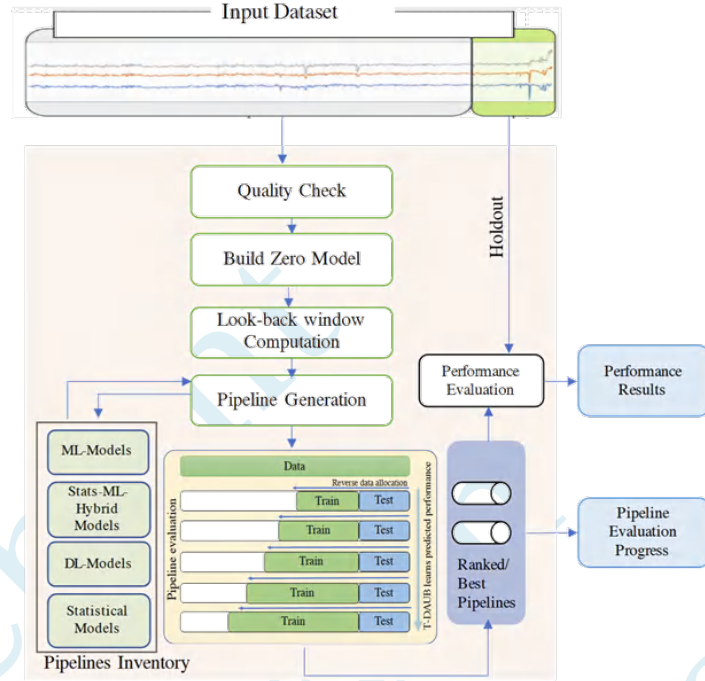


图 6.5: AutoAI-TS 框架示意图，AutoAI-TS 通过阶段性增加训练数据量，预测全量数据训练下的模型性能，舍弃性能较差的模型，避免资源的浪费 [15]

等数据集的划分是对某一个数据集进行划分，而对于模型选择来说划分的方式是将一部分的数据集整体作为训练集，一部分数据集整体作为测试集。基于元学习的方法可划分为三个阶段：训练阶段、推理阶段以及评估阶段。

1. **训练阶段**：本阶段的目标是构建“数据特征—模型性能”的元学习映射关系。首先需要构建元学习所需的数据集，每个样本包含一个历史数据集的特征与候选模型（包括其超参数配置）在该数据集上的评估结果。为了获取这些信息，需对候选模型在多个不同的历史数据集上进行参数搜索，并记录其性能表现。与此同时，提取每个数据集的特征，通常包括统计特征（如均值、方差、自相关等）或基于深度学习的隐表征。最终，以数据特征作为输入，模型性能作为学习目标，训练得到元学习器。这一过程可以形式化地表示为：

$$\text{Predictor}(m_i, x_j) = \hat{p}_j^i, \quad (238)$$

其中， m_i 表示候选模型（及其超参数配置）， x_j 表示数据集， $\hat{p}_j^i$ 表示元学习器预测的模型 m_i 在数据集 x_j 上的性能指标。

2. **推理阶段**：在推理阶段，对于一个目标数据集，首先按照训练阶段相同的方法提取数据集的特征，并输入到训练好的元学习器中，以获得所有候选模型在该目标数据集上的性能预测。最终，选择预测性能最优的模型作为本次任务的推荐模型。

3. **评估阶段**：在目标数据集上的评估过程，通常会采用交叉验证 (Cross-Validation) 方法，以缓解单一目标数据集带来的评估偏差，即将整个数据集分为 F 折，每次用其中的 $F-1$ 训练元学习器，剩下的 1 折作为目标数据集，这样就可以获取在整个数据集上的评测结果。评估指标与基于搜索的模型自动选择方法一致，为 **Top- k 命中率 (Hit@ k)** 和 **平均排名 (Average Rank)**，此处不再赘述。

与依赖系统性搜索的基于搜索方法不同，元学习方法强调对历史经验的提炼与迁移，从而在新任务上高效选取最优模型。该策略主要关注两个研究重点：其一是**元特征设计**，即如何提取能够充分表征数据集特性的统计特征、频域特征或嵌入特征，为模型性能预测提供可靠输入；其二是**元学习器设计**，即如何将元特征映射到候选模型的性能指标，并通过随机森林、梯度提升树或神经网络等方法训练预测器，从而实现对目标数据集性能的准确预测。总体而言，基于元学习的策略在候选模型池规模较大或模型训练代价较高的情况下，能够显著降低模型自动选择的计算成本，同时保持较高的模型选择准确率。

- **元特征的设计**：元特征设计旨在充分捕捉数据集的结构信息，以支持元学习器对模型性能的准确预测。总体上，需要提取能够描述时间序列分布、周期性、趋势及模型适配性的特征。主要可以分为统计特征和深度学习表征两类。

- **统计特征**——直接计算时间序列的描述性特征，反映其分布、波动和周期特性，图 6.6 展示了部分的时间序列统计特征。统计特征可解释性高，计算效率高，但是对复杂模式捕捉有限，依赖人工定义。**tsfresh** [3] 工具包可以方便地计算此类统计特征。**Abdallah** 等人 [1] 使用了统计特征，并对提取后的特征使用 PCA 降维，以减小冗余并突出主要信息。**Fischer** 等人 [6] 则进一步将模型复杂度和推理功耗纳入特征，实现对预测性能与计算效率的综合考虑。

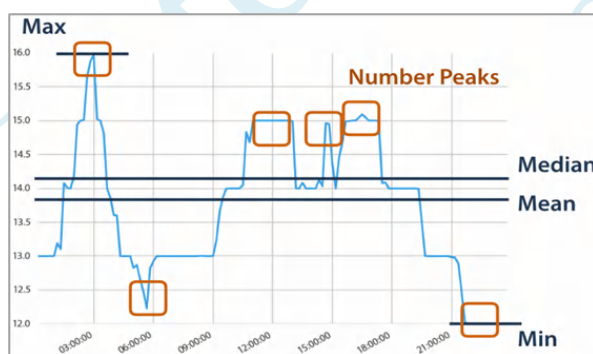


图 6.6: tsfresh 时间序列统计特征示例 [3]

- **深度学习表征**——可以捕捉复杂的特征，但是可解释性较低，计算开销较高，比较有代表性的工作是 TS2VEC [25]，图 6.7 展示了 TS2VEC 的框架。**Yao** 等人 [24]

使用了 TS2Vec 将整个时间序列映射为低维向量表示，用以捕捉序列的全局与局部模式，从而为元学习器提供有效输入。

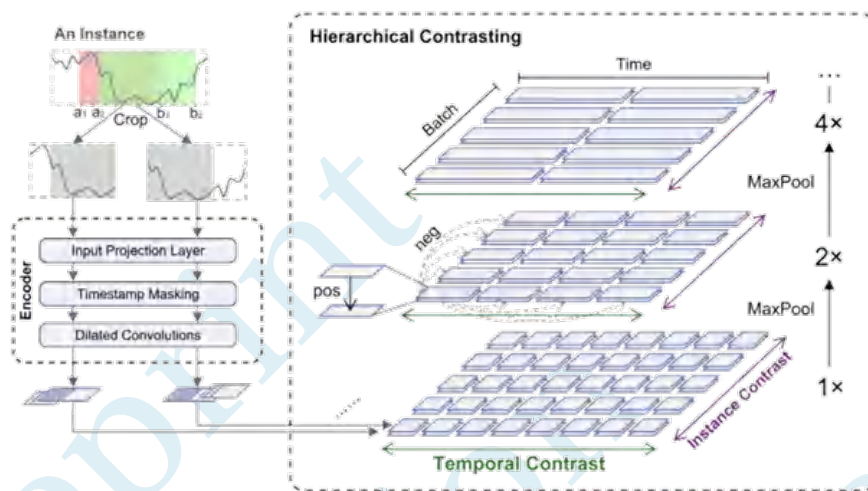


图 6.7: TS2VEC 框架示意图，TS2VEC 为每个时间戳生成向量表示，利用空洞卷积进一步对向量表示编码，并采用层次化对比学习，为时间序列生成具有上下文感知能力的通用向量表示 [25]

- **元学习器的设计：**元学习器的设计旨在利用元特征预测候选模型在目标数据集上的性能，并实现高效选择。总体上，需要合理选择学习模型与训练策略，以充分挖掘元特征与模型性能之间的映射关系。下面介绍几种具有代表性的方法。
 - Abdallah 等人 [1] 构建了两类元学习器：一类采用多输出回归模型（如线性回归、神经网络），直接预测候选模型在当前时间片段的表现；另一类使用 LSTM，将历史性能特征序列与当前数据特征作为输入，捕捉时间依赖性，并通过联合损失函数优化预测效果。
 - Yao 等人 [24] 通过先对候选模型在少量训练数据上的性能进行聚类，确定每个聚类中性能最优的模型类别，再结合 TS2Vec [25] 对数据集进行编码，将编码结果作为特征训练分类器，从而实现对目标数据集的高效模型选择。
 - Fischer 等人 [6] 则使用可解释回归方法（如随机森林），通过组合得分对模型进行排序，兼顾预测性能与可解释性。

6.3 时间序列分析模型自动集成

6.3.1 模型自动集成的定义及流程

模型自动集成，是指在面对时间序列分析任务（如：预测）时，通过一定的模型集成策略，将多个模型的预测结果进行融合，以提升整体预测性能和鲁棒性。与模型选择相比，模型集成更侧重于通过组合多个模型的优势，减少单一模型可能带来的偏

差。具体而言, 给定模型池 $M = \{m_1, m_2, \dots, m_n\}$, m_{ensemble} 为集成模型。对于输入数据集 x_j , 模型自动集成的目标是学习一组权重 $\{a_1, a_2, \dots, a_n\}$, 使得最终的集成预测结果为:

$$m_{\text{ensemble}}(x_j) = \sum_{m_i \in M} a_i \cdot m_i(x_j), \quad (239)$$

其中权重 $\{a_1, a_2, \dots, a_n\}$ 可通过多种策略自动确定, 其中部分模型可能会自然地被弱化或舍弃 ($a_i = 0$ 即表示没有选 m_i), 变相起到了模型选择的作用。理想情况下, 集成模型 m_{ensemble} 的性能应优于集合中任一单模型, 以预测任务为例, P 为某种误差指标, 则有:

$$P(m_{\text{ensemble}}, x) \leq \min_{m \in M} P(m, x), \quad (240)$$

模型自动集成通过整合多模型的优势, 能够在降低误差波动的同时, 提升模型的泛化能力与稳定性。

6.3.2 模型自动集成策略

如图 6.8 所示, 模型自动集成策略通常包括四个阶段: 数据处理、模型选择、输出集成和评估。其中数据处理、模型选择和输出集成为核心阶段, 数据处理为模型提供不同视角的数据; 模型选择则用于筛选适合集成的模型子集, 以避免模型集成时引入效果过差的模型; 输出集成阶段则对各子模型的预测结果进行融合, 生成最终输出。在核心阶段中, 仅输出集成为必要步骤。数据处理步骤为可选, 因为不同模型本身已

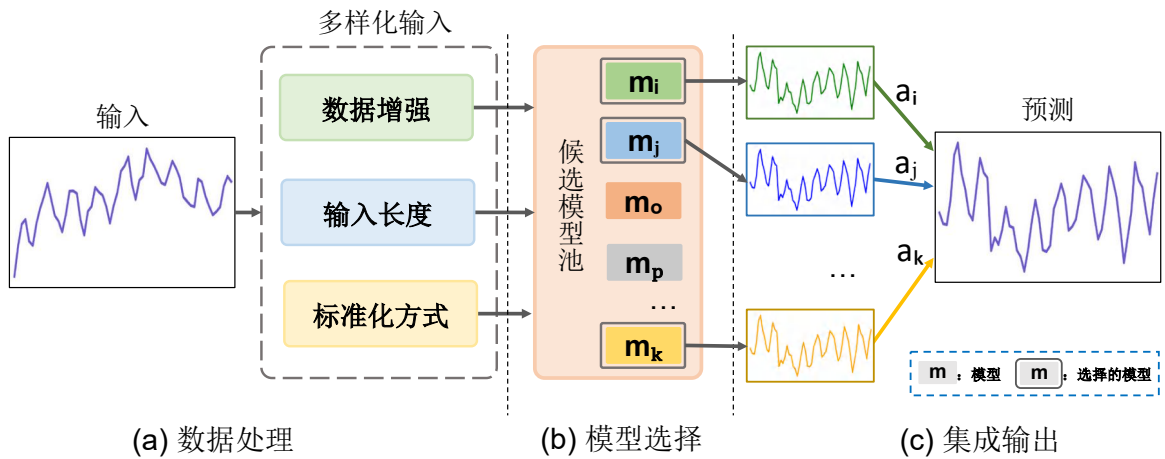


图 6.8: 模型自动集成策略

提供了多样化的视角, 无需额外的视角转换; 模型选择步骤也是可选, 因为部分方法的集成模型组合在设计时已固定, 并且部分输出集成方法可以直接弱化或舍弃一部分

模型，变相起到模型选择的作用，因此无需额外筛选。下面将对这几个阶段进行详细介绍：

1. **数据处理阶段**：数据处理阶段的核心目标是为模型提供不同视角的数据。在时间序列预测任务中，这种多样性可以通过多种方式实现。例如，最直接的方式是为模型提供不同长度的历史序列作为输入，使其在不同时间窗口下进行预测；此外，常见的数据增强手段包括对原始序列添加噪声、在部分时间步进行掩码处理，以及通过降采样或上采样改变时间粒度。这些操作能够模拟真实世界中数据的变动性与不确定性。同时，不同的标准化方法（如 Z-score 标准化、对数变换、差分变换等）也可用于构造多种输入表示形式，从而促使候选模型从不同视角学习时间序列的结构与特征。
2. **模型选择阶段**：该阶段的目标是在候选模型池中选出一组适合集成且互补的子模型，避免集成中引入效果过差的模型。不同于传统的自动模型选择方法仅保留性能最优的单个模型，模型自动集成通常会选取前 k 个性能较优且差异性较大的模型进行融合。相关的自动选择策略已在前一节中详细介绍，合理的模型选择不仅有助于降低冗余，还能避免模型间输出高度一致而导致的集成无效问题。
3. **输出集成阶段**：输出集成是自动集成流程中的核心环节，负责将多个子模型的预测结果融合为最终输出，直接决定集成模型的最终性能。最常用的融合方式是加权平均，其中权重的设定可基于不同策略获得。最简单的做法是对所有模型预测结果取平均，即赋予每个模型相同的权重。更复杂的方式包括启发式方法（如贪心搜索，在验证集上逐步添加子模型以最大化集成性能），监督学习方法（如基于岭回归的权重回归器），以及深度学习方法（例如训练一个门控神经网络，根据输入或模型性能动态分配权重）。
4. **评估阶段**：模型集成的评估阶段包括两部分的评估，包括模型选择效果的评估以及最终集成数据的输出。一方面，对于模型选择性能的评估，可直接采用前述自动化模型选择中引入的指标（如 Hit@k 和 Average Rank）。然而，由于自动化模型集成通常选择的是 Top- k 个候选模型而非单一模型，因此更合适的评估方式应考虑所选模型集合中与真实 Top- k 最优模型的重合度。具体计算如下：

$$\text{Accuracy@k} = \frac{|\hat{M}_k \cap M_k|}{k} \quad (241)$$

其中， $\hat{M}_k$ 表示元学习器预测得到的 Top- k 模型集合， M_k 表示在真实性能下性能最优的 Top- k 模型集合， $|\hat{M}_k \cap M_k|$ 表示两者的交集大小。Accuracy@ k 衡量预测结果中命中的最优模型比例，数值越高代表模型选择效果越优。此外，由于所选模型之间可能存在互补性，单独评估模型选择的效果并不能完全反映集成模型的整体性能。最直接的方法是通过集成模型的预测结果计算评价指标，例如：

(a) 平均绝对误差 (MAE):

$$\text{MAE} = \frac{1}{F} \sum_{i=t+1}^{t+F} |x_i^{\text{test}} - \hat{x}_i^{\text{test}}| \quad (242)$$

(b) 均方误差 (MSE):

$$\text{MSE} = \frac{1}{F} \sum_{i=t+1}^{t+F} (x_i^{\text{test}} - \hat{x}_i^{\text{test}})^2 \quad (243)$$

(c) 平均绝对百分比误差 (MAPE):

$$\text{MAPE} = \frac{1}{F} \sum_{i=t+1}^{t+F} \left| \frac{x_i^{\text{test}} - \hat{x}_i^{\text{test}}}{x_i^{\text{test}}} \right| \times 100\% \quad (244)$$

表 6.3总结了现有的模型集成方法。现有的模型集成方法主要关注于**数据预处理方法**、**模型选择方法**和**模型集成方法**，其中模型选择策略在章节 6.2 已介绍，本章节不再赘述。

表 6.3: 时间序列预测自动化模型集成策略总结

策略名称	集成模型	数据处理	模型选择	输出集成
Zhang et al. [26]	DL	多种噪声增强	-	均值集成策略
AutoGluon [16]	SL/ML/DL	-	元学习 + 搜索	堆叠 + 贪心加权集成
Zhang et al. [27]	DL	-	-	基于遗传算法的集成策略
Lv et al. [12]	ML/DL	-	-	类 Boosting 集成策略
Torgo et al. [13]	ML	输入变换	-	均值集成策略
He et al. [8]	DL	-	-	均值集成策略
Merlion [2]	SL/ML/DL	-	网格搜索	多种可选基础集成策略

注：SL 表示传统统计模型 (Statistical Learning)，ML 表示机器学习模型 (Machine Learning)，DL 表示深度学习模型 (Deep Learning)。

6.3.3 数据预处理方法

为了促使候选模型从不同视角学习时间序列的结构与特征，数据预处理通常通过为模型提供同一数据的多种变换视角来实现。下面介绍几种具有代表性的方法。

- Zhang [26] 等人通过在原始数据集上加入不同噪声生成多个变体数据集，然后在这些数据集上分别训练同一基础模型，最后将各模型的预测结果进行集成，以提升整体预测的稳健性和泛化能力。

- Torgo [13] 等人则采用输入变换策略，如调整模型的回看窗口长度或修改输入标准化方式，从而生成多个输入版本，再在这些输入上训练模型并集成其预测结果，以充分捕捉时间序列的多尺度特征和潜在非线性模式。这类预处理方法的核心目标是通过多样化输入引导模型学习不同层次的信息，增强集成模型对复杂序列特征的适应能力。

6.3.4 输出集成方法

模型集成旨在通过组合多个子模型的预测结果，提高整体预测准确性并降低单模型的不稳定性。具体而言，常见的简单方法包括直接使用平均或加权平均，对各子模型输出按固定或动态权重进行组合，从而实现稳健预测。融合过程中，部分模型自然地被弱化或舍弃，可变相起到模型选择的作用。下面介绍几种具有代表性的方法。

- Bhatnagar 等人 [2] 提供了多种可选策略，如加权平均、中位数等，使用户能够根据任务需求灵活选择。

- Zhang 等人 [27] 利用遗传算法逐步筛选候选模型。如图 6.9所示，该方法将个体定义为不同的模型选择，个体选择的模型预测结果，使用核岭回归集成获取一个最终的输出，这个结果与真实值的偏差和方差为该个体的适应度。通过遗传算法不断迭代选择、交叉和变异，逐步优化个体，最终获得最优模型组合。

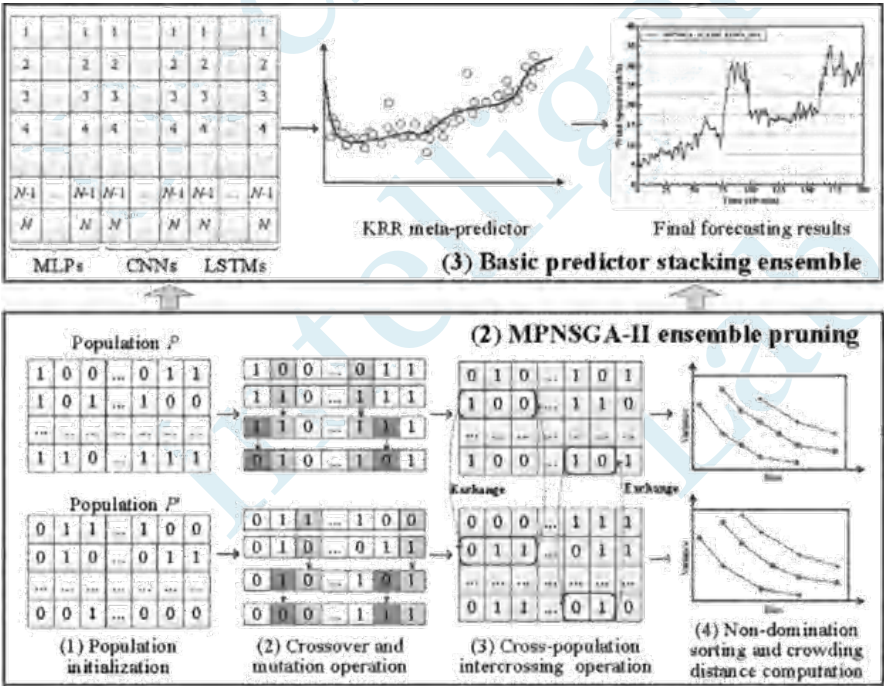


图 6.9: 基于遗传算法的模型集成 [27]

- Lv 等人 [12] 提出一种迭代优化方法。如图 6.10所示，通过计算每个模型预测值与当前集成预测残差的关系，评估模型对整体误差的贡献程度。然后动态调整模型权重：

贡献较大的模型权重增加，贡献较小的模型权重减少，若权重降为零则剔除该模型。通过重复这一过程，不断优化集成模型的组合，使整体预测性能逐步提升。

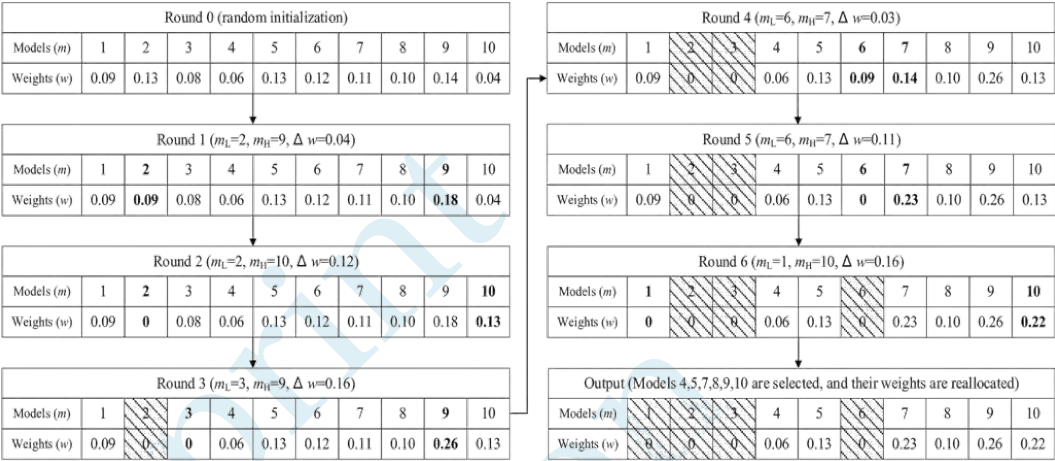


图 6.10: 基于贪心算法的模型集成 [12]

6.4 时间序列分析模型自动设计

6.4.1 模型自动设计的定义及流程

模型自动设计在深度学习的语境下又称神经架构搜索（Neural Architecture Search）和神经架构-超参数联合搜索（Joint Neural Architecture and Hyperparameter Search），主要是指通过抽象出人工神经网络中架构和超参数的规律，构造相应的搜索空间，进而构建一个不存在于当前模型池 M 中的全新模型 $m_x \notin M$ 。如图 6.11 所示，神经架构搜索的三要素是构造合理的搜索空间，采用合适的搜索策略，对搜索出的架构进行性能评估，从而支持高效迭代，逐步找到最合适的架构。

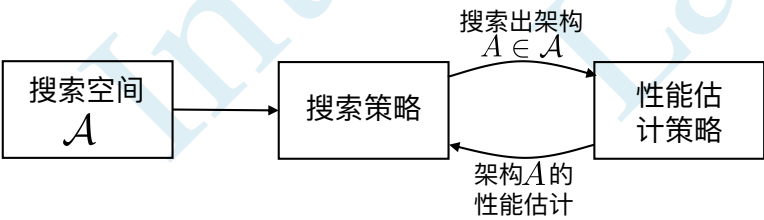


图 6.11: 神经架构搜索的三要素 [5]

由于时间序列领域的数据来自于多个领域，例如金融、气象、医疗等，其数据模式变动较大，统计特性各不相同，往往同一模型难以适应不同场景下的预测或异常检测任务。而模型自动设计能够为不同领域的数据搜索最适合的模型结构，从而在更多应用场景中发挥作用，有助于人工智能的大众化。面对具体的时序预测任务 $T = (D, S)$,

其中 D 表示数据集， S 表示预测步长等任务设定，神经架构搜索算法往往利用特殊设计的搜索策略，并借助训练集部分 D_{train} 感知任务特点，从预定义好的搜索空间中迭代地选择最适合当前场景的神经网络架构，一般而言，会借助验证集部分 D_{valid} 高效评估选择出的模型的性能，从而决定搜索过程需要持续多久。

6.4.2 模型自动设计策略

如图 6.12所示，自动模型设计方法大致可以分为两种范式，**任务特定的自动模型设计**和**可泛化的自动模型设计**。前者在每个任务上从头开始搜索，而后者利用元学习知识在数据集或任务之间进行泛化。具体来说：

- 1. **任务特定的自动模型设计**：任务特定的模型自动设计是一种端到端的自动化流程，无需利用先验知识。其核心在于直接在特定目标场景的数据集上进行完整搜索，训练，评估，最终自动输出为该任务量身定制的最合适的神经网络架构。
- 2. **可泛化的自动模型设计**：通过学习不同任务下，模型配置和性能之间的关系，预训练一个模型性能预测网络。进而面对目标场景能快速评估在新任务下不同模型配置的性能，进行高效地选择最优模型配置。

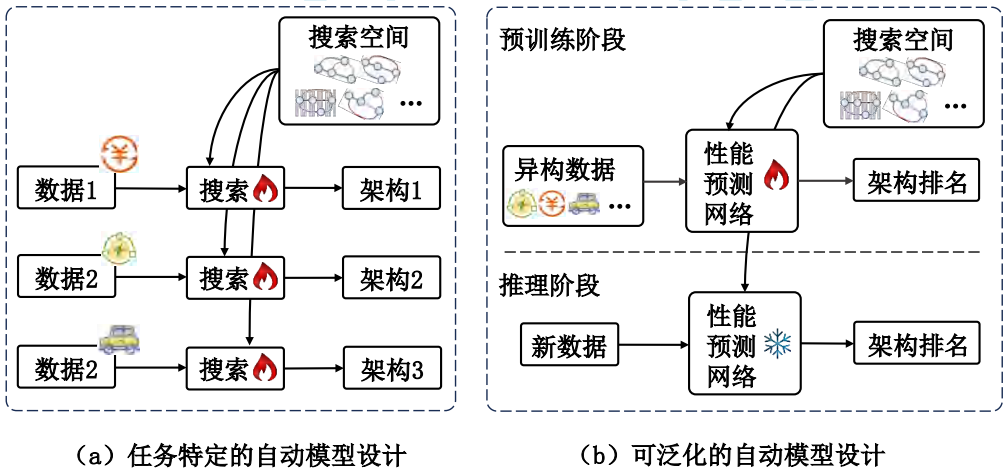


图 6.12: 自动模型设计方法

现有的模型自动设计策略可以根据搜索空间、搜索策略以及模型评估三个角度总结为表6.4 所示。

表 6.4: 模型自动设计方法总结

方法名	类型	搜索空间	搜索策略	模型评估	创新点
AutoST(2020)[10]	任务特定	架构参数	DARTS	完整训练, 剥离出最优子模型	首次在时序预测领域应用 DARTS 的方法
AutoSTG[14]	任务特定	架构参数	DARTS + meta-learn	完整训练, 剥离出最优子模型	结合元学习技巧进行加速
AutoCTS [19]	任务特定	架构参数	DARTS	完整训练, 剥离出最优子模型	构建更加精细的搜索空间
AutoST(2022)[9]	任务特定	架构参数	DARTS	完整训练, 剥离出最优子模型	考虑具体任务对于模型结构的影响
RS-STGNN[23]	任务特定	架构参数 + 超参数	cherry re-gion+Random Search	批量采样, 比较出最优子模型	利用统计学方法缩小搜索空间
AutoCTS+ [20]	任务特定	架构参数 + 超参数	Random Search + 比较器快速排序	批量采样, 比较出最优子模型	设计比较器, 增加方法的迁移能力
AutoCTS++ [17]	可泛化	架构参数 + 超参数	Random Search + 比较器快速排序	批量采样, 比较出最优子模型	设计感知任务特点的比较器, 具有多任务泛化能力
FACTS [18]	可泛化	架构参数 + 超参数	Random Search + 比较器快速排序	批量采样, 比较出最优子模型	设计感知任务特点的比较器, 具有多任务泛化能力

6.4.3 任务特定的模型自动设计

任务特定的模型自动设计是一种端到端的自动化流程, 其核心在于直接在特定目标场景的数据集上进行完整搜索, 训练, 评估, 最终自动输出为该任务量身定制的最合适的神经网络架构。任务特定的模型自动设计主要包括两个阶段: 模型搜索阶段和评估阶段。

1. **模型搜索阶段**: 模型搜索过程通过交替更新的方式进行: 首先在训练集 D_{train} 上优化网络的**权重参数** (即可学习参数), 随后在验证集 D_{valid} 上优化网络的**架构**

参数（即决定网络拓扑结构的参数）。权重参数与架构参数的交替更新，共同驱动着模型架构向高性能方向演进。

2. **评估阶段**：神经架构搜索方法的评估与传统的模型一致，将得到的模型在当前任务的训练集 D_{train} 上进行训练，利用验证集 D_{valid} 进行早停等操作，从而获取最优的检查点，用以在测试集上推理。

任务特定的模型自动设计早期的进展主要围绕应用 DARTS，而 DARTS 方法会带来不可规避的空间效率和时间效率问题，因此研究进展主要聚焦于如何提升搜索效率。下面将按照模型自动设计发展过程介绍几种具有代表性的方法。

AutoST[10] 中，第一次将 DARTS 中的方法应用到时序预测之中，这篇文章针对的主要是人流的预测，采用网格化的方法把城市切分成若干块，收集每一块的人口流动数据，应用 DARTS[11] 的方法论设计搜索空间。但是由于其研究的对象不够广泛，所以采用的算子还是较为简单的卷积层和残差连接，创新点主要在于算子选择方面，通过将卷积和残差分开考虑（图6.13），按照一定的策略控制生成网络的结构，从而降低了模型搜索的时间复杂度。

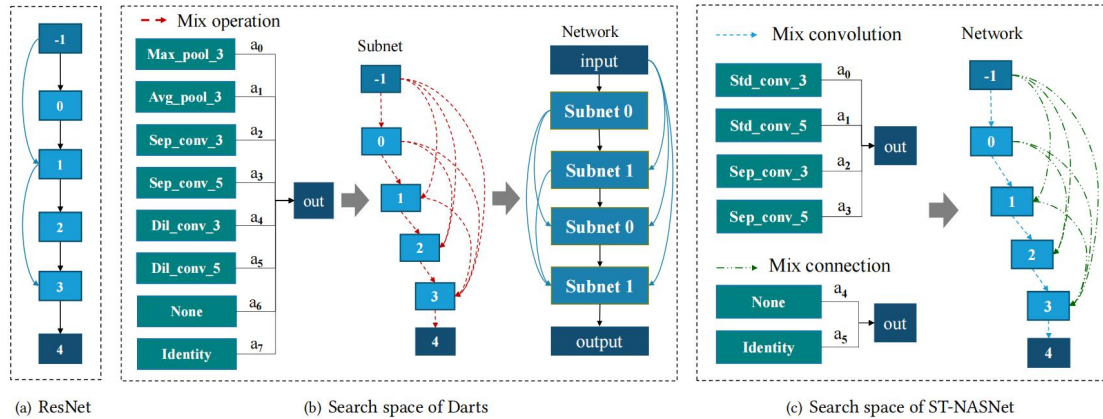


图 6.13: AutoST(2020) 结构总览 [10]

DARTS 是一种利于梯度下降的 One-Shot NAS[7] 算法，当一个路径上可选算子有数种时，考虑用权值待定的方式将他们加和：

$$\bar{o}^{(i,j)} = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x) \quad (245)$$

经过上述过程，原先离散的架构参数搜索空间被连续化，所以可以通过梯度下降的方式更新 $\alpha_o^{(i,j)}$ ，问题就被转化成了一个可微的双重优化过程，如公式 246 和 247 所示。训练完成之后，架构参数被训练充分，按照权值在每条路径下仅留下两条路径，

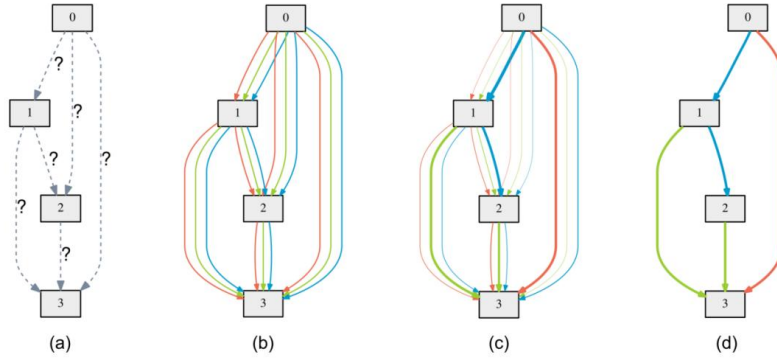


图 6.14: DARTS 结构总览 [7]

就得到了剥离出来的目标子模型，在原数据集上重新训练该模型即可。

$$\min_{\theta} \mathcal{L}_{val}(w^*(\alpha_{\theta}), \alpha_{\theta}) \quad (246)$$

$$s.t. \quad w^*(\alpha_{\theta}) = \underset{w}{\operatorname{argmin}} \mathcal{L}_{train}(w, \alpha_{\theta}) \quad (247)$$

Algorithm 1: DARTS – Differentiable Architecture Search

Create a mixed operation $\bar{\sigma}^{(i,j)}$ parametrized by $\alpha^{(i,j)}$ for each edge (i, j)

while not converged do

1. Update architecture α by descending $\nabla_{\alpha} \mathcal{L}_{val}(w - \xi \nabla_w \mathcal{L}_{train}(w, \alpha), \alpha)$
($\xi = 0$ if using first-order approximation)
2. Update weights w by descending $\nabla_w \mathcal{L}_{train}(w, \alpha)$

Derive the final architecture based on the learned α .

图 6.15: DARTS 中提出的双重优化算法 [7]

AutoSTG[14] 将 DARTS 的方法推广到了一般的时间序列预测问题之上，首次提出在自动化方法中考虑时空图建模。在算子选择上按照人工设计的时序预测模型，明确时间算子和空间算子这两大范畴。为了加速模型搜索的速度，采用了元学习的方式利用额外数据为算子生成初始参数。虽然 AutoSTG 规范化了基于 DARTS 的时序预测自动化算法的设计流，但在算子选择方面，AutoSTG 仅在时空算子中各选取了一个：空间算子为朴素的 Graph Convolution，时间算子则可以视为一组 1-D 卷积。

AutoCTS[19] 的提出主要从两个层面解决了 AutoSTG 的一些不足之处，首先是算子的选用方面，AutoCTS 详细分析了 CNN、RNN 和 Transformer 三族算子的优劣，选取了其中最优秀的算子加入到算子库中（图6.16）。

	Family	Operator	Literature	Equation
T-Operators	CNN	1D Convolution	[14]	$H^{(i)} = Z^{(i)} * W \quad (8)$
		Gated Dilated Causal Convolution (GDCC)	[9, 17, 51]	$H^{(i)} = (Z^{(i)} * W_1) \odot \sigma(Z^{(i)} * W_2) \quad (9)$
	RNN	Long Short Term Memory (LSTM)	[23, 38]	$H_t^{(i)} = LSTM(Z_t^{(i)}, H_{t-1}^{(i)}) \quad (10)$
		Gated Recurrent Unit (GRU)	[1, 4, 28]	$H_t^{(i)} = GRU(Z_t^{(i)}, H_{t-1}^{(i)}) \quad (11)$
	Attention	Transformer	[34, 46]	$H^{(i)} = SoftMax(\frac{(Z^{(i)} W_Q)(Z^{(i)} W_K)^{\top}}{\sqrt{D^i}})(Z^{(i)} W_V) \quad (12)$
		Informer (INF-T)	[53]	$H^{(i)} = SoftMax(\frac{smip(Z^{(i)} W_Q)(Z^{(i)} W_K)^{\top}}{\sqrt{D^i}})(Z^{(i)} W_V) \quad (13)$
S-Operators	GCN	Chebyshev GCN	[9, 11, 14, 17, 51]	$H_t = \sum_{k=0}^{K-1} W_k T_k(\tilde{L}) Z_t \quad (14)$
		Diffusion GCN (DGCN)	[28, 33, 45]	$H_t = \sum_{k=0}^K (D_O^{-1} A)^k Z_t W_1^k + (D_I^{-1} A^{\top})^k Z_t W_2^k \quad (15)$
	Attention	Transformer	[34, 46]	$H_t = SoftMax(\frac{(Z_t W_Q)(Z_t W_K)^{\top}}{\sqrt{D^i}})(Z_t W_V) \quad (16)$
		Informer (INF-S)	None	$H_t = SoftMax(\frac{smip(Z_t W_Q)(Z_t W_K)^{\top}}{\sqrt{D^i}})(Z_t W_V) \quad (17)$

图 6.16: AutoCTS 中使用的时空算子 [19]

其次，AutoSTG 使用的是 DARTS 原文提出的方法，只考虑同一路径上不同算子的权重，而 AutoCTS 将这个过程进行了优化，使用了 PC-DARTS[22] 中的方法，细化考虑隐藏表达式之间的权重关系，在此基础上，AutoCTS 还额外考虑了不同时空模块 (ST-block) 之间连接的权重关系，搜索空间被区分为三个层次：1. 算子选用层面，考虑算子库中具体选择哪些算子。2. 微观层面（图6.17），考虑同一时空模块之中存在的拓扑关系，具体表现为隐藏表达式之间的权重以及同一路径上不同算子的权重。3. 宏观层面（图6.18），架构 (ST-backbone) 的拓扑结构，具体表现为不同时空模块之间的连接权重。

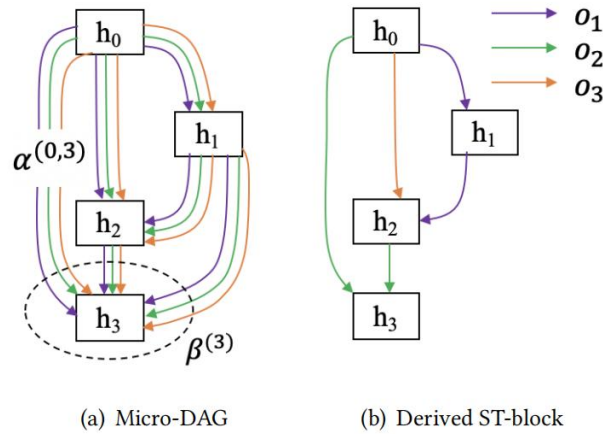


图 6.17: AutoCTS 的微搜索空间 [19]

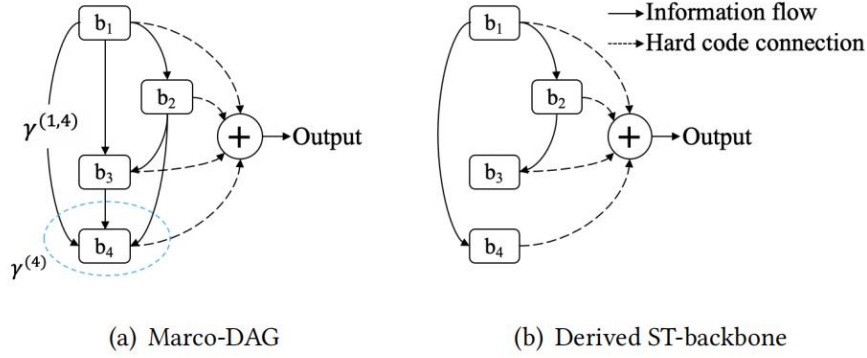


图 6.18: AutoCTS 的宏搜索空间 [19]

RS-STGNN[23] 中提出了一类方法，即通过优化搜索空间从而实现改善搜索出的子模型的质量。该方法考虑时序预测自动化常关注的一些问题：时空算子的拓扑关系、时空算子的具体选用和设计、残差连接的数量和位置以及超参数的选取。作者通过大量实验建立起统计学依据，设计了几个标准用以标定更优的子空间（Cherry Region）。因此可以将架构参数和超参数的待选区域都缩小，在缩小后的区域内，优秀的模型参数集中于此，从而在搜索时可以更快更精准地找到最优模型。然而该方法也一定的问题，当目标数据集更换之后，最优子空间（cherry region）并不能够迁移，这就需要大量的时间训练统计，重新生成目标数据集上的最优子空间。

AutoCTS+[20] 由于基于 DARTS 的方法不能联合搜索超参数，必须人为指定一些结构，所以存在非完全自动化的问题，例如时空模块中的隐藏表达式数量，在 AutoCTS 中规定为 4。而且该方法在模型搜索的过程中将所有算子纳入训练，存在内存过度占用的问题。对于新的目标任务，也无法实现快速迁移，而是需要基于数据集重新拟合。

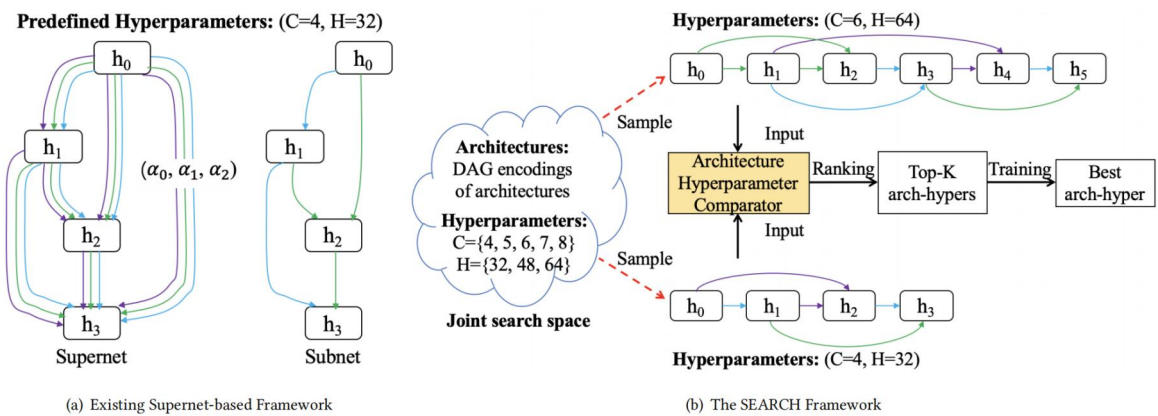


图 6.19: AutoCTS 存在的问题 [20]

AutoCTS+ 给出的解决方案是通过提高模型评估的效率加速模型搜索的过程，方法是通过设计架构-超参数联合搜索的对偶图空间（图6.20），通过随机采样网络训练，得到架构-超参数-性能的三元组作为原始数据。作者还设计了架构-超参数比较器，原理是利用 GIN(Graph Isomorphism Network)[21] 这个具备强大的区分异构图能力的网络，监督训练一个能够比较不同网络架构间性能差距的比较器。（图6.21）。

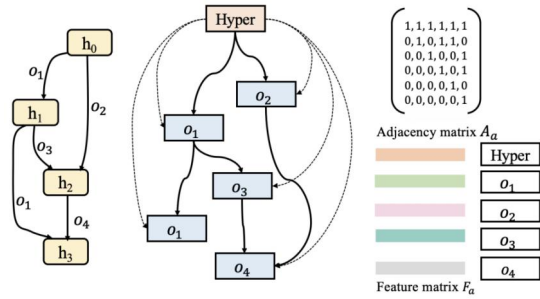


图 6.20: 架构-超参数对偶图 [20]

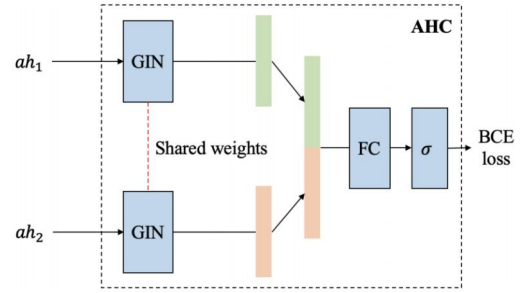


图 6.21: 架构-超参数比较器 [20]

训练好的比较器具备直接比较不同架构模型之间性能优劣的能力，无需重新评测。并且，训练好的比较器还具备可迁移性，当更换目标任务时，只需要先根据目标任务的数据集随机采样少量架构，训练得到原始数据，然后使用这部分数据微调比较器，即可将比较器迁移至新的任务上。

6.4.4 可泛化的模型自动设计

可泛化的模型自动设计方法首先从预定义的搜索空间中采样大量网络架构，并在不同的数据集上训练一个通用的性能预测网络。在推理阶段，该预测网络能够直接对搜索空间中所有架构的性能进行快速评估并排序，最终无需重新训练即可为当前特定场景的数据集自动选取最佳架构。可泛化的模型自动设计方法主要可以分为三个阶段：数据处理阶段，模型搜索阶段和评估阶段。

- 1. 数据处理阶段：**为了让神经架构搜索算法具备泛化能力，从而为不同的任务场景自适应地构造最优模型，常用的做法是先从预定义好的搜索空间中采集一部分模型，获取其性能评估指标，再训练一个能根据模型结构以及任务场景输出性能表现的神经网络，用以根据当前任务高效衡量整个搜索空间中模型的性能。故而在数据处理阶段，往往花费一定的代价从搜索空间中采样并训练模型，获得（模型结构，任务场景，性能指标）的三元组。
- 2. 模型搜索阶段：**将搜索策略提前预训练好，并且在训练搜索策略的同时，已经将性能评估的标准通过训练的方式灌输给了搜索策略，从而具备直接在未见任务场

景下做零样本推理，直接输出适配当前场景的最优模型的能力。预训练的数据格式参照数据处理阶段得到的三元组。经过预训练后，该类方法的搜索策略具备了一定的泛化能力，能够根据搜索空间中架构的特点，以及任务场景下的数据特征，输出这些待选架构的性能推测，并直接排序找到最优架构。

3. **评估阶段**：神经架构搜索方法的评估与传统的模型一致，将得到的模型在当前任务的训练集 D_{train} 上进行训练，利用验证集 D_{valid} 进行早停等操作，从而获取最优的检查点，用以在测试集上推理。

可泛化的模型自动设计方法主要聚焦于如何设计模型的性能预测网络以及如何通过缩减搜索空间、加速训练等方法进一步提升效率。下面介绍几种具有代表性的可泛化的模型自动设计方法。

AutoCTS++ [17] 作为 AutoCTS+ 的扩展版本，进一步围绕泛化能力展开研究。在 AutoCTS+ 中，如果要将训练好的比较器适配新的任务场景（不同的数据集或预测步长），那么需要在新场景下先收集一部分架构的性能，再微调比较器，实现迁移。该过程主要的开销是在新场景下收集架构的性能，因为涉及到对于模型的全量训练，一旦新场景的数据集规模较大，训练成本将会显著提高。如图 6.22 所示，AutoCTS++ 通过预训练支持了对于多任务场景的泛化，从而可以在推理时直接根据下游任务的表征，从搜索空间中选出最适合的架构。

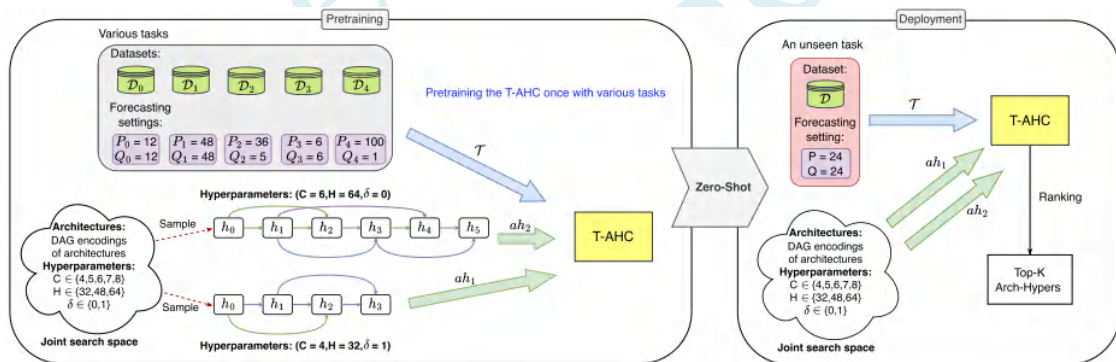


图 6.22: AutoCTS++ 的架构 [17]

具体而言，我们通过改进 AutoCTS+ 中的比较器完成这个功能，如图 6.23 所示，改进后的比较器 T-AHC (Task-aware Architecture-Hyperparameter Comparator) 能够接收数据集作为输入，通过预训练好的 TS2Vec 抽取数据集的统一表征，然后经过双层的 Set-Transformer 完成表征的维度压缩和时空关联提取，得到任务表征。最后，任务表征和架构参数以及超参数被统一考虑，由于判别在具体任务场景下，不同架构之间的性能关系。为了预训练好 T-AHC，需要提前采集不同任务上的架构超参组合，并通过 AutoCTS+ 中的代理指标，降低得到他们性能评估的开销。在具备多任务下的

元数据后，T-AHC 通过 BCE Loss 进行预训练，在推理过程中可以直接泛化到相近的任务上，从而节省了迁移的成本，可以做 zero-shot 推理。

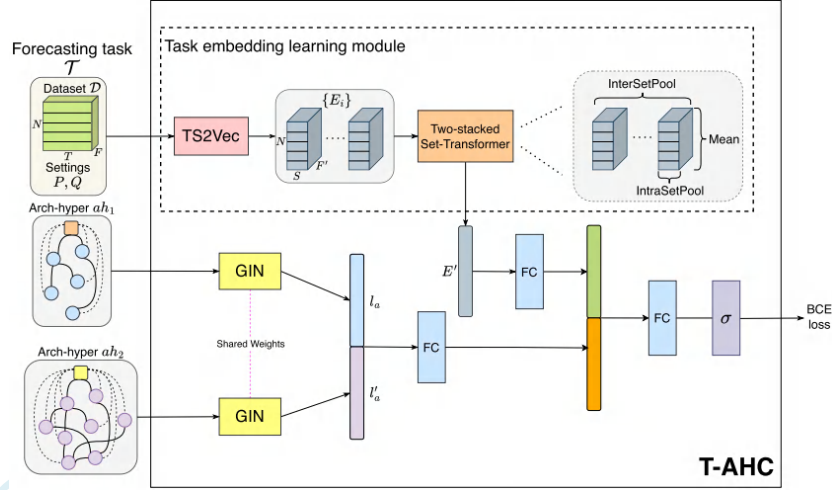


图 6.23: T-AHC [17]

FACTS [18] 在 AutoCTS++ 的基础上进一步改进。[18] 虽然 AutoCTS++ 已经基本解决了时序预测领域神经架构搜索的大部分不足，但是其精度受限于元数据的规模。元数据收集的成本较高，且较难广泛地覆盖搜索空间，AutoCTS++ 的架构-超参联合搜索空间规模高达 10^{20} ，但是由于成本问题，往往只能在每种任务上收集数百个架构，这可能会导致细粒度的架构性能差异难以被判别，从而产生次优结果。为了缓解上述问题，FACTS 提出了搜索空间剪枝的思路，见图 6.24，核心思想是设计模块，自适应地根据任务特点进行搜索空间剪枝，找到优秀架构富集的子空间，再遍历较小的子空间，搜索出最优架构，并通过参数继承的方式，加速搜索出的架构的训练过程。

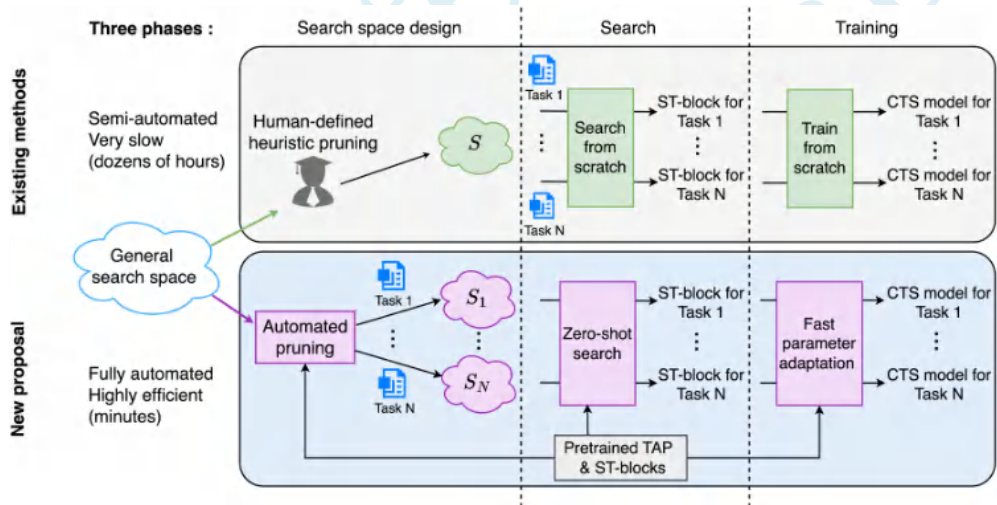


图 6.24: 搜索空间剪枝 [18]

如图 6.25 所示，在训练过程中，FACTS 在不同任务上收集架构，训练出用来剪枝搜索空间的 **comb predictor**，功能是预测哪一块子空间的架构平均性能较高，随后用该部件进行搜索空间裁剪，保留高性能区域并从中采样新一批架构，以此类推，最后可以得到极小的最优子空间。最后，从中采样一批架构，用以训练用来与 AutoCTS++ 中的 T-AHC 中类似的排序器 TAP，衡量整个最优子空间的架构，并返回最优架构。该训练过程可以得到具有泛化能力的 **comb predictor** 和 TAP，分别能根据不同的下游热任务，完成搜索空间裁剪和模型架构排序的功能，从而在面对完全未见的任务时，**comb predictor** 可以根据任务特点高效裁剪搜索空间，TAP 再遍历最优子空间，获取更有性能保证的模型架构。此外，FACTS 为了加速搜索出的最优架构的训练，还设计了参数继承机制，在前期预训练过程中，将训练过的架构的参数存储好，搜索出新的架构后，与这个库中的架构进行匹配，找到之前较为相似的架构，并将他们的参数进行加权继承，继承到搜索出的架构中。这可以有效加速新架构在未见任务上的训练速度。

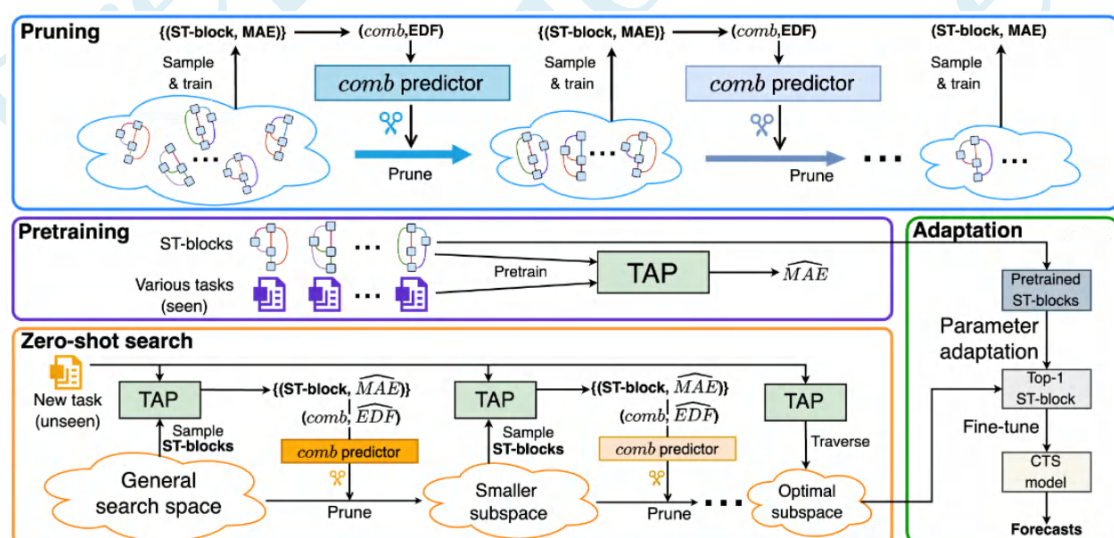


图 6.25: FACTS 的架构 [18]

参考文献

- [1] Mustafa Abdallah, Ryan Rossi, Kanak Mahadik, Sungchul Kim, Handong Zhao, and Saurabh Bagchi. AutoForecast: Automatic time-series forecasting model selection. In *CIKM*, pages 5–14, 2022.
- [2] Aadyot Bhatnagar, Paul Kassianik, Chenghao Liu, Tian Lan, Wenzhuo Yang, Rowan Cassius, Doyen Sahoo, Devansh Arpit, Sri Subramanian, Gerald Woo, Amrita Saha, Arun Kumar Jagota, Gokulakrishnan Gopalakrishnan, Manpreet Singh, K C Krithika, Sukumar Maddineni, Daeki Cho, Bo Zong, Yingbo Zhou, Caiming Xiong, Silvio Savarese, Steven Hoi, and Huan Wang. Merlion: A machine learning library for time series. 2021.
- [3] Maximilian Christ, Nils Braun, Julius Neuffer, and Andreas W Kempa-Liehr. Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package). *Neurocomputing*, 307:72–77, 2018.
- [4] Siem Morten Johannes Dahl. Tspo: an automl approach to time series forecasting. Master’s thesis, Universidade NOVA de Lisboa (Portugal), 2020.
- [5] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *J. Mach. Learn. Res.*, 20(1):1997–2017, 2019.
- [6] Raphael Fischer and Amal Saadallah. AutoXPCR: Automated multi-objective model selection for time series forecasting. In *SIGKDD*, pages 806–815, 2024.
- [7] Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. Single path one-shot neural architecture search with uniform sampling. In *ECCV*, pages 544–560. Springer, 2020.
- [8] Kaijian He, Qian Yang, Lei Ji, Jingcheng Pan, and Yingchao Zou. Financial time series forecasting with the deep learning ensemble model. *Mathematics*, 11(4):1054, 2023.
- [9] Jianxin Li, Shuai Zhang, Hui Xiong, and Haoyi Zhou. Autost: Towards the universal modeling of spatio-temporal sequences. *NeurIPS*, 35:20498–20510, 2022.
- [10] Ting Li, Junbo Zhang, Kainan Bao, Yuxuan Liang, Yexin Li, and Yu Zheng. Autost: Efficient neural architecture search for spatio-temporal prediction. In *SIGKDD*, pages 794–802, 2020.

- [11] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.
- [12] Sheng-Xiang Lv, Lu Peng, Huanling Hu, and Lin Wang. Effective machine learning model combination based on selective ensemble strategy for time series forecasting. *Inf. Sci.*, 612:994–1023, 2022.
- [13] Mariana Oliveira and Luis Torgo. Ensembles for time series forecasting. In *ACML*, pages 360–370. PMLR, 2015.
- [14] Zheyi Pan, Songyu Ke, Xiaodu Yang, Yuxuan Liang, Yong Yu, Junbo Zhang, and Yu Zheng. Autostg: Neural architecture search for predictions of spatio-temporal graph. In *WWW*, pages 1846–1855, 2021.
- [15] Syed Yousaf Shah, Dhaval Patel, Long Vu, Xuan-Hong Dang, Bei Chen, Peter Kirchner, Horst Samulowitz, David Wood, Gregory Bramble, Wesley M Gifford, et al. AutoAI-TS: Autoai for time series forecasting. In *SIGMOD*, pages 2584–2596, 2021.
- [16] Oleksandr Shchur, Ali Caner Turkmen, Nick Erickson, Huibin Shen, Alexander Shirkov, Tony Hu, and Bernie Wang. AutoGluon-timeseries: AutoML for probabilistic time series forecasting. In *International Conference on Automated Machine Learning*, pages 9–1. PMLR, 2023.
- [17] Xinle Wu, Xingjian Wu, Bin Yang, Lekui Zhou, Chenjuan Guo, Xiangfei Qiu, Jilin Hu, Zhenli Sheng, and Christian S Jensen. AutoCTS++: zero-shot joint neural architecture and hyperparameter search for correlated time series forecasting. *The VLDB Journal*, pages 1–28, 2024.
- [18] Xinle Wu, Xingjian Wu, Dalin Zhang, Miao Zhang, Chenjuan Guo, Bin Yang, and Christian S Jensen. Fully automated correlated time series forecasting in minutes. In *PVLDB*, volume 18, pages 144–157, 2025.
- [19] Xinle Wu, Dalin Zhang, Chenjuan Guo, Chaoyang He, Bin Yang, and Christian S Jensen. Autocts: Automated correlated time series forecasting. *PVLDB*, 15(4):971–983, 2021.
- [20] Xinle Wu, Dalin Zhang, Miao Zhang, Chenjuan Guo, Bin Yang, and Christian S Jensen. Autocts+: Joint neural architecture and hyperparameter search for correlated time series forecasting. *Proceedings of the ACM on Management of Data*, 1(1):1–26, 2023.

- [21] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.
- [22] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong. Pc-darts: Partial channel connections for memory-efficient architecture search. *arXiv preprint arXiv:1907.05737*, 2019.
- [23] Zhen Xu, Quanming Yao, Yong Li, and Qiang Yang. Understanding and simplifying architecture search in spatio-temporal graph neural networks. *Trans. Mach. Learn. Res.*, 2023, 2023.
- [24] Yuanyuan Yao, Dimeng Li, Hailiang Jie, Hailiang Jie, Tianyi Li, Jie Chen, Jiaqi Wang, Feifei Li, and Yunjun Gao. SimpleTS: An efficient and universal model selection framework for time series forecasting. *PVLDB*, 16(12):3741–3753, 2023.
- [25] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. TS2Vec: Towards universal representation of time series. In *AAAI*, volume 36, pages 8980–8987, 2022.
- [26] G Peter Zhang. A neural network ensemble method with jittered training data for time series forecasting. *Inf. Sci.*, 177(23):5329–5346, 2007.
- [27] Shuai Zhang, Yong Chen, Wenyu Zhang, and Ruijun Feng. A novel ensemble deep learning model with dynamic error correction and multi-objective ensemble pruning for time series forecasting. *Inf. Sci.*, 544:427–445, 2021.

7 时间序列基础模型

7.1 节作者为：王益杭、邱钰莹、张睿桐、汪琳丰、郭晨娟；

7.2 节作者为：李哲、陆骏凯、陈鹏、郭晨娟。

本章主编为郭晨娟。

在第3至6章中，我们介绍了多种任务的端到端时间序列模型的训练流程及其关键设计。然而，这类在单一数据集上训练的端到端模型往往在处理新数据时表现出有限的泛化能力，并伴随较高的训练开销。为克服这些局限性，近年来时间序列基础模型（Time Series Foundation Model, TSFM）的开发呈现出显著增长。

根据时间序列基础模型的预训练数据的类型可以被分为两类：**基于预训练的时间序列基础模型（Time Series Pre-trained Foundation Model）**和**基于大语言模型的时间序列基础模型（LLM-based Time Series Foundation Model）**。本章将在后续的第7.1章和第7.2章中分别展开介绍。

7.1 基于预训练的时间序列基础模型

7.1.1 模型定义及流程

时间序列预训练基础模型（Time Series Pre-trained Foundation Model）是指通过大规模多源时间序列数据预训练，能有效应用于下游各种时间序列任务上的模型。随着时间序列数据量的增长，时间序列预训练基础模型用海量数据学习通用的表示，展现出强大的跨领域泛化能力，能显著降低下游任务对数据和计算资源的需求。

时间序列预训练基础模型在多源时间序列数据集 $\mathbf{D}_{\text{pre-train}} = (\mathbf{X}^N, \hat{\mathbf{X}}^N)$ 上进行预训练，其中 N 为数据集的数量。在下游任务中，时间序列预训练基础模型在下游任务训练集 $\mathbf{D}_{\text{train}} = (\mathbf{X}^{\text{train}}, \hat{\mathbf{X}}^{\text{train}})$ 上进行微调，并在测试集 $\mathbf{D}_{\text{test}} = (\mathbf{X}^{\text{test}}, \hat{\mathbf{X}}^{\text{test}})$ 上测试效果，其中 $\mathbf{D}_{\text{pre-train}}$ 、 $\mathbf{D}_{\text{train}}$ 以及 $\mathbf{D}_{\text{test}}$ 三个集合两两无交集。此外，部分时间序列预训练基础模型可以直接在 $\mathbf{D}_{\text{test}}$ 上进行测试而无需使用 $\mathbf{D}_{\text{train}}$ 进行微调。

时间序列预训练基础模型与任务专用的小模型存在根本性差异。在数据维度上，时间序列预训练基础模型依赖大规模预训练数据，对下游任务微调所需数据量要求较低；而任务专用模型需要为每个下游任务单独收集大量训练数据。在模型架构维度上，Transformer 及其变体因其强大的序列建模能力和并行计算效率，成为目前时间序列预训练基础模型的主流架构；而任务专用模型多样化，通常针对特定任务或特定数据集设计。在跨域泛化的维度上，时间序列预训练基础模型具备跨领域泛化能力，可应用于下游多种场景，而任务专用模型在相似数据上表现好，跨领域泛化能力有限。从训练与维护成本上，基础模型预训练成本高，但一次预训练，后期多次微调成本低，重复部署成本低。而任务专用模型需要对每个场景模型从头训练，单独部署成本高。

图7.1展示了时间序列预训练基础模型的通用框架，其训练过程通常包括以下几个步骤：1) 预训练数据收集，从服务器以及其他数据源收集大规模多源的时间序列；2) 预训练数据预处理，对海量的多源时间序列数据进行筛选，调整数据具有多样性且不同领域数据量均衡，并通过数据清洗、缺失值处理、归一化和标准化等操作提高数据质量；3) 模型预训练，设置合适的模型架构和预训练任务，通过前向传播和反

向传播优化模型参数，从多源数据集中学习通用的表征；4) 下游数据收集与预处理，从服务器或其他数据源获取目标领域的时间序列并通过数据清洗、缺失值处理、归一化和标准化等操作提高数据质量；5) 模型微调，使用下游数据训练集对预训练好的模型参数进行微调，微调完成后使用测试集进行性能评估；6) 将模型部署到服务器进行实际应用，并持续监控其性能，必要时使用新的数据再进行模型微调。

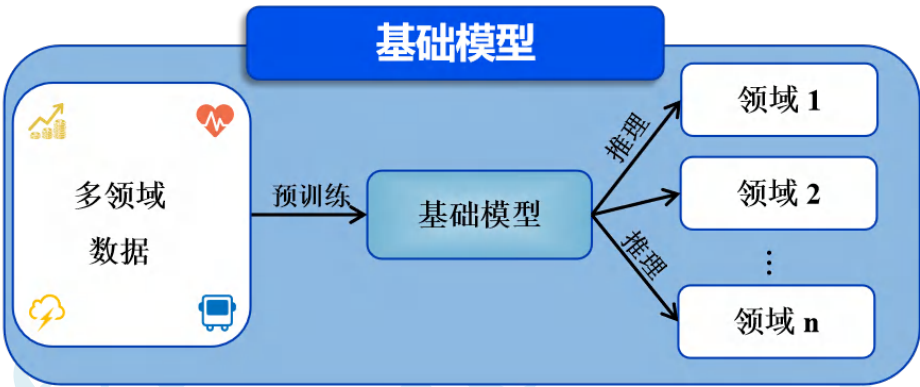


图 7.1: 时间序列预训练基础模型的流程

7.1.2 模型训练及评估

1. 预训练阶段：常见的预训练任务有：
- (a) **重构任务**：在时间序列中随机选择一部分数据点或数据段，将其值替换为掩码标记（如零值或特定的掩码符号）。让模型利用未被掩码的数据，重建被掩码的部分，从而学习时间序列中的模式和依赖关系，提升模型对时间序列的表征能力和鲁棒性。
 - (b) **预测任务**：利用时间序列的自回归特性，使用历史序列预测未来序列值。通过这种方式，模型可以学习时间序列中的长期依赖关系和模式，从而为下游预测任务提供更好的初始模型。
 - (c) **混合任务**：结合重构任务以及预测任务，设置双目标预训练任务，从而使得模型在具有泛化性的同时，在预测任务上具备出色的零样本迁移能力。
2. 微调阶段：以下是常见微调方法：
- (a) **线性探测 (Linear Probing)**：只训练最后添加的线性层，而冻结预训练模型的其他参数。适用于任务不相似或者数据稀缺的情况，更侧重于评估预训练模型的特征表征能力。

(b) **端到端微调 (End-to-end Fine-tuning)**：在预训练模型的基础上，继续更新整个模型的参数来适应新的下游任务。通常用于目标任务和预训练任务相似的情况。

3. **评估阶段**：与第 3 至 6 章节中端到端模型的评估方式类似，时间序列预训练基础模型的评估过程同样使用 MSE, MAE, Accuracy, Recall, Hit@k, Average Rank 等评估指标来评估模型性能。与不同的是，时间序列预训练基础模型更关注模型的泛化性和少样本（甚至零样本）迁移能力：

(a) **全样本评估 (full-shot)**：利用全部下游训练样本对基础模型进行微调，使模型在预训练的基础上进一步学习下游数据的全面特征与模式，从而验证模型在不同领域数据集上的泛化性以及其在特定任务上的优化能力。

(b) **少样本评估 (few-shot)**：在仅使用少量下游训练样本微调的情况下，验证模型在不同领域数据集上的泛化性以及对新领域的快速适应能力。

(c) **零样本评估 (zero-shot)**：不使用下游训练样本对模型进行微调，仅依靠预训练的特征表示完成下游任务的推理，以此验证模型在不同领域上的通用性。

表 7.1: 时间序列预训练基础模型分类总结

模型	年份	模型架构	多变量处理	内生多模态	预训练数据量	模型参数量
Timer [11]	ICML2024	Decoder-only	通道独立	×	28B	67M
MOIRAI [22]	ICML2024	Encoder-only	通道混合	×	231B	311M
TimesFM [5]	ICML2024	Decoder-only	通道独立	×	100B	200M
TTMs [6]	NeurIPS2024	MLP	通道独立	×	1B	1M
MOMENT [7]	ICML2024	Encoder-only	通道独立	×	1.23B	385M
ROSE [20]	ICML2025	Encoder-only	通道独立	×	0.89B	7.4M
Chronos [1]	TMLR2025	Encoder-Decoder	通道独立	×	84B	710M
LightGTS [19]	ICML2025	Encoder-Decoder	通道独立	×	1B	4M
SymTime [18]	NeurIPS2025	Encoder-only	通道独立	✓	-	-
Time-MoE [15]	ICLR2025	Decoder-only	通道独立	×	300B	2.4B
DADA [14]	ICLR2025	TCN	通道独立	×	-	-
VisionTS [3]	ICML2025	Encoder-only	通道独立	✓	-	-
Sundial [10]	ICML2025	Encoder-only	通道独立	×	1T	128M
AimTS [4]	ICDE2025	Encoder-only	通道独立	✓	-	-

7.1.3 模型关键技术

表7.1从不同角度总结了现有的时间序列预训练基础模型的关键技术，本小节将详细展开介绍。

1. **模型架构**：与第 3 至 6 章节中端到端模型的评估方式类似，时间序列预训练基础模型的评估过程同样使用 MSE, MAE, Accuracy, Recall, Hit@k, Average Rank 等评估指标来评估模型性能。与其不同的是，时间序列预训练基础模型更关注模型的泛化性和少样本（甚至零样本）迁移能力：

- (a) **基于 Encoder-only 架构的时间序列预训练基础模型** Encoder-only 架构在时间序列预训练中主要采用编码器结构（如 Transformer Encoder）提取多层次特征，并通过掩码重建等预训练任务学习通用表示。该类模型的核心特点是利用双向注意力机制全面捕捉时间点之间的依赖关系。典型代表 MOIRAI、MOMENT、ROSE，这些模型通过在大量时序数据上预训练，显著提升了下游任务的样本效率与泛化能力。

MOIRAI [22]：MOIRAI 是一套专为通用时间序列预测设计的开源基础模型。该模型采用 Encoder-only 的 Transformer 架构，通过多尺度 patch 编码与掩码预测任务进行统一训练，能够有效建模时间序列中的多粒度结构特征。为了提升性能，MOIRAI 在原始 Transformer 架构基础上引入了三项关键改进：多尺度切片嵌入层支持不同频率的动态适配，MOIRAI 使用多种 patch 大小并行投影嵌入，使模型能够在一个架构中同时捕捉短期与长期依赖特征，提升其对跨频率时序结构的适应能力；通道关联建模机制支持任意维度变量输入建模，使模型不仅能够捕捉时间维度的依赖关系，还能显式建模不同变量之间的交互与相关性；混合分布预测头通过多种概率分布混合建模预测不确定性，有效提升了模型对真实世界复杂时间序列的建模能力。MOIRAI 支持多种时间序列预测场景，具备出色的零样本推理能力。MOIRAI 的提出突破了传统时间序列模型难以统一、泛化能力弱的限制，为大规模、通用的时间序列预测任务提供了一个开放、可扩展的高性能解决方案。

MOMENT [7]：MOMENT 是一套专为通用时间序列分析设计的开源基础模型，旨在实现跨任务、跨领域的高效迁移和泛化能力。该模型采用 Encoder-only 架构，使用掩码重构任务进行无监督预训练，该方法通过随机均匀遮蔽时间序列中的部分片段，训练模型根据上下文对缺失部分进行还原，从而在无监督环境下有效捕捉时间序列中的局部依赖与全局结构，提升模型泛化能力。为了提升性能，MOMENT 在 Transformer 架构上进行了关键改进：采用轻量级预测头，替代传统与编码器相同大小的解码器，显著减少参数量，便于在下游任务中进行高效微调。MOMENT 支持多种时间序列任务，包括分类、回归、异常检测、预测与填补等，并具备一定的零样本推理能力，在未见过的任务或领域中也能展现出稳健的迁移表现。MOMENT 的提出突破了以往模型任务割裂、难以迁移等局限，为各类时间序列任务提供了可扩展、高性能的解决方案。

ROSE [20]: ROSE 是一种用于时间序列预测的新型预训练模型，通过多领域时间序列数据的预训练来提升泛化能力，解决了利用多域数据集提高下游预测任务性能的挑战。ROSE 采用 **Encoder-only** 架构，使用分解频率学习，通过基于频率的掩蔽和重建分解时间序列中的耦合语义信息，具体而言，ROSE 利用频率掩蔽与重建机制，将时间序列表示解耦为不同频率成分，从而捕捉跨域共享的结构特征，并构建统一的表征空间，提升在下游任务中的迁移能力。同时 ROSE 设计了时间序列寄存器机制，该机制通过在预训练阶段动态学习多个领域的特征寄存器，并在下游任务上选择相关的寄存器达到增强域自适应转移的目的。这种设计不仅提升了模型在多源异构数据上的泛化能力，还显著提高了其在小样本和零样本场景下的预测性能。整体而言，ROSE 在保持结构简洁的同时，有效解决了多领域时间序列建模中泛化困难与领域迁移效率低的问题，为通用时序建模提供了一种高效可扩展的解决方案。

- (b) **基于 Decoder-only 架构的时间序列预训练基础模型** Decoder-only 架构采用自回归方式逐步生成未来值，其核心是掩码自注意力机制（仅允许访问历史信息），天然适配时间序列预测任务。该类模型通过大规模预训练学习时间动态模式，并可通过零样本或少样本推理直接应用于新数据集。典型模型如 TimeGPT（首个大规模时间序列预测基础模型）、Lag-Llama（基于滞后特征与 Llama 架构）以及 TimesFM（Google 研发的时序预测 Decoder 模型）。这些模型在长时序预测中展现强鲁棒性，但通常对分类等非生成任务适配性较弱。

Timer [11]: Timer 是一款大规模通用时间序列模型，为时间序列建模引入了生成式预训练方法，旨在开发大型时间序列模型（LTSM），以克服当前小型深度学习模型在数据稀缺场景下的表现瓶颈。该模型基于 **Transformer decoder-only** 架构，借鉴 GPT 模型的设计，将时序建模转化为自回归生成任务，使得模型能够逐步生成未来时间步，有效建模复杂的时间依赖关系。通过在大规模无标签数据上进行预训练，Timer 显著提升了模型在多个任务上的泛化能力，使其在数据稀缺的情况下依然能保持良好效果。Timer 支持多种时间序列任务，在预测精度、异常检测、缺失值填补等任务中均展现出优异性能，在少样本场景下依然保持稳健表现，具备良好的迁移能力和 zero/few-shot 推理潜力。Timer 的提出为未来构建高性能、可泛化的时间序列基础模型提供了新路径与技术支持。

TimesFM [5]: TimesFM 是一套基于 Transformer 的通用时间序列建模框架，旨在在保持预测准确率的同时，提升远程预测的效率与泛化能力。TimesFM 采用 **Decoder-only** 架构，将时间序列划分为 patch 单元，增强了模型对序列结构的建模能力。为了适应时间序列任务的上下文特性，TimesFM 设计

了片段遮蔽机制，使模型能够灵活感知不同范围的上下文信息，提升对长期依赖关系的捕捉能力。相较于传统语言模型常见的逐步自回归输出方式，TimesFM 支持大跨度预测，能够一次性生成多个未来 patch，有效提升了预测效果。TimesFM 专注于时间序列预测任务，并展现出一定的 zero-shot 推理能力。TimesFM 的提出推动了时间序列建模向更高效、更通用方向的演进。通过对语言模型结构的合理借鉴与针对时间序列特征的专属适配，TimesFM 在保留预测精度的基础上显著优化了建模效率，为广泛时间序列任务提供了一个具备可迁移性与高性能的统一解决方案。

Time-MoE [15]: Time-MoE 是一套面向通用时间序列预测任务设计的开源基础模型，旨在兼顾大规模建模能力与推理效率，在跨领域场景中实现强泛化能力与零样本适应性。该模型采用 Decoder-only 架构，并引入 Mixture-of-Experts (MoE) 稀疏激活机制，在每层仅激活部分专家网络，从而在不显著增加推理成本的前提下扩展模型容量至数十亿参数级别。Time-MoE 以自回归方式进行多尺度预测预训练，结合 Huber 损失与负载均衡损失，有效增强模型的鲁棒性与专家间协同。为进一步提升泛化能力，Time-MoE 通过多尺度预测头设计支持不同预测步长，提升了对各类序列特征的建模能力并在多个标准数据集上展现出领先的零样本预测性能，显著优于现有主流模型，其提出为时间序列基础模型的发展提供了新范式，也为大模型在时序领域的落地奠定了技术基础。

- (c) **基于 Encoder-Decoder 架构的时间序列预训练基础模型** Encoder-Decoder 架构通过编码器压缩输入序列的语义表示，再通过解码器生成目标输出（如预测值、重构信号或异常评分），适用于需要复杂映射的时序任务（如预测、修复、跨模态生成）。预训练常采用编码器-解码器联合训练，例如通过掩码重建或去噪任务学习鲁棒表示。典型模型包括 LightGTS（基于周期 Tokenization 和周期非自回归解码策略）以及 T5 风格的时间序列生成模型（如 Chronos）。该架构灵活性高，但训练复杂度与计算成本通常高于前两类。

Chronos [1]: Chronos 是一款专为时间序列预测设计的通用模型，其核心创新在于将连续时间序列通过缩放与量化映射为有限的离散 token 空间，使得原始数值序列能够被类比为自然语言序列，从而借助语言模型架构进行高效建模。在预测策略上，Chronos 不再局限于传统的点预测方式，而是输出对应的概率分布，并通过采样机制生成多条潜在的未来路径，刻画时间序列演化过程中的不确定性。该方法有效融合了概率推理与生成式建模的优势，使模型更具表现力与鲁棒性。Chronos 聚焦于时间序列预测任务，并具备一定的 zero-shot 推理能力，在面对未知任务或领域时依然能够展现出良好的迁移性能。其离散化策略与分布建模思路，为未来构建统一、可泛化的时间

序列预测框架提供了新的方向。

LightGTS [19]: LightGTS 是一款兼具轻量化与通用性的时间序列预测模型，其核心在于融合周期化分词、弹性投影层与周期化并行解码器，构建出高效精简的预测框架。该模型通过将时间序列划分为能够完整捕捉周期结构的片段，并在不同时间尺度下保持语义一致，从而有效增强跨尺度的泛化能力。在预测阶段，LightGTS 采用周期化并行解码替代传统的自回归方式，在提升推理效率的同时保持周期一致性与预测稳定性。模型支持 zero-shot 推理，在陌生任务或领域中同样具备良好的泛化能力，具备极高的计算效率与部署灵活性。LightGTS 的提出打破了传统模型在性能与资源开销之间的权衡限制，为轻量级深度时序建模提供了全新的设计范式与实践路径。

- (d) **其余架构模型介绍 Tiny Time Mixers [6]:** Tiny Time Mixers (TTMs) 是一种面向多变量时间序列预测的极其轻量级预训练模型，旨在以最小的模型容量实现高效泛化与迁移性能。该模型构建于轻量级的 TSMixer 架构之上，融合了自适应切片、多分辨率采样以及分辨率前缀调优等关键技术，使其能够灵活适应不同时间分辨率的数据分布。具体而言，TTMs 采用 MLP 架构，通过自适应切片机制，针对每个数据集的时间粒度动态调整输入片段策略，从而增强模型对不同时间尺度的适应能力。为进一步提升跨分辨率的泛化能力，TTMs 对高分辨率数据进行下采样，合成多种时间粒度的版本，并在多分辨率混合训练过程中引入“分辨率前缀调优”机制。该策略通过在模型输入中嵌入表示时间粒度的前缀 token，使模型能够显式感知当前输入的时间分辨率，从而实现更精准的特征提取与迁移。尽管 TTMs 拥有远低于传统模型的参数量，但在多个真实世界数据集上，仍在 zero-shot 与 few-shot 场景中展现出优于大型模型的表现，验证了其在实际应用中极具潜力的轻量化与可迁移性。

2. 多变量处理

- (a) **通道独立** 在时间序列基础模型的预训练阶段，通道独立处理核心思想是将多变量时间序列中的各个通道（变量）视为相互独立的单变量序列，分别输入模型进行特征提取与预测，从而避免不同变量之间的复杂交互对训练过程带来的干扰。通道独立处理能够显著提升模型的训练稳定性与泛化能力。一方面，多变量时间序列往往存在采样频率、尺度分布及信号噪声水平的差异，直接联合建模可能导致模型在高相关性变量上过拟合，而在低相关性或高噪声变量上学习不足。通过通道独立处理，每个变量都能在统一的结构下实现自适应建模，使模型更好地捕捉其自身的动态规律与时间依赖结构。另一方面，该策略也大幅简化了模型结构，减少了参数量与计算复杂度，使得预训

练过程更加高效。然而，由于缺乏跨通道的协同建模，其在捕捉变量间交互模式与全局依赖关系方面存在一定局限。通道独立方法以简化结构与稳健泛化为特征，为大规模多维时间序列的快速建模提供了一种高效的基础策略。在现有主流的时间序列预训练基础模型中，大多数模型在预训练阶段考虑通道独立，比如：Timer [11]、ROSE [20]、LightGTS [19]。

(b) **通道混合** 在时间序列基础模型的预训练阶段，通道混合处理旨在对多变量时间序列中的不同通道（变量）进行联合建模与交互融合，以充分挖掘变量之间的相关性与动态依赖关系。其核心思想是将多个通道的时序信号映射至共享的表示空间中，通过跨通道注意力、特征融合层或混合投影等机制，实现变量间信息的交互与协同建模，从而捕捉系统整体的动态演化特征。通道混合处理能够显著提升模型对全局依赖关系与多源交互模式的理解能力。现实世界中的多变量时间序列常表现出复杂的协同变化与因果耦合，单变量独立建模难以揭示其整体结构。通道混合通过在特征空间中显式建模变量间的交互，使模型能够从多维信号中提取出潜在的共性模式与依赖结构，从而增强预测的准确性与语义解释性。另一方面，通道混合有助于模型在面对多模态或异构数据时建立统一的表征基础，使不同来源、不同语义层次的变量能够在预训练阶段形成协同的嵌入结构。然而，通道混合也带来更高的计算复杂度与建模难度。由于变量数量可能较多，交互建模会引入额外的注意力计算与参数开销，且在高维场景下可能引发过拟合或特征冗余问题。在现有主流的时间序列预训练基础模型中，少数模型在预训练阶段考虑通道混合，比如：MOIRAI [22]、MOMENT [7]。

3. **时序内生多模态** 多模态学习旨在融合来自不同感知源的数据模态，如图像、语音、文本、时间序列等，以实现跨模态的信息互补、统一理解与知识共享。其核心在于突破单一模态的表征限制，通过在特征层、语义层或决策层实现多源信息的深度融合，从而获得更具语义一致性和认知全面性的表示能力。随着多模态研究在视觉、语言等领域的成功，时间序列领域也逐渐引入多模态思想，以期利用外部模态的先验知识来弥补纯时序建模在语义表达、结构建模与跨域泛化方面的不足。

然而，相较于图文多模态任务中已有的海量公开数据集，真实且高质量的时序多模态数据集仍然十分稀缺。为此，研究者从时间序列本身出发，生成或映射出具有语义或空间表征能力的衍生模态，例如将时间序列转化为图像（如频谱图、时频图、相空间重构图等）或文本（如符号表达、公式描述、语义标签等），以实现模态扩展和语义重构。这一方向不仅保留了时间序列的动态结构特征，还引入了可解释的多模态对齐空间，为跨模态表征学习提供了新的途径。

基于此，现有的多模态时间序列方法主要包括三类：基于时序-文本的多模态方法、基于时序-图像的多模态方法，以及同时融合时序、文本与图像的多模态联合建模方法。这些方法试图借助外部模态在语义理解或空间建模上的优势，提升时间序列任务在表达能力、泛化性能和复杂场景适应性方面的表现。

- (a) **结合图像** 基于时序图像的多模态方法主要通过将时间序列数据转换为图像形式，借助现有视觉模型（如 CNN、ViT 等）对图像空间结构的建模能力间接实现对时间模式的理解。早期工作中，如基于格拉姆角场（GAF）、马尔可夫过渡场（MTF）等转换方式，将时间序列映射到二维图像，通过提取纹理、形状等视觉特征辅助完成分类与识别任务。然而这些转换方法往往依赖复杂的先验理论设计，通用性与可解释性较弱。近期研究则表明，简单直接的图像构建方式同样可以取得优异效果。ViTST 直接将时间序列绘制为折线图并输入视觉变换器模型进行分类，取得了良好性能，表明直观图像表达与高效视觉建模的协同潜力。VisionTS [3] 通过构造二值图像并从图像末列重建时间序列，实现了视觉驱动的预测建模。HCR-AdaAD 结合时间序列图像表示用于异常检测，而 AimTS [4] 提出了一种时序图像-原始数据的对比融合策略，从视觉角度捕捉序列结构特征，提升了模型的泛化能力。
- (b) **结合文本** 基于时序文本的多模态方法主要利用外源文本为时序模型提供额外视角的信息。这类方法通常通过设计多模态特征提取和融合机制，提升时序分析能力和模型的通用性。例如 SymTime 提出一种能够无限制生成高质量时间序列及其对应的符号表达式的数据生成机制，主要引入四种不同的预训练目标，分别是时间序列掩码建模、自然语言掩码建模、时间序列与符号表达式之间的跨模态对比以及动量蒸馏损失。

4. 模型数据规模

- (a) **大规模**：图 7.2 展示了现有的时间序列预训练基础模型的模型参数和预训练参数量以及该模型在 ETT 数据集上的平均 MSE 值。时间序列预训练基础模型呈现出“大规模”的特点，这种“大规模”体现在模型参数规模以及预训练数据集规模上。其一，模型参数有 10M、100M、1B 等不同量级，部分模型如 Timer(67M)、Time-MOE(2.4B) 等参数规模较大；其二，预训练数据集规模跨度从 1B 到 1T 不等，如 Sundial 的预训练数据集达到 1T。然而，并非模型参数和预训练数据集规模越大，模型的性能就越好。例如，Time-MOE 模型参数规模较大且预训练数据集规模较大，但平均 MSE 并非最低。在时间序列预训练基础模型中，“大规模”的模型参数与预训练数据集是模型的重要特征，但并非越大越好。
- (b) **轻量化**：现有的时间序列预训练基础模型往往拥有庞大的参数规模，并依赖

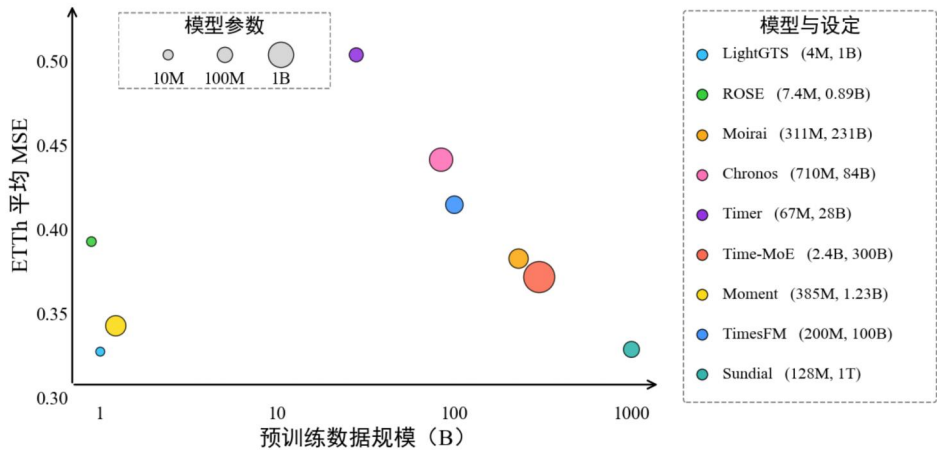


图 7.2: 时间序列预训练基础模型的流程

于大规模多源数据集进行预训练，以学习具有强泛化能力的通用时序表示。这类“大模型”虽然在性能上表现优异，但其训练与推理成本极高，对计算资源、存储空间和部署环境均提出了严苛要求。模型轻量化能使模型在资源少的情况下不受限。现有的模型轻量化方法主要从两方面考虑，一种是模型架构上轻量化；一种是从 Tokenization 上轻量化。TTMs [6] 从模型架构上考虑轻量化，通过采用基于 MLP-Mixer 的 TSMixer 架构代替 Transformer，去除了计算量巨大的自注意力模块，因此 LightGTS [19] 从 Tokenization 上考虑轻量化极大降低了计算复杂度与显存需求。其通过自适应的周期化切片机制，以时间序列的内在周期长度为单位，将序列划分为完整的周期片段，使每个 token 都对应一个完整的周期模式。这样模型在多源数据预训练中能保持周期语义一致性和跨尺度对齐，从而减少了为适应尺度差异所需的冗余参数。

5. 输出方式

(a) **预测：**常见的预测输出方式如下：

- i. **自回归：**自回归是一种按照时间顺序逐步生成未来值的预测方式，模型在每一个时刻的输出都依赖于之前的真实观测或已生成的预测结果。该方法严格遵守时间因果关系，能够较好地捕捉长期依赖特征，因此在金融序列、文本生成等具有明显时间递进特性的任务中应用广泛。然而，由于自回归需要逐步滚动预测，其推理速度较慢，且容易出现误差在多步预测中累积的问题，例如 Timer [11]。
- ii. **展平输出头：**展平输出头是一种并行预测方式，它将时序模型的编码输出进行展平或池化处理，通过全连接层一次性生成固定长度的预测窗口。

这种方法能在一次前向计算中获得完整的预测序列，计算效率高、实现简洁，适合固定地平线的批量预测场景，如工业控制、电力负荷或交通流量预测等。其局限在于预测步长通常是固定的，对长序列依赖或地平线可变的任务泛化性较弱，例如 Time-MoE [15]。

- iii. **掩码重建**：掩码重建方法通过在输入序列中随机掩盖部分时间步或特征，让模型根据可见部分重建被掩盖的内容，从而在训练中学习全局时序结构。这是一种典型的自监督学习策略，常用于时序基础模型的预训练阶段，能够有效提升模型在下游任务中的泛化能力。掩码重建无需额外标签，对缺失或噪声数据具有较强的鲁棒性，但由于其重建目标与具体预测任务存在差异，通常需要通过微调来适配实际应用，例如 MOIRAI [22]。
- iv. **非自回归**：非自回归方法与自回归不同，它不再依赖历史生成逐步预测，而是一次性并行地输出整个未来时间窗口。该方式能够显著提升推理速度，避免逐步生成带来的误差累积，并且在长地平线预测中具有更好的全局一致性。然而，其训练过程通常更复杂，对模型结构与损失函数设计要求更高。非自回归预测多用于需要快速、多步并行预测的场景，如能源管理、生产调度与多变量时序控制等，例如 LightGTS [19]。

(b) **概率预测**：常见的概率预测输出方式如下：

- i. **分位数**：分位数预测方法直接输出目标变量在若干分位点（如 0.1、0.5、0.9）处的估计值，从而刻画未来预测的不确定性分布。这种方法不依赖任何特定分布假设，能够灵活地反映数据的偏态或多峰特征，并且通过不同分位数之间的差值即可形成预测区间。其训练通常采用分位数回归损失函数（Pinball Loss），方法稳定且实现简单。但分位数预测可能出现分位线交叉的现象，需要通过单调性约束或后处理加以修正。该方法广泛用于风险评估、区间预测等需要提供上下置信界的场景，例如 Chronos [1]。
- ii. **预设分布**：预设分布方法假定未来目标服从某种已知的参数化分布（如高斯分布、学生 t 分布或负二项分布），模型的输出为该分布的参数，如均值与方差。训练时通常采用极大似然或负对数似然损失，使预测结果具备严格的概率解释。该方法能够直接生成完整的条件分布，便于进行采样、计算置信区间或进行概率评估。但若所假设的分布与实际数据差异较大，会导致模型校准性下降。因此，在连续数据中多采用高斯或学生 t 分布，在计数或离散数据中则常用泊松或负二项分布。该方法尤其适用于需要密度建模、概率打分或风险量化的应用领域，例如 MOIRAI [22]。

(c) **异常检测**：常见的异常检测输出方式如下：

- i. **基于预测的异常检测**：基于预测的异常检测核心是通过模型预测正常时间序列的未来走势，再将预测结果与实际观测值对比，差异超过阈值的

部分判定为异常，本质是用“预测正常”来反向识别“实际异常”。Timer 将异常检测统一为“下一个 token 预测”任务，例如 Timer [11]。

ii. **基于重构的异常检测**：基于重构的异常检测通过模型学习正常数据的特征规律，再利用其“重构正常数据”与“无法准确重构异常数据”的差异来识别异常。这种方法不直接定义异常，而是让模型从正常样本中学习数据分布，最终通过重构误差的大小判断样本是否异常，例如 DADA [14]。

(d) **分类**：在时序分类任务中，基于线性头微调是一种常用且高效的迁移学习策略。该方法通过将预训练好的时间序列编码器迁移至下游任务，在冻结或微调编码器参数的基础上，于其输出特征上接入一个简单的线性分类头，用于实现最终的类别判别。具体而言，时间序列编码器负责将原始时间序列映射为具有判别性的潜在表示，而线性头则学习从这些表示中提取出与具体分类标签相关的决策边界。由于仅需在小规模标注数据上训练少量参数，该方法具有参数高效、训练稳定、泛化能力强等优点，广泛应用于工业监测、健康预测、金融行为识别等实际场景中，例如 AimTS [4]。

7.2 基于大语言模型的时间序列基础模型

本章介绍基于大语言模型的时间序列基础模型的相关知识。7.2.1 章节给出了基于大语言模型的时间序列基础模型的定义与工作流程。7.2.2 章节给出了基于大语言模型的时间序列基础模型的训练流程和评估策略。7.2.3 章节给出了基于大语言模型的时间序列基础模型的关键技术。

7.2.1 模型定义及流程

基于大语言模型的时间序列基础模型 (LLM-based Time Series Foundation Model) 是指利用大规模预训练的语言模型 (LLM) 或其变体，在大规模、多领域、多模态时间序列数据上学习通用表征，并通过**提示词设计 (Prompt Design)**、**模态对齐 ((Modality Alignment))** 以及**参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT)** 等方式适配下游任务的建模范式。

与传统在单一数据集上端到端训练的预测模型不同，基于 LLM 的时间序列基础模型通过利用大预言模型强大序列理解能力和推理能力显著提升了时间序列模型在未知数据上的泛化能力与训练效率。形式化地，给定历史观测序列 $\mathbf{X} = (\mathbf{x}_{t-H+1}, \mathbf{x}_{t-H+2}, \dots, \mathbf{x}_t) \in \mathbb{R}^{H \times N}$ ，通过一个预训练模型 $\mathcal{M}_\Theta : \mathbb{R}^{H \times N} \rightarrow \mathbb{R}^{F \times N}$ ，将输入序列映射到未来预测序列 $\hat{\mathbf{X}} = (\hat{\mathbf{x}}_{t+1}, \hat{\mathbf{x}}_{t+2}, \dots, \hat{\mathbf{x}}_{t+F}) \in \mathbb{R}^{F \times N}$ ，其表达式为：

$$\hat{\mathbf{X}} = \mathcal{M}_\Theta(\mathbf{X}; \mathcal{S}), \quad (248)$$

其中， Θ 为 LLM 的预训练参数， $\mathcal{S}$ 表示提示或参数高效微调策略，用于引导模型适配特定任务。

图 7.3展示了基于 LLM 的时间序列预测通用框架，其核心是通过语言建模范式重构时间序列预测任务。该流程包含三个关键阶段：1) 时间序列数据语言化，将连续时间序列转化为离散符号（如数值字符串，被切分为子序列的表征等），使 LLM 能够直接处理原始时间序列数据；2) 多模态输入构造，融合时间序列符号与自然语言指令（如"预测未来 24 小时负荷"），形成包含任务语义的增强输入（可选步骤）；3) LLM 微调，通过选择部分 LLM 主干参数或额外添加的模型组件进行微调，实现 LLM 在时间序列各种下游任务上的应用。

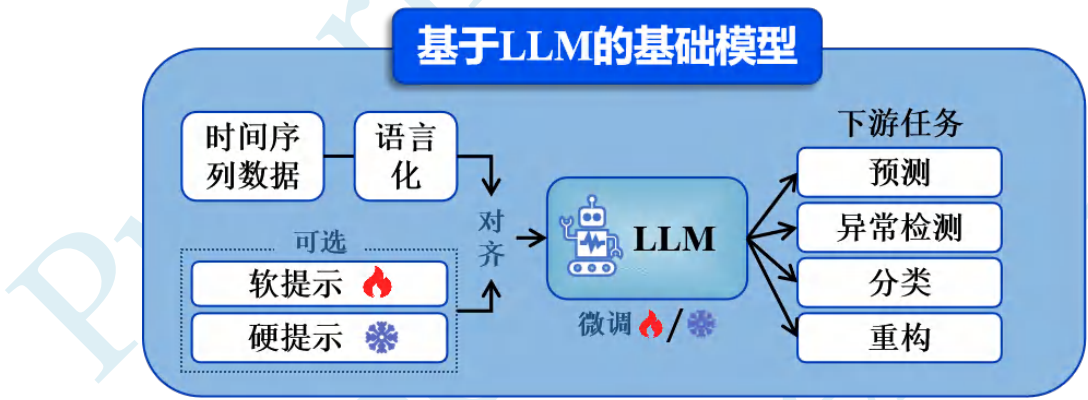


图 7.3: 基于 LLM 的时间序列模型的流程

7.2.2 模型训练及评估

- 1. **训练阶段：**与第 3 至第 6 章节中端到端时间序列模型的训练方式不同，基于 LLM 的时间序列基础模型通常采用“**预训练-微调**”的两阶段策略，以实现从通用知识到任务知识的迁移。具体而言，**预训练阶段**通过在大规模文本或多模态数据上进行自监督学习，训练得到具备通用序列建模与语义理解能力的基础模型，例如 GPT-2 [13]、LLaMA [17] 等；而在**微调阶段**，则通过多样化的技术，将预训练模型进一步调整以适应时间序列的特定下游任务。
 - (a) **预训练阶段：**LLM 的预训练通常依赖大规模文本或多模态数据，采用自回归建模（Autoregressive Modeling）或掩码建模（Masked Modeling）等目标，使模型能够学习通用的语言或序列表征。
 - (b) **微调阶段：**在下游任务中，通过以下三类方式进行高效迁移：
 - **提示词设计（Prompt Design）：**通过人工或自动化方式设计与数据特征、任务目标相关的提示（prompt），引导预训练模型执行特定的时间序列

任务。提示可显式描述预测目标、时间范围或数据属性，从而充分利用 LLM 的语义理解与推理能力 [24, 26]。

- 模态对齐 (Modality Alignment): 由于时间序列数据与语言文本在表示空间、结构特征及语义形式上存在显著差异，模态对齐旨在构建从时间序列到语言表示的映射关系。常见做法包括将时间序列编码为离散符号、语义嵌入或特征 token，以实现与 LLM 输入空间的统一，从而促进跨模态知识迁移与理解 [27, 8]。
- 参数高效微调 (Parameter-efficient Fine-Tuning, PEFT): 通过在冻结大部分预训练参数的基础上，仅引入少量可学习模块（如可学习提示、LoRA 等）实现任务适配。该策略显著降低训练和存储开销，同时保持甚至提升模型在时间序列任务中的泛化性能 [2, 9]。

2. **评估阶段**：与第 3 至 6 章节中端到端模型的评估方式类似，基于 LLM 的时间序列基础模型的评估过程同样使用 MSE, MAE, Accuracy, Recall, Hit@k, Average Rank 等评估指标来评估模型性能。更重要的是，基于 LLM 的时间序列基础模型更关注模型的泛化性和效率：

- (a) 多任务评估：在预测、分类、异常检测等任务上验证模型的通用性。
- (b) 跨领域评估：在金融、医疗、交通、气象等不同领域数据集上测试模型的泛化能力。
- (c) 效率评估：从参数规模或训练代价（通常用浮点运算数 FLOPs 或训练总计算量来衡量）等角度衡量模型的实用性。

7.2.3 模型关键技术

随着 LLM 的快速发展，其强大的语义理解与逻辑推理能力为时间序列建模带来了新的研究范式。为了充分发挥 LLM 在通用知识建模和跨模态理解方面的潜力，时间序列基础模型通常依赖三项关键技术：提示词设计 (Prompt Design)、模态对齐 (Modality Alignment) 与参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT)。当前模型所使用的具体关键技术如表 7.2。

1. **提示词设计 (Prompt Design)**：提示词工程决定了时间序列知识如何被注入或提取，是连接任务需求与 LLM 推理能力的关键桥梁。此类方法通过精心设计的文本模板或上下文构造策略，引导模型理解时间序列的输入语义与预测目标。例如 PromptCast [24] 使用自然语言模板显式描述输入，使 LLM 能在无需微调的情况下直接进行预测任务；而 ReasonTSC [26] 一类的工作，将数据集描述、时序的统计信息等额外信息加入上下文来帮助 LLM 进行理解。

表 7.2: 基于 LLM 的时间序列基础模型的技术总结

模型	年份	提示词设计	模态对齐	参数高效微调	技术细节
GPT4TS [27]	2023	-	✓	✓	可学习映射 + 部分参数更新
S ² IP-LLM [12]	2024	✓	✓	✓	分解驱动的多尺度融合 + 部分参数更新 + 前缀微调
TEMPO [2]	2024	-	✓	✓	分解驱动的多尺度融 + Prompt 微调 + LoRA
Time-FFM [9]	2024	-	✓	✓	可学习映射 + 前缀微调
PromptCast [24]	2024	✓	-	-	提示词设计 + 完全文本化
Time-LLM [8]	2024	✓	✓	✓	提示词设计 + 可学习映射 + 部分参数更新
TEST [16]	2024	-	✓	-	显示对齐
ReasonTSC [26]	2025	✓	✓	-	提示词设计 + 完全文本化
ChatTS [23]	2025	✓	✓	-	提示词设计 + 显示对齐
ITFormer [21]	2025	✓	✓	✓	提示词设计 + 显示对齐 + 部分参数更新
TimeMaster [25]	2025	✓	✓	-	提示词设计 + 显示对齐

2. **模态对齐 (Modality Alignment)**: 时间序列数值空间与语言模型语义空间之间存在鸿沟，其会削弱 LLM 对于时序任务的理解推理能力。因此，需要模态对齐来弥合时间序列数值空间与语言模型语义空间之间的鸿沟，使 LLM 能够理解并推理连续时间模式。现有工作主要从文本化、可学习映射、显示对齐、分解融合等角度展开。

- (a) **完全文本化 (Textualization)**: 这类方法直接将数值序列转换为自然语言，最小化模态转换成本，是时间序列文本化的早期探索。例如 PromptCast [24] 首次提出基于提示的时间序列预测范式，将序列完全文本化，以适配 LLM 的输入输出格式来让 LLM 适配时序任务。
- (b) **可学习映射 (Mapping)**: 该方向通过在 LLM 前端插入可学习的映射层，以实现时间序列和文本的语义对齐。例如 GPT4TS [27] 直接将时间序列数据分块变为时序 Token 后通过映射层映射到文本语义空间，从而进行推理；而 Time-LLM [8] 在引入映射层将数值序列映射到文本空间的基础上，结合任务的文本描述来执行任务。
- (c) **显示对齐 (Explicit Alignment)**: 此类方法通过在 LLM 的语义空间中设置若干“参照点”，引导时间序列的表示向这些“参照点”的语义方向对齐，从而让两种模态共享一致的表示空间。例如 TEST [16] 提出文本对齐嵌入机制，通过对比学习引导时序 Token 与文本嵌入空间对齐，避免嵌入漂移的同时也激活了 LLM 的结构化推理能力；S²IP-LLM [12] 则在时间序列分解后引入语义参照点，利用 LLM 词向量作为对齐参照，引导趋势、季节性、残差分量的语义一致嵌入。
- (d) **分解驱动的多尺度融合 (Decomposition-Driven Fusion)**: 这类工作通过显示分解时间序列的趋势、季节性、残差等模式特征，来提升时序语义可

分性与模态对齐精度。例如 TEMPO [2] 将时间序列分解为独立分量，并以 GPT-2 [13] 为主干进行分量级预测，利用基于检索的提示实现语义层面对齐，同时提升可解释性；S²IP-LLM [12] 进一步将分解信息与语义对齐模块融合，实现语义空间与时间结构的双重一致性。

- (e) **原生时序模态对齐 (Native Time Series Modality Alignment)**: 该类方法保留专用时序编码器或多模态融合模块，将时间序列作为独立模态输入，从根本上扩展 LLM 的模态边界。例如 ChatTS [23] 将时间序列视为原生模态，通过专用时序编码器与 LLM 语义空间对齐，并使用合成时序-文本数据进行多模态指令微调；ITFormer [21] 则通过轻量跨模态融合模块连接时间序列编码器与 LLM；TimeMaster [25] 将时序模态变为图像模态并引入强化学习框架，通过复合奖励函数优化时序推理的准确性与可解释性，实现模态对齐。

3. 参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT): 旨在在保持大部分预训练模型参数冻结的前提下，仅调整少量或新增的可训练参数。该策略显著降低了微调阶段的计算与存储开销，同时在时间序列任务中通常能获得与全量微调相当的性能。更重要的是，部分 PEFT 方法在跨域或分布偏移场景下表现出更强的泛化能力，优于传统的全量微调方式。在时间序列领域，PEFT 技术主要可以分为以下三大类

- (a) **部分参数更新 (Partial Parameter Update)**: 指在微调过程中，仅对预训练模型中部分参数进行更新，而将其他参数保持冻结，从而在降低计算成本的同时实现任务适配。例如，GPT4TS [27] 和 S²IP-LLM [12] 仅更新归一化层与位置编码层，以适应时间序列数据的分布特性和时序结构，同时保持预训练语言模型的通用表征能力。
- (b) **结构式注入 (Structural Injection)**: 在冻结主干网络的同时，插入轻量级的可训练结构，通过局部参数学习实现任务特化。

- **Prompt 微调 (Prompt Tuning)** 是一种通过设计和优化输入提示(prompt)来引导预训练模型生成特定任务输出的策略。与直接修改模型参数不同，Prompt 微调通过对输入空间的轻量调整，实现任务适配与知识迁移。TEMPO [2] 引入了一种半软提示 (semi-soft prompting) 策略，旨在充分利用时间序列中蕴含的丰富语义信息。该方法为每个主要的时间序列成分 (趋势、季节性和残差) 分别生成独立的提示表征，从而实现更细粒度的建模。为了进一步提升模型对非平稳性和分布偏移的适应能力，TEMPO [2] 设计了提示池 (prompt pool) 机制。提示池包含多个软提示向量，模型可根据输入序列的模式或任务需求动态选择或组合提示，以增强泛化能力。

- **前缀微调 (Prefix Tuning)** 通过在输入序列或中间层前添加可学习的前缀向量，引导预训练模型产生符合特定任务需求的输出。例如，**S²IP-LLM** [12] 通过一个通用映射函数从 LLM 的词嵌入中提取少量关键语义锚点 (semantic anchors)，并利用余弦相似度将这些锚点与时间序列嵌入对齐，从中选取前 K 个最相关的锚点作为前缀提示，以强化时间序列表示；而 **Time-FFM** [9] 则将前缀设计为领域指令 (domain instructions)，让模型基于 LLM 的语义理解自动选择最优领域提示，从而注入领域知识，提升模型在特定任务中的可解释性与泛化性能。
- (c) **低秩重参数化 (Low-Rank Reparameterization)**：采用如 LoRA 等方法，将大矩阵分解为低秩近似，仅训练其中的小规模参数，从而在不显著增加计算成本的情况下实现高效适配。例如，在 **TEMPO** [2] 中，LoRA 被应用于 Transformer 架构的关键权重矩阵，允许模型在不同任务和数据上进行快速调整。

参考文献

- [1] Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Türkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke Schneider, and Bernie Wang. Chronos: Learning the language of time series. *Trans. Mach. Learn. Res.*, 2024.
- [2] Defu Cao, Furong Jia, Sercan Ö. Arik, Tomas Pfister, Yixiang Zheng, Wen Ye, and Yan Liu. TEMPO: prompt-based generative pre-trained transformer for time series forecasting. In *ICLR*, 2024.
- [3] Mouxiang Chen, Lefei Shen, Zhuo Li, Xiaoyun Joy Wang, Jianling Sun, and Chenghao Liu. Visionts: Visual masked autoencoders are free-lunch zero-shot time series forecasters. 2025.
- [4] Yuxuan Chen, Shanshan Huang, Yunyao Cheng, Peng Chen, Zhongwen Rao, Yang Shu, Bin Yang, Lujia Pan, and Chenjuan Guo. Aims: Augmented series and image contrastive learning for time series classification. In *ICDE*, 2025.
- [5] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.
- [6] Vijay Ekambaram, Arindam Jati, Pankaj Dayama, Sumanta Mukherjee, Nam Nguyen, Wesley M. Gifford, Chandra Reddy, and Jayant Kalagnanam. Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. In *NeurIPS*, 2024.
- [7] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. MOMENT: A family of open time-series foundation models. In *ICML*, 2024.
- [8] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-llm: Time series forecasting by reprogramming large language models. In *ICLR*, 2024.
- [9] Qingxiang Liu, Xu Liu, Chenghao Liu, Qingsong Wen, and Yuxuan Liang. Time-ffm: Towards llm-empowered federated foundation model for time series fore-

- casting. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *NeurIPS*, 2024.
- [10] Yong Liu, Guo Qin, Zhiyuan Shi, Zhi Chen, Caiyin Yang, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Sundial: A family of highly capable time series foundation models. 2025.
- [11] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. In *ICML*, 2024.
- [12] Zijie Pan, Yushan Jiang, Sahil Garg, Anderson Schneider, Yuriy Nevmyvaka, and Dongjin Song. S2IP-LLM: semantic space informed prompt learning with LLM for time series forecasting. In *ICML*, 2024.
- [13] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, page 9, 2019.
- [14] Qichao Shentu, Beibu Li, Kai Zhao, Yang Shu, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. Towards a general time series anomaly detector with adaptive bottlenecks and dual adversarial decoders. In *ICLR*, 2025.
- [15] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-moe: Billion-scale time series foundation models with mixture of experts. In *ICLR*, 2025.
- [16] Chenxi Sun, Hongyan Li, Yaliang Li, and Shenda Hong. TEST: text prototype aligned embedding to activate llm’s ability for time series. In *ICLR*, 2024.
- [17] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schel-

- ten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023.
- [18] Wenxuan Wang, Kai Wu, Yujian Betterest Li, Dan Wang, Xiaoyu Zhang, and Jing Liu. Mitigating data scarcity in time series analysis: A foundation model with series-symbol data generation. 2025.
- [19] Yihang Wang, Yuying Qiu, Peng Chen, Yang Shu, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. Lightgts: A lightweight general time series forecasting model. 2025.
- [20] Yihang Wang, Yuying Qiu, Peng Chen, Kai Zhao, Yang Shu, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. Towards a general time series forecasting model with unified representation and adaptive transfer. In *ICML*, 2025.
- [21] Yilin Wang, Peixuan Lei, Jie Song, Yuzhe Hao, Tao Chen, Yuxuan Zhang, Lei Jia, Yuanxiang Li, and Zhongyu Wei. Itformer: Bridging time series and natural language for multi-modal QA with large-scale multitask dataset. In *ICML*, 2025.
- [22] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *ICML*, 2024.
- [23] Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. Chatts: Aligning time series with llms via synthetic data for enhanced understanding and reasoning. *Proc. VLDB Endow.*, pages 2385–2398, 2025.
- [24] Hao Xue and Flora D. Salim. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE Trans. Knowl. Data Eng.*, 36(11):6851–6864, 2024.
- [25] Junru Zhang, Lang Feng, Xu Guo, Yuhan Wu, Yabo Dong, and Duanqing Xu. Timemaster: Training time-series multimodal llms to reason via reinforcement learning. *CoRR*, 2025.

- [26] Jiahui Zhou, Dan Li, Lin Li, Zhuomin Chen, Shunyu Wu, Haozheng Ye, Jian Lou, and Costas J. Spanos. Enhancing LLM reasoning for time series classification by tailored thinking and fused decision. *CoRR*, 2025.
- [27] Tian Zhou, Peisong Niu, Xue Wang, Liang Sun, and Rong Jin. One fits all: Power general time series analysis by pretrained LM. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *NeurIPS*, 2023.

Preprint
Decision
Intelligence
Lab

8 时间序列评测基准

8.1-8.2节作者为：邱翔飞、胡吉林、杨彬；

8.3与8.6节作者为：李哲、邱翔飞、胡吉林、杨彬；

8.4-8.5与8.7节作者为：邱翔飞、李哲、胡吉林、杨彬。

本章主编为胡吉林。

在人工智能研究中，测评基准（Benchmark）扮演着至关重要的角色。它不仅是算法性能比较的共同语言，更是推动学术研究标准化与产业落地化的核心动力。从图像识别到自然语言处理，标准化的 Benchmark 体系促进了领域的快速迭代与技术积累，使得研究成果能够被客观、公正地评估与复现。

然而，在时间序列分析（Time Series Analysis）这一广泛应用于金融、工业、医疗、气象等领域的重要方向中，长期以来却缺乏统一、系统且公正的测评基准体系。不同研究工作往往使用不一致的数据集处理方法、实验设置与评估策略，使得算法间的比较缺乏可重复性与公平性。在一定程度上阻碍了时间序列领域整体的发展。

本章的结构安排如下：在第 8.1 节，我们首先介绍测评基准的背景与意义。接着在第 8.2 节，我们回顾时间序列领域 Benchmark 的发展脉络。随后，在第 8.3 节中，我们将介绍时间序列 Benchmark 的一般测评框架，分析其核心模块，包括数据层、方法层、评估层与报告层。随后，第 8.4 节详细介绍数据层的构建过程，第 8.5 节说明方法层的设计原则，第 8.6 节阐述评估层的评价策略与评价指标，第 8.7 节介绍报告层的组织与展示方式。通过本章的系统论述，读者将全面了解如何设计、组织与实现一个系统化、可复现且公平的时间序列测评基准。

8.1 测评基准的背景与意义

背景：在深度学习与机器学习领域，Benchmark（测评基准）是用来评估、比较和验证不同算法、模型或系统性能的标准化工具。Benchmark 通常包括预定义的数据集、基线方法、评估指标、评估策略与结果报告，旨在为研究人员、开发人员及从业者提供一个统一的测评平台，以确保不同方法之间的可比性。通过基准测试，研究者能够判断新提出的算法或模型在特定任务中的表现，并与现有的最优方法进行比较，从而推动技术的进步。

意义：在深度学习这一快速发展的领域，每年都会涌现出大量新算法和新技术。然而，在这些多样且迅速的创新背后，如何有效比较新方法的优劣，避免过度的理论探讨和空泛宣传，成为一个难题。而 Benchmark 的出现恰好解决了这一问题。Benchmark 往往有以下的研究意义：

- **客观评估：**Benchmark 通过提供标准化的测试数据集和评估指标、评估策略，为每种新方法创造了相同的实验条件，从而确保研究结果的客观性和可重复性。借助 Benchmark，研究人员能够清晰了解某一模型在标准数据集上的表现，并基于这些结果作出相对准确的技术判断。可以说，Benchmark 为技术发展搭建了一个竞争的舞台，而技术的进步往往正是源于对现有方法的挑战与超越。
- **促进竞争：**通过设定标准任务和明确的评估标准，Benchmark 激励研究者不断优化算法、提升模型性能。这种竞争机制推动了技术的快速迭代与创新。例如，在

计算机视觉领域，ImageNet [5] 的推出激发了大量深度学习模型的发展与优化，直接促成了卷积神经网络（CNN）等技术的重大突破。类似地，在图表示学习（Graph Representation Learning）领域，Open Graph Benchmark (OGB [7]) 的建立为图神经网络（GNN）提供了系统化的测评平台，推动了从模型结构设计到泛化性能验证的标准化研究。这种“竞赛式”的模式显著加速了学术界的进步。

- 识别优缺点: 与此同时, **Benchmark** 还为实际应用提供了重要参考。工业界的企业和开发者常常面临如何选择适合自身需求的技术和模型的问题。通过 **Benchmark** 的标准化测评, 他们可以直观地比较不同模型的性能表现, 从而选择最优解决方案。这使得 **Benchmark** 不仅在学术研究中具有重要意义, 也在产业实践中展现了巨大价值。
- 行业标准: 此外, **Benchmark** 对标准化和透明性的推动作用不可忽视。在科技领域, 标准化是保证技术可复现性和可推广性的关键。公开的 **Benchmark** 使研究者能够验证他人的工作, 并基于相同的标准重新实验。这种公开、透明的测评方式有助于消除研究之间的偏差, 提升了科学研究的可信度。同时, 它还能帮助研究者发现模型和算法的潜在不足, 进一步推动学术界的持续进步。

然而, 随着深度学习和机器学习领域的不断演进, 传统 **Benchmark** 有时会暴露出一些问题, 例如任务设定过于单一或测评环境过于理想化。因此, **Benchmark** 的更新与创新显得尤为重要。新 **Benchmark** 任务的设计往往反映了技术发展的新需求, 引导研究者探索更具挑战性的解决方案。随着技术的复杂化和应用场景的多样化, **Benchmark** 的多样性和综合性也在不断增强, 使其能够更好地适应复杂多变的实际需求, 从而持续推动技术的进步。

8.2 时间序列测评基准的发展脉络

在理解了 **Benchmark** 的概念及其在人工智能研究中的重要作用之后, 本节将聚焦于时间序列分析领域 (重点是广受关注的时间序列预测以及异常检测领域) 的 **Benchmark** 体系, 回顾其从萌芽到体系化发展的演进过程。

2002 年, E. Keogh 首次介绍了 UCR 时间序列分类数据挖掘档案 [4]。该档案包含超过 128 个数据集, 覆盖金融、医学、自然科学等领域, 并提供了元数据、预处理代码和相关论文等资源, 推动了时间序列数据挖掘的研究。

2018 年, Anthony Bagnall 教授领导创建了 UEA 多元时间序列分类数据集 [2]。该数据集包含 80 个数据集和数万个时间序列实例, 覆盖运动传感器、医疗监测等多个领域, 目标是推动时间序列分类算法的比较与评估。

同年, Vincent Jacob 教授团队推出了 Exathlon [8], 一个专注于时间序列异常检测可解释性的基准框架。该框架通过多种数据集和评估指标, 帮助研究人员客观评估

表 8.1: 时间线 (Benchmark 发展脉络)

2002	《The UCR time series archive》(JAS) [4]
2018	《The UEA multivariate time series classification archive, 2018》(arXiv) [2]
2021	《Exathlon: A Benchmark for Explainable Anomaly Detection over Time Series》(PVLDB) [8]
2021	《Monash Time Series Forecasting Archive》(NeurIPS) [6]
2021	《Libra: A Benchmark for Time Series Forecasting Methods》(ACM/SPEC) [3]
2021	《Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress》(TKDE) [18]
2021	《Revisiting Time Series Outlier Detection: Definitions and Benchmarks》(NeurIPS) [9]
2022	《TSB-UAD: An End-to-End Benchmark Suite for Univariate Time-Series Anomaly Detection》(PVLDB) [12]
2024	《The Elephant in the Room: Towards A Reliable Time-Series Anomaly Detection Benchmark》(NeurIPS) [11]
2024	《TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods》(PVLDB 最佳研究论文奖提名) [13]
2024	《GIFT-Eval: A Benchmark For General Time Series Forecasting Model Evaluation》(arXiv) [1]
2024	《ProbTS: Benchmarking Point and Distributional Forecasting across Diverse Prediction Horizons》(NeurIPS) [19]
2025	《TSFM-Bench: A Comprehensive and Unified Benchmark of Foundation Models for Time Series Forecasting》(KDD) [10]
2025	《Exploring progress in multivariate time series forecasting: Comprehensive benchmarking and heterogeneity analysis》(TKDE) [15]
2025	《TAB: Unified Benchmarking of Time Series Anomaly Detection Methods》(PVLDB) [14]
2025	《Deep Time Series Models: A Comprehensive Survey and Benchmark》(TPAMI) [17]
2025	《fev-bench: A Realistic Benchmark for Time Series Forecasting》(arXiv) [16]

算法性能和可解释性。

2021 年, Rob Hyndman 教授创建了 Monash Time Series Forecasting Archive [6]。该数据集涵盖金融、环境等多个领域的时间序列数据, 用于测试和比较不同预测算法的性能, 成为时间序列预测研究的重要参考。

同样在 2021 年, IBM 研究人员发布了 Libra [3], 一个用于评估时间序列预测方法的基准数据集。它包含来自金融、物流等领域的时间序列数据, 具有多样性和复杂性, 为算法比较提供了统一平台。

此外, 2021 年的文献 [18] 指出当前时间序列异常检测基准存在缺陷, 呼吁开发更具代表性的数据集和新的评估指标, 以更准确地反映算法的实际表现。

另一项研究于 2021 年由 Lai 等人提出 [9], 该工作重新审视并定义了时间序列异常检测 (Time Series Anomaly Detection, TSAD) 中的“异常”概念。同时, 作者构建并发布了一组广泛应用的基准数据集 (Benchmark Datasets), 为后续研究提供了统一的实验平台, 用于系统评估不同算法的性能。

2022 年, TSB-UAD [12] 发布, 这是一个专为单变量时间序列异常检测设计的端到端基准套件。包含 18 个单变量异常检测数据集, 覆盖网络、环境等领域, 并提供开源实现和统一评估框架, 助力算法开发与测试。

2024 年, TSB-AD [11] 正式发布。该工作在此前 TSB-UAD [12] 的基础上进行了重要扩展, 不仅覆盖了多变量时间序列异常检测任务 (Multivariate Time Series Anomaly Detection), 还系统化地支持了时间序列异常检测基础模型的测评与比较。

2024 年, 华东师范大学决策智能实验室正式发布了 TFB [13]。该工作旨在构建一个全面、公平且可复现的时间序列预测测评体系, 系统解决以往研究中存在的数据不一致、评估标准不统一及实验复现困难等问题。TFB 整合了来自 10 个不同行业领域的 25 个真实世界数据集, 覆盖能源、交通、气象、经济、医疗等多个应用场景, 并统一了数据预处理流程、任务划分方式与性能度量指标。

2024 年, 来自 Salesforce、新加坡国立大学的研究者引入了 GIFT-Eval [1] 这一新的综合基准, 旨在评估通用的时间序列预测能力, 特别针对时序预测基础模型。该基准引入了一个多样化的数据集集合, 其中包括 28 个数据集, 涵盖 144,000 个时间序列, 总共包括 1.77 亿个数据点。GIFT-Eval 的数据覆盖了 7 个不同领域, 10 种频率, 单/多变量输入, 预测长度从短期到长期不等。

2024 年, 微软亚洲研究院发布 ProbTS [19], 首个统一评估点预测与分布预测的时序基准框架。它通过量化趋势强度、季节性和非高斯性等数据特性, 揭示了自回归与非自回归解码方案在不同预测场景中的性能边界, 并为时间序列基础模型的零样本能力评估提供了标准化平台。

2025 年, 华东师范大学决策智能实验室正式发布了 TSFM-Bench [10]。该工作针对近年来迅速兴起的时间序列预测基础模型进行系统化测评。TSFM-Bench 收录了

多种主流时间序列基础模型，涵盖了基于大型语言模型与预训练机制的模型架构。该基准支持多种预测情景，包括零样本（Zero-shot）、少样本（Few-shot）与全样本（Full-shot）设置。

2025 年，中科院团队发布 BasicTS+ [15]，构建了统一的训练流程，有效解决了因数据处理、训练配置和评估实现差异导致的性能不一致问题。在数据处理环节，默认采用 z-score 归一化，并添加外部时间特征；训练流程中，统一使用 masked MAE 作为损失函数，并集成了课程学习和梯度裁剪等训练技巧；评估组件则提供了 MAE、RMSE、MAPE、WAPE 等多种指标的标准实现。

2025 年，华东师范大学决策智能实验室正式发布了 TAB [14]。TAB 包含 29 个公共多元数据集和 1635 个来自不同领域的单变量时间序列，便于对不同数据集进行更全面的评估。在方法层面，TAB 涵盖了各种时间序列异常检测方法，包括非学习、机器学习、深度学习、基于 llm 的方法和时序预训练方法。此外，TAB 具有统一的自动化评估框架，可以公平、轻松地对时间序列异常检测方法，进行评估。

2025 年，清华团队发布 TSLib [17]，首个统一评估点预测与分布预测的时序基准框架。它通过量化趋势强度、季节性和非高斯性等数据特性，揭示了自回归与非自回归解码方案在不同预测场景中的性能边界，并为时间序列基础模型的零样本能力评估提供了标准化平台。

2025 年，fev-bench [16] 是一个融合了协变量、多变量任务以及基于统计原则的结果聚合方法的时间序列预测基准。通过在多个领域中实现具有统计意义的公平比较，fev-bench 为推动预训练预测模型的发展提供了可靠基础，并能够系统地评估这些模型在处理现实世界需求（如协变量建模与多变量结构）方面的能力。

8.3 时间序列测评基准框架

随着时间序列分析任务在预测、分类、异常检测等领域中的广泛应用，模型设计与算法改进呈现爆发式增长。然而，不同研究常使用各自的数据集、预处理方式与评估标准，导致实验结果难以复现和横向比较。因此，建立一个系统化、可扩展、可复现的时间序列测评基准框架（Benchmark），成为推动该领域标准化和持续进步的重要基础 [13, 19, 11]。该框架的核心目标在于：

- 提供统一的数据，实现不同时间序列数据的标准化管理；
- 支撑多类型方法的灵活调用与比较，涵盖统计学习、机器学习与深度学习模型；
- 建立可量化的评价体系，确保模型性能评估的客观性与一致性；
- 形成可追踪的实验报告与可视化输出。

由于在时间序列领域中，**预测 (Forecasting)** 与 **异常检测 (Anomaly Detection)** 任务最受关注，因此在此后的小节中，我们将重点介绍这两类任务的测评基准框架设计与实现细节。

如图 8.1 所示，对于时间序列来说，无论是预测还是异常检测任务，整个时间序列测评基准由四个核心模块组成：**数据层 (Data Layer)**、**方法层 (Model Layer)**、**评估层 (Evaluation Layer)** 和 **报告层 (Reporting Layer)**。这四个层次共同构成了一个自下而上、可复现的测评体系结构。每一层都承担不同的功能模块，彼此通过标准化接口进行信息交互，从而实现时间序列建模、评估与结果呈现的一体化流程。

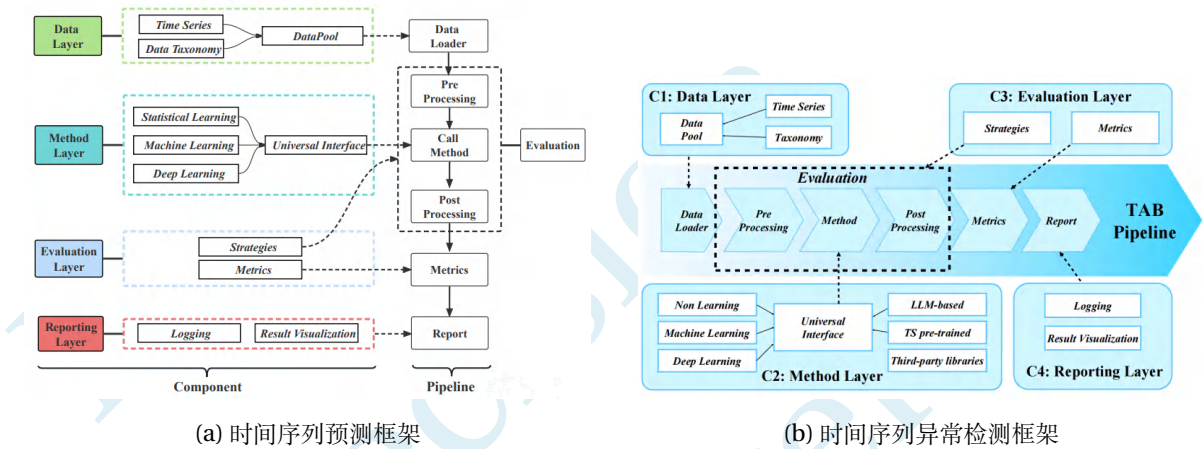


图 8.1: 时间序列测评基准框架

- **数据层 (Data Layer)** : 数据层是整个框架的基础，负责时间序列数据的组织、管理与标准化，为后续测评提供可靠的输入。在此层中，系统首先从不同领域（如气象、交通、金融、医疗等）收集原始时间序列数据，并通过数据分类体系 (Data Taxonomy) 进行结构化管理，形成统一的数据池 (DataPool)。此外，数据层还负责数据预处理与划分，包括缺失值填充、异常检测、归一化及窗口切分等操作。最终，数据按照固定比例划分为训练集、验证集与测试集，确保评估的可重复性与公平性。本书将在第 8.4 节对数据层的构建进行详细介绍。
- **方法层 (Model Layer)** : 方法层定义了模型调用与训练的标准接口，是连接数据与算法的核心桥梁。该层支持多种时间序列建模范式，包括统计学习 (Statistical Learning)、机器学习 (Machine Learning) 与深度学习 (Deep Learning)。通过设计统一的通用接口 (Universal Interface)，研究者可以将不同模型无缝集成到同一评估框架中，而无需修改数据加载或评估逻辑。此设计不仅提高了实验复现性，也为大规模模型比较与自动化实验提供了灵活支持。本书将在第 8.5 节对方法层的构建进行详细介绍。

- **评估层 (Evaluation Layer)**：评估层是整个框架中衡量模型性能的关键部分。它定义了一系列评估策略 (Strategies) 与评价指标 (Metrics)，例如 MAE、RMSE、MAPE、CRPS 等，用以全面刻画模型在不同场景下的预测精度、稳定性与泛化能力。评估层会接收来自方法层的模型输出，并与真实值进行比较，从而生成标准化的性能统计结果。该层的设计保证了模型评估过程的客观性和一致性。本书将在第 8.6 节对评估层的构建进行详细介绍。
- **报告层 (Reporting Layer)**：报告层是测评框架的输出端，负责结果记录、可视化与分析。其主要包括两个功能模块：日志记录 (Logging) 与结果可视化 (Result Visualization)。日志模块用于追踪实验的元信息（如数据版本、模型配置、训练过程等），确保实验可追溯；可视化模块则通过曲线图、表格等形式展示模型表现。最终，生成测评结果排行榜 (Leaderboard)，为模型性能提供定量分析。本书将在第 8.7 节对报告层的构建进行详细介绍。

8.4 数据层

在时间序列测评基准 (Benchmark) 中，数据层 (Data Layer) 是整个体系的基础与核心。一个高质量的时间序列基准不仅需要涵盖多领域、多种特征的数据集，还应保证数据的真实性、代表性与可比性。数据层的设计直接决定了测评结果的可靠性，同时影响算法在不同场景下的泛化表现。

通常，构建时间序列 Benchmark 的数据层主要包括三个阶段：

- **数据集的收集**：从多领域、多来源系统性地汇聚原始时序数据；
- **数据集的筛选**：通过标准化流程筛除质量不高或分布异常的数据集；
- **数据集的划分**：依据统一协议完成训练、验证与测试集的比例划分，以支持可复现的实验。

接下来将分别介绍这三个关键环节。

8.4.1 时序数据集收集

数据集收集一般有以下几种渠道：

- **开源数据集的收集汇总**：研究者通常会从已有文献、竞赛平台或其他研究者的开源项目中获取数据。一方面，许多学术论文在实验部分选取了具有代表性的数据集，这些数据集经过整理与复现，可为新的研究提供坚实的基础。另一方面，来自各类竞赛平台的时间序列数据也极具研究价值，例如 M-competition、KDD Cup、Santa Fe Competition、Kaggle 时间序列竞赛、Neural Network Competition、

Global Energy Forecasting Competition 等，它们涵盖了从宏观经济到电力负荷预测等多个领域。此外，一些企业或研究机构也出于科研促进目的开放了高质量数据集。例如，云智慧（北京）科技有限公司发布的 GAIA 数据集，为智能运维领域的时间序列异常检测研究提供了重要支撑。

- 人工合成数据集: 人工合成的时间序列数据可以基于真实场景或特定异常模式构造。例如，研究者可以在真实的传感器数据中人为注入故障信号或缺失片段，以模拟工业设备故障或传输异常；也可以针对突变、趋势漂移、周期性扰动等特定类型异常生成序列，从而研究模型对不同异常类型的敏感性。
- 真实或模拟环境产生的数据集: 这类数据往往通过传感器网络或实验平台在真实环境中采集。例如 PEMS（Performance Measurement System）交通数据集的创建过程包括多个步骤：在道路网络不同节点安装微波、红外或电感式传感器，以获取车辆速度、流量、占有率等指标；通过采集器实时收集数据并传输至中心数据库；随后对数据进行清洗、去重和插值等处理，确保其准确性与完整性；最终，整理后的数据被存储并向公众开放，为交通流预测与管理研究提供支撑。
- 人工改动数据集: 这类数据来源于已有数据集的重新加工与任务转化。例如，研究者可以将时间序列分类数据集中的部分类别标注改为“异常”，构造出时间序列异常检测任务；或者将图像类数据（如心电图波形图像）转化为一维时间序列信号，通过标注关键特征点来构建新的分类或检测数据集。这类改动通常用于扩展任务边界、验证模型的迁移能力或提升特定领域的样本多样性。
- 混合类型: 混合类型指的是数据集由以上几种情况混合组成。

8.4.2 时序数据集筛选

数据集的筛选旨在确保 Benchmark 中的数据高质量且统计特性合理。在这一阶段，主要执行以下三个层次的筛选与规范化步骤：

- 完整性筛查：去除标签错误、严重采样不连续或含有异常噪声的数据。
- 均衡性调整：确保数据在各种特性（趋势性、周期性、平稳性、漂移性、转移性、相关性）上面分布均匀，而不是被某一种特性所主导。例如，TFB [13] 中采用 Hexbin 图（见图 8.2）对最终选取的单变量预测数据集进行可视化，展示其在由 PCA 生成的低维特征空间中，各特性（趋势、季节性、平稳性、移位与过渡）的归一化密度分布均匀，体现了样本在多维特征维度上的代表性和平衡性。

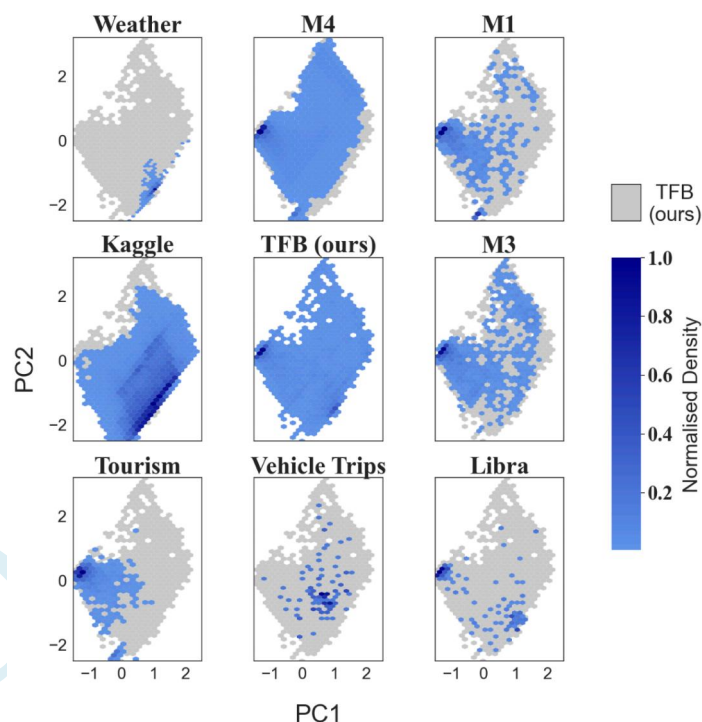


图 8.2: TFB 中的均衡性调整

- 合规性检查：此外，在 **Benchmark** 设计中还需考虑数据版权、隐私合规与许可问题，优先使用可公开共享的数据源，并明确其许可证（如 CC-BY、MIT 等），以保证学术研究的可复现性与合规性。

8.4.3 时序数据集划分

数据集划分是指将整个数据集分成训练集、验证集和测试集三个子集，用于不同的目的。训练集用于训练模型，验证集用于调整模型超参数和评估模型的泛化能力，测试集则用于最终评估模型的性能。

合理的时间序列数据划分应遵循以下原则：

- 时间一致性：时间序列划分应严格按照时间顺序进行，禁止打乱时间步，以避免信息泄漏；
- 比例统一：常见划分比例为训练集: 验证集: 测试集 = 7:1:2 或 6:2:2，应保持固定且一致的切分比例；
- 场景适配性：针对零样本 (Zero-shot)、少样本 (Few-shot) 与全样本 (Full-shot) 等不同实验场景，需提供多种划分策略；

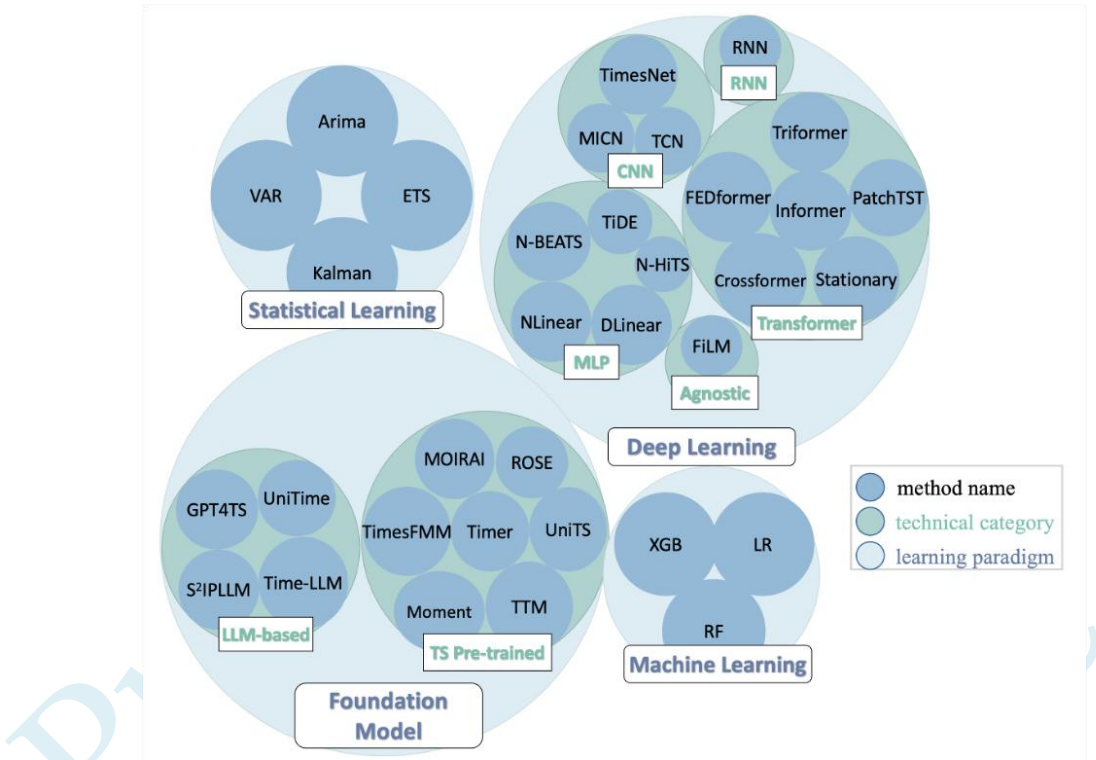


图 8.3: 时间序列预测方法分类

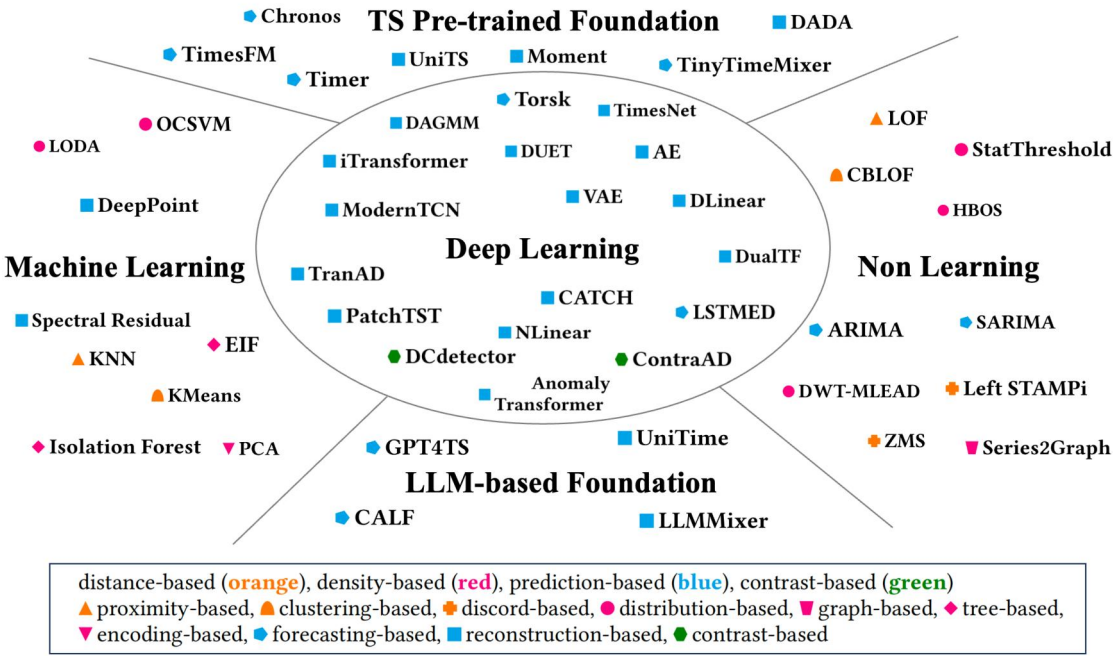


图 8.4: 时间序列异常检测方法分类

8.5 方法层

方法层 (Method Layer) 是时间序列评测基准 (Benchmark) 中最为核心的组成部分之一，它直接决定了模型评估的广度、深度与公平性。

在一个完整的时间序列 Benchmark 体系中，方法层通常包含从非学习算法到深度学习模型、再到预训练与大模型 (Foundation Models) 等多种方法类别。该层的设计目标不仅是对比模型性能，更重要的是揭示不同方法在数据特征、任务类型与场景复杂度上的适用边界，为模型选择与未来算法创新提供系统化依据。

8.5.1 设计原则

方法层的设计通常遵循以下三个基本原则：

- **全面性**: 评测基准应尽可能覆盖多种类型的方法，包括传统统计模型、机器学习方法、深度学习结构以及近期兴起的时间序列基础模型。这种全面性确保了 Benchmark 不仅服务于前沿研究，也能兼容传统基线，从而在横向与纵向维度上形成统一的比较框架。
- **公平性**: 不同算法在超参数调优策略、训练数据规模、模型容量等方面存在显著差异。方法层的设计需要通过统一的实验协议与标准化的训练流程来确保结果具有可比性。
- **可扩展性**: 随着时间序列研究的快速演化，新的架构 (Diffusion 模型、LLM 适配器等) 不断涌现。因此，方法层应具备良好的可扩展性，支持研究者在现有框架下快速添加新算法进行评估。

8.5.2 方法类别

方法类别是测评体系中核心的组成部分之一，它决定了整个基准所能覆盖的算法范畴与研究广度。时间序列分析方法的发展经历了从统计模型到机器学习模型，再到深度学习与预训练模型的演进，不同阶段的技术体现了对时间序列规律理解的不断深化与表达能力的逐步增强。

(1) 时间序列预测方法分类：

图 8.3 展示了 TFB [13] 中的时间序列预测方法分类情况。其总体可划分为四大类：统计学习 (Statistical Learning)、机器学习 (Machine Learning)、深度学习 (Deep Learning) 以及基础模型 (Foundation Model)。

- **统计学习 (Statistical Learning)** : 统计学习方法是时间序列分析的早期基石，依赖于显式建模与概率推断。代表性方法包括：ARIMA (自回归积分滑动平均模

型), VAR (向量自回归), Kalman Filter (卡尔曼滤波器), ETS (指数平滑模型) 等这类方法通常假设数据平稳性或线性可分, 具有较强的可解释性, 但在处理复杂非线性或多变量依赖结构时存在局限。

- 机器学习 (Machine Learning): 机器学习方法通过数据驱动方式替代显式统计假设, 提升了对复杂特征的拟合能力。代表方法包括: XGBoost (XGB), Random Forest (RF), Linear Regression (LR) 等, 但是这些方法仍然需要手动特征工程和模型设计。
- 深度学习 (Deep Learning): 深度学习已成为时间序列预测的主流方向, 其核心特征在于自动特征提取与端到端建模能力。根据模型架构可进一步细分为多个技术类别: RNN 系列: 包括 RNN、LSTM、GRU 等, 擅长捕捉时间依赖关系; CNN 系列: 如 TCN、MICN、TimesNet, 通过卷积操作提取局部时序模式; Transformer 系列: 如 Informer、FEDformer、PatchTST、Crossformer、Triformer 等, 依赖注意力机制以建模全局依赖; MLP 系列: 包括 NLinear、DLinear、N-BEATS、TiDE、N-HiTS 等轻量化架构, 强调简洁结构与快速收敛; Agnostic 类方法: 如 FiLM 等结构无关方法, 侧重通用的时间序列建模能力。
- 基础模型 (Foundation Model): 随着大模型范式的兴起, 时间序列领域出现了一系列基础模型, 可细分为两类: LLM-based 模型: 包括 GPT4TS、Time-LLM、S\$IIPLLM、UniTime 等, 将大型语言模型应用于时间序列理解与生成任务, 探索跨模态与跨领域泛化能力。TS Pre-trained 模型: 包括 MOIRAI、ROSE、TimesFMM、Timer、UniTS、TTM、Moment 等, 通过在大规模时间序列数据上进行自监督或多任务预训练, 增强模型的通用表示与下游适应能力。

(2) 时间序列异常检测方法分类:

图 8.4展示了 TAB [14] 中的时间序列异常检测方法分类情况。其总体可划分为四大类: 非学习范式 (Non Learning)、机器学习 (Machine Learning)、深度学习 (Deep Learning) 以及基础模型 (Foundation Model)。

- 非学习方法 (Non Learning): 非学习类方法主要依赖信号处理与统计分析原理, 无需模型训练, 具有较高的可解释性和低计算开销。代表方法包括: ARIMA、SARIMA: 基于残差统计建模; HBOS、StatThreshold、DWT-MLEAD: 基于分布或小波阈值检测; Left STAMPi、ZMS、Series2Graph: 通过形状相似性或图结构实现异常定位。这类方法适用于小样本与实时场景, 但难以捕获复杂的非线性或多通道时序模式。
- 机器学习方法 (Machine Learning): 机器学习类方法通过数据驱动的方式学习异常模式, 通常依赖特征提取与统计量构建。其典型代表包括: KNN、LOF、

CBLOF、KMeans：基于距离或密度的异常检测；Isolation Forest、EIF、OCSVM、PCA：基于树结构或降维空间的分布学习；Spectral Residual、DeepPoint：结合信号频谱与统计残差进行异常定位。这类方法在灵活性与泛化性上优于传统统计方法，但仍需人工设计特征，难以端到端优化。

- 深度学习方法 (Deep Learning)：深度学习方法已成为 TSAD 研究的主流方向，通过自编码、注意力或对比学习等机制实现端到端的异常模式学习。根据建模策略，可分为以下几类：预测型 (Prediction-based)：如 DLinear、NLinear、ModernTCN、iTransformer、TimesNet 等，通过预测误差检测异常；重构型 (Reconstruction-based)：如 DUET、CATCH、AE、VAE、DAGMM 等，通过重构误差判断异常；对比型 (Contrast-based)：如 DCdetector、ContraAD、CATCH、Anomaly Transformer，通过对比损失捕捉潜在异常分布；
- 基础模型 (Foundation Models)：随着大模型范式的兴起，时间序列领域出现了一系列基础模型，可细分为两类：LLM-based Foundation：如 GPT4TS、LLMMixer、UniTime、CALF，将大语言模型的通用理解能力迁移到时序异常检测任务；TS Pre-trained Foundation：如 TimesFM、Chronos、TinyTimeMixer、Moment、Timer、UniTS、DADA 等，通过大规模预训练构建通用时序表示，再适配异常检测任务。

8.6 评估层

模型性能评估主要包括两方面内容：**评估设置与评估指标**。评估设置规定了模型训练和测试的策略，以保证不同方法的比较公平性；评估指标则用于量化模型在具体任务上的表现，包括预测精度和异常检测能力以及模型效率等指标。通过系统地结合评估设置与指标，可以全面、可靠地衡量模型在各类时间序列任务中的性能。

8.6.1 评估设置

评估设置部分用于规定不同模型在测评过程中的训练与测试策略，确保比较结果的公平性与可复现性。由于时间序列模型在规模与预训练能力上的差异，其评估方式根据**端到端小模型与时间序列基础模型**的不同分为三类，如图 8.5 所示。这三种设置分别对应从充足监督到完全无监督的不同学习情形，反映模型在数据驱动学习与知识迁移能力上的差异。具体定义如下：

- **Zero-shot**：该场景评估模型在完全无已见样本条件下的通用推理能力。模型仅依赖自然语言提示或任务描述进行预测与判断，体现其预训练知识的迁移能力与语义泛化水平，尤其适用于跨领域或跨任务评估，如图 8.5a。

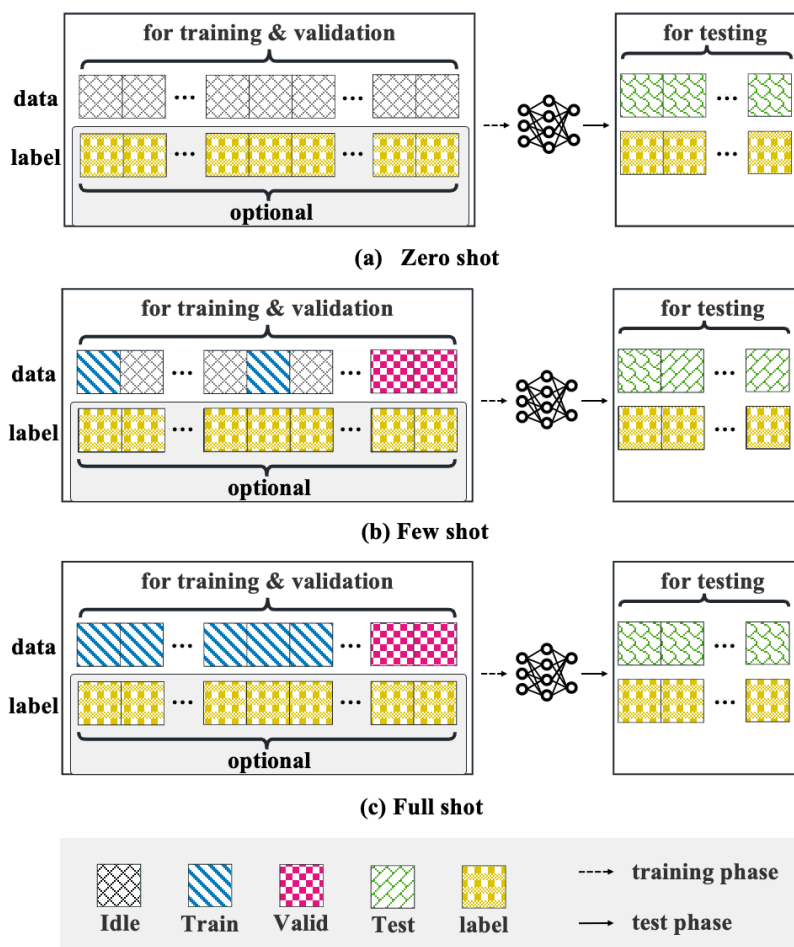


图 8.5: 时间序列测评评估场景

- **Few-shot:** 该场景仅提供少量标注样本，用于让模型学习任务格式与预测目标。由于训练样本有限，样本的选择和分布对模型性能具有显著影响，如图 8.5b。
- **Full-shot:** 该场景假设模型可充分访问训练集中的全部样本，通过多轮迭代学习时间序列的动态模式与依赖结构，从而在测试阶段执行预测或检测任务，如图 8.5c。

在第 3 到中 6 章节介绍的端到端时间序列模型通常在 Full-shot 场景下进行训练与评估，以充分发挥其对大量监督数据的建模能力。而第 7 章中的时间序列基础模型则更关注 Few-shot 与 Zero-shot 条件下的泛化与迁移能力。其中，在 Few-shot 场景中，少量样本的选择方式对模型性能具有显著影响，常用的采样策略如图 8.6 所示。

- **窗口采样 (Window Sampling):** 以连续时间窗口为单元提取片段，适配大模型的序列建模能力：
 - 前端窗口采样：选取时间轴前段窗口片段，捕获初始模式特征。

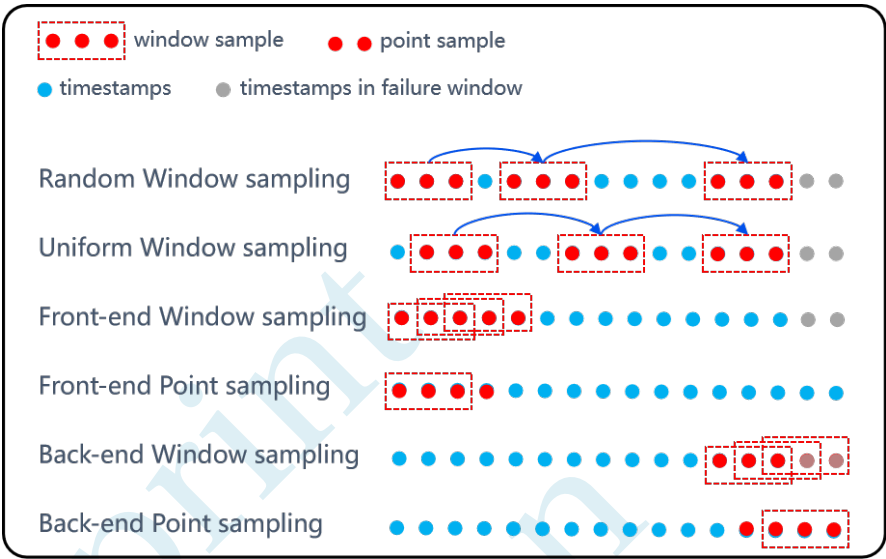


图 8.6: 时间序列大模型在 Few-shot 场景下的数据划分方式

- 后端窗口采样：聚焦时间轴末端窗口片段，提取最新动态特征。
- 均匀窗口采样：等间隔选取窗口片段，保证全局模式覆盖。
- 随机窗口采样：随机锚点生成窗口片段，增强分布鲁棒性。
- **点采样 (Point Sampling)**：由于需保证单个样本的时间连续性，在固定采样率（如 5%）下无法随机离散选择点，因此仅采用位置策略。
 - 前端点采样：在时间轴前段连续采样指定采样率的时间点。
 - 后端点采样：在时间轴末端连续采样指定采样率的时间点。

此外,在预测类任务中,预测目标通常分为两种形式:**固定预测(Fixed Prediction)**和**滚动预测(Rolling Prediction)**,如图 8.7所示。前者在固定时间窗口上进行单次预测,而后者在连续时间步上迭代执行预测,从而更接近真实应用场景。相较之下,异常检测任务的目标更为直接,即识别输入序列中的异常片段(或异常点),而无需进一步区分。

- **固定预测**: 如图8.7a, 固定预测中, 总共 n 个时间点, 定义预测步长为 f 个时间点, 则 $(n-f)$ 个时间点作为训练数据。
- **滚动预测**: 如图8.7b, 蓝色正方形代表训练集中的观测值, 绿色正方形代表测试集中的观测值, 白色正方形代表时间序列中的其余观测值。除了最后一次迭代, 每次迭代训练集都会增加一个固定数量的观测值(滚动步长)。在最后一次迭代中, 训练集被扩大, 以使两个集合一起覆盖整个时间序列。对于每次迭代, 预测方法基于当前训练集执行预测。最后, 每个预测的计算指标被平均。

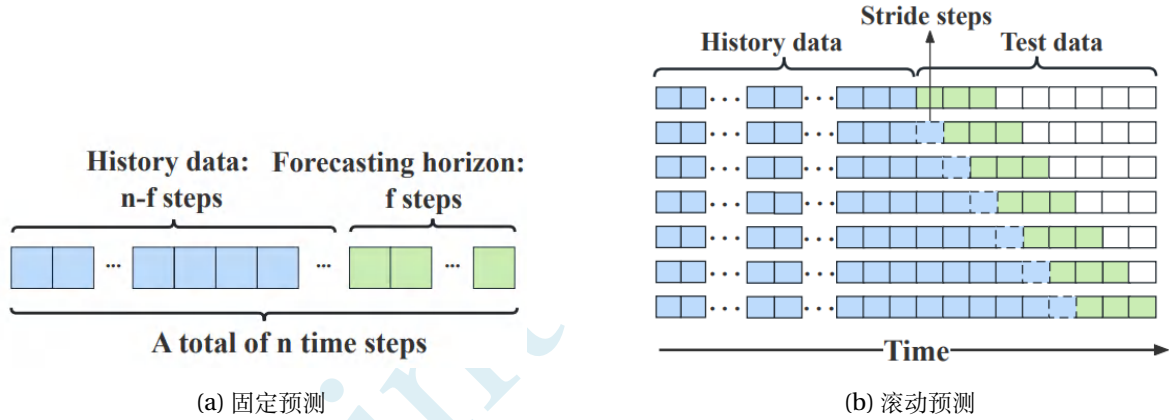


图 8.7: 时间序列预测策略

8.6.2 评估指标的选择

为了全面评估时间序列模型的能力，本文将评估指标分为**性能指标 (Performance Metrics)**与**效率指标 (Efficiency Metrics)**两大类。性能指标主要用于衡量模型在预测精度、异常检测准确性等方面的表现，反映模型的任务完成质量；效率指标则关注模型在计算资源消耗、推理速度及参数规模等方面的表现，用于评估其在实际应用中的可部署性与运行效率。二者相结合，可全面刻画模型在“效果-成本”维度上的综合性能。

性能指标 (Performance Metrics)

预测任务指标用于衡量模型对未来数值的预测精度，如 MAE、RMSE、MAPE 和 MASE 等；异常检测指标用于评估模型在识别异常点上的能力，如精确率、召回率、F1 值以及 AUC-ROC 等。通过分别考察预测和异常检测性能，可以更全面地理解模型在不同任务上的表现。

- 在**时间序列预测任务**中，常用的评估指标主要衡量模型在未来时间点上的数值偏差。设训练序列中共有 M 个数据点，预测范围为 h ，预测值为 F_k ，真实值为 Y_k ，则常用指标包括：

– Mean Squared Error (MSE):

$$MSE = \frac{1}{h} \sum_{k=1}^h (F_k - Y_k)^2 \quad (249)$$

– Mean Absolute Scaled Error (MASE):

$$MASE = \frac{\sum_{k=M+1}^{M+h} |F_k - Y_k|}{\frac{h}{M-S} \sum_{k=S+1}^M |Y_k - Y_{k-s}|} \quad (250)$$

- Mean Percentage Error (MPE):

$$MPE = \frac{1}{h} \sum_{k=1}^h \frac{(Y_k - F_k)}{Y_k} \times 100\% \quad (251)$$

- symmetric Mean Absolute Percentage Error (sMAPE):

$$sMAPE = \frac{100\%}{h} \sum_{k=1}^h \frac{|F_k - Y_k|}{(|Y_k| + |F_k|)/2} \quad (252)$$

- Mean Absolute Error (MAE):

$$MAE = \frac{\sum_{k=1}^h |F_k - Y_k|}{h} \quad (253)$$

- Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{\frac{\sum_{k=1}^h |F_k - Y_k|^2}{h}} \quad (254)$$

- 在时间序列异常检测任务中，模型的输出通常为每个时间点的异常分数或二值标签。常用指标用于衡量检测的准确性与鲁棒性，包括：

- Precision 所有预测的正类中有多少真实标签为正样本。

$$Precision = \frac{TP}{TP + FP} \quad (255)$$

- Recall 所有真实标签为正的样本有多少被预测为正。

$$Recall = \frac{TP}{TP + FN} \quad (256)$$

其中, TP (True Positive) : 正确的正例, 一个实例是正类并且也被判定成正类。FN (False Negative) : 错误的反例, 漏报, 本为正类但判定为假类。

- Accuracy 正确的样本数量占全部样本数量的比例。

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (257)$$

其中, TP (True Positive) : 正确的正例, 一个实例是正类并且也被判定成正类。FN (False Negative) : 错误的反例, 漏报, 本为正类但判定为假类。FP (False Positive) : 错误的正例, 误报, 本为假类但判定为正类。TN (True Negative) : 正确的反例, 一个实例是假类并且也被判定成假类。

- F1 score 综合考虑 Precision (精确率) 与 Recall (召回率) 的评估指标, 用于在类别不平衡的时间序列任务中衡量模型的整体判别性能。

$$F_{Score} = (1 + \beta^2) \frac{Precision \times Recall}{\beta^2 \times Precision + Recall} \quad (258)$$

其中，当 $\beta = 0$ 时， $F\text{-Score} = \text{Precision}$ ，此时为高精准率；当 $\beta \rightarrow \infty$ 时， $F\text{-Score} = \text{Recall}$ ，此时为高召回率；当 $\beta = 1$ 时，得出精准率和召回率的调和平均值；当精准率比较重要时， $0 \leq \beta < 1$ ；当召回率比较重要时， $\beta > 1$ 。

- ROC-AUC: AUC (Area Under Curve) 被定义为 ROC 曲线下的面积---见图 8.8。往往使用 AUC 值作为模型的评价标准是因为很多时候 ROC 曲线并不能清晰的说明哪个分类器的效果更好，而作为一个数值，对应 AUC 更大的分类器效果更好。
- AUC-PR (精确召回曲线下面积)：P-R 曲线刻画查准率和查全率之间的关系---见图 8.8，查准率指的是在所有预测为正例的数据中，真正例所占的比例，查全率是指预测为真正例的数据占有所有正例数据的比例。

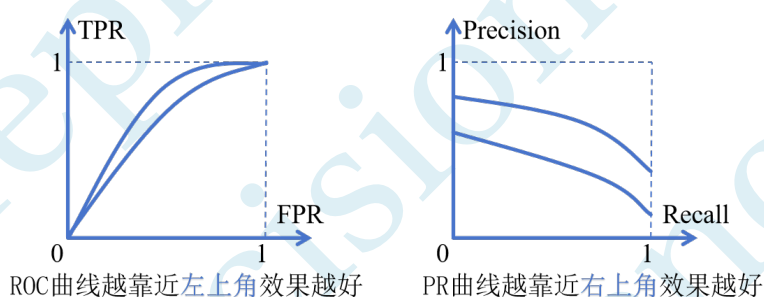


图 8.8: ROC-AUC & AUC-PR

- AP: Average Precision 简称 AP，这是一个在检索任务和回归任务中经常使用的指标，实际等于 Precision-Recall 曲线下的面积。

效率指标 (Efficiency Metrics)

旨在衡量时间序列模型在训练与推理过程中的**计算开销与资源利用水平**，是评估模型在实际应用中可部署性与扩展性的关键指标。与仅关注预测精度的性能指标不同，模型效率强调算法的运行速度、计算资源占用以及硬件适配能力，特别是在需要实时响应或大规模部署的场景下，其重要性尤为突出。常用的模型效率指标包括：

- 训练时间 (Training Time)：模型训练的时间，通常以秒或分钟为单位。
- 预训练时间 (Pretraining Time)：在大规模通用数据集上训练基础模型所消耗的时间，通常远长于微调时间，以小时、天甚至月为单位
- 推理时间 (Inference Time)：模型预测的时间，通常以秒或毫秒为单位。
- 批处理时间 (Batch Processing Time)：每批次数据处理的时间，通常以秒或毫秒为单位。

OpenTS-Bench: 测评结果

为了帮助用户深入了解各算法的性能，我们提供了全面的测评结果。我们支持高度自定义的视图，您可以根据需求筛选特定的数据集、评估指标和算法进行对比。此外，所有结果均支持一键下载及全屏展示，以优化您的查阅体验。请注意：若结果显示为 NaN，则表示该项指标因特定原因未能成功计算。

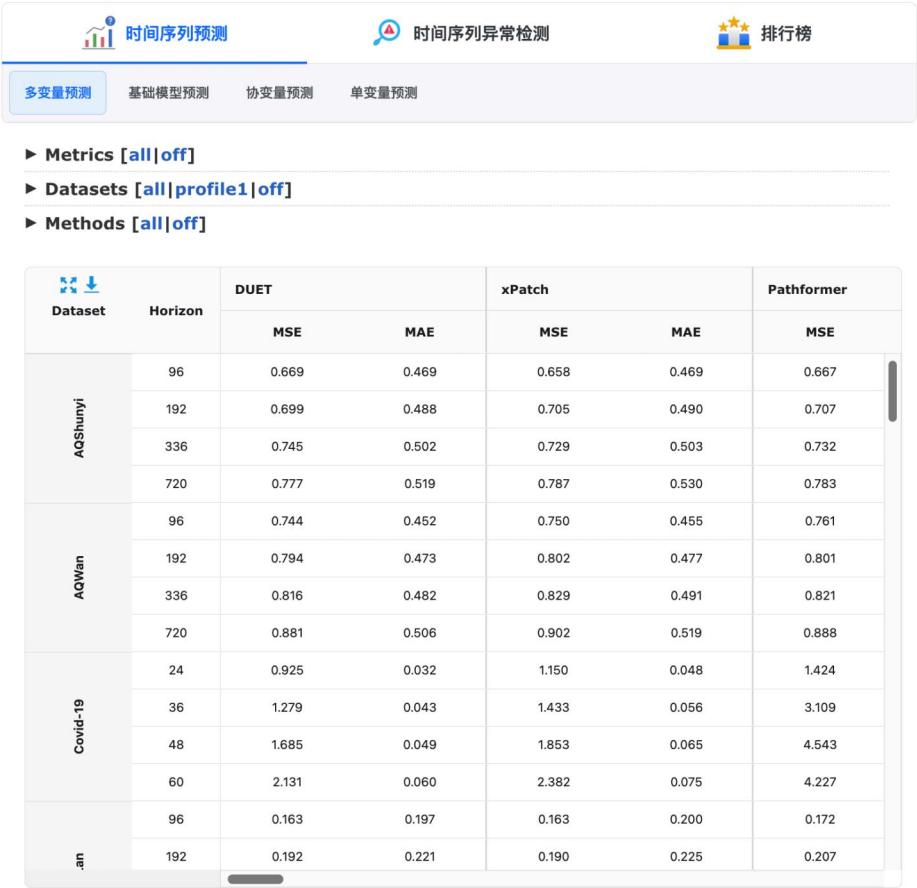


图 8.9: OpenTS-Bench 测评结果

- 每秒推理次数 (Inference per Second, IPS): 模型每秒钟可以处理的预测请求次数，通常以次数为单位。
- GPU 利用率 (GPU Utilization): GPU 在训练或预测过程中的利用率，通常以百分比的形式表示。
- 内存占用率 (Memory Usage): 模型在训练或预测过程中占用的内存大小，通常以 GB 为单位。

8.7 报告层

报告层 (Reporting Layer) 是测评体系的最上层，主要负责结果的标准化呈现、统计显著性验证、可视化展示与可复现性支持。它直接面向研究者与用户，旨在让测

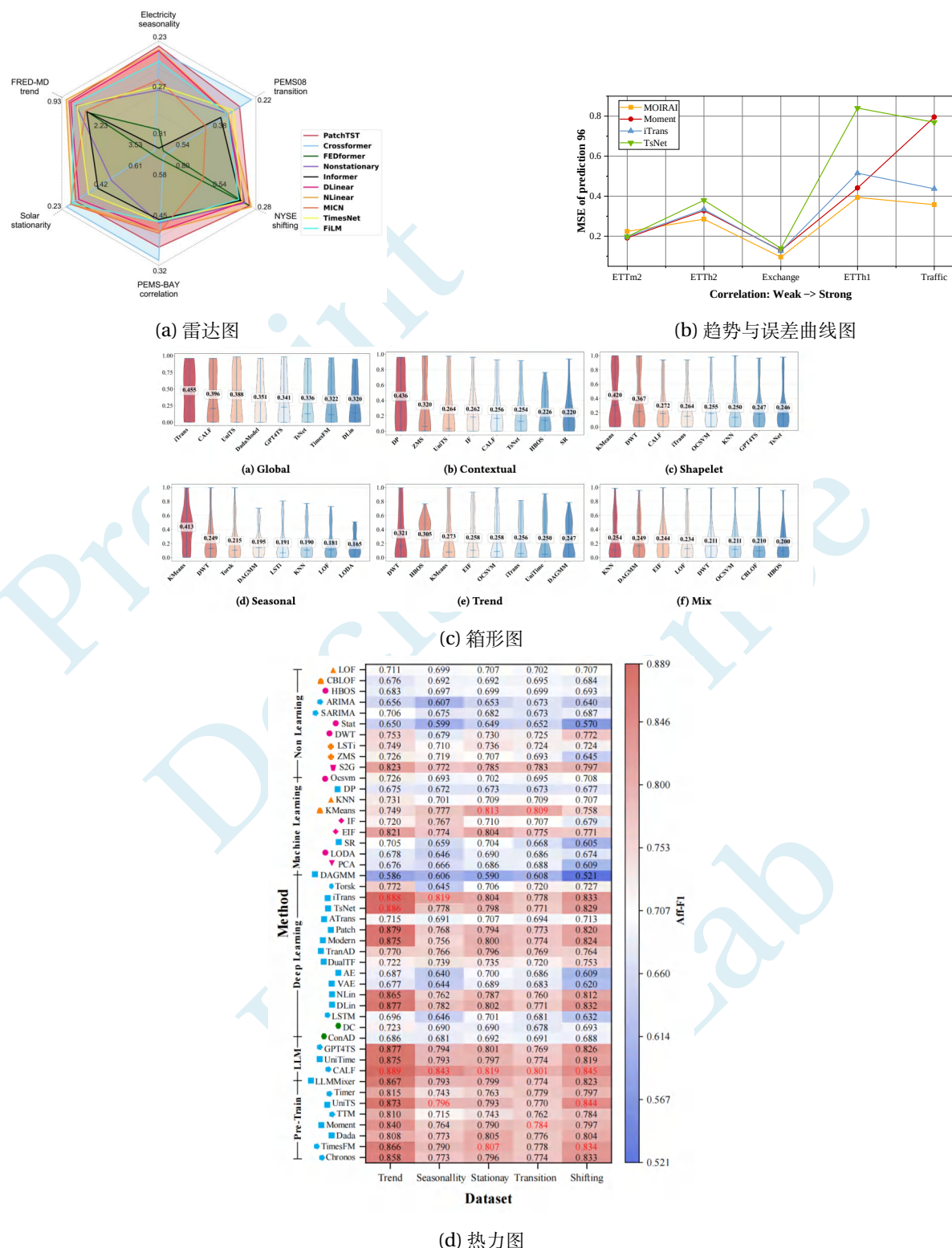


图 8.10: 可视化展示

OpenTS-Bench: 排行榜

为了促进时间序列分析领域的学术交流与技术进步，我们构建了面向时间序列预测及异常检测任务的算法排行榜。该项目的所有基准测试代码、实验结果与可复现脚本现已公开发布，以期为相关研究提供便利。

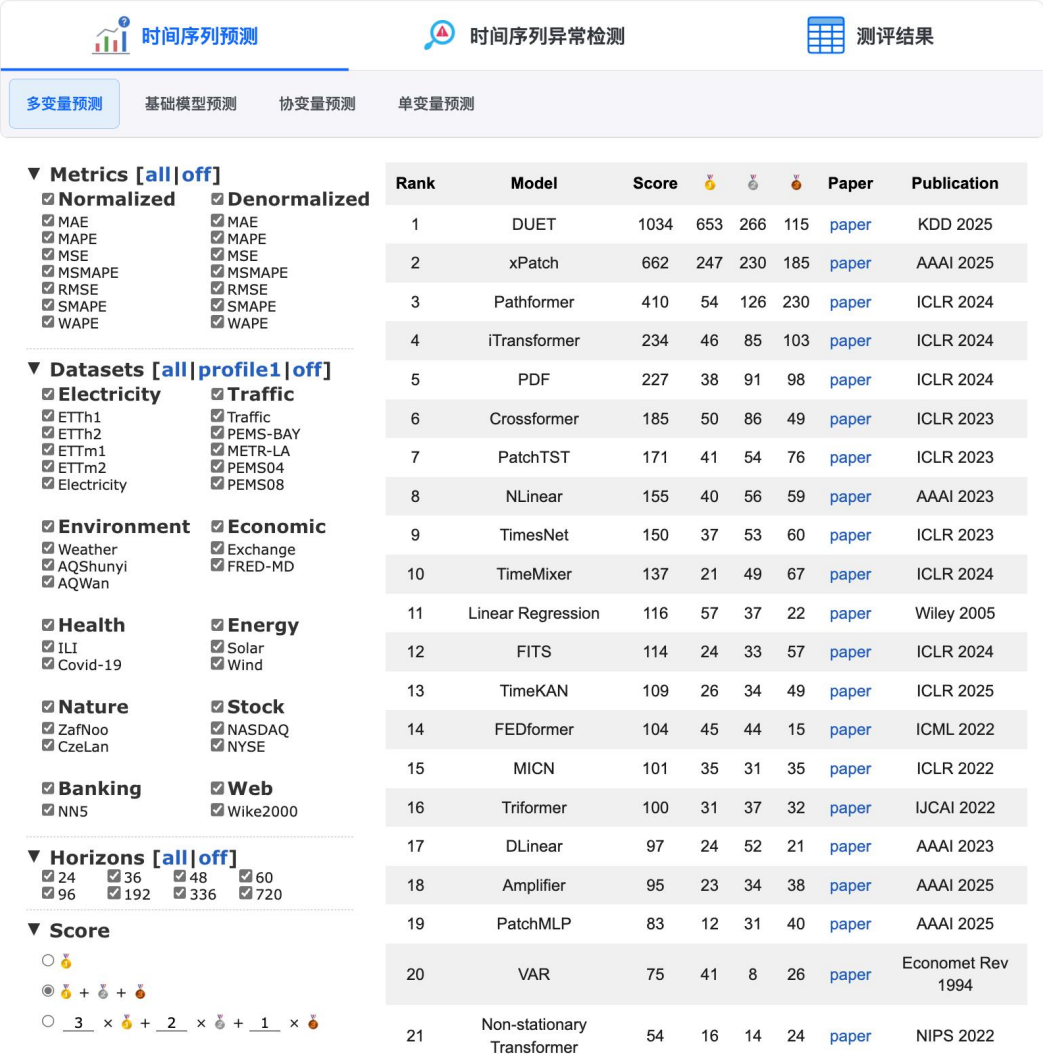


图 8.11: OpenTS-Bench 排行榜

评结果“可读、可信、可比、可复现”。一个完善的报告层不仅是结果输出的终点，也是研究复现与社区交流的起点。

报告层的设计应该考量以下几点内容：

- 结果标准化呈现：报告层首先需确保结果在格式与指标上的统一性与规范性---见图 8.9, 这种标准化不仅便于横向对比，也为自动化可视化与分析提供基础支撑。
- 可视化展示：报告层通过系统化的可视化模块，将复杂的实验结果以直观形式展示。典型可视化包括：性能雷达图 8.10a：展示模型在多指标下的平衡性；箱线

图 8.10c：刻画模型在不同数据集下的稳定性与离群情况；热力图 8.10d：反映不同模型与数据集间的性能分布规律；趋势与误差曲线图 8.10b：用于展示模型预测结果与真实值的动态对比。

- 可复现性支持: 应该记录实验配置文件、运行脚本、模型参数与日志。研究者可一键重现任意实验，确保实验的透明性与可验证性。这一机制符合当前学术界关于“开放科学 (Open Science)”的要求，也促进了时间序列社区的公平竞争与持续改进。
- 社区共享: 最后，报告层支持结果的在线共享与版本管理。研究者可选择将结果纳入公共排行榜---见图 8.11。

参考文献

- [1] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. Gift-eval: A benchmark for general time series forecasting model evaluation. *CoRR*, abs/2410.10393, 2024.
- [2] Anthony Bagnall, Hoang Anh Dau, Jason Lines, Michael Flynn, James Large, Aaron Bostrom, Paul Southam, and Eamonn Keogh. The uea multivariate time series classification archive, 2018. *arXiv preprint arXiv:1811.00075*, 2018.
- [3] André Bauer, Marwin Züfle, Simon Eismann, Johannes Grohmann, Nikolas Herbst, and Samuel Kounev. Libra: A benchmark for time series forecasting methods. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, pages 189–200, 2021.
- [4] Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The ucr time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.
- [5] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255, 2009.
- [6] Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I Webb, Rob J Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive. *arXiv preprint arXiv:2105.06643*, 2021.
- [7] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.
- [8] Vincent Jacob, Fei Song, Arnaud Stiegler, Bijan Rad, Yanlei Diao, and Nesime Tatbul. Exathlon: A benchmark for explainable anomaly detection over time series. *arXiv preprint arXiv:2010.05073*, 2020.
- [9] Kwei-Herng Lai, Daochen Zha, Junjie Xu, Yue Zhao, Guanchu Wang, and Xia Hu. Revisiting time series outlier detection: Definitions and benchmarks. In *Thirty-fifth conference on neural information processing systems datasets and benchmarks track (round 1)*, 2021.

- [10] Zhe Li, Xiangfei Qiu, Peng Chen, Yihang Wang, Hanyin Cheng, Yang Shu, Jilin Hu, Chenjuan Guo, Aoying Zhou, Qingsong Wen, et al. TSFM-Bench: A comprehensive and unified benchmark of foundation models for time series forecasting. In *SIGKDD*, 2025.
- [11] Qinghua Liu and John Paparrizos. The elephant in the room: Towards A reliable time-series anomaly detection benchmark. In *NeurIPS*, 2024.
- [12] John Paparrizos, Yuhao Kang, Paul Boniol, Ruey S Tsay, Themis Palpanas, and Michael J Franklin. Tsb-uad: an end-to-end benchmark suite for univariate time-series anomaly detection. *Proceedings of the VLDB Endowment*, 15(8):1697–1711, 2022.
- [13] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. TFB: Towards comprehensive and fair benchmarking of time series forecasting methods. In *Proc. VLDB Endow.*, pages 2363–2377, 2024.
- [14] Xiangfei Qiu, Zhe Li, Wanghui Qiu, Shiyan Hu, Lekui Zhou, Xingjian Wu, Zhengyu Li, Chenjuan Guo, Aoying Zhou, Zhenli Sheng, Jilin Hu, Christian S. Jensen, and Bin Yang. Tab: Unified benchmarking of time series anomaly detection methods. In *Proc. VLDB Endow.*, pages 2775–2789, 2025.
- [15] Zezhi Shao, Fei Wang, Yongjun Xu, Wei Wei, Chengqing Yu, Zhao Zhang, Di Yao, Guangyin Jin, Xin Cao, Gao Cong, Christian S. Jensen, and Xueqi Cheng. Exploring progress in multivariate time series forecasting: Comprehensive benchmarking and heterogeneity analysis. *TKDE*, 37:291–305, 2023.
- [16] Oleksandr Shchur, Abdul Fatir Ansari, Caner Turkmen, Lorenzo Stella, Nick Erickson, Pablo Guerron, Michael Bohlke-Schneider, and Yuyang Wang. fev-bench: A realistic benchmark for time series forecasting. *CoRR*, 2025.
- [17] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. Deep time series models: A comprehensive survey and benchmark. *TPAMI*, 2024.
- [18] Renjie Wu and Eamonn Keogh. Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 2021.

- [19] Jiawen Zhang, Xumeng Wen, Zhenwei Zhang, et al. ProbTS: Benchmarking point and distributional forecasting across diverse prediction horizons. In *NeurIPS*, 2024.

Preprint
Decision
Intelligence
Lab

9 神经微分方程时间序列分析

本章作者为：丁逸飞、田锦东、杨彬、郭晨娟。

本章主编为杨彬。

本章将介绍基于神经微分方程的时间序列建模方法。与第3章至第7章所讨论的传统时序模型不同，神经微分方程通过使用神经网络的方法对微分方程进行参数化，能够直接建模时序变量的连续动态特性，更适用于描述复杂时序系统的演化规律。本章内容由浅入深，首先从基础微分方程入手，逐步引出神经微分方程的核心思想及建模机制，最后介绍该方法在时间序列领域中的前沿研究与实际应用。具体结构安排如下：9.1章节介绍微分方程与时间序列之间的关系。9.2章节讲解神经微分方程的基本形式及其反向传播算法。9.3章节列举近年来神经微分方程在时序建模中的典型应用与研究进展，供读者进一步参考。

9.1 微分方程与时间序列

9.1.1 微分方程简介

很多动态系统都是随时间的延续而不断变化的，如单摆运动、卫星轨道、化学反应和物种竞争，而微分方程则是描述动态系统连续变化过程的数学工具。要理解微分方程，首先需要认识它的基本形式。以一阶线性微分方程为例，如下所示：

$$\frac{dy}{dx} = ky, \quad (259)$$

该方程刻画了因变量 x 关于自变量 y 的变化率，即 $\frac{dy}{dx}$ 与因变量 y 之间的正比关系 ky 。

例1：在金融学中，若银行存款以连续复利计息，设 $x(t)$ 表示时刻 t 的存款额， k 为利率， x_0 为初始存款额，则每一个瞬时的存款增长量正比于当前余额，即

$$\frac{dx}{dt} = kx, \quad x(0) = x_0, \quad (260)$$

先将变量分离得到

$$\frac{dx}{x} = kdt, \quad (261)$$

再两边同时积分，得

$$\begin{aligned} \int_0^t \frac{dx}{x} &= \int_0^t kdt, \\ \ln \frac{x(t)}{x_0} &= kt. \end{aligned} \quad (262)$$

于是得到存款 x 与时间 t 的关系为

$$x(t) = x_0 e^{kt}, \quad (263)$$

这一结果揭示了复利增长的内在机制：利息在不断积累的同时，持续参与后续利息的计算，从而形成递增加速的指数型增长趋势。

例 2：在运动学中，质量为 m 的质点通过理想弹簧沿一维往复振动，弹簧的劲度系数为 k 。设质点在时间 t 距离平衡位置的位移为 $x(t)$ ，根据胡克定律弹簧受力为

$$F_s = -kx, \quad (264)$$

其方向为指向平衡位置的方向。由牛顿第二定律得：

$$\begin{cases} m \frac{dv}{dt} = -kx, \\ \frac{dx}{dt} = v, \end{cases} \quad \begin{cases} x(0) = x_0, \\ v(0) = v_0. \end{cases} \quad (265)$$

结合以上两个例子，我们可以给出微分方程的定义：

定义 1：一个 n 阶常微分方程 (Ordinary differential equation, ODE) 是关于单个自变量 t 、因变量 x 以及它的导数 $x', x'', \dots, x^{(n)}$ 之间的关系

$$F(t, x, x', \dots, x^{(n)}; k) = 0. \quad (266)$$

其中， k 是未知参数，往往需要通过数据估算得到。在例 1 中，因变量 x 关于自变量 t 的导数存在函数关系，利率 k 为未知参数，可以通过存款变化推算得到。在例 2 中，弹簧振子的位移 x 和速度 v 关于时间 t 的导数存在函数关系，弹簧的劲度 k 为未知参数。

当自变量的个数大于 1 时，该方程称为偏微分方程 (Partial differential equation, PDE)。常微分方程和偏微分方程统称为微分方程。当微分方程可以显式地表示为导数关于其变量的函数时，常将其写为如下标准形式：

$$x^{(n)} = f(t, x, x', \dots, x^{(n-1)}; k), \quad (267)$$

其中，等号右端的函数 f 被称为该系统的动力学函数 (dynamics function)。这种形式特别适合用于建模系统的演化过程，以及数值求解与神经微分方程等现代方法的实现。例 2 弹簧振子模型按照上述形式可写为：

$$x'' = x^{(2)} = -\frac{kx}{m}. \quad (268)$$

基于上述定义，生活中处处存在可被微分方程建模的动态过程：

- 钟摆的摆动可以用二阶微分方程刻画，揭示它与重力、摆长的关系；
- 疫情期间的感染人数预测，常基于包含传播率的微分方程模型；
- 天气预报中大气运动的模拟，则依赖于复杂的偏微分方程组。

9.1.2 微分方程求解方法

除了在例 1 中提到的分离变量法，常见的求解微分方程解析解的方法还有积分因子法、拉普拉斯变换法、变量代换法等方法，有兴趣的读者可以参考 [11, 12] 自行学习，这里就不再赘述。

但是在实际应用中，能求出解析解的微分方程只占少数，当方程无法求得解析解或解析解形式复杂时，通常采用数值求解的方法。对于初值问题

$$\frac{dx}{dt} = f(t, x), \quad x(t_0) = x_0, \quad (269)$$

现代科学计算软件中提供了多种高精度的数值积分求解算法，常用的有以下两类：

1. 欧拉 (Euler) 方法

欧拉方法是最基本的一阶数值积分方法，其核心思想是：在每一个小步长区间内，用当前斜率 $f(t_n, x_n)$ 来近似函数的增量。对于步长 Δt ,

$$x_{n+1} = x_n + \Delta t f(t_n, x_n), \quad t_{n+1} = t_n + \Delta t. \quad (270)$$

欧拉方法也可以理解为在点 (t_n, x_n) 处作切线，并沿切线前进一步 Δt 得到下一个近似值。图 9.1 中展示了欧拉方法的具体流程，由于欧拉法求解过程简单，随着步长的增加，求解精度会随之下降。

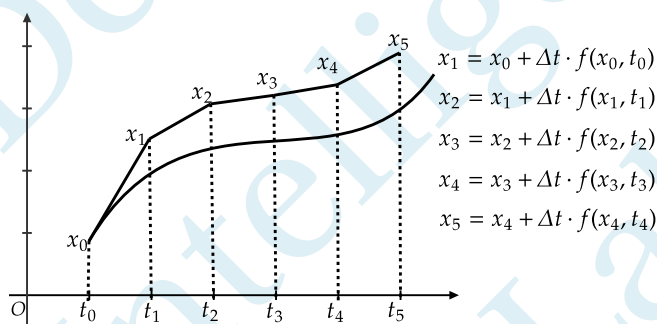


图 9.1: Euler 方法流程示意图。

2. Runge-Kutta 方法

Runge-Kutta 方法是一类高阶精度数值积分方法，它在欧拉方法的基础上，使用若干个点处的斜率的加权平均来近似函数的增量。最常用的是四阶 Runge-Kutta

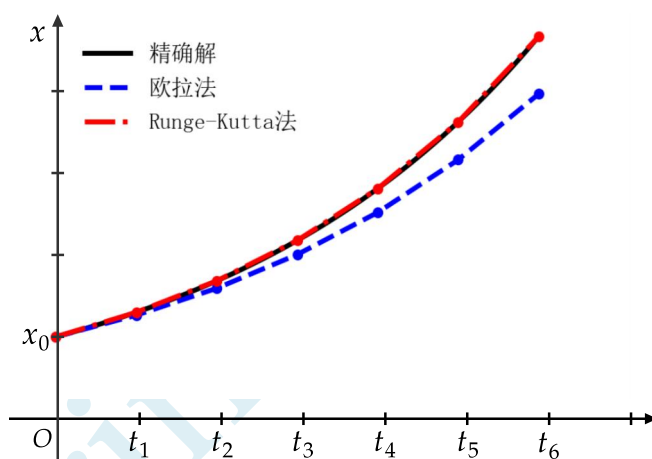


图 9.2: Euler 方法与 Runge-Kutta 方法的精度对比示意图。

方法（RK4），即选取了四个点处的斜率并进行加权平均。对于步长 Δt ，

$$\begin{aligned}
 k_1 &= f(t_n, x_n), \\
 k_2 &= f\left(t_n + \frac{\Delta t}{2}, x_n + \frac{\Delta t}{2} k_1\right), \\
 k_3 &= f\left(t_n + \frac{\Delta t}{2}, x_n + \frac{\Delta t}{2} k_2\right), \\
 k_4 &= f(t_n + \Delta t, x_n + \Delta t k_3), \\
 x_{n+1} &= x_n + \Delta t \cdot \frac{(k_1 + 2k_2 + 2k_3 + k_4)}{6}.
 \end{aligned} \tag{271}$$

图 9.2 中展示了欧拉法和 Runge-Kutta 法求解例 1 中公式 (260) 的具体流程，以及两种方法之间的精度对比，可以发现：欧拉法虽然简单但并没有 Runge-Kutta 法准确。另外，Python 中还提供了 Solve_ivp RK45、Solve_ivp LSODA、Odeint 等数值微分方程求解器，用于求解不同类型的微分方程 [6]。

9.1.3 时间依赖型微分方程与时间序列

当自变量为时间 t 时，此类微分方程被称为**时间依赖型微分方程**。它与时间序列之间存在密切联系，二者从不同角度揭示了同一系统随时间演化的规律。微分方程作为一种连续时间的动态建模工具，通过描述变量变化率与变量自身或其他量之间的关系，揭示系统的演化机制；而时间序列则是系统在离散时刻上的观测结果，体现了这些连续变化在时间维度上的外在表现。换言之，微分方程提供了时间序列的生成规律，而时间序列则是微分方程解的离散化表达。

从趋势性的角度来看，微分方程能够刻画时间序列的长期变化趋势。例如，在连

续复利模型中，存款额 $x(t)$ 满足

$$\frac{dx}{dt} = kx, \quad (272)$$

其解为

$$x(t) = x_0 e^{kt}. \quad (273)$$

如图9.3(a)所示，该解函数反映出存款额随时间呈指数型增长，显著展示了时间序列的趋势性特征，揭示了系统在时间上具有持续积累或衰减的趋势。

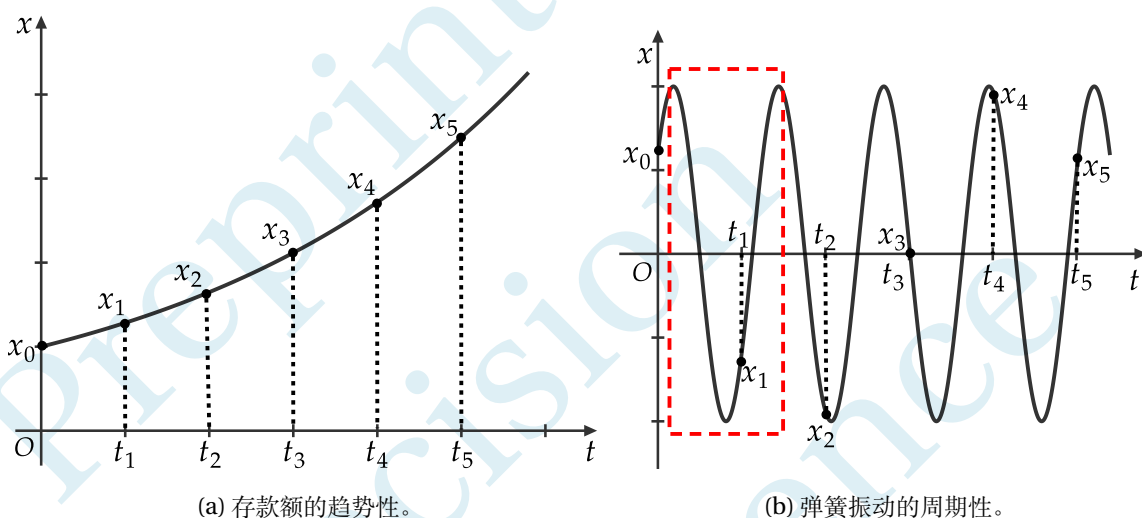


图 9.3: 时间依赖型微分方程解函数中的时序特性。

从周期性的角度来看，微分方程同样能够揭示时间序列的周期变化规律。例如，质点的弹簧振动微分方程如下所示：

$$\begin{cases} m \frac{dv}{dt} = -kx, \\ \frac{dx}{dt} = v. \end{cases} \quad (274)$$

其解为

$$\begin{cases} x(t) = A \cos\left(\sqrt{\frac{k}{m}} t\right) + B \sin\left(\sqrt{\frac{k}{m}} t\right), \\ v(t) = -A \sqrt{\frac{k}{m}} \sin\left(\sqrt{\frac{k}{m}} t\right) + B \sqrt{\frac{k}{m}} \cos\left(\sqrt{\frac{k}{m}} t\right). \end{cases} \quad (275)$$

如图9.3(b)所示，质点位移 $x(t)$ 表现出典型的周期性振荡行为。该模型说明，当系统中存在回复力或平衡机制时，变量会围绕某个平衡位置周期性波动。如果进一步考虑阻尼或外力的作用，系统还会表现出衰减振荡或受迫振荡等更复杂的周期模式。

9.2 神经微分方程

9.2.1 神经微分方程简介

虽然微分方程能够描述时间序列的动态演化过程，并且通过求解方程即可实现对未来序列的预测，但在大多数实际问题中，微分方程的具体形式往往是未知的。另一方面，考虑到现代传感器技术的发展使我们能够收集大量时序数据。因此，研究者开始尝试从数据中学习微分方程的形式。随着近年来神经网络建模能力的显著提升，一种新兴的方法是使用神经网络构建可学习的微分方程，通过梯度下降等优化技术，使其逐步逼近能够拟合数据分布的动力学系统 [5]。

在这一背景下，**神经微分方程 (Neural Ordinary Differential Equation, Neural ODE)** 应运而生。如公式(276)所示，其核心思想是用神经网络对微分方程的右端函数进行参数化，从而构建一个端到端可学习的、连续时间的动态系统。NeuralODE 提供了一种自然的连续时间建模框架，能够更精确地刻画系统状态随时间的演化，尤其适用于不规则采样、稀疏观测等实际应用场景中常见的问题。形式如下：

$$\frac{d\mathbf{x}(t)}{dt} = f_{\theta}(\mathbf{x}(t), t). \quad (276)$$

其中， $\mathbf{x}(t) \in \mathbb{R}^N$ 表示系统中 N 个变量在时间 t 的状态， f_{θ} 是由神经网络参数化的函数，学习目标是拟合数据中蕴含的隐含动力学规律。通过给定初始状态 $\mathbf{x}(t_0) \in \mathbb{R}^N$ ，可以通过数值微分方程求解器（如 Euler 法、Runge-Kutta 法等）计算任意时间点的状态，从而生成一条连续轨迹。当求解器为欧拉法时，图 9.4 展示了微分方程与神经微分方程的不同之处，即迭代增量为确定性函数还是可学习的神经网络。

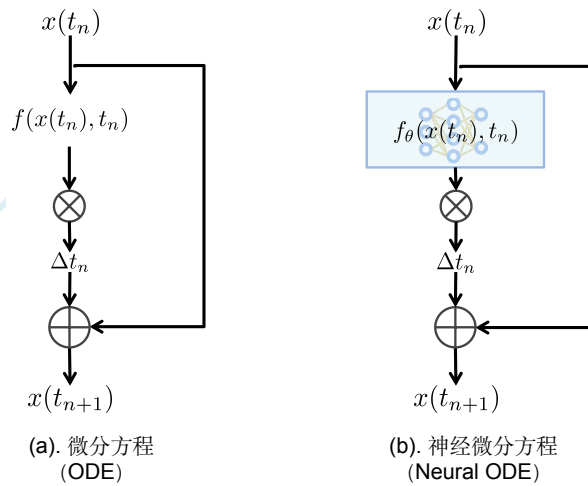


图 9.4: 微分方程与神经微分方程的区别。

与传统时序模型（RNN, Transformer）相比，NeuralODE 建模时序背后的动态

系统具备天然优势。因为其能够避免固定时间步长的限制，提供更强的建模灵活性和解释能力。图 9.5(a) 展示了传统 RNN 的工作流程，其计算公式为：

$$\begin{aligned}\hat{x}_{t_1}, h_1 &= \text{RNN}(x_{t_0}, h_0), \\ \hat{x}_{t_{i+1}}, h_{i+1} &= \text{RNN}(\hat{x}_{t_i}, h_i),\end{aligned}\quad (277)$$

其中， h_i 为隐状态。RNN 对应的均方损失函数为

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (x_{t_i} - \hat{x}_{t_i})^2. \quad (278)$$

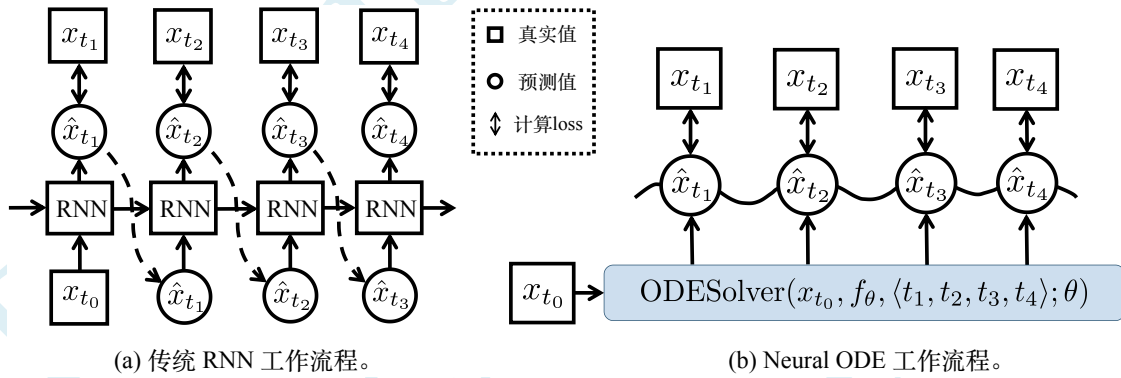


图 9.5: RNN 与 Neural ODE 的不同工作流程

给定目标求解的时间戳序列 $\langle t_1, \dots, t_r \rangle$ ，我们可以调用第 9.1.2 章节中介绍的微分方程求解器，从初始时刻 t_0 出发，计算未来时序。形式如下：

$$\hat{x}_{t_1}, \hat{x}_{t_2}, \dots, \hat{x}_{t_n} = \text{ODESolver}(x_{t_0}, f_\theta, t_0, \langle t_1, \dots, t_n \rangle). \quad (279)$$

其中， $\text{ODESolver}(\cdot)$ 表示数值求解器，它根据初始状态 $x(t_0)$ 、神经网络参数化的动力学函数 f_θ 、起始时间 t_0 以及目标时间戳列表，输出 Neural ODE 在指定时刻的状态估计。求解出的状态轨迹通常对应于感兴趣的观测时刻，或是待预测的未来时间序列。图 9.5(b) 展示了 Neural ODE 的工作流程，其均方损失函数同样为

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (x_{t_i} - \hat{x}_{t_i})^2. \quad (280)$$

在下一节中，我们将进一步讨论如何训练神经微分方程模型，使其能够从已有的时序数据中学习出最契合的动态系统。

9.2.2 反向传播算法

与传统神经网络类似，Neural ODE 也需要通过反向传播算法计算梯度，通过梯度下降算法来优化其中的可学习参数。

1. 直接法

直接法通过对 ODE 求解器中每步前向的计算过程进行梯度累积，但是由于求解一次 ODE 需要大量计算，反向传播会占用大量的内存，造成巨大的空间复杂度和额外的数值误差。因此，[1] 提出了邻接伴随法，通过牺牲时间复杂度来降低空间复杂度并保持数值稳定性。

2. 邻接伴随法

邻接伴随法通过构建方程求解来计算求解器中反向传播的梯度，使得每个时间求解步的梯度不用被缓存，从而大大降低了空间复杂度。具体来说，考虑优化一个损失函数 $L(\cdot)$ ，其输入为 ODE 求解器的结果：

$$L(\mathbf{x}(t_1)) = L\left(\mathbf{x}(t_0) + \int_{t_0}^{t_1} f(\mathbf{x}(t), t, \theta) dt\right) = L(\text{ODESolve}(\mathbf{x}(t_0), f, t_0, t_1, \theta)). \quad (281)$$

为了优化 L ，我们需要求出其关于 θ 的梯度。第一步是确定损失函数的梯度在每个时刻如何依赖于隐状态 $\mathbf{x}(t)$ 。这个量被称为伴随态 $\mathbf{a}(t) = \partial L / \partial \mathbf{x}(t)$ 。其动力学方程为：

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \mathbf{x}}, \quad (282)$$

可将其视为链式法则的瞬时模拟。可以通过再次调用微分方程求解器计算 $\partial L / \partial \mathbf{x}(t_0)$ ，该求解器需反向运行，从初始值 $\partial L / \partial \mathbf{x}(t_1)$ 出发进行反向积分。需要特别注意的是，求解该微分方程需掌握 $\mathbf{x}(t)$ 在整个轨迹上的状态信息。但在实际计算中，我们可以将状态量 $\mathbf{x}(t)$ 与伴随变量 $\mathbf{a}(t)$ 按时间逆向联合求解，初始值设定为 $\mathbf{x}(t)$ 的终值。此时参数 θ 的梯度计算即为：

$$\frac{dL}{d\theta} = - \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta} dt. \quad (283)$$

图 9.6 展示了 Neural ODE 反向传播的具体过程，[1] 发布了基于 PyTorch 框架的实现代码，其中包含若干标准常微分方程求解器的 GPU 加速版本，代码仓库地址为 github.com/rtqichen/torchdiffeq。

表 9.1 展示了直接法与邻接伴随法间的时空复杂度的差异，其中 RK-Net 代表直接根据 Runge-Kutta 求解器的计算过程计算梯度，ODE-Net 表示使用邻接伴随法进行梯度的计算， $\tilde{L}$ 代表运行一次 ODE 求解器所需要的计算次数。

9.3 相关应用研究

神经微分方程 (Neural ODE) 凭借其连续时间动态建模的核心特性，正在重塑时序预测的研究范式。与传统离散模型（如 RNN、LSTM）相比，Neural ODE 将隐空

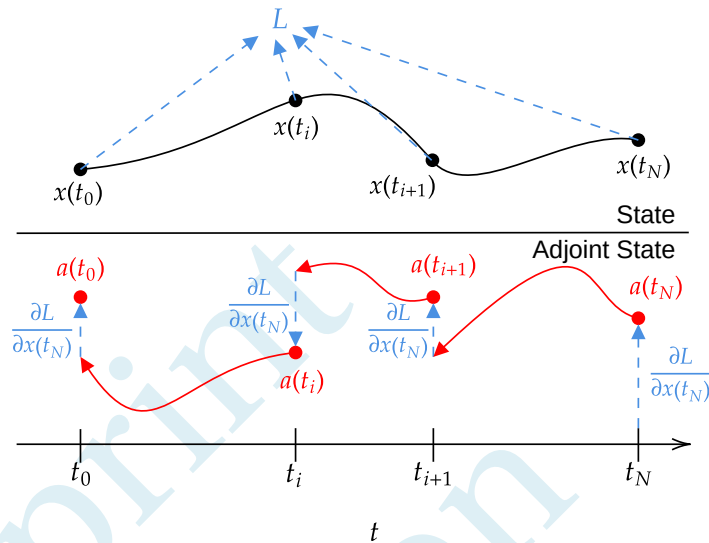


图 9.6: 神经常微分方程的反向传播过程 [1]。

表 9.1: 直接法与邻接伴随法之间的时空复杂度差异。

模型	空间复杂度	时间复杂度
RK-Net	$\mathcal{O}(\tilde{L})$	$\mathcal{O}(\tilde{L})$
ODE-Net	$\mathcal{O}(1)$	$\mathcal{O}(\tilde{L})$

间状态的演化过程表述为微分方程 $\frac{d\mathbf{h}(t)}{dt} = f_{\theta}(\mathbf{h}(t), t)$ 的求解问题，从而在不规则采样、长期依赖建模以及小样本泛化等任务中展现出显著优势。不仅如此，神经微分方程还构建了具有物理意义的微分方程与神经网络之间的联系，这为知识-数据双驱动的深度学习提供了一个可行的技术路径。本节将结合近年论文中的典型案例，探讨其实际应用价值。

1. 医疗健康

在电子健康记录（EHR）中，患者生理指标（如心率、血糖）的监测常伴随非均匀采样与数据缺失问题。Chen 等人 [2] 提出的 MP-ODE 模型以神经常微分方程（Neural ODE）为核心，能够自然地建模连续时间动态，从而有效应对不规则序列。其主要创新包括：融合差分算子以显式计算速度和加速度等运动信息，引入位置编码以注入时间特征，并且设计 ODE-GRU 单元以结合离散观测与连续演化过程；此外，模型采用四元数损失函数以提升训练稳定性。实验结果表明，MP-ODE 在标准数据集上无论面对固定帧率输入还是存在严重丢帧的不规则序列，均表现出优异且领先的性能，显著增强了模型在实际应用中的鲁棒性。

2. 工业物联网

工业传感器数据的非均匀性与噪声干扰长期以来是时序预测的主要瓶颈。MIT 团队 [3] 提出的 CfC (Closed-form Continuous-time Networks) 模型, 通过为连续时间神经网络的微分方程推导近似闭式解, 从根本上消除了对数值求解器的依赖及其计算瓶颈。得益于此, CfC 在训练与推理阶段的速度比传统方法快 100 至 100,000 倍, 同时在多种复杂时序任务上, 其精度可达到或超越现有最优模型 (如 RNN、Transformer 和 Neural ODE)。该模型尤其适用于需高效处理非均匀采样或快速响应的时序数据场景。

3. 气候与交通

气候预测任务需要同时建模时间演化过程与空间关联特征。卡塔尔人工智能中心 [9] 提出的 TV-ODE 框架在损失函数中引入基于总变差 (Total Variation, TV) 的感知正则化项, 从而显式地控制模型在预测精度与时间动态保真度之间的平衡。实验结果表明, 该方法在交通 (Traffic4cast)、出租车需求 (TaxiBJ) 和气象 (WeatherBench) 等数据集上, 能够在保持预测精度较高的同时, 显著提升预测的感知质量 (例如时间波动保真度最高提升 45.9%), 并揭示了模型架构中隐含的权衡机制。

4. 小样本场景

当训练数据稀缺时, 纯数据驱动模型往往容易出现过拟合问题。中国科学院团队 [10] 提出了灰色神经 ODE (GM-ODE) 模型, 该模型将灰色 GM(1, N) 的白化方程嵌入 Neural ODE 框架中, 利用灰色理论在小样本建模中的优势来增强 Neural ODE 的泛化能力。同时, 模型采用 SoftPlus 激活函数与四阶 Runge-Kutta 数值积分方法, 以确保解的唯一性与数值稳定性。在两项能源预测任务 (中国电力消费与煤炭占比) 中, GM-ODE 的测试集平均 MAPE 分别达到 0.82% 和 1.13%, 显著优于十种基准模型 (如 GM、LSTM、ARIMA), 验证了其在小样本非线性预测中的高精度与强鲁棒性。

5. 知识-数据双驱动的深度学习

在很多应用场景中, 知识的表现形式为各式各样的数学物理方程 (例如: 流体力学中的纳维斯托克斯方程, 传染病动力学方程, 污染物传播的扩散-对流方程)。然而, 数据驱动的神经网络往往只能从数据中学习这些机理, 这种对数据的极高依赖会降低可泛化性与建模精确性。因此, 需要引入知识-数据双驱动来解决该问题。神经微分方程恰巧建立了实际微分方程与神经网络之间的联系, 这为知识-数据双驱动提供了新的技术支持。AirPhyNet [4] 通过神经微分方程建立了污染物传播机理与神经网络的关联, 成功将微分方程作为知识嵌入到神经网络中。

ClimODE [8] 和 Air-DualODE [7] 分别建立了数据驱动与知识驱动模块，通过融合的方式建立知识-数据双驱动模型，有效展现了数学物理方程知识对数据驱动模型的促进作用。

尽管成果显著，当前研究仍面临明显挑战。首先在计算效率层面，高精度求解器（如 Dopri5）的长序列计算开销大，CfC 等闭式解模型是优化方向。其次，面向复杂动态建模，尤其是混沌系统（如湍流）的预测需结合随机微分方程（SDE）扩展。最后，在评估标准革新层面，需设计超越均方误差（MSE）的时空敏感指标（如动态时间规整 DTW、物理守恒律约束）。

参考文献

- [1] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [2] Yang Chen, Hong Liu, Pinhao Song, and Wenhao Li. Neural ordinary differential equation for irregular human motion prediction. *Pattern Recognition Letters*, 178:76–83, 2024.
- [3] Ramin Hasani, Mathias Lechner, Alexander Amini, Lucas Liebenwein, Aaron Ray, Max Tschaikowski, Gerald Teschl, and Daniela Rus. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 4(11):992–1003, Nov 2022.
- [4] Kethmi Hirushini Hettige, Jiahao Ji, Shili Xiang, Cheng Long, Gao Cong, and Jingyuan Wang. Airphynet: Harnessing physics-guided neural networks for air quality prediction. In *International Conference on Learning Representations (ICLR)*, 2024.
- [5] Patrick Kidger. On neural differential equations. *University of Oxford*, 2021.
- [6] Seonkyu Lim, Jaehyeon Park, Seojin Kim, Hyowon Wi, Haksoo Lim, Jinsung Jeon, Jeongwhan Choi, and Noseong Park. Long-term time series forecasting based on decomposition and neural ordinary differential equations. In *2023 IEEE International Conference on Big Data (BigData)*, pages 748–757. IEEE, 2023.
- [7] Jindong Tian, Yuxuan Liang, Ronghui Xu, Peng Chen, Chenjuan Guo, Aoying Zhou, Lujia Pan, Zhongwen Rao, and Bin Yang. Air quality prediction with physics-guided dual neural odes in open systems. In *International Conference on Learning Representations (ICLR)*, 2025.
- [8] Yogesh Verma, Markus Heinonen, and Vikas Garg. Climode: Climate and weather forecasting with physics-informed neural odes. In *International Conference on Learning Representations (ICLR)*, 2024.
- [9] Shehel Yoosuf, Hamza Baali, and Abdesselam Bouzerdoum. Improving perceptual quality in spatiotemporal timeseries forecasting. *Engineering Applications of Artificial Intelligence*, 156:111062, 2025.

- [10] Fangxue Zhang, Xinping Xiao, and Mingyun Gao. An extended neural ordinary differential equation network with grey system and its applications. *Neuro-computing*, 576:127343, 2024.
- [11] 同济大学数学系. 高等数学. 高等教育出版社, 2014.
- [12] 王高雄. 常微分方程. 高等教育出版社, 2020.

Preprint
Decision
Intelligence
Lab